TKS v8.0: A Typed Finite-Function Kernel for Cognitive-Architecture Semantics

Little Henderson

Research monograph v0.3, 2026-08. Artifact status: DRAFT / OPEN / RING 4 / NOT LICENSED / HELD. This monograph expands the corresponding v0.2 research paper over the same Project Coherence Gate 1 / Gate 4 source artifacts; every status label, proof status, hash identity, and open obligation of those sources governs this text. Drafting this monograph promotes no claim; independent review remains open.

Contents

Chapter 1. Introduction 1
1. The TKS/CAS formal program and its compiler 1
2. Kinds are not types 3
3. Failure mode I: untyped composition 3
4. Failure mode II: invalid power inference 5
5. Failure mode III: nondeterministic identity 6
6. The scenarios replayed against the kernel 8
7. Design philosophy of the kernel 9
8. Contributions and their evidential status 10
9. Positioning among neighboring disciplines 11
10. Reading guide 12
11. Notation and conventions 13

Chapter 2. Mathematical Preliminaries 15
1. Finite sets and total functions 15
2. Images, fibers, and the classification of maps 16
3. Counting total maps 18
4. The algebraic ladder: semigroups to categories 20
5. Functional graphs of endomaps 22
6. Finite-dimensional Hilbert spaces 25
7. Words and the shortlex order 28
8. Summary of the toolkit 31

Chapter 3. Objects, Arrows, and the Determinism Discipline 32
1. Objects 32
2. Name-inclusive identity 33
3. The ordered-input rule 35
4. Hash-order nondeterminism as an evidence problem 37
5. Arrow judgments and extensional equality 40
6. Presentation names are not identity 42
7. Negative-fixture testing of the constructors 43
8. Summary 44

Chapter 4. Typed Composition and the Category Laws 46
1. Identity and Composition 46
2. The Category-Laws Theorem 49
3. The Mismatch-Rejection Contract 52
4. The Verification Story I: Bounded Exhaustive Testing 54
5. The Verification Story II: The Lean Formal Seed 57
6. Exercises 58

7. Summary of Claims and Their Status 59

Chapter 5. Endomaps, Powers, and Finite Idempotence 60
1. The Endomap Construction 60
2. Monoid Power Laws 62
3. Functional-Graph Structure Theory 62
4. The Factorial Idempotence Theorem 67
5. Sharper Variants, Stated Honestly 68
6. Verification of the Idempotence Theorem 69
7. The Power API Contract 71
8. Binary Exponentiation 72
9. Bounded Presentation Names 73
10. Exercises 74
11. Summary of Claims and Their Status 75

Chapter 6. Semigroupoid Paths and the Invalid-Inference Analysis 76
1. The Path Construction 76
2. Path Algebra 78
3. The Invalid Inference 79
4. Three Case Studies 80
5. Type-Level Enforcement 83
6. Boundary: What Paths Do Not Claim 85
7. Exercises 86
8. Summary 87

Chapter 7. The Deterministic Standard-Basis Lift 88
1. Overview and evidential status 88
2. Finite-dimensional preliminaries 89
3. The lift: definition, matrix form, and running examples 90
4. Three derived operators, kept distinct 93
5. The exact norm theorem 95
6. Corollaries with proofs 99
7. Composition, fibers, and submultiplicativity 100
8. The permanent counterexample 102
9. Functoriality 103
10. Formal verification scope: verified versus open 105
11. What the lift is not 106
12. Summary and status table 107

Chapter 8. Denotational Normal Forms 109
1. The canonicalization problem 109
2. Generator catalogs 109
3. Typed words and denotations 110
4. The shortlex order 111
5. Normal forms 112
6. The representative-first closure algorithm 113
7. Soundness of pruning 114
8. Finiteness and complexity 117
9. Resource semantics 118

10. Generator-order relativity 119
11. Extended worked example: a two-object catalog, by hand 120
12. Two permanent boundaries 122
13. Status summary 123

Chapter 9. Implementation 124
1. The package and its documentary boundaries 124
2. `category.py`: the typed finite-function category 125
3. `deterministic_lift.py`: the standard-basis lift 130
4. `normal_forms.py`: the shortlex closure engine 132
5. Manifest binding of the implementation 136
6. Summary 137

Chapter 10. Verification Methodology 139
1. The bounded-exhaustive philosophy 139
2. The four suites 141
3. Coverage summary 144
4. Trusting the tests 144
5. The Gate 1 verifier 145
6. What a green suite does not establish 146

Chapter 11. The Formal Seed 147
1. Kernel-checking as a trust story 147
2. The pinned toolchain 149
3. The verifier discipline 150
4. Verified scope 150
5. Open scope 151
6. What kernel-checking does not verify 152
7. The PO-07 partial status 153

Chapter 12. Boundaries, Downstream Roles, and the Research Program 154
1. Excluded transfers 154
2. The three downstream consumers 156
3. The obligation ledger, row by row 157
4. Related work 161
5. Conclusion 162

Appendix A. Worked Examples 163
1. A three-carrier endomap and its powers 163
2. The lift of f, its operators, and its norm 164
3. A closure run with the shortlex table 166

Appendix B. The Proof-Obligation Ledger 168
1. Reference table 168
2. Per-row status and evidence 169
3. Reading conventions 171

Appendix C. Glossary and Notation Index 172
1. Terms 172
2. Symbols 175

3. Status vocabulary 176

Bibliography 177

CHAPTER 1

Introduction

This monograph develops, in full and at reference depth, a deliberately small executable kernel for the typed-composition layer of the Tootra Knowledge System (TKS) and its secular derivative, the Cognitive Architecture System (CAS). The kernel — TKS v8.0 — supplies named finite objects with explicitly ordered carriers, total finite-function arrows with declared source and target, partial type-checked categorical composition, a hard type separation between one-carrier endomaps and heterogeneous semigroupoid paths, a deterministic standard-basis lift into finite-dimensional Hilbert spaces, and stable denotational normal forms computed by shortlex minimization over a completed finite closure. Everything the kernel does is either proved here, tested by bounded exhaustive enumeration, kernel-checked in a pinned proof assistant, or explicitly recorded as open. Nothing in this monograph is peer-reviewed; the artifact status of the entire program is DRAFT / GATE 1 PROTOTYPE / RING 4 / NOT LICENSED / NOT FROZEN, and we will repeat that status wherever a reader might otherwise infer more.

The present chapter explains why so small a kernel deserves so long a book. The short answer is that the kernel exists to make three specific reasoning failures *impossible to write down*, and that the discipline required to achieve this — in the mathematics, in the implementation, and in the evidence — turns out to be instructive far beyond the kernel's size. The three failures are untyped composition, invalid power inference, and nondeterministic identity. Each is developed below with an extended worked scenario showing exactly how the failure produces a wrong conclusion from locally reasonable steps. Each scenario is then revisited at the end of the chapter to show which kernel rule makes the failing step a type error rather than a silent miscomputation.

1. The TKS/CAS formal program and its compiler

1.1. The ontology in brief. The TKS/CAS formal program, in its v7.4 form and the v7.5 through v7.7 extensions, defines a rich ontology for describing cognitive architecture: ten Noetic operators, four processing Domains, seven Core Drives, four Registers introduced in the later extensions, and a disciplined operation notation in which compound cognitive descriptions are written as expressions over these primitives. The ontology is served by a production notation compiler that parses, validates, and evaluates expressions of the notation. This monograph takes the ontology entirely as *context*: the kernel developed here deliberately encodes none of it. The refusal is not modesty for its own sake; it is a gate. Mapping the canonical Noetics or any CAS construct onto kernel arrows is a separately tracked research obligation (PO-TKS8-CORE-08 in the ledger of Appendix B), it has not been performed, and it is recorded as OPEN / NOT LICENSED. Until that mapping is reviewed, every statement in this book is a statement about named finite sets and total functions — no more.

1.2. Kinds in the production compiler. The production notation compiler enforces what we will call *value kinds*. Each value flowing through an expression carries a tag drawn

from a small fixed vocabulary — state, drive, relation, program, and so on — and the compiler rejects expressions that combine kinds in ways the grammar forbids. Kind checking is genuinely useful: it stops a drive value from being used where a relation is expected, and it catches a large class of transcription errors at parse time.

What a kind system does not do — cannot do, at this granularity — is track *categorical type*: the exact source object and exact target object of each operation. Two operations may both carry the kind "relation" while one of them is a map out of a seven-element carrier and the other is a map into a four-element carrier. The kind system sees two relations and permits the combination; the composition is nevertheless meaningless, because the intermediate objects disagree. In categorical language [2]: a kind system checks membership in a class of arrows, whereas composition legality is a statement about the *objects* at the interface, and objects are precisely what kinds erase.

1.3. The artifact and its evidence infrastructure. Because later chapters constantly refer to the kernel's evidence apparatus, we fix the vocabulary here. The *kernel* is the Python package `src/tks_v8_core`, comprising three modules: `category.py` (objects, arrows, composition, endomaps, paths), `deterministic_lift.py` (the standard-basis lift and its derived operators), and `normal_forms.py` (generator catalogs, typed words, the shortlex closure). A *manifest* binds the exact bytes of these modules and their test suites by SHA-256 hash. The *Gate 1 verifier* (`scripts/verify_tks_v8_gate1.py`) checks artifact hashes against the manifest and reruns the executable suite; the *Lean verifier* (`scripts/verify_psr8_lean_gate1.py`) rebuilds a pinned, isolated Lean 4.32.1 formal seed [16] under an audited axiom footprint. "Gate 1" names the project's first review gate: the kernel is a Gate 1 *candidate* — the manifest does not hash itself, no reviewed commit or acceptance receipt yet binds the manifest hash, and until one does, nothing here is a freeze receipt. The validation snapshot of 2026-08-04 records PASS on all 20 tests of the suite.

Two terms of art recur. A claim's *evidential status* is its entry in the proof-obligation ledger (Appendix B), drawn from a fixed vocabulary explained in Remark 1.4 below. The *ring/license* annotations (RING 4 / NOT LICENSED) are the program's admission-control labels: they mark the entire artifact as an unreviewed research prototype whose claims may not be transferred into any production, marketing, or canonical-ontology context. This monograph inherits both disciplines wholesale and adds no claim beyond what the ledger registers.

1.4. Provenance of the failure analysis. The three failure modes below were identified by the Project Coherence mathematical red-team audit of the TKS/CAS formal layer. The audit's findings are inherited here as the kernel's requirements; we restate them in our own notation and expand each into a worked scenario. The scenarios are constructed for this monograph — they are illustrative models of the failure classes, not transcripts of observed production incidents, and no claim about the behavior of any deployed system is made or implied.

ADVERSE FINDING 1.1 (Audit finding, restated). The production compiler's kind system does not enforce categorical source and target objects. Consequently three classes of invalid inference are expressible without error: (i) composition of operations whose intermediate objects disagree; (ii) application of single-operator iteration theorems to heterogeneous generator sequences; (iii) dependence of canonical representatives and digests on interpreter hash state. TKS v8.0 is the response: the smallest kernel in which each class is a type error.

2. Kinds are not types

Before the scenarios, it is worth being precise about the gap, because the gap is the entire motivation for the kernel.

A *kind*, as the production compiler uses the term, is a unary predicate on values: "this value is a relation." A *categorical type*, as the kernel uses the term, is a pair of objects: "this arrow goes from X to Y," written $f : X \to Y$. The difference matters at exactly one place: composition. Kind-level checking validates $g \circ f$ whenever f and g have composable kinds. Type-level checking validates $g \circ f$ only when the declared target object of f *equals* the declared source object of g — equality of objects, not of kinds, not of cardinalities, not of element sets.

Three progressively stricter equality tests could be imagined at the interface, and the kernel adopts the strictest:

(1) *Cardinality equality*: $|Y| = |Y'|$. This licenses the index coincidence failure of Scenario I below and is rejected.
(2) *Carrier equality*: the carriers of Y and Y' are equal as ordered sequences. This still coerces semantically distinct objects that happen to share elements — two registers both carrying $(0, 1)$ — and is rejected.
(3) *Name-and-carrier equality*: the declared names and the ordered carriers both agree. This is the kernel's rule (Chapter 3), and the anti-coercion argument for it is given at length in Section 2.

A natural question is why the response to the gap is a separate kernel rather than an extension of the production compiler's kind system into a full type system. The answer is procedural as much as technical. The production compiler is a live artifact with existing users and existing semantics; retrofitting object-level types into it would change the acceptance behavior of existing expressions, and doing so *before* the categorical layer has been independently specified, proved, and reviewed would repeat the exact mistake the audit flagged — semantics adopted by implementation accident rather than by reviewed decision. The kernel therefore stands beside the compiler, not inside it: a reference model whose small size makes complete specification and bounded exhaustive validation feasible, and whose integration with the compiler is deliberately held open as a separately gated obligation (PO-TKS8-CORE-09, OPEN). If and when integration is designed, the kernel provides the semantics to integrate *to*; until then it provides something almost as useful — a precise statement of what the kind system does not check, with executable witnesses.

The remainder of this chapter shows what goes wrong when the interface check is weaker than test (3), and, in the third scenario, what goes wrong when even test (3) is undermined by nondeterministic carrier order.

3. Failure mode I: untyped composition

3.1. The failure class. Nothing in a kind system prevents composing operations whose intermediate objects disagree. The result is a well-formed expression — it parses, it kind-checks, it may even execute — with no denotation. The most dangerous member of this class is not the case where execution crashes; it is the case where a cardinality coincidence lets an index-based implementation run to completion and return a value. The value is then read as if it meant something, and a conclusion is drawn that no step of the computation ever licensed.

3.2. Worked scenario. We construct the scenario with kernel-style objects so that the reader can see exactly which datum the weak checker ignores. Take three finite objects:

- $D = (\texttt{DriveProfile}, (d_1, d_2, d_3, d_4, d_5, d_6, d_7))$, a seven-element carrier whose positions are read, in the surrounding analysis, as seven drive labels;
- $E = (\texttt{EvalSummary}, (e_0, e_1, e_2, e_3))$, a four-element carrier of evaluative summary labels;
- $Q = (\texttt{QuadState}, (q_0, q_1, q_2, q_3))$, a four-element carrier of unrelated quadrant labels used by a different analysis pipeline.

Two total maps are on hand, both carrying the kind "relation" in a kind-checked setting:

- $p : D \to E$, a summarization step; say p has mapping tuple $(0, 0, 2, 2, 2, 1, 3)$, meaning $p(d_1) = e_0$, $p(d_2) = e_0$, $p(d_3) = e_2$, and so on.
- $r : Q \to D$, a reading step from the quadrant pipeline; say r has mapping tuple $(4, 6, 5, 0)$, meaning $r(q_0) = d_5$, $r(q_1) = d_7$, $r(q_2) = d_6$, $r(q_3) = d_1$.

An analyst wants to chase a drive profile through summarization and back into drive space, and writes the composite $r \circ p$. The composite is ill-typed: the target of p is E and the source of r is Q, and $E \neq Q$ — different names, different carriers. But both carriers have four elements. An implementation that represents maps as bare index tuples, with no declared source and target, evaluates $r \circ p$ without complaint: it takes the output index of p and uses it to index into the table of r.

Run the numbers. Start from d_3. Then $p(d_3) = e_2$, held internally as index 2. The untyped composition feeds index 2 into the table of r, whose entry at position 2 is 5 — but position 2 of the table of r is the row for q_2, not for e_2. The machine returns d_6. The analyst reads off: "chasing d_3 through summarization and reading yields d_6," and proceeds to build on the association between d_3 and d_6.

The conclusion is wrong in the strongest possible sense: not incorrect by a computable error term, but *underived*. No step of any analysis ever asserted what the reading map r does to the evaluative label e_2, because r is not defined on evaluative labels at all. The number 5 that surfaced mid-computation is the image of q_2, an element of a carrier from an unrelated pipeline, smuggled in through nothing but the coincidence $|E| = |Q| = 4$. Had $|E|$ been 5, the composition would have crashed on some inputs and the error would have been caught; the four-versus-four coincidence is exactly what makes the failure silent.

Note carefully what each checking regime says about this composite:

- *Kind checking*: p and r are both relations; relations compose; accepted.
- *Cardinality checking*: $|E| = |Q| = 4$; accepted.
- *Carrier checking*: $(e_0, e_1, e_2, e_3) \neq (q_0, q_1, q_2, q_3)$; rejected — but only because the element sets happen to differ. Rename the quadrant labels to $e_0, \ldots, e_3$ during a refactor and carrier checking silently accepts again.
- *Kernel checking*: the target object of p is $(\texttt{EvalSummary}, \ldots)$ and the source object of r is $(\texttt{QuadState}, \ldots)$; the names differ; the composition raises a typed error naming both objects. No refactor of element spellings can reintroduce the silent acceptance without also merging the declared object identities, which is a visible, reviewable act.

3.3. A taxonomy of the failure's presentations. It is worth separating the three ways an untyped composition can present, because they have very different detection profiles:

(1) *Loud*: the cardinalities disagree and some input's index falls outside the second table. The composition crashes — on those inputs, in that run. Detection is likely but not certain: if the offending inputs never occur in testing, the crash ships.
(2) *Silent-total*: the cardinalities agree, as in the worked scenario. Every input executes, every output is a well-formed element, and no observable signal distinguishes the underived result from a correct one. Detection requires semantic review of a computation that looks healthy.
(3) *Silent-partial*: the cardinalities disagree but the observed inputs happen to land inside the smaller table. The pipeline runs clean for weeks and crashes on the first unusual input — combining the worst of both: a latency-bombed crash *and* a history of silently underived outputs before it.

Only (1) is reliably caught by ordinary testing, and an adversarially unlucky data distribution converts (1) into (3). A type check at composition time collapses all three into a fourth presentation: *immediate, deterministic, input-independent rejection at construction* — the only presentation whose detection probability is 1 and whose detection latency is zero.

3.4. The lesson. The failure is not exotic. Index-tuple representations of finite maps are the natural implementation choice, and cardinality coincidences among small carriers are the norm, not the exception — the TKS/CAS ontology alone has a four-element Domain axis, a four-element Register axis, and numerous four-valued summaries. Any two of these will silently interchange under an untyped composition. The kernel's response (Chapter 3 and Chapter 4) is that an arrow *is* its judgment: source object, target object, and mapping tuple travel together, and composition is a partial operation defined only when the middle objects are equal as named, ordered objects.

4. Failure mode II: invalid power inference

4.1. The failure class. Finite endomaps obey strong iteration theorems. The one this monograph proves as Theorem 5.1 (Chapter 5; the underlying mathematics is Theorem 2.32 of the primer) is the safe finite power: for any total $f : X \to X$ on a finite carrier, the power $f^{|X|!}$ is idempotent, $f^{|X|!} \circ f^{|X|!} = f^{|X|!}$. Results of this shape license fixed-point and stabilization conclusions: iterate long enough and the process reaches an idempotent regime.

Every such theorem has a hypothesis that informal operator notation makes easy to lose: the iterated thing must be *one* endomap — a single element of the finite monoid $\mathrm{End}(X)$, applied repeatedly. A workflow that applies generators $g_1, g_2, g_3, \ldots$ in temporal order is not, in general, iterating one endomap. If the g_t differ from step to step, the composite $g_t \circ \cdots \circ g_1$ is an element of a *path* through the category, and if the state spaces change over time it is not even an endomap. Writing the sequence as "g^t" is a notational pun, and importing an idempotence theorem through that pun produces conclusions that are false — not approximately false, but exactly false, refutable on a three-element carrier.

4.2. Worked scenario. Let X be a three-element carrier, positions $\{0, 1, 2\}$. A session applies two alternating generators, both honest endomaps of X:

- a, the cyclic rotation with mapping tuple $(1, 2, 0)$: it sends $0 \mapsto 1$, $1 \mapsto 2$, $2 \mapsto 0$;
- b, the swap with mapping tuple $(1, 0, 2)$: it exchanges 0 and 1 and fixes 2.

The session trace is $a, b, a, b, a, b, \ldots$: at odd steps apply a, at even steps apply b. Write s_t for the composite state map after t steps, so $s_1 = a$, $s_2 = b \circ a$, $s_3 = a \circ b \circ a$, and in general s_t is the left-to-right alternating product.

An analyst models the session as "the session operator applied t times" — a single f with $s_t = f^t$ — and invokes the safe finite power with $|X| = 3$, hence $|X|! = 6$: f^6 is idempotent, so $f^{12} = f^6 \circ f^6 = f^6$, and the analyst concludes that the state map after twelve steps equals the state map after six steps. *The process has stabilized by step six*; monitoring beyond that point is declared redundant.

Now compute what actually happens. The two-step round is

$$h = b \circ a : \quad 0 \xrightarrow{a} 1 \xrightarrow{b} 0, \qquad 1 \xrightarrow{a} 2 \xrightarrow{b} 2, \qquad 2 \xrightarrow{a} 0 \xrightarrow{b} 1,$$

so h has mapping tuple $(0, 2, 1)$: it fixes 0 and swaps 1 and 2. Then $s_6 = h^3$ and $s_{12} = h^6$. Since h is a transposition, $h^2 = \mathrm{id}_X$, hence

$$s_6 = h^3 = h \neq \mathrm{id}_X = h^6 = s_{12}.$$

The predicted equality $s_{12} = s_6$ is false: after six steps the state map swaps positions 1 and 2; after twelve it is the identity. Any downstream conclusion keyed to "the stabilized map" — for instance, that position 1 has permanently migrated to position 2 — is refuted six steps later by the process itself.

The instructive point is that *every hypothesis of the theorem holds for each generator separately.* Both a and b are single finite endomaps; $a^6 = \mathrm{id}_X$ and $b^6 = \mathrm{id}_X$, and the identity is idempotent, so the safe finite power theorem is true of a and true of b. It fails for the session because the session is not a power of any single endomap: there is no $f : X \to X$ with $f^t = s_t$ for all t (already $s_1 = a$ and $s_2 = b \circ a$ would force $f = a$ and $f^2 = b \circ a$, but $a^2 = (2, 0, 1) \neq (0, 2, 1)$). The pun "$g^t$" asserted the existence of such an f, silently, in the notation itself.

The heterogeneous variant is even sharper. If the generators cross carriers — $g_1 : X_0 \to X_1$, $g_2 : X_1 \to X_2$ with $X_0 \neq X_2$ — then "the session operator squared" is not merely a false claim but an ill-typed expression: the composite has type $X_0 \to X_2$, and composing it with itself requires $X_2 = X_0$. There is no monoid in which the power lives.

4.3. The lesson. The kernel's response (Chapters 5 and 6) is a type-level separation: the `Endomap` constructor is available only for arrows whose source and target are the *same* object, and only `Endomap` carries a power operation. Heterogeneous or time-varying sequences are represented as `SemigroupoidPath` values, which have a typed composite and *deliberately no power operation* — the method is absent, and the runtime types are distinct, so a path cannot be passed where an endomap is required. The invalid inference of this scenario is not discouraged by documentation; it is unwritable. One cannot ask the kernel for s^6 because the object s on which one would ask it does not have the method, and constructing an `Endomap` from the session requires exhibiting the single arrow that the pun only pretended exists.

5. Failure mode III: nondeterministic identity

5.1. The failure class. The first two failures are semantic. The third is evidential, and it undermines even a correctly typed system if left unaddressed. Everything in the kernel downstream of an object — basis indices of the Hilbert-space lift, mapping tuples of arrows, shortlex ranks of words, canonical representatives of denotations, and every digest computed over any of these — depends on the *order* of the object's carrier. If that order is left to the iteration order of a hash-based container, it depends on interpreter state: in CPython, on the per-process string hash randomization controlled by `PYTHONHASHSEED`. Two runs of the same program on the same input may then disagree about which element sits at index 2, and therefore about every downstream artifact, and therefore about every downstream digest.

Replayability — the ability of an auditor to rerun a computation and obtain bit-identical evidence — dies at the first hash-ordered iteration.

5.2. Worked scenario. A well-meaning pipeline builds a carrier from a Python set literal:

```
states = {"calm", "alert", "overload"}
carrier = tuple(states)
```

The programmer reasons: the set has three fixed elements, so the tuple has three fixed elements; nothing here is random. The reasoning is wrong in one load-bearing detail: `tuple(states)` fixes the *membership* but takes the *order* from set iteration, and set iteration order for strings is a function of the process's hash seed.

Run A of the pipeline, under one seed, obtains the carrier order (`alert`, `calm`, `overload`): the element `calm` has standard index 1. An endomap f intended to send every state to `calm` is then constructed by looking up indices, producing mapping tuple $(1, 1, 1)$. The pipeline serializes the object and arrow — carrier order and mapping tuple — into a canonical byte string and records its SHA-256 digest in an evidence log, alongside the conclusions drawn from the run.

Run B, the auditor's replay on another day (a fresh process, hence a fresh seed), obtains carrier order (`overload`, `alert`, `calm`): now `calm` has standard index 2, the same intended map serializes with mapping tuple $(2, 2, 2)$ and a permuted carrier sequence, and the recorded digest does not match. The auditor is left with exactly the wrong dichotomy: either the original evidence was corrupted, or the pipeline is nondeterministic — and no local inspection of the code distinguishes the benign cause (hash-order iteration of a three-element set) from a malignant one (actual tampering or an actual semantic bug). The evidentiary value of the digest is zero, not reduced: a digest that honest replays fail to reproduce cannot certify anything.

The failure generalizes beyond digests. If two subsystems each build "the same" carrier from set iteration within one process, they may agree by luck; across processes, machines, or language versions they will not. Basis-dependent quantities — the matrix of the lift A_f, the shortlex rank of a word, the identity of the canonical representative selected by normal-form computation — all inherit the nondeterminism. A "canonical" form that varies by process is a contradiction in terms.

Chapter 3 develops this scenario mechanically — what `PYTHONHASHSEED` actually controls, how randomization enters `set` and `dict` iteration, why the problem is a general software-evidence problem rather than a Python quirk — and states the kernel's ordered-input rule: every order-sensitive constructor input rejects sets, mappings, and one-shot iterables outright, at construction time, with a typed error. The caller must supply a replayable ordered sequence, which makes the ordering decision explicit, reviewable, and versionable.

5.3. The lesson. Determinism is not an implementation nicety here; it is a precondition for the entire evidence discipline of the program. The kernel's artifacts are bound by SHA-256 hashes in a manifest, its test outcomes are recorded against those hashes, and its formal seed audits exact source hashes before building. All of that machinery presupposes that honest reruns reproduce bytes. A single hash-ordered iteration anywhere upstream of a digest silently voids the presupposition. The ordered-input rule is therefore enforced at the lowest level of the kernel — inside the constructors — rather than in documentation or review checklists, on the general principle that a property this load-bearing must be impossible to violate accidentally.

6. The scenarios replayed against the kernel

Before stating the design philosophy in the abstract, it is worth replaying each scenario against the kernel's actual rules, because the three outcomes have a common shape: the failing step does not execute wrongly — it does not execute at all, and the error it raises names the exact judgment that failed.

6.1. Scenario I under typed composition. In the kernel, the summarization and reading steps are arrow values carrying their full judgments:

`p : DriveProfile -> EvalSummary` with mapping $(0, 0, 2, 2, 2, 1, 3)$
`r : QuadState -> DriveProfile` with mapping $(4, 6, 5, 0)$

The composite request `compose(r, p)` is checked before any index is touched: the declared target object of `p` is $(\texttt{EvalSummary}, (e_0, e_1, e_2, e_3))$, the declared source object of `r` is $(\texttt{QuadState}, (q_0, q_1, q_2, q_3))$, and the two are unequal because their names differ. The kernel raises `CompositionTypeError` with a message of the form

```
cannot compose p:DriveProfile->EvalSummary then
r:QuadState->DriveProfile
```

The number 5 — the smuggled image of q_2 that the untyped implementation returned — is never computed, because the guard runs before the pointwise formula. Note also what the fix looks like on the kernel's terms: if the analyst genuinely intends an identification of evaluative labels with quadrant labels, she must write it as an explicit arrow `EvalSummary` $\to$ `QuadState` and insert it into the composite. The identification then exists as a reviewable object with an author and a version — exactly the audit record whose absence made the original failure silent (Remark 3.6 returns to this point).

6.2. Scenario II under the endomap/path separation. The alternating session is representable in the kernel in exactly one way: as a path value,

```
session = SemigroupoidPath((a, b, a, b, a, b))
```

which validates (all six arrows are endo on the same object, so all adjacent pairs match) and offers `composite()` — the honest six-step composite, which evaluates to the transposition $h^3 = h$ computed in Section 4.2. What the path does *not* offer is any power operation: there is no `session.power(n)`, by design, and the runtime type `SemigroupoidPath` is not accepted where an `Endomap` is required. The analyst who wants to invoke the factorial idempotence theorem must first construct `Endomap(g)` for some single arrow `g` — and for the alternating session no such single arrow exists with the session's semantics, a fact the analyst now confronts at the type level instead of discovering, twelve steps into production, through a falsified stabilization claim. If she constructs `Endomap(session.composite())` she gets a legitimate object — the one-round map h composed out to six steps — and every theorem she then invokes is true *of that map*, which is the point: the type system has forced the quantifier back over single endomaps, where the theorems actually hold.

The cross-carrier variant fails even earlier: a path whose adjacent objects mismatch raises `CompositionTypeError` at construction, naming the offending segment, and a mismatched `Endomap` construction — an arrow whose source and target are different objects — likewise raises at construction with a message of the form `"g' has type X0->X2, so it is not an endomap"`.

6.3. Scenario III under the ordered-input rule. The pipeline's first line, transplanted into the kernel, is already illegal:

```
FiniteObject(name="States", elements={"calm", "alert", "overload"})
```

raises `ValidationError`: finite-object elements must be an ordered sequence; sets, mappings, and one-shot iterables are not accepted. The programmer is forced to write the order:

```
FiniteObject(name="States", elements=("calm", "alert", "overload"))
```

and from this point Invariant 3.8 takes over: the constructed object, the arrow built over it, the serialization, and the digest are pure functions of the written program text. Run A and Run B now agree byte-for-byte, on any machine, under any `PYTHONHASHSEED`, because the hash seed no longer participates in any datum the kernel emits. The auditor's replay reproduces the digest, and the digest is evidence again. The cost — one explicit ordering decision, visible in the diff — is the entire price, and Section 4 argues it is the minimum price any solution can charge, since the ordering decision existed all along and was merely being made, invisibly and differently each run, by the hash table.

6.4. The common shape. In all three replays the kernel's contribution is the same: a judgment that informal notation left implicit — the middle objects of a composite, the single-endomap hypothesis of an iteration theorem, the enumeration order of a carrier — is reified as explicit, checkable data, and the check runs *before* any computation that would depend on the judgment being true. The failures do not become error codes to handle; they become programs that cannot be written. This is what "make the failure a type error" means concretely, and the remainder of the monograph is the systematic execution of that idea across the kernel's five capabilities.

7. Design philosophy of the kernel

Four commitments shape everything that follows.

7.1. Smallest kernel, hardest boundary. TKS v8.0 is the *smallest* kernel that turns each of the three failures into a type error. Smallness is a feature with teeth: the implementation is three modules (`category.py`, `deterministic_lift.py`, `normal_forms.py`, totalling under 26 kilobytes of source), the mathematics is elementary and completely proved, and the bounded exhaustive test families can genuinely enumerate their domains (19,683 associativity triples, 289 endomaps, 499 typed maps, 1,678 lift-composition pairs; Chapter 10). A larger kernel would dilute each of these properties. In particular the kernel is an *independent reference model*: it does not modify, replace, or certify the active CAS notation compiler, it has no production runtime dependency, and a manifest-boundary regression test asserts exactly that.

7.2. Refusal to encode canon before review. The kernel deliberately encodes no Noetic, no Domain, no Drive, no Register, and no CAS construct of any sort. This is the program's most frequently misread decision, so we state its logic plainly. The mapping from the canonical ontology onto a categorical substrate is itself a research claim — arguably the central one — and admitting it by construction, as a side effect of writing the kernel, would launder an unreviewed claim into infrastructure. The mapping is instead held as an explicit open obligation (PO-TKS8-CORE-08, OPEN / NOT LICENSED), to be performed, if at all, under separate review against the corrected v7.4 theorem records. Until then, an arrow in this book means a total function between named finite carriers, and nothing else.

CLAIM BOUNDARY 1.2. No result in this monograph is a claim about cognition, about the TKS/CAS ontology, about the production compiler's semantics, or about any empirical

system. The kernel maps no canonical construct onto arrows. Where a scenario in this chapter uses ontology-flavored names (`DriveProfile`, `EvalSummary`), the names are illustrative strings; the mathematics sees only carriers and index tuples.

7.3. Fail closed. Wherever the kernel cannot deliver a complete, correct answer, it raises a typed error rather than returning a best effort. Composition with mismatched middle objects raises rather than coercing; constructors reject unordered inputs rather than guessing an order; the normal-form closure raises `ClosureLimitExceeded` rather than returning a partially explored table as "canonical." The alternative — silently truncated canonical forms, silently coerced compositions — is precisely the raw material of the three failure modes.

7.4. Evidence as a first-class artifact. Every mathematical claim in the program carries a registered evidential status, and the statuses are heterogeneous on purpose: some claims have complete informal proofs plus bounded exhaustive tests; some have partial formal verification in a pinned Lean 4.32.1 seed [16] whose verifier admits the single standard axiom `propext` and rejects project axioms and `sorry`; some are standard results cited to the literature with formalization open; some are open outright. The full ledger is reproduced in Appendix B. The monograph never averages these statuses into an impression of overall verifiedness: bounded testing is not proof-assistant verification of the running code, the Lean seed's scope is exactly enumerated (Chapter 11), and the equivalence between the Python implementation and the Lean model is itself an open obligation. This stance follows the small-scope tradition in software modeling [15]: exhaustive checking of a bounded model is a powerful falsifier and a weak verifier, and conflating the two is how disciplines rot.

8. Contributions and their evidential status

The kernel supplies five capabilities. For each we state the capability, the chapters that develop it, and the exact evidential status, quoted from the ledger.

(1) *Named finite objects with explicitly ordered carriers* (Chapter 3). Object identity is the pair of declared name and ordered carrier; all order-sensitive constructor inputs reject sets, mappings, and one-shot iterables. Status: executable validation — constructor and negative-fixture tests; this is a design discipline, not a theorem.

(2) *Total finite-function arrows with partial, type-checked composition satisfying the category laws* (Chapters 3 and 4). Status: partial formal verification present — pointwise proof, bounded exhaustive tests over 19,683 associativity triples and 499 typed maps, and exact Lean lemmas for the Lean model; Python-to-Lean equivalence open.

(3) *Type separation of endomap powers from semigroupoid paths* (Chapters 5 and 6), with the safe finite power theorem $f^{|X|!} \circ f^{|X|!} = f^{|X|!}$. Status: the separation is executable validation (distinct runtime types, negative tests); the theorem is a standard finite-semigroup result [13, 14] verified exhaustively for all 289 endomaps on carriers through size four, with formalization open.

(4) *The deterministic standard-basis lift* $A_f e_x = e_{f(x)}$ (Chapter 7), with the exact norm formula $\|A_f\|^2 = \max_y |f^{-1}(y)|$, its trace, rank, and isometry corollaries, functoriality, and a permanent noncontractivity counterexample. Status: informal proof present; bounded exhaustive structural tests (1,678 exact-matrix composition pairs); Lean kernel-checks the exact two-state counterexample and the identity and basis-composition laws; universal formalization and the finite-sum matrix bridge open.

(5) *Stable denotational normal forms* by shortlex minimization over a completed finite closure (Chapter 8), with soundness of representative-first pruning and fail-closed

resource semantics. Status: informal correctness argument present; regression tests; formalization open.

Chapters 9 through 11 document the implementation, the executable evidence (the validation snapshot of 2026-08-04 records PASS on 20 tests), and the formal seed. Chapter 12 states the integration boundaries and the downstream program. Nothing in the list above is peer-reviewed (PO-TKS8-CORE-10, OPEN); no empirical, cognitive-dynamics, channel-contractivity, or CAS-efficacy claim follows from any item; and the mathematical content is standard — the contribution claimed is the *engineering discipline* binding it together, not new theorems.

The correspondence between capabilities, obligations, and statuses is summarized in Table 1; the authoritative record is the ledger itself (Appendix B), and where this table and the ledger could ever disagree, the ledger governs.

Capability	Chapters	Ledger items	Status (abbreviated)
Objects, ordered carriers	3	PO-02 (part)	executable validation
Typed composition, laws	3, 4	PO-01, PO-02	partial formal verif.
Endomap/path separation	5, 6	PO-03, PO-04	exec. valid.; std. result
Deterministic lift, norm	7	PO-05, PO-11	informal proof; partial
Shortlex normal forms	8	PO-06	informal argument
Lean formal seed	11	PO-07	partial, present
Semantic mapping to CAS	12	PO-08	open / not licensed
Compiler integration	12	PO-09	open
Specialist review	—	PO-10	open

TABLE 1. Capabilities against ledger obligations (PO-TKS8-CORE numbers abbreviated). "Partial formal verif." always means: exact Lean lemmas about the Lean model, bounded exhaustive tests of the Python model, and an open bridge between the two.

9. Positioning among neighboring disciplines

The kernel's ideas are borrowed, individually, from well-established practice; the claimed contribution is their joint enforcement in one small, evidence-bound artifact. Situating each borrow both credits the sources and sharpens what the kernel does not attempt.

Typed composition is the oldest idea here: that the legality of combining two operations should be checked against declared interfaces rather than discovered at runtime is the founding insight of static type systems, and its categorical formulation — composition as a partial operation governed by objects — is textbook category theory [**2**]. The kernel adds only strictness choices: name-inclusive object identity and the refusal of every coercion.

The endomap/path separation is elementary semigroup theory [**14**, **13**] read as a typing discipline: powers live in endomorphism monoids and nowhere else, so the type system should make "nowhere else" unrepresentable. The mathematical content (Chapter 2) is classical; the engineering content is the decision to encode a quantifier's hypothesis as a runtime type.

Determinism as an evidence requirement is the reproducible-builds insight applied to mathematical artifacts: a digest certifies nothing unless honest replays reproduce it. The kernel's ordered-input rule is a library-level instance of that doctrine, and Chapter 3 argues the general case.

Bounded exhaustive testing follows the small-scope tradition [15]: enumerate *all* instances of a small model rather than sampling a large one, on the hypothesis that most defects manifest at small scope. The kernel's test families are sized by exact counts (Chapter 2, Section 3) so that "all" is a theorem, not an aspiration.

Kernel-checked formalization follows standard proof-assistant practice [16]: a small trusted checker, a pinned toolchain, an audited axiom footprint. The program's honesty rule — the Lean model and the Python implementation are distinct artifacts whose equivalence is an open obligation — is itself standard practice in careful verification work, and Chapter 11 maintains it without exception.

What the kernel does not attempt, even within these disciplines: it is not a general-purpose proof assistant, not a rewriting engine, not a model checker, and not a type-system extension of any existing language — it is a library whose constructors refuse, which is the smallest mechanism that suffices for the three failures.

9.1. Falsifiability of the kernel's claims. A reference kernel should say what would refute it. Each capability has a concrete falsifier: a composable triple of typed arrows on which the implementation's composition disagrees with pointwise evaluation; a constructible arrow accepted with mismatched middle objects; a `SemigroupoidPath` value on which any power-like operation succeeds; an endomap f on a carrier of size at most four with $f^{|X|!} \circ f^{|X|!} \neq f^{|X|!}$; a typed pair (f, g) with $A_{g \circ f} \neq A_g A_f$ as exact integer matrices; a catalog and denotation for which the closure returns a word that is not shortlex-least among well-typed representatives, or returns a partial table without raising; and any kernel output that differs between two runs with identical constructor arguments. The bounded exhaustive families of Chapter 10 are precisely systematic searches for the first five falsifiers within their bounds; none has been found, which is evidence within the bounds and silence beyond them (Remark 1.4).

10. Reading guide

The monograph is organized so that a reader can enter at several depths.

Part I: Foundations. Chapter 2 is a self-contained mathematical primer — finite sets and total functions, counting, the semigroup-to-category ladder, functional graphs of endomaps, finite-dimensional Hilbert spaces, and shortlex orders — cited throughout as standard background [17, 2, 13, 14, 12]. A reader fluent in these topics may skim it for notation. Chapter 3 defines objects and arrows and develops the determinism discipline at full depth.

Part II: The kernel's mathematics. Chapter 4 proves the category laws. Chapter 5 develops endomap theory and the safe finite power. Chapter 6 develops semigroupoid paths and the impossibility of path powers. Chapter 7 develops the lift: norm theorem, corollaries, functoriality, and the permanent counterexample. Chapter 8 develops normal forms: shortlex minimization, the representative-first closure algorithm, its soundness, and its resource semantics.

Part III: Evidence. Chapter 9 walks the implementation. Chapter 10 specifies the bounded exhaustive test families and what they do and do not establish. Chapter 11 documents the pinned Lean seed, its axiom audit, and its exact verified and open scope.

Part IV: Boundaries. Chapter 12 states what is excluded, the three downstream boundary objects that consume the kernel strictly through its types, and the reproduction procedure.

Appendices. Appendix A collects worked examples end to end. Appendix B reproduces the complete proof-obligation ledger (PO-TKS8-CORE-01 through 11) with statuses. Appendix C is a glossary.

Readers primarily interested in *why* the kernel exists should read this chapter and Chapter 12. Readers auditing the mathematics should read Chapters 2 through 8. Readers replaying the evidence should read Chapters 9 through 11 and Appendix B.

11. Notation and conventions

NOTATION 1.3. Throughout the monograph:

- Finite objects are written X, Y, Z; $|X|$ is the carrier length; carrier positions are the *standard indices* $0, \ldots, |X| - 1$.
- Arrows are written $f : X \to Y$; id_X is the identity; $g \circ f$ is composition, defined iff the target object of f equals the source object of g; $\mathrm{im}(f)$ is the image; $f^{-1}(y)$ is the fiber over y.
- $\mathrm{End}(X)$ is the monoid of endomaps of X; f^n is the n-th monoid power, $f^0 = \mathrm{id}_X$.
- H_X is the complex Hilbert space with orthonormal basis $(e_x)_{x \in X}$ indexed by the ordered carrier; A_f is the deterministic standard-basis lift of f; A_f^* its adjoint; $\|\cdot\|$ the operator norm; tr and rank the trace and rank.
- $[\![w]\!]$ is the denotation of a typed word w; $\mathrm{NF}_G(a)$ is the shortlex-least well-typed word denoting a relative to the declared generator catalog G; $<_{\mathrm{slex}}$ is the shortlex order.
- Code identifiers appear in typewriter face (`FiniteObject`, `denotation_key`); they name entities of the reference implementation in `src/tks_v8_core`.
- Claim-status words — OPEN, NOT LICENSED, PARTIAL FORMAL VERIFICATION PRESENT, EXECUTABLE VALIDATION, STANDARD RESULT — are quoted from the obligation ledger and are never paraphrased.

Table 2 collects the recurring symbols with the location of their governing definitions.

Symbol	Meaning	Defined
$X = (\mathrm{name}, \mathrm{carrier})$	finite object	Def. 3.1
$\|X\|$	carrier length	Def. 3.1
$f : X \to Y$	arrow judgment	Def. 3.11
id_X, $g \circ f$	identity, composition	Ch. 2, Ch. 4
$\mathrm{im}(f)$, $f^{-1}(y)$	image, fiber	Def. 2.4
$\mathrm{End}(X)$, f^n	endomorphism monoid, power	Prop. 2.22
T_X	full transformation monoid	Def. 2.27
H_X, $(e_x)_{x \in X}$	lift space, standard ONB	Ch. 2 Sec. 7, Ch. 7
A_f, A_f^*	lift, adjoint	Ch. 7
$\|\cdot\|$, tr, rank	operator norm, trace, rank	Def. 2.40
J_m	all-ones $m \times m$ matrix	Prop. 2.43
$[\![w]\!]$	denotation of a typed word	Ch. 8
$<_{\mathrm{slex}}$, NF_G	shortlex order, normal form	Def. 2.50, Ch. 8
$\mathrm{rank}_{\mathrm{slex}}(w)$	shortlex rank of a word	Prop. 2.54
$\mathcal{C}$, Ob, Hom	category, objects, hom-sets	Def. 2.20

TABLE 2. Recurring symbols. References into Chapters 4–8 point at the governing definitions of Part II; primer references are exact.

Remark 1.4 (How to read the status vocabulary). Executable validation means: the property is enforced by constructors and demonstrated by tests, including negative fixtures; it is a statement about the finite executable model only. Standard result means: the mathematics is textbook material, cited as background and proved here for self-containment, with no novelty claimed. Partial formal verification present means: a pinned proof assistant kernel-checks stated lemmas about a Lean model of the definitions, under an audited axiom footprint, and the bridge from that model to the running Python is open. Open means open: no evidence is claimed. Not licensed means that even drafting an argument for the claim is out of scope for this lane until a separate review admits it.

With the failures, the philosophy, and the notation in hand, we turn to the mathematics. Everything in the next chapter is standard; everything after it is discipline.

CHAPTER 2

Mathematical Preliminaries

This chapter assembles, at tutorial depth, every piece of mathematics the kernel uses: finite sets and total functions; images, fibers, and the classification of maps, with the counting facts that make bounded exhaustive testing possible; the algebraic ladder from semigroups through monoids and semigroupoids to categories; the structure theory of functional graphs of endomaps, culminating in eventual periodicity and the factorial idempotent power; the linear algebra of finite-dimensional Hilbert spaces, adjoints, operator norms, and positive semidefinite matrices, including a complete spectral computation for the all-ones matrix; and shortlex orders on words, with the well-order property proved. All of it is standard. We cite it as background — to Hammack for sets and functions [17], to Mac Lane for category theory [2], to Pin and to Howie for finite semigroups and transformation monoids [13, 14], and to Horn and Johnson for matrix analysis [12] — and we prove what we use, so that the monograph is self-contained and so that later chapters can point at exact statements rather than folklore. No novelty is claimed anywhere in this chapter.

1. Finite sets and total functions

1.1. Finite sets. A set X is *finite* if it is empty or in bijection with $\{0, 1, \ldots, n-1\}$ for some positive integer n; in either case the number $n \geq 0$ is unique and is written $|X|$, the *cardinality* of X. We take the elementary theory of finite sets as given [17] and record only the facts we lean on: a subset of a finite set is finite with $|A| \leq |X|$, and $|A| = |X|$ for $A \subseteq X$ forces $A = X$; a union of two finite sets is finite with $|A \cup B| = |A| + |B| - |A \cap B|$; and the product of finite sets is finite with $|X \times Y| = |X| \cdot |Y|$.

Throughout the monograph, finite sets come to us with more structure than bare sethood: the kernel's objects carry an *ordered* finite carrier, a sequence $(x_0, x_1, \ldots, x_{n-1})$ of pairwise distinct elements. The order is data, not decoration — Chapter 3 is largely about why — but for the purposes of the present chapter, whenever we say "finite set" we permit ourselves to fix an enumeration and to identify the set with its index set $\{0, \ldots, n-1\}$ when convenient. Every construction in this chapter is compatible with that identification, and the kernel performs it explicitly via standard indices.

1.2. Total functions.

Definition 2.1 (Total function). A *total function* $f : X \to Y$ between sets assigns to *every* element $x \in X$ exactly one element $f(x) \in Y$. The set X is the *domain*, Y the *codomain*. Two total functions $f, g : X \to Y$ with the same domain and codomain are *equal* iff $f(x) = g(x)$ for every $x \in X$; this is *extensional* equality.

Three points deserve emphasis, because each is load-bearing later.

First, *totality is part of the definition.* A partial assignment — defined on some of X — is not a function $X \to Y$ in our usage. The kernel enforces totality syntactically: an arrow's

mapping tuple must supply one target index for each of the $|X|$ source indices, and a tuple of the wrong length is rejected at construction.

Second, *the codomain is part of the data.* The same assignment rule with a larger codomain is a different function; surjectivity, for instance, is a property of the pair (rule, codomain), not of the rule alone. This foreshadows the kernel's stricter stance, where even the codomain's *name* is part of an arrow's type.

Third, *equality is extensional.* Two functions given by wildly different descriptions — one by a formula, one by a table, one as a composite of seventeen other functions — are equal the moment they agree pointwise on a shared domain and codomain. The kernel mirrors this exactly: arrow equality is equality of source, target, and mapping tuple, and the presentation name attached to an arrow for diagnostics never participates (Chapter 3).

For finite X with a fixed enumeration $(x_0, \dots, x_{n-1})$ and finite Y with enumeration $(y_0, \dots, y_{m-1})$, a total function $f : X \to Y$ is the same data as a tuple $(t_0, \dots, t_{n-1}) \in \{0, \dots, m-1\}^n$ via $f(x_i) = y_{t_i}$. This *tuple representation* is the kernel's concrete representation of arrows and will be used freely in proofs.

1.3. Composition and identity.

Definition 2.2 (Composition). Given $f : X \to Y$ and $g : Y \to Z$, the *composite* $g \circ f : X \to Z$ is the total function $(g \circ f)(x) = g(f(x))$. The *identity* $\mathrm{id}_X : X \to X$ is the total function $\mathrm{id}_X(x) = x$.

Proposition 2.3 (Associativity and neutrality). *For $f : X \to Y$, $g : Y \to Z$, $h : Z \to W$:*

$$h \circ (g \circ f) = (h \circ g) \circ f, \qquad f \circ \mathrm{id}_X = f, \qquad \mathrm{id}_Y \circ f = f.$$

Proof. All four sides of the first equation are total functions $X \to W$; for each $x \in X$, both evaluate to $h(g(f(x)))$, so they are extensionally equal. Similarly $(f \circ \mathrm{id}_X)(x) = f(\mathrm{id}_X(x)) = f(x)$ and $(\mathrm{id}_Y \circ f)(x) = \mathrm{id}_Y(f(x)) = f(x)$ for all x. □

The proof is one line, and deliberately so: the entire content of the category laws for total functions is pointwise evaluation plus extensional equality. What the kernel adds in Chapter 4 is not mathematical depth but *typing*: the requirement that the middle objects of a composite agree as declared objects, checked before the pointwise formula is ever consulted.

2. Images, fibers, and the classification of maps

2.1. Image and fibers.

Definition 2.4 (Image, fiber). For $f : X \to Y$: the *image* is $\mathrm{im}(f) = \{\, f(x) \mid x \in X \,\} \subseteq Y$; for $y \in Y$, the *fiber* over y is $f^{-1}(y) = \{\, x \in X \mid f(x) = y \,\} \subseteq X$.

Fibers may be empty — $f^{-1}(y) = \emptyset$ iff $y \notin \mathrm{im}(f)$ — and the notation $f^{-1}(y)$ does *not* presuppose an inverse function.

Lemma 2.5 (Fiber partition). *The nonempty fibers of $f : X \to Y$ partition X: every $x \in X$ lies in exactly one fiber, namely $f^{-1}(f(x))$. Consequently, for finite X,*

$$\sum_{y \in Y} \left| f^{-1}(y) \right| = |X|.$$

Proof. By totality every x has an image $f(x)$, so $x \in f^{-1}(f(x))$ and every element lies in at least one fiber. If $x \in f^{-1}(y)$ and $x \in f^{-1}(y')$ then $y = f(x) = y'$, so fibers over distinct points are disjoint and every element lies in at most one. The counting identity is then the

statement that a partition's block sizes sum to the size of the whole; empty fibers contribute zero and may be included in the sum harmlessly. □

This innocuous lemma does real work in Chapter 7: the trace identity $\operatorname{tr}(A_f^* A_f) = |X|$ for the lift is precisely Lemma 2.5 read off the diagonal of a Gram matrix, and the norm theorem is a statement about the *largest* block of the fiber partition.

2.2. Injections, surjections, bijections.

Definition 2.6. A total function $f : X \to Y$ is *injective* if $f(x) = f(x')$ implies $x = x'$; *surjective* if $\operatorname{im}(f) = Y$; *bijective* if it is both.

The fiber language rephrases all three: f is injective iff every fiber has at most one element; surjective iff every fiber is nonempty; bijective iff every fiber has exactly one element. The kernel's isometry criterion for lifts (Chapter 7) is the first of these in linear-algebraic clothing.

Proposition 2.7 (Finite endomap trichotomy collapses). *For a total function $f : X \to Y$ between finite sets with $|X| = |Y|$, the following are equivalent: (i) f is injective; (ii) f is surjective; (iii) f is bijective.*

Proof. (i) ⇒ (ii): if f is injective the nonempty fibers are singletons, so by Lemma 2.5 the number of nonempty fibers is $|X| = |Y|$; hence all $|Y|$ fibers are nonempty and f is surjective. (ii) ⇒ (i): if f is surjective all $|Y|$ fibers are nonempty; since the fiber sizes are positive integers summing to $|X| = |Y|$, each must equal 1, so f is injective. Either direction gives (iii), and (iii) trivially gives (i) and (ii). □

Remark 2.8. Proposition 2.7 is the pigeonhole principle in functional dress, and it fails instantly for infinite sets (the successor map on the natural numbers is injective, not surjective). Its relevance to the kernel is indirect but real: on a finite carrier, a non-bijective endomap must *merge* somewhere and *miss* somewhere, and repeated iteration therefore shrinks the image until it stabilizes — the qualitative fact behind the eventual-periodicity theory of Section 5.

2.3. Composites and cancellation.

Proposition 2.9. *Let $f : X \to Y$ and $g : Y \to Z$. (i) If f and g are injective, so is $g \circ f$; if both are surjective, so is $g \circ f$. (ii) If $g \circ f$ is injective, f is injective. (iii) If $g \circ f$ is surjective, g is surjective.*

Proof. (i) If $g(f(x)) = g(f(x'))$, injectivity of g gives $f(x) = f(x')$ and injectivity of f gives $x = x'$. For surjectivity, any $z \in Z$ is $g(y)$ for some y, and $y = f(x)$ for some x, so $z = g(f(x))$. (ii) If $f(x) = f(x')$ then $g(f(x)) = g(f(x'))$, so $x = x'$ by injectivity of the composite. (iii) Any $z \in Z$ is $(g \circ f)(x)$ for some x, hence $z = g(f(x)) \in \operatorname{im}(g)$. □

2.4. Sections and retractions. Injectivity and surjectivity admit a second characterization — by one-sided inverses — that translates directly into the operator language of Chapter 7, where it becomes the isometry and coisometry structure of lifts.

Proposition 2.10 (One-sided inverses). *Let $f : X \to Y$ with X nonempty and both sets finite.*

1. *f is injective iff it has a* retraction*: some $r : Y \to X$ with $r \circ f = \mathrm{id}_X$.*
2. *f is surjective iff it has a* section*: some $s : Y \to X$ with $f \circ s = \mathrm{id}_Y$.*
3. *f is bijective iff it has a two-sided inverse, which is then unique and equals every retraction and every section of f.*

PROOF. (1) If $r \circ f = \mathrm{id}_X$ then f is injective by Proposition 2.9(ii) applied to the injective composite id_X. Conversely, if f is injective, fix any $x_* \in X$ (here nonemptiness is used) and define $r(y) = x$ when $y = f(x)$ for the (unique, by injectivity) such x, and $r(y) = x_*$ when $y \notin \mathrm{im}(f)$. Then r is total and $r(f(x)) = x$ for every x.

(2) If $f \circ s = \mathrm{id}_Y$ then f is surjective by Proposition 2.9(iii). Conversely, if f is surjective, every fiber $f^{-1}(y)$ is nonempty; choose for each y an element $s(y) \in f^{-1}(y)$ (a finite sequence of choices, no choice axiom needed: pick the element of least standard index). Then $f(s(y)) = y$ for every y.

(3) If g satisfies $g \circ f = \mathrm{id}_X$ and $f \circ g = \mathrm{id}_Y$, then f is bijective by (1) and (2). Conversely a bijection's inverse assignment $y \mapsto$ (the unique x with $f(x) = y$) is a two-sided inverse. For uniqueness and the coincidence claim: if $r \circ f = \mathrm{id}_X$ and $f \circ s = \mathrm{id}_Y$, then $r = r \circ \mathrm{id}_Y = r \circ (f \circ s) = (r \circ f) \circ s = \mathrm{id}_X \circ s = s$; so any retraction equals any section, and in particular two two-sided inverses are equal. □

REMARK 2.11 (Choice of section is data). The proof of (2) had to *choose* an element of each fiber, and made the choice canonical by taking least standard indices. This is a small early instance of the chapter-spanning theme: where mathematics says "choose arbitrarily," a deterministic kernel must say *which*, and the ordered carrier is what makes "least index" meaningful. An implementation whose carriers were unordered sets could not even state this canonical section.

3. Counting total maps

Bounded exhaustive testing — the kernel's principal executable evidence — rests on knowing exactly how many objects of each shape exist, so that "all of them" is a checkable claim. This section proves the counts.

THEOREM 2.12 (Number of total maps). *For finite sets X and Y, the number of total functions $X \to Y$ is $|Y|^{|X|}$, with the conventions $m^0 = 1$ (one empty function out of the empty set, for any Y, including $Y = \emptyset$) and $0^n = 0$ for $n \geq 1$ (no total function from a nonempty set into the empty set).*

PROOF. Write $n = |X|$, $m = |Y|$, and fix an enumeration $x_0, \ldots, x_{n-1}$ of X. We induct on n.

Base case $n = 0$. A total function must assign an image to every element of the domain; with no elements there is nothing to assign, and two such functions are extensionally equal vacuously. So there is exactly one function $\emptyset \to Y$ — the empty function — and $m^0 = 1$ counts it correctly, even when $m = 0$.

Inductive step. Suppose the count is m^{n-1} for domains of size $n-1$, and let $|X| = n \geq 1$. If $m = 0$ there is no possible image for x_0, hence no total function, and $0^n = 0$ is correct. If $m \geq 1$, a total function $f : X \to Y$ is uniquely determined by the pair (its restriction to $X \setminus \{x_0\}$, its value $f(x_0)$), and every such pair arises from exactly one f: given a function g on $X \setminus \{x_0\}$ and a value $y \in Y$, define $f(x_0) = y$ and $f(x_i) = g(x_i)$ for $i \geq 1$; this f is total, restricts to g, and takes the value y; and two distinct pairs yield extensionally distinct functions. The correspondence is therefore a bijection between functions $X \to Y$ and pairs, so the count is $m^{n-1} \cdot m = m^n$ by the induction hypothesis and the product rule. □

COROLLARY 2.13 (Counts used by the test families). *(i) The number of endomaps of an n-element set is n^n; in particular there are $3^3 = 27$ endomaps of a three-element carrier and hence $27^3 = 19{,}683$ ordered triples of such endomaps — the exhaustive associativity family of Chapter 10. (ii) Summing over carrier sizes zero through four, $\sum_{n=0}^{4} n^n = 1+1+4+27+256 =$*

289 *endomaps — the exhaustive endomap family (here* $0^0 = 1$ *counts the empty function). (iii) The number of injective functions from an* n*-set to an* m*-set is the falling factorial* $m(m-1)\cdots(m-n+1)$ *for* $n \le m$ *and* 0 *otherwise; the number of bijections of an* n*-set with itself is* $n!$*.*

PROOF. (i) and (ii) are immediate from Theorem 2.12. For (iii): choose images for $x_0, x_1, \ldots$ in order; injectivity means each choice must avoid the previously chosen images, leaving $m-i$ options at step i; the product rule gives the falling factorial, which is 0 as soon as $n > m$ by pigeonhole (Proposition 2.7); bijections are the case $n = m$. □

THEOREM 2.14 (Number of surjections). *For finite sets with* $|X| = n$ *and* $|Y| = m$*, the number of surjective total functions* $X \to Y$ *is*

$$\operatorname{Surj}(n, m) \;=\; \sum_{k=0}^{m} (-1)^k \binom{m}{k} (m-k)^n.$$

PROOF. For each $y \in Y$ let B_y denote the set of total functions $X \to Y$ that *miss* y, i.e. with $y \notin \operatorname{im}(f)$. The surjections are the functions in none of the B_y, so by inclusion-exclusion over the finite index set Y,

$$\operatorname{Surj}(n, m) = \sum_{S \subseteq Y} (-1)^{|S|} \Big| \bigcap_{y \in S} B_y \Big|.$$

The intersection $\bigcap_{y \in S} B_y$ consists of the total functions $X \to Y \setminus S$, of which there are $(m - |S|)^n$ by Theorem 2.12. Grouping the 2^m subsets by size $k = |S|$, of which there are $\binom{m}{k}$ each, gives the stated sum. □

EXAMPLE 2.15. On a three-element carrier: 27 endomaps in all (Theorem 2.12); of these, $\operatorname{Surj}(3,3) = 27 - 3 \cdot 8 + 3 \cdot 1 - 0 = 6$ are surjective — and indeed the surjective endomaps of a finite set are exactly its bijections (Proposition 2.7), of which there are $3! = 6$; the two counts confirming each other is a useful sanity identity, $\operatorname{Surj}(n, n) = n!$. The remaining 21 endomaps of the three-element carrier are the non-bijections, each of which merges at least two points and misses at least one.

REMARK 2.16 (Exhaustibility, and where it stops). The counts explain the exact shape of the kernel's bounded exhaustive test families (Chapter 10). Endomap families grow as n^n: sizes 0 through 4 contribute $1 + 1 + 4 + 27 + 256 = 289$ cases — trivially exhaustible — while size 5 alone would add 3,125 and size 8 would add 16,777,216. Triples for associativity grow as $(n^n)^3$: on the three-element carrier this is $27^3 = 19{,}683$, run in milliseconds, while on a four-element carrier it would already be $256^3 = 16{,}777{,}216$. The chosen bounds are the largest at which *complete* enumeration remains cheap enough to run on every test invocation; beyond them the suite would have to sample, and sampled evidence is a different, weaker kind of claim. Small-scope testing doctrine [**15**] holds that most implementation defects are already expressible in small models; the kernel adopts that doctrine explicitly and records its bounds rather than hiding them.

REMARK 2.17 (Finiteness of the normal-form closure). Theorem 2.12 is also the finiteness engine for Chapter 8: the denotations reachable from a finite generator catalog over a finite object set live inside the union of the sets of total maps between catalog objects, of which there are at most $|Y|^{|X|}$ for each ordered pair of objects, hence finitely many overall. The closure algorithm's termination is exactly this observation.

4. The algebraic ladder: semigroups to categories

The kernel's central type distinction — endomaps have powers, paths do not — is a distinction between positions on a small ladder of algebraic structures. We define each rung and prove the elementary facts; the reader is referred to Howie [14] and Pin [13] for semigroup theory and to Mac Lane [2] for categories.

Definition 2.18 (Semigroup, monoid). A *semigroup* is a set S with an associative binary operation $(s,t) \mapsto st$. A *monoid* is a semigroup with a two-sided identity element 1: $1s = s1 = s$ for all s.

Proposition 2.19 (Uniqueness of identity; unambiguous powers). *(i) A monoid's identity is unique. (ii) In a semigroup, the power s^n $(n \geq 1)$, defined by $s^1 = s$ and $s^{n+1} = s^n s$, satisfies $s^{m+n} = s^m s^n$ and $(s^m)^n = s^{mn}$; in a monoid the same laws hold for all $m, n \geq 0$ with $s^0 = 1$.*

Proof. (i) If 1 and $1'$ are both identities, $1 = 1 \cdot 1' = 1'$. (ii) $s^{m+n} = s^m s^n$ is induction on n: the case $n = 1$ is the definition, and $s^{m+(n+1)} = s^{(m+n)+1} = s^{m+n}s = (s^m s^n)s = s^m(s^n s) = s^m s^{n+1}$ using associativity. Then $(s^m)^n = s^{mn}$ is induction on n using the first law. The monoid extension is immediate from $1s = s$. □

The point of belaboring (ii): the expression s^n is meaningful *because* all ways of parenthesizing n copies of s agree (generalized associativity, itself a routine induction), and it presupposes that the n factors are *the same element* of *one* semigroup. Both presuppositions fail for the generator sequences of Section 4.2, and the kernel's refusal to give paths a power operation is the refusal to notate a power whose defining laws have no ambient semigroup in which to hold.

Definition 2.20 (Category). A *category* $\mathcal{C}$ consists of a collection of *objects* $\mathrm{Ob}(\mathcal{C})$; for each ordered pair of objects (X, Y) a set $\mathrm{Hom}(X, Y)$ of *arrows*; for each object an identity arrow $\mathrm{id}_X \in \mathrm{Hom}(X, X)$; and a composition assigning to $f \in \mathrm{Hom}(X, Y)$ and $g \in \mathrm{Hom}(Y, Z)$ an arrow $g \circ f \in \mathrm{Hom}(X, Z)$, such that composition is associative wherever defined and identities are neutral: $f \circ \mathrm{id}_X = f = \mathrm{id}_Y \circ f$ [2].

Definition 2.21 (Semigroupoid). A *semigroupoid* is defined as a category is, but *without* the requirement of identity arrows: objects, hom-sets, and an associative partial composition. Every category is a semigroupoid; a semigroupoid with all identities is a category.

Proposition 2.22 (The ladder). *(i) A monoid is exactly a category with one object: the elements are the arrows on that object, the operation is composition, the identity element is the identity arrow. (ii) A semigroup is exactly a semigroupoid with one object. (iii) In any category, each* $\mathrm{Hom}(X, X)$ *is a monoid under composition, called the* endomorphism monoid $\mathrm{End}(X)$*; in any semigroupoid, each* $\mathrm{Hom}(X, X)$ *is a semigroup.*

Proof. (i) Given a monoid M, define a category with a single object $\star$, $\mathrm{Hom}(\star, \star) = M$, composition the monoid operation, and $\mathrm{id}_\star = 1$; the category axioms are precisely associativity and the identity laws. Conversely a one-object category's arrows form a monoid by the same reading. (ii) is the same argument without identities. (iii) Composition restricted to $\mathrm{Hom}(X, X)$ is total (source and target both X), associative because the ambient composition is, and has identity id_X (or, in a semigroupoid, is merely associative). □

Remark 2.23 (Where powers live). Proposition 2.22 locates the power operation precisely: f^n makes sense exactly when f inhabits some $\mathrm{Hom}(X, X)$ — a monoid — and makes

no sense for $f \in \mathrm{Hom}(X, Y)$ with $X \neq Y$, since already $f \circ f$ requires the target of the inner f to be the source of the outer. A composable sequence $X_0 \to X_1 \to \cdots \to X_n$ of arrows in a category has a composite, by iterated use of the composition operation and associativity, but the sequence itself is an element of no single hom-set unless all the objects coincide, and even then the *sequence* is not an *element*: it composes to one, which is a different thing. The kernel's `Endomap` / `SemigroupoidPath` type distinction (Chapters 5 and 6) is this remark made mechanical.

4.1. Generalized associativity. Both the power notation s^n and the composite of a semigroupoid path rest on the same silent convention: an n-fold product may be written without parentheses. That convention is a theorem, and since the kernel's path composite (Chapter 6) is computed by one particular parenthesization — left-to-right folding — while the mathematics quantifies over all parenthesizations, we prove it.

THEOREM 2.24 (Generalized associativity). *Let $a_1, a_2, \ldots, a_n$ ($n \geq 1$) be a composable sequence in a semigroupoid (in particular: any sequence in a semigroup, or a typed path's arrow sequence). Then every way of fully parenthesizing the product $a_n \cdots a_2 a_1$ yields the same element, which may therefore be written without parentheses.*

PROOF. Define the *left-folded* product L_k by $L_1 = a_1$ and $L_{k+1} = a_{k+1} L_k$. We show by strong induction on n that every fully parenthesized product P of $a_n, \ldots, a_1$ (in this order) equals L_n. For $n = 1$, $P = a_1 = L_1$. For $n \geq 2$, the outermost multiplication of P splits the sequence at some position j: $P = Q \cdot R$, where R is a parenthesized product of $a_j, \ldots, a_1$ and Q one of $a_n, \ldots, a_{j+1}$, with $1 \leq j \leq n-1$. By induction $R = L_j$ and $Q = M$, where M is the left-folded product of $a_n, \ldots, a_{j+1}$. If $j = n-1$ then $M = a_n$ and $P = a_n L_{n-1} = L_n$ directly. Otherwise write $M = a_n M'$ with M' the left-folded product of $a_{n-1}, \ldots, a_{j+1}$ (definition of left folding, since $n - j \geq 2$). Using binary associativity,

$$P = (a_n M') L_j = a_n (M' L_j),$$

and $M' L_j$ is a parenthesized product of $a_{n-1}, \ldots, a_1$, hence equals L_{n-1} by induction. So $P = a_n L_{n-1} = L_n$. (All composites appearing are defined: in a semigroupoid, definedness of a parenthesized product depends only on adjacent source-target matching of the underlying sequence, which is given.) □

REMARK 2.25. Theorem 2.24 is what entitles the kernel to *implement* the path composite as a left fold while *claiming* the parenthesization-independent composite: the two coincide. It is also the induction behind Proposition 2.19(ii), made explicit.

4.2. Structure-preserving maps: homomorphisms and functors. One more definition from the standard toolkit is needed — not for the kernel's core, but for stating what Chapter 7 proves about the lift.

DEFINITION 2.26 (Monoid homomorphism; functor). A *monoid homomorphism* $\varphi : M \to N$ satisfies $\varphi(1_M) = 1_N$ and $\varphi(st) = \varphi(s)\varphi(t)$. More generally, a *functor* $F : \mathcal{C} \to \mathcal{D}$ between categories assigns to each object X of $\mathcal{C}$ an object $F(X)$ of $\mathcal{D}$ and to each arrow $f : X \to Y$ an arrow $F(f) : F(X) \to F(Y)$, such that $F(\mathrm{id}_X) = \mathrm{id}_{F(X)}$ and $F(g \circ f) = F(g) \circ F(f)$ whenever $g \circ f$ is defined [**2**].

By Proposition 2.22(i), a functor between one-object categories is exactly a monoid homomorphism; the two clauses of Definition 2.26 are the two *functor laws*. Chapter 7's deterministic lift is an assignment $X \mapsto H_X$, $f \mapsto A_f$ claimed to satisfy exactly these laws ($A_{\mathrm{id}_X} = I$ and $A_{g \circ f} = A_g A_f$); the claim's executable verification — exact matrix equality over all 1,678

valid composition pairs on carriers through size three — and its partially formalized status are detailed in Chapters 10 and 11. A functor restricted to one endomorphism monoid is a monoid homomorphism $\mathrm{End}(X) \to \mathrm{End}(H_X)$; in particular functors carry powers to powers, $F(f^n) = F(f)^n$, by induction from the composition law — which is why the lift of an idempotent power is an idempotent operator, a fact Chapter 7's counterexample then sharpens (idempotent, yes; projector, no).

4.3. The full transformation monoid.

Definition 2.27. For a finite set X, the *full transformation monoid* T_X is the monoid of all total functions $X \to X$ under composition, with identity id_X [14].

By Theorem 2.12, $|T_X| = |X|^{|X|}$. The invertible elements of T_X are exactly the bijections (Proposition 2.7 shows one-sided inverses suffice in the finite case), forming the symmetric group of order $|X|!$. Everything the kernel proves about endomap powers is a statement about elements of T_X; the classical theory of finite transformation semigroups [14, 13] contains far more (Green's relations, ideal structure, generation), none of which the kernel needs or encodes.

5. Functional graphs of endomaps

Fix a finite set X, $|X| = n \geq 1$, and an endomap $f : X \to X$. This section develops the combinatorial structure of iteration — the *functional graph* — and proves the eventual-periodicity facts that Chapter 5 packages as the safe finite power. The material is classical [13, 14].

5.1. The rho shape.

Definition 2.28 (Functional graph; orbit). The *functional graph* of f is the directed graph with vertex set X and one edge $x \to f(x)$ for each x; every vertex has out-degree exactly one (totality and single-valuedness). The *orbit* of x is the sequence $x, f(x), f^2(x), \ldots$.

Definition 2.29 (Periodic and eventually periodic points). A point x is *periodic* if $f^k(x) = x$ for some $k \geq 1$; the least such k is its *period*. A point is *eventually periodic* if $f^i(x)$ is periodic for some $i \geq 0$; the least such i is the *tail length* (or index) of x.

Lemma 2.30 (Every orbit enters a cycle). *Let $|X| = n$ and $x \in X$. Then:*

1. *The orbit of x repeats within n steps: there exist $0 \leq i < j \leq n$ with $f^i(x) = f^j(x)$.*
2. *x is eventually periodic, with tail length $t(x) \leq n - 1$ and the period $\ell(x)$ of the periodic point it reaches satisfying $1 \leq \ell(x) \leq n$, and moreover $t(x) + \ell(x) \leq n$.*
3. *For all $a \geq t(x)$ and all $b \geq 0$: $f^{a+b}(x) = f^a(x)$ if and only if $\ell(x) \mid b$.*

Proof. (1) The $n+1$ points $x, f(x), \ldots, f^n(x)$ lie in the n-element set X; by pigeonhole two coincide, say $f^i(x) = f^j(x)$ with $0 \leq i < j \leq n$.

(2) Take $i < j$ as in (1) with j minimal and, for that j, i minimal. Set $t = i$ and $\ell = j - i$. The point $y = f^t(x)$ satisfies $f^\ell(y) = f^j(x) = f^i(x) = y$, so y is periodic and x is eventually periodic with tail length at most t. Minimality of the pair makes the points $x, f(x), \ldots, f^{j-1}(x)$ pairwise distinct (a repeat among them would contradict minimality of j, or of i for the given j); there are $t + \ell = j$ of these distinct points in an n-set, so $t + \ell \leq n$, which gives both $t \leq n - 1$ (since $\ell \geq 1$) and $\ell \leq n$ (since $t \geq 0$). Finally the period of y divides ℓ and its tail is 0, so the stated bounds hold for the true tail length and true period as well.

(3) Let $y = f^{t(x)}(x)$, a periodic point of period $\ell = \ell(x)$, and let $a \geq t(x)$, say $a = t(x) + c$. Then $f^a(x) = f^c(y)$, and $f^{a+b}(x) = f^{c+b}(y)$. Since y has period ℓ, the points $y, f(y), \ldots, f^{\ell-1}(y)$ are pairwise distinct (a coincidence $f^u(y) = f^v(y)$ with $0 \leq u < v < \ell$ would give $f^{u+(\ell-v)}(y) = f^\ell(y) = y$ with $1 \leq u + \ell - v < \ell$, contradicting minimality of the period), and $f^m(y) = f^{m'}(y)$ iff $m \equiv m' \pmod{\ell}$ for $m, m' \geq 0$: the "if" direction is $f^\ell(y) = y$ iterated, and the "only if" direction follows because both sides reduce to points of the distinct list. Applying this with $m = c + b$, $m' = c$ gives $f^{a+b}(x) = f^a(x)$ iff $\ell \mid b$. □

The picture painted by Lemma 2.30 is the classical *rho* (ρ) shape: each orbit runs along a tail of at most $n - 1$ steps and then winds forever around a cycle of length at most n. Globally, the functional graph decomposes into weakly connected components, each consisting of exactly one directed cycle with rooted trees hanging off the cycle vertices, edges oriented toward the cycle; the periodic points are precisely the cycle vertices.

5.2. Index and period of the endomap. Iteration also acts on f itself, inside the finite monoid T_X.

THEOREM 2.31 (Eventual periodicity of powers; index and period). *Let $f \in T_X$ with $|X| = n \geq 1$. There exist a least $t \geq 0$ (the* index*) and a least $p \geq 1$ (the* period*) such that $f^{t+p} = f^t$; and then $f^a = f^b$ for $a, b \geq t$ iff $a \equiv b \pmod{p}$. Moreover $t \leq n - 1$ and p equals the least common multiple of the cycle lengths of the functional graph, so $p \leq \operatorname{lcm}(1, 2, \ldots, n)$ and, crudely, $p \mid n!$.*

PROOF. The powers $f^0, f^1, f^2, \ldots$ live in the finite monoid T_X of size n^n, so by pigeonhole some two coincide, $f^i = f^j$ with $i < j$; then $f^a = f^{a+(j-i)}$ for all $a \geq i$ (compose both sides of $f^i = f^j$ with f^{a-i}), so the sequence of powers is eventually periodic as a sequence in T_X; define t as the least index from which some repetition recurs and p as the least positive b with $f^{t+b} = f^t$. The biconditional for $a, b \geq t$ follows exactly as in the proof of Lemma 2.30(3), applied to the orbit of id_X under composition with f in T_X.

For the bound on t: by Lemma 2.30(2) every point has tail length at most $n - 1$, so for every x the value $f^{n-1}(x)$ is already periodic; hence for any b that is a multiple of all point-periods, $f^{(n-1)+b} = f^{n-1}$ pointwise, showing the index is at most $n - 1$.

For the identification of p: let L be the least common multiple of the cycle lengths (equivalently, of the periods $\ell(x)$ over all x, since each $\ell(x)$ is the length of the cycle its orbit enters). If $a \geq t$ then in particular $a \geq t(x)$ for every x (the pointwise tails are dominated by the index — indeed $f^a(x)$ must be periodic for repetition of the map to occur at a), and Lemma 2.30(3) gives: $f^{a+b} = f^a$ as maps iff $\ell(x) \mid b$ for every x, iff $L \mid b$. Hence $p = L$. Each cycle length is at most n, so L divides $\operatorname{lcm}(1, \ldots, n)$, which in turn divides $n!$ since every factor $1 \leq k \leq n$ divides $n!$. □

5.3. The factorial idempotent power.

THEOREM 2.32 (Factorial idempotent power). *For every finite set X and every $f \in T_X$, the power $f^{|X|!}$ is idempotent:*

$$f^{|X|!} \circ f^{|X|!} = f^{|X|!}.$$

PROOF. Write $N = |X|$. For $N = 0$ the only endomap is the empty function, which is its own composite, and $f^{0!} = f^1 = f$; the claim holds. For $N = 1$ the only endomap is id_X, idempotent. Assume $N \geq 2$; then $N! \geq N > N - 1$. Fix any $x \in X$. By Lemma 2.30(2), the tail length of x is at most $N - 1 < N!$, so $f^{N!}(x)$ is a periodic point; let $\ell \leq N$ be the length of its cycle. Since every integer from 1 to N divides $N!$, we have $\ell \mid N!$, and Lemma 2.30(3)

(with $a = N!$, $b = N!$) gives $f^{2N!}(x) = f^{N!}(x)$. As x was arbitrary, $f^{2N!} = f^{N!}$ extensionally, which is the claim. □

REMARK 2.33 (Sharpness and cost). The exponent $N!$ is chosen for the simplicity of its correctness proof, not for minimality: Theorem 2.31 shows the least idempotent power of f is f^a for the least $a \geq t$ divisible by p, which can be far smaller than $N!$. What $N!$ buys is *uniformity* — one exponent that works for every endomap of every N-element carrier, computable without inspecting the functional graph. The kernel computes $f^{N!}$ by binary powering, so the cost is $O(\log(N!)) = O(N \log N)$ compositions, each $O(N)$ array operations; Chapter 5 details the implementation and the bounded presentation-name scheme for such large exponents. The exhaustive verification of Theorem 2.32 over all 289 endomaps on carriers of size at most four (Chapter 10) is possible precisely because Corollary 2.13 counts that family exactly.

REMARK 2.34 (What the theorem does not say). The theorem quantifies over single elements f of the monoid T_X. It says nothing about products of *distinct* endomaps — the alternating scenario of Section 4.2 satisfies the theorem for each generator and still refutes the stabilization conclusion for the sequence — and nothing at all in the heterogeneous case, where no monoid contains the sequence. This is the exact boundary the kernel's type system draws.

5.4. Idempotents and the eventual image.

PROPOSITION 2.35 (Characterization of idempotents). *For $e \in T_X$, the following are equivalent: (i) $e \circ e = e$; (ii) e restricts to the identity on its image, i.e. $e(y) = y$ for every $y \in \mathrm{im}(e)$; (iii) $\mathrm{im}(e)$ is exactly the set of fixed points of e.*

PROOF. (i) ⇒ (ii): for $y \in \mathrm{im}(e)$ write $y = e(x)$; then $e(y) = e(e(x)) = e(x) = y$. (ii) ⇒ (iii): every image point is fixed by (ii), and every fixed point $y = e(y)$ is an image point. (iii) ⇒ (i): for any x, $e(x) \in \mathrm{im}(e)$ is a fixed point, so $e(e(x)) = e(x)$; extensionally $e \circ e = e$. □

PROPOSITION 2.36 (The eventual image). *Let $f \in T_X$, $|X| = n \geq 1$. The images of the powers form a descending chain $\mathrm{im}(f) \supseteq \mathrm{im}(f^2) \supseteq \cdots$ that stabilizes at some $k \leq n - 1$ to the* eventual image *$X_\infty = \mathrm{im}(f^k) = \mathrm{im}(f^m)$ for all $m \geq k$. Moreover X_∞ is exactly the set of periodic points of f, f restricts to a bijection of X_∞, and the idempotent $e = f^{n!}$ of Theorem 2.32 has $\mathrm{im}(e) = X_\infty$.*

PROOF. $\mathrm{im}(f^{m+1}) = f(\mathrm{im}(f^m)) \subseteq \mathrm{im}(f^m)$ gives the chain. A strictly decreasing chain of nonempty subsets of an n-set has length at most n, and once two consecutive images coincide, all later ones do (apply f to the equality); since $|\mathrm{im}(f)| \leq n$ and each strict step drops the size by at least one, stabilization occurs by index $n - 1$ at the latest (if no strict drop ever occurs, f is bijective and the chain is constant from the start).

Periodic points lie in every $\mathrm{im}(f^m)$ (a point of period ℓ is $f^{m\ell}$ of itself), hence in X_∞. Conversely, take $m \geq n - 1$ and $x \in X$: the tail of x has length at most $n - 1 \leq m$ by Lemma 2.30(2), so $f^m(x)$ lies on a cycle and is periodic. Hence $\mathrm{im}(f^m)$ consists of periodic points for all such m, and X_∞ is exactly the periodic set. On X_∞, f is injective: a merge $f(u) = f(v)$ with $u \neq v$ both periodic would give, for a common multiple P of their periods with $P \geq 1$, $u = f^P(u)$ and $v = f^P(v)$, yet $f^P(u) = f^{P-1}(f(u)) = f^{P-1}(f(v)) = f^P(v)$, forcing $u = v$. An injective self-map of the finite set X_∞ (note $f(X_\infty) = X_\infty$ by stabilization) is a bijection of it (Proposition 2.7). Finally $\mathrm{im}(f^{n!}) = X_\infty$ since $n! \geq n - 1$ puts $n!$ beyond the stabilization index. □

EXAMPLE 2.37 (A worked functional graph). Let X have six elements with standard indices $0, \dots, 5$ and let f have mapping tuple $(1, 2, 3, 1, 5, 5)$, i.e.

$$0 \mapsto 1, \quad 1 \mapsto 2, \quad 2 \mapsto 3, \quad 3 \mapsto 1, \quad 4 \mapsto 5, \quad 5 \mapsto 5.$$

The functional graph has two components. Component one contains the cycle $1 \to 2 \to 3 \to 1$ of length 3, with the tail $0 \to 1$ hanging on it: point 0 has tail length 1, points $1, 2, 3$ are periodic of period 3. Component two contains the fixed point 5 (a cycle of length 1) with the tail $4 \to 5$: point 4 has tail length 1, point 5 is periodic of period 1.

Reading off the theory: tail lengths are 1, comfortably within the bound $n - 1 = 5$ of Lemma 2.30; the cycle lengths are 3 and 1, so the period of f is $p = \operatorname{lcm}(3, 1) = 3$ (Theorem 2.31).

Compute the powers as mapping tuples, each a mechanical fold of lookups:

$$f = (1, 2, 3, 1, 5, 5), \qquad f^2 = (2, 3, 1, 2, 5, 5), \qquad f^3 = (3, 1, 2, 3, 5, 5), \qquad f^4 = (1, 2, 3, 1, 5, 5).$$

Thus $f^4 = f^1$ while f^1, f^2, f^3 are pairwise distinct and none equals $f^0 = \mathrm{id}_X$: the index of f is $t = 1$ and the period is confirmed as $p = 3$, with $f^a = f^b$ for $a, b \geq 1$ iff $a \equiv b \pmod 3$.

The least idempotent power is the least $a \geq t$ divisible by p, namely $a = 3$. Check directly against Proposition 2.35: f^3 sends $0 \mapsto 3$, $1 \mapsto 1$, $2 \mapsto 2$, $3 \mapsto 3$, $4 \mapsto 5$, $5 \mapsto 5$; its image is $\{1, 2, 3, 5\}$, which it fixes pointwise — an idempotent whose image is exactly the periodic set, as Proposition 2.36 requires. The uniform exponent of Theorem 2.32 is $6! = 720$; since $720 \geq 1$ and $720 \equiv 0 \pmod 3$, we get $f^{720} = f^3$: the factorial power lands on the same idempotent, as it must, at the cost of a much larger exponent — the price of uniformity noted in Remark 2.33.

The example also illustrates a methodological point the kernel takes seriously: powers are computed and compared as mapping tuples, never chased through orbits in prose. The tuple representation makes each power an auditable mechanical object, and the exhaustive suite of Chapter 10 compares exactly these tuples; informal orbit-chasing is precisely where a wrong entry survives review.

6. Finite-dimensional Hilbert spaces

The kernel's lift (Chapter 7) sends finite functions to linear maps between finite-dimensional complex Hilbert spaces. We assemble the needed linear algebra; the standard reference for this material is Horn and Johnson [**12**].

6.1. Inner products and orthonormal bases.

DEFINITION 2.38. A *finite-dimensional (complex) Hilbert space* is a finite-dimensional complex vector space H with an inner product $\langle \cdot, \cdot \rangle : H \times H \to \mathbb{C}$, linear in the second argument, conjugate-linear in the first, conjugate-symmetric, and positive definite ($\langle v, v \rangle > 0$ for $v \neq 0$). The norm is $\|v\| = \sqrt{\langle v, v \rangle}$. A family $(e_1, \dots, e_d)$ is an *orthonormal basis* (ONB) if $\langle e_i, e_j \rangle = \delta_{ij}$ and the family spans H.

In finite dimension no completeness axiom is needed: every finite-dimensional inner-product space is complete, so "Hilbert space" adds nothing beyond the inner product, and we use the term for continuity with the lift literature. Every such space has an ONB (Gram–Schmidt), and relative to an ONB every vector expands as $v = \sum_i \langle e_i, v \rangle e_i$ with $\|v\|^2 = \sum_i |\langle e_i, v \rangle|^2$ (Parseval; a two-line computation from orthonormality).

PROPOSITION 2.39 (Cauchy–Schwarz). *For all $v, w \in H$: $|\langle v, w \rangle| \leq \|v\| \|w\|$, with equality iff v and w are linearly dependent.*

PROOF. If $w = 0$ both sides vanish and $w = 0 \cdot v$ gives dependence. Otherwise let $\lambda = \langle w, v\rangle / \|w\|^2$ and expand, using sesquilinearity and conjugate-symmetry:

$$0 \le \|v - \lambda w\|^2 = \|v\|^2 - \overline{\lambda}\,\langle w, v\rangle - \lambda\,\overline{\langle w, v\rangle} + |\lambda|^2\,\|w\|^2 = \|v\|^2 - \frac{|\langle w, v\rangle|^2}{\|w\|^2},$$

because each of the three subtracted-or-added terms evaluates to $\pm|\langle w, v\rangle|^2/\|w\|^2$ with signs $-, -, +$. Rearranging gives $|\langle w, v\rangle|^2 \le \|v\|^2\|w\|^2$, and $|\langle v, w\rangle| = |\langle w, v\rangle|$ by conjugate-symmetry. Equality holds iff $\|v - \lambda w\| = 0$, i.e. $v = \lambda w$; conversely dependence gives equality by direct substitution. □

Cauchy–Schwarz is used silently whenever the operator norm is manipulated (for instance, it makes $\|A\| = \max_{\|v\|=\|w\|=1} |\langle w, Av\rangle|$ an equivalent definition); we record it once so that no later step is unproved.

For the kernel the relevant spaces are concrete: for a finite object X with ordered carrier $(x_0, \dots, x_{n-1})$, the space H_X is $\mathbb{C}^n$ with the standard inner product and the standard ONB $(e_{x_0}, \dots, e_{x_{n-1}})$, where the basis vector attached to a carrier element is determined by that element's *standard index*. The order of the carrier is thereby welded to the geometry: permuting the carrier permutes the basis, changing every matrix below. This is one of the three places the ordered-input rule of Chapter 3 is load-bearing.

6.2. Linear maps as matrices; adjoints. Fix ONBs $(e_1, \dots, e_n)$ of H_1 and $(f_1, \dots, f_m)$ of H_2. A linear map $A : H_1 \to H_2$ is determined by its values on the basis, and its *matrix* has entries $A_{ji} = \langle f_j, Ae_i\rangle$, so that $Ae_i = \sum_j A_{ji} f_j$; composition of maps is multiplication of matrices. The *adjoint* $A^* : H_2 \to H_1$ is the unique linear map with $\langle A^* w, v\rangle = \langle w, Av\rangle$ for all $v \in H_1$, $w \in H_2$; existence and uniqueness in finite dimension follow by defining A^* on the basis via $(A^*)_{ij} = \overline{A_{ji}}$ — the conjugate transpose — and checking the defining identity on basis pairs. The adjoint reverses composition, $(AB)^* = B^* A^*$, and $(A^*)^* = A$.

A map U is an *isometry* if $\|Uv\| = \|v\|$ for all v; polarizing, this is equivalent to $\langle Uv, Uw\rangle = \langle v, w\rangle$ for all v, w, and hence to $U^* U = I$.

6.3. Operator norm.

DEFINITION 2.40. The *operator norm* of a linear map $A : H_1 \to H_2$ between finite-dimensional Hilbert spaces is

$$\|A\| = \max_{\|v\|=1} \|Av\|,$$

the maximum being attained because the unit sphere of a finite-dimensional space is compact and $v \mapsto \|Av\|$ is continuous.

Immediately from the definition: $\|Av\| \le \|A\|\|v\|$ for all v; $\|AB\| \le \|A\|\|B\|$ (submultiplicativity); and $\|A\| = 0$ iff $A = 0$.

6.4. Positive semidefinite matrices and spectral facts.

DEFINITION 2.41. A linear map $P : H \to H$ (equivalently its matrix in an ONB) is *self-adjoint* if $P^* = P$, and *positive semidefinite* (PSD) if it is self-adjoint and $\langle v, Pv\rangle \ge 0$ for all v.

For any $A : H_1 \to H_2$, the map $A^* A : H_1 \to H_1$ is PSD: $(A^* A)^* = A^* A^{**} = A^* A$ and $\langle v, A^* Av\rangle = \langle Av, Av\rangle = \|Av\|^2 \ge 0$.

We use the spectral theorem for self-adjoint maps in the following form, cited as standard [12]: *every self-adjoint map P on a finite-dimensional Hilbert space has an ONB of eigenvectors with real eigenvalues; P is PSD iff all eigenvalues are nonnegative.* From it we prove the two facts the lift theory consumes.

PROPOSITION 2.42 (Norm squared is the top eigenvalue of the Gram map). *For any linear $A : H_1 \to H_2$,*

$$\|A\|^2 \;=\; \lambda_{\max}\big(A^*A\big),$$

the largest eigenvalue of the PSD map A^*A.

PROOF. Let $(u_1, \dots, u_n)$ be an ONB of H_1 consisting of eigenvectors of A^*A, with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$. For a unit vector $v = \sum_i c_i u_i$ (so $\sum_i |c_i|^2 = 1$),

$$\|Av\|^2 = \langle Av, Av\rangle = \langle v, A^*Av\rangle = \Big\langle \sum_i c_i u_i, \sum_i c_i \lambda_i u_i \Big\rangle = \sum_i \lambda_i |c_i|^2,$$

a convex combination of the eigenvalues, hence at most λ_1; and the value λ_1 is attained at $v = u_1$. Taking the maximum over unit v and then square roots gives the claim. □

PROPOSITION 2.43 (Spectrum of the all-ones matrix). *Let J_m be the $m \times m$ matrix ($m \geq 1$) with every entry equal to 1. Then J_m is PSD,* $\operatorname{rank}(J_m) = 1$, *$J_m^2 = mJ_m$, and the spectrum of J_m is: eigenvalue m with multiplicity 1 (eigenvector the all-ones vector $\mathbf{1} = (1, \dots, 1)^\mathsf{T}$), and eigenvalue 0 with multiplicity $m-1$ (eigenspace the hyperplane $\{v : \sum_i v_i = 0\}$). Consequently* $\operatorname{tr}(J_m) = m$ *and* $\lambda_{\max}(J_m) = m$.

PROOF. $J_m = \mathbf{1}\mathbf{1}^\mathsf{T}$, an outer product, so every column is a multiple of $\mathbf{1}$ and $\operatorname{rank}(J_m) = 1$; also $J_m^* = J_m$ (real symmetric) and $\langle v, J_m v\rangle = |\mathbf{1}^\mathsf{T} v|^2 = \big|\sum_i v_i\big|^2 \geq 0$, so J_m is PSD. Compute $J_m\mathbf{1} = \mathbf{1}(\mathbf{1}^\mathsf{T}\mathbf{1}) = m\mathbf{1}$: eigenvalue m on the line spanned by $\mathbf{1}$. For any v with $\sum_i v_i = 0$, $J_m v = \mathbf{1}(\mathbf{1}^\mathsf{T} v) = 0$: eigenvalue 0 on a hyperplane of dimension $m - 1$. The line and the hyperplane are orthogonal complements, so together they exhaust $\mathbb{C}^m$, and the listed spectrum is complete. Then $J_m^2 = \mathbf{1}(\mathbf{1}^\mathsf{T}\mathbf{1})\mathbf{1}^\mathsf{T} = mJ_m$, the trace is the eigenvalue sum $m + 0 \cdot (m-1) = m$, and the largest eigenvalue is m. □

Two further pieces of standard machinery are needed to *assemble* spectra, because Chapter 7's Gram matrices arrive as block structures only after a basis reordering.

LEMMA 2.44 (Unitary and permutation similarity preserve spectra). *Let U be unitary ($U^*U = UU^* = I$) and $P = U^*QU$. Then P and Q have the same eigenvalues with the same multiplicities, and P is PSD iff Q is. In particular this applies when U is a* permutation matrix *— the matrix of a basis reordering, with entries $U_{ij} = 1$ if the reordering sends position j to position i and 0 otherwise, which is unitary since its columns are distinct standard basis vectors.*

PROOF. If $Qv = \mu v$ with $v \neq 0$, then $P(U^*v) = U^*QUU^*v = U^*Qv = \mu(U^*v)$ and $U^*v \neq 0$ (U^* is invertible), so μ is an eigenvalue of P; the map $v \mapsto U^*v$ is a linear bijection between the eigenspaces, preserving multiplicities, and the argument reverses via $Q = UPU^*$. For positivity: $\langle v, Pv\rangle = \langle Uv, Q\,Uv\rangle$, and $v \mapsto Uv$ ranges over all vectors, so the two quadratic forms are nonnegative together; self-adjointness transfers by $P^* = U^*Q^*U$. □

LEMMA 2.45 (Spectrum of a direct sum). *Let $P = \bigoplus_{r=1}^{s} P_r$ be block-diagonal with square blocks $P_1, \dots, P_s$. Then the spectrum of P is the multiset union of the spectra of the blocks, and $\lambda_{\max}(P) = \max_r \lambda_{\max}(P_r)$ when all blocks are self-adjoint.*

PROOF. The ambient space decomposes as an orthogonal direct sum $H = \bigoplus_r H_r$ with P mapping each H_r to itself by P_r. A vector $v = (v_1, \ldots, v_s)$ satisfies $Pv = \mu v$ iff $P_r v_r = \mu v_r$ for every r; for $v \neq 0$ some $v_r \neq 0$, so every eigenvalue of P is an eigenvalue of some block, and conversely a block eigenvector extends by zeros to an eigenvector of P. For multiplicities: the μ-eigenspace of P is the direct sum of the μ-eigenspaces of the blocks, so dimensions add. The $\lambda_{\max}$ statement follows since the multiset union's maximum is the maximum of the blockwise maxima. □

6.5. Projectors and partial isometries. Chapter 7's permanent counterexample — an idempotent lift that is *not* a projector — needs the following standard vocabulary to be stated precisely.

DEFINITION 2.46. A linear map $P : H \to H$ is an *orthogonal projector* if $P^2 = P$ and $P^* = P$. A linear map $A : H_1 \to H_2$ is a *partial isometry* if A^*A is an orthogonal projector.

PROPOSITION 2.47 (Elementary projector facts). *(i) An orthogonal projector satisfies $\|P\| \leq 1$, with $\|P\| = 1$ unless $P = 0$. (ii) An idempotent that is not self-adjoint can have norm strictly greater than 1; idempotence alone bounds nothing. (iii) An isometry is a partial isometry ($A^*A = I$ is a projector).*

PROOF. (i) P is self-adjoint, so by the spectral theorem it has an ONB of eigenvectors; $P^2 = P$ forces each eigenvalue μ to satisfy $\mu^2 = \mu$, so $\mu \in \{0, 1\}$. By Proposition 2.42 applied to $P^*P = P^2 = P$, $\|P\|^2 = \lambda_{\max}(P) \leq 1$, with equality iff the eigenvalue 1 occurs, i.e. iff $P \neq 0$. (ii) is witnessed by any concrete example; the canonical one, the 2×2 matrix with rows $(1, 1)$ and $(0, 0)$, is exactly the lifted constant map of Chapter 7, where its norm is computed to be $\sqrt{2}$. (iii) is immediate from the definitions since I is a projector. □

REMARK 2.48 (Why these two facts). Chapter 7 shows that for the lift A_f of a finite function f, the Gram map $A_f^* A_f$ is, after grouping the input basis by fibers, a direct sum of blocks J_{m_y} with $m_y = |f^{-1}(y)|$ the fiber sizes. The grouping is a basis reordering — a permutation similarity, harmless to spectra by Lemma 2.44 — and the direct sum's spectrum is the multiset union of the block spectra by Lemma 2.45. Feeding Proposition 2.43 through Proposition 2.42 then yields the exact norm formula $\|A_f\|^2 = \max_y |f^{-1}(y)|$ with no analysis beyond what this section contains. The trace identity $\operatorname{tr}(A_f^* A_f) = \sum_y m_y = |X|$ is Lemma 2.5. The primer is arranged so that Chapter 7's proofs are assembly, not invention.

7. Words and the shortlex order

The kernel's normal forms (Chapter 8) select, for each reachable denotation, the least word denoting it under the *shortlex* order. For that selection to be well defined and stable, shortlex must be a total order with no infinite descent — a well-order — on the relevant word sets. This section defines the order and proves those properties.

7.1. Words over a ranked alphabet.

DEFINITION 2.49 (Words). Let G be a finite set of *letters* equipped with a fixed total order, presented as a *ranking*: a bijection $\rho : G \to \{0, 1, \ldots, |G| - 1\}$. A *word* over G is a finite (possibly empty) sequence $w = g_1 g_2 \cdots g_k$ of letters; $|w| = k$ is its *length*, and ε denotes the empty word. G^* is the set of all words.

In the kernel the letters are generator labels from a declared catalog, and the ranking is the catalog's declared order — explicitly supplied, never inferred from container iteration

(Chapter 3). Words are additionally *typed* (their letters' arrows must compose in sequence from a declared start object), which restricts G^* to a subset; every property proved below is inherited by subsets, so we work with G^* and note the inheritance at the end.

DEFINITION 2.50 (Shortlex). For words $u = u_1 \cdots u_k$ and $v = v_1 \cdots v_l$ over (G, ρ), say $u <_{\text{slex}} v$ iff either

(1) $k < l$ (shorter words come first); or
(2) $k = l$ and there is a position $i \leq k$ with $\rho(u_j) = \rho(v_j)$ for all $j < i$ and $\rho(u_i) < \rho(v_i)$ (equal lengths compare lexicographically by rank).

PROPOSITION 2.51 (Shortlex is a strict total order). *On G^*, the relation $<_{\text{slex}}$ is irreflexive, transitive, and total: for distinct words $u \neq v$, exactly one of $u <_{\text{slex}} v$, $v <_{\text{slex}} u$ holds.*

PROOF. *Irreflexivity*: $u <_{\text{slex}} u$ would require $|u| < |u|$ or a position where $\rho(u_i) < \rho(u_i)$; both are impossible.

Totality: let $u \neq v$. If $|u| \neq |v|$ the length clause orders them one way and (by irreflexivity of $<$ on integers) not the other. If $|u| = |v| = k$, then since $u \neq v$ there is a least position i with $u_i \neq v_i$; all earlier positions agree, and since ρ is injective, exactly one of $\rho(u_i) < \rho(v_i)$, $\rho(v_i) < \rho(u_i)$ holds, ordering the words one way and not the other.

Transitivity: suppose $u <_{\text{slex}} v <_{\text{slex}} w$. If $|u| < |v|$ or $|v| < |w|$ then $|u| < |w|$ (lengths are ordered integers and are nondecreasing along $<_{\text{slex}}$), giving $u <_{\text{slex}} w$ unless $|u| = |w|$, which can happen only when $|u| = |v| = |w|$. So assume all lengths equal k and let i (resp. j) be the deciding position for u against v (resp. v against w). Put $h = \min(i, j)$. For all positions $< h$, all three words agree. At position h: if $h = i < j$ then u_h has smaller rank than $v_h = w_h$; if $h = j < i$ then $u_h = v_h$ has smaller rank than w_h; if $h = i = j$ then $\rho(u_h) < \rho(v_h) < \rho(w_h)$. In every case h witnesses $u <_{\text{slex}} w$. □

7.2. The well-order property.

THEOREM 2.52 (Shortlex is a well-order). *Every nonempty subset $S \subseteq G^*$ has a $<_{\text{slex}}$-least element. Equivalently, there is no infinite strictly $<_{\text{slex}}$-descending sequence of words.*

PROOF. The key counting observation: for each fixed length k there are exactly $|G|^k$ words of length k (Theorem 2.12, reading a word as a function from positions to letters), a finite number; and every word of length k precedes every word of length $k + 1$ in shortlex. Hence for any word w, the set $\{\, u \in G^* \mid u <_{\text{slex}} w \,\}$ is contained in the words of length at most $|w|$, of which there are $\sum_{k=0}^{|w|} |G|^k$ — finitely many. (For $|G| = 0$ the only word is ε and everything below is trivial; assume $|G| \geq 1$.)

Now let S be nonempty and pick any $w_0 \in S$. The set $S' = \{\, u \in S \mid u \leq_{\text{slex}} w_0 \,\}$ is nonempty (contains w_0) and finite (contained in the finite set of words at most w_0, plus w_0 itself). A nonempty finite subset of a strict total order has a least element — induct on the size: remove any element; the remainder's least element (or, at size one, the sole element) compared with the removed one yields the least, using totality (Proposition 2.51). Let m be the least element of S'. For any $s \in S$: if $s \leq_{\text{slex}} w_0$ then $s \in S'$ and $m \leq_{\text{slex}} s$; otherwise $m \leq_{\text{slex}} w_0 <_{\text{slex}} s$ by transitivity. So m is least in S.

For the equivalence with no infinite descent: an infinite strictly descending sequence would be a nonempty set with no least element (its every member is exceeded from below by a later member); conversely if some nonempty S had no least element one could choose a strictly descending sequence by repeatedly picking a smaller member. We only use the first direction, which is constructive given the finiteness above. □

COROLLARY 2.53 (Minimal representatives exist and are unique). *Let $W \subseteq G^*$ be any set of words and let $\sim$ be any equivalence on W (in Chapter 8: "denotes the same finite function"). Every nonempty equivalence class C has a unique $<_{\text{slex}}$-least element, which may serve as the canonical representative* $\mathrm{NF}(C)$.

PROOF. Existence is Theorem 2.52 applied to $C \subseteq G^*$. Uniqueness: two least elements are mutually $\leq_{\text{slex}}$, and antisymmetry — immediate from totality and irreflexivity of the strict order — forces them equal. □

7.3. Enumeration and the shortlex rank. Shortlex is not only a well-order; it is a computable enumeration, and the position of any word in that enumeration has a closed form. This matters operationally: Chapter 8's closure explores words in exactly this enumeration order, and the formula below is what makes "explored in shortlex order" an auditable claim rather than an implementation hope.

PROPOSITION 2.54 (Shortlex rank formula). *Let $|G| = g \geq 1$ and let $w = w_1 \cdots w_k$ be a word of length k. The number of words strictly $<_{\text{slex}}$-below w is*

$$\mathrm{rank}_{\text{slex}}(w) \;=\; \sum_{j=0}^{k-1} g^{\,j} \;+\; \sum_{i=1}^{k} \rho(w_i)\, g^{\,k-i},$$

the first sum counting all strictly shorter words and the second the length-k words that are lexicographically smaller. The map $w \mapsto \mathrm{rank}_{\text{slex}}(w)$ is the unique order-isomorphism from $(G^*, <_{\text{slex}})$ onto $(\mathbb{N}, <)$.

PROOF. Words shorter than w: there are g^j words of each length $j < k$ (Theorem 2.12), all below w by the length clause, giving the first sum. Words of length exactly k below w: such a word u agrees with w on some (possibly empty) prefix of length $i-1$ and has $\rho(u_i) < \rho(w_i)$ at the first disagreement i; for each choice of $i \in \{1, \ldots, k\}$ there are $\rho(w_i)$ choices of the smaller letter and g^{k-i} free choices for the remaining positions $i+1, \ldots, k$, and these cases are disjoint (distinct first-disagreement positions) and exhaustive. Summing gives the second sum.

For the order-isomorphism claim: $\mathrm{rank}_{\text{slex}}$ is strictly monotone (if $u <_{\text{slex}} w$ then every word below u is below w and u itself is below w, so $\mathrm{rank}_{\text{slex}}(u) < \mathrm{rank}_{\text{slex}}(w)$), hence injective; and it is surjective onto $\mathbb{N}$ because the count of words below the $(n+1)$-st word of the enumeration is exactly n (each natural number is achieved as one steps through the enumeration, which exists by Theorem 2.52: the least word has rank 0, and the least word above any w has rank $\mathrm{rank}_{\text{slex}}(w)+1$ since finitely many words lie below any word). Uniqueness of an order-isomorphism between two linear orders with no greatest element in which every element has finitely many predecessors is standard: any two such isomorphisms agree by induction along the enumeration. □

EXAMPLE 2.55 (Shortlex over a two-letter catalog). Let $G = \{a, b\}$ with declared ranking $\rho(a) = 0$, $\rho(b) = 1$. The shortlex enumeration begins

$$\varepsilon,\ a,\ b,\ aa,\ ab,\ ba,\ bb,\ aaa,\ aab,\ aba,\ abb,\ baa,\ \ldots$$

and, for instance, $\mathrm{rank}_{\text{slex}}(ba) = (1+2)+(1\cdot 2+0\cdot 1) = 5$: three shorter words (ε, a, b) plus two smaller words of length two $(aa,\ ab)$, in agreement with the listing. Reversing the declared ranking ($\rho(b) = 0$, $\rho(a) = 1$) reverses every lexicographic tiebreak: the enumeration becomes $\varepsilon, b, a, bb, ba, ab, aa, \ldots$, and a denotation reachable by both ab and ba (should the generators

commute on their carrier) changes its canonical representative from ab to ba. Nothing mathematical broke — both enumerations are lawful well-orders — but the *selection* changed, which is why the kernel treats the catalog order as declared, versioned data (Remark 2.56 below).

Remark 2.56 (Inheritance by typed words; dependence on the ranking). Both Proposition 2.51 and Theorem 2.52 are statements about arbitrary subsets of G^*, so they hold verbatim for the kernel's typed words, which form such a subset once start objects are fixed (formally, one takes a disjoint union over start objects, ordering within each; Chapter 8 makes this precise). Note also what the results do *not* give: canonicity is relative to the ranking ρ. Two catalogs with the same generators in different declared orders induce different shortlex orders and may select different representatives for the same denotation. This is a feature the kernel documents rather than a defect it hides — representatives are canonical *relative to a declared, versioned catalog*, and comparing normal forms across catalogs without a catalog identity is meaningless. The determinism discipline of Chapter 3 exists precisely so that ρ is always explicit data, never an accident of container iteration.

8. Summary of the toolkit

For ease of later reference, the primer's yield is:

- *Extensionality and the tuple representation* (Section 1): finite total functions are mapping tuples; equality is componentwise; composition and identity satisfy the category laws pointwise (Proposition 2.3).
- *Fibers* (Section 2): nonempty fibers partition the domain (Lemma 2.5); injectivity, surjectivity, and bijectivity are fiber-size conditions; on equal finite cardinalities the three notions coincide (Proposition 2.7).
- *Counting* (Section 3): there are $|Y|^{|X|}$ total maps (Theorem 2.12); the test families' cardinalities 19,683, 289, and the falling factorials are exact (Corollary 2.13).
- *The ladder* (Section 4): monoid = one-object category; semigroup = one-object semigroupoid; powers live in $\mathrm{End}(X)$ and nowhere else (Proposition 2.22, Remark 2.23); all parenthesizations of a composable sequence agree (Theorem 2.24); functors preserve identities, composites, and powers (Definition 2.26).
- *Functional graphs* (Section 5): orbits are rho-shaped with tail $\leq n-1$ and cycle $\leq n$ (Lemma 2.30); powers of f have an index $\leq n-1$ and a period equal to the lcm of cycle lengths, dividing $n!$ (Theorem 2.31); $f^{n!}$ is idempotent (Theorem 2.32).
- *Hilbert spaces* (Section 6): adjoints, operator norm, PSD Gram maps; $\|A\|^2 = \lambda_{\max}(A^*A)$ (Proposition 2.42); the all-ones matrix J_m has spectrum $\{m, 0^{(m-1)}\}$ (Proposition 2.43); permutation similarity and direct sums assemble spectra (Lemmas 2.44–2.45); projectors have norm at most one, bare idempotents do not (Proposition 2.47).
- *Shortlex* (Section 7): a strict total well-order on words over a ranked alphabet (Proposition 2.51, Theorem 2.52); least representatives exist and are unique per class (Corollary 2.53), relative to the declared ranking; the rank formula makes the enumeration computable and auditable (Proposition 2.54).

Every later chapter draws on this list and on nothing outside it. The mathematics being standard is the point: the kernel's claims are meant to be auditable by any reader with this chapter in hand, and the program's open formalization obligations (Chapter 11, Appendix B) are obligations to mechanize precisely these statements, no more and no less.

CHAPTER 3

Objects, Arrows, and the Determinism Discipline

This chapter defines the kernel's two ground-level notions — objects and arrows — and develops the discipline that makes everything built on them replayable. The mathematical content is thin by design: an object is a named, explicitly ordered finite carrier, and an arrow is a total function between two declared objects, equal to another arrow exactly when the declared data agree. The engineering content is thick, and it is the chapter's real subject. Three decisions do the work: object identity includes the declared name, so that semantically distinct objects can never be silently coerced; every order-sensitive constructor input must be a replayable ordered sequence, with sets, mappings, and one-shot iterables rejected at construction; and arrow equality is extensional over declared data only, with presentation names excluded from equality and hashing. Each decision is argued, not merely announced, and each is enforced in the constructors of the reference implementation (`category.py` in `src/tks_v8_core`) and demonstrated by negative fixtures — tests whose entire assertion is that a forbidden construction fails with the right typed error. The chapter closes with the deepest of the three arguments: a treatment of hash-order nondeterminism as a general software-evidence problem, of which the kernel's ordered-input rule is a local solution.

Throughout, statements about what the implementation does are statements about the Gate 1 prototype sources hashed in the project manifest; their evidential status is EXECUTABLE VALIDATION (ledger items PO-TKS8-CORE-02 and PO-TKS8-CORE-03), which is a claim about the finite executable model and its test suite, not a proof about all executions of all Python interpreters.

1. Objects

1.1. Definition.

DEFINITION 3.1 (Finite object). An *object* is a pair

$$X = (\text{name}_X,\ \text{carrier}_X)$$

where name_X is a non-blank string and carrier_X is a finite, explicitly ordered sequence $(x_0, x_1, \ldots, x_{n-1})$ of pairwise distinct, hashable elements. We write $|X| = n$ for the carrier length. The position of an element in the carrier is its *standard index*; standard indices run from 0 to $|X| - 1$.

Two objects are *equal* iff their names are equal *and* their carrier sequences are equal element-by-element in order.

The implementation realizes Definition 3.1 as the frozen dataclass `FiniteObject` with fields `name` and `elements`. Construction validates all four conditions — non-blank string name, ordered-sequence input (Section 3), hashability, pairwise distinctness — and raises the typed error `ValidationError` on any violation. The class exposes `index(element)`, returning the standard index of a carrier element and raising `ValidationError` for a non-element; this method is the single bridge between element-level talk and index-level talk, and

everything numerical downstream (mapping tuples, basis vectors, matrix entries) lives on the index side.

Three consequences of the definition deserve immediate notice.

First, *the carrier is a sequence, not a set.* The elements are required to be distinct, so the carrier determines an underlying set, but the object is strictly more data: the same three elements in two different orders yield two *unequal* objects. This is deliberate. The standard indices are the coordinates against which arrows' mapping tuples, the Hilbert-space basis of Chapter 7, and the shortlex machinery of Chapter 8 are all defined; an object that forgot its order would make all of them ill-defined (Section 3).

Second, *hashability of elements is required but hash values are never consulted for order.* Elements must be hashable so that fiber computations, deduplication tables, and denotation keys can use them in dictionaries — as keys whose *lookup* is hash-based but whose *enumeration* the kernel never performs. The distinction between using a hash table for membership and using it for iteration is exactly the line drawn in Section 4.

Third, *the name is load-bearing.* It is not a comment. It participates in object equality, and through object equality in composition legality, endomap constructibility, path well-typedness, and every downstream judgment. The next section argues that this is the right choice.

2. Name-inclusive identity

2.1. The anti-coercion argument. Consider the alternative: *carrier-only identity*, under which two objects are equal whenever their ordered carriers are equal. This is the natural default — it is what "a finite set with an enumeration" means mathematically — and it is wrong for a typed kernel, for one reason: composition legality must track *intent*, and carriers systematically underdetermine intent.

The failure pattern is silent coercion between semantically distinct objects of coincidentally equal carrier. Three escalating examples:

Example 3.2 (Two registers over the same alphabet). A model has two binary registers, `GateA` and `GateB`, each carrying the ordered carrier $(0, 1)$. Under carrier-only identity the two objects are *the same object*, and the kernel would accept the composite of an arrow into `GateA` with an arrow out of `GateB` — a data-flow that the modeler never asserted exists. Under name-inclusive identity the two objects are distinct, the composite raises `CompositionTypeError`, and connecting the registers requires writing an explicit arrow `GateA` $\to$ `GateB` — an auditable modeling decision, made once, in one reviewable place.

Example 3.3 (Coincident carriers across pipelines). The scenario of Section 3.2 ended with a four-element evaluative carrier and a four-element quadrant carrier. Suppose a refactor renames the quadrant labels so that both carriers become literally (e_0, e_1, e_2, e_3). Carrier-only identity now merges the two objects, and the underived composite of that scenario — the one that smuggled a quadrant row into an evaluative conclusion — becomes legal again, silently, as a side effect of a renaming that touched no arrow and no analysis. Name-inclusive identity is immune: `EvalSummary` and `QuadState` remain distinct however their elements are spelled, and the only way to re-enable the composite is to merge the declared object identities themselves — a change no reviewer would mistake for cosmetic.

Example 3.4 (Same mathematics, different units). Two objects carry $(0, 1, 2, 3)$: one is a count of retries, one an index into a rubric. Every mathematical property of the two carriers is identical. No inspection of elements, cardinalities, or even attached arrows need

reveal the difference — the difference *is* the intent, and the name is the only place intent lives. Dimensional-analysis disciplines in physical computation make the same move for the same reason: quantities with identical numeric representations are kept apart by a nominal tag because the cost of silent unit coercion is unbounded.

RULE 3.5 (Name-inclusive identity). Object equality is equality of the pair (declared name, ordered carrier). No operation of the kernel — composition, endomap construction, path validation, lift construction, catalog registration — ever identifies two objects with different names, or two objects with the same name and different carriers. There is no coercion path, implicit or explicit, between distinct objects; the only way to relate them is an arrow.

The second half of the rule — same name, different carriers, still unequal — closes the converse hole: a name is not a *claim* that overrides data, it is one component of a pair. Two versions of "the same" object with different carriers (an element added, an order changed) are different objects, and arrows attached to the old version do not type-check against the new. This is again deliberate: a carrier change alters standard indices, hence the meaning of every mapping tuple that references the object, and forcing the type mismatch to surface is strictly safer than letting stale arrows flow into new coordinates.

REMARK 3.6 (Cost of the rule, and why it is paid). Name-inclusive identity has a real cost: it refuses some identifications a mathematician would make without thought, and it obliges the modeler to write explicit conversion arrows (often identities-on-carrier with a new name) at boundaries where two vocabularies genuinely coincide. The kernel pays this cost knowingly. The conversion arrow is not bureaucracy; it is the audit record of the identification — who asserted it, in which version, reviewable and revocable. Every silent coercion the rule forbids is a place where that record would otherwise not exist.

2.2. Carrier evolution and object versioning. The same-name-different-carrier clause has a practical corollary worth developing: it prescribes how models *evolve*. Suppose an analysis adds a fourth state to a three-state carrier. The tempting move — mutate "the" object in place — is unavailable twice over: the implementation's objects are frozen values, and the discipline's objects are identity-carrying declarations. The correct move is to declare a new object (in practice with a versioned name, say `States_v2`, though the kernel imposes no naming scheme) and to relate it to the old one by explicit arrows: an inclusion `States` $\to$ `States_v2` embedding the old states, and, where meaningful, a retraction collapsing the new state onto an old one. Every existing arrow remains valid against the old object — history does not retype — and participates in the new regime only through the declared bridges, which are ordinary arrows subject to ordinary review.

The alternative regimes are both worse in characteristic ways. Silent in-place mutation retypes history: mapping tuples written against the old indices are reinterpreted against the new carrier, which is Scenario I of Chapter 1 induced deliberately. And carrier-only identity with mutation tracking ("the object is whatever the carrier currently is") makes object identity a function of time, destroying the replayability of any digest computed before the change. Name-inclusive identity with explicit versioning is the unique regime in which every artifact ever produced remains exactly reinterpretable — its objects are immutable values that still mean what they meant. The cost, once again, is explicitness; the payment, once again, is an audit trail.

2.3. Relation to the categorical picture. Nothing in Rule 3.5 conflicts with the mathematics of Chapter 2; it refines it. The kernel's category (Chapter 4 proves it is one)

simply has *more objects* than the category of bare finite carriers: one object per declared (name, carrier) pair, with hom-sets empty between objects the modeler has not connected. Categorically, names implement a discrete indexing of otherwise isomorphic objects — the standard device of working in a category where isomorphic objects are deliberately not identified, because the identification is exactly what must be controlled [2]. The mathematical content of any construction is unchanged by the extra tags; the *legality* of compositions is what the tags govern.

3. The ordered-input rule

3.1. Statement and enforcement point.

Rule 3.7 (Ordered input). Every constructor input whose order affects any downstream artifact must be supplied as a replayable ordered sequence. Sets, mappings, and one-shot iterables are rejected at construction time with `ValidationError`. The rule is enforced for exactly seven input positions:

(1) carrier elements (of a `FiniteObject`);
(2) arrow mappings (the image-index tuple of a `FiniteArrow`);
(3) path arrows (the arrow sequence of a `SemigroupoidPath`);
(4) word labels (the generator-label sequence of a typed word);
(5) catalog objects (the object sequence of a generator catalog);
(6) catalog generators (the generator sequence of a catalog, whose declared order is the shortlex ranking);
(7) lift coordinate vectors (coefficient sequences fed to the deterministic lift).

The enforcement point is a single shared validator, `_ordered_tuple`, called by every constructor listed. Its acceptance condition is deliberately conservative: the input must be a sequence in the collections sense — indexable, re-iterable, with an order that is part of its contract — and must not be one of the explicitly excluded types. Excluded are:

- *sets* (any `Set` instance): iteration order is not part of the contract and, for hash-randomized element types, varies by process (Section 4);
- *mappings* (any `Mapping` instance): even where a concrete mapping type happens to iterate deterministically, the abstract contract does not promise an order, and accepting mappings would invite callers to encode order-sensitive data in a container whose ordering guarantees vary by type, language version, and implementation;
- *one-shot iterables* (generators, iterators, streams): a consumed iterator re-iterates as empty, so validation and storage would observe different data than a second traversal; nothing about the values is stable enough to constitute a declaration;
- *strings and byte strings* (`str`, `bytes`, `bytearray`): these *are* ordered sequences, and they are rejected anyway, because a string supplied where a sequence of elements is expected almost always signals an error — it would silently explode into a carrier of single characters, an object the caller never intended. Rejecting the container forces the caller to write the character sequence explicitly if that is truly what is meant.

What remains — lists, tuples, and other genuine sequences — is converted to an immutable tuple at the constructor boundary, so that the stored order can never drift after validation.

3.2. Why exactly these seven positions. The rule is not a blanket ban on unordered containers; the kernel freely uses sets and dictionaries *internally* where order does not reach

any output — membership tests, duplicate detection, fiber grouping keyed by element. The seven positions of Rule 3.7 are exactly the places where order is *semantics*:

Carrier elements fix the standard indices, and standard indices are the coordinate system for everything: an arrow's mapping tuple is read against them (position i of the tuple is the image of carrier element i), and the Hilbert space H_X of Chapter 7 takes its orthonormal basis $(e_{x_0}, \ldots, e_{x_{n-1}})$ in carrier order, so that permuting the carrier conjugates every lift matrix by a permutation. Two enumerations of the same element set yield different objects with different geometry — correctly so, and only tolerably so if the enumeration was chosen by the caller rather than by a hash table.

Arrow mappings are the function itself: the tuple $(t_0, \ldots, t_{n-1})$ *is* the total function in the tuple representation of Section 1. An unordered or re-orderable source for this data is an unordered function — a contradiction.

Path arrows fix which composite a path denotes; composition is not commutative, and a path delivered as a set of arrows would denote as many composites as orderings.

Word labels fix both the denotation of a typed word (the composite of its arrows in sequence) and its identity as a word — the thing shortlex compares letter by letter.

Catalog objects and catalog generators fix the closure's universe and, critically, the *ranking* ρ of Definition 2.50: the declared generator order is the lexicographic tiebreak of shortlex, so the canonical representative of every denotation depends on it (Remark 2.56). A catalog fed from a set would have a per-process canonical form — the contradiction in terms already noted in Section 5.2.

Lift coordinate vectors pair coefficients with basis vectors by position; an unordered coefficient container does not determine a vector.

In every one of the seven, the downstream artifact is either an equality judgment, a digest, or a canonical form. That is the criterion. Order that reaches none of these may be, and is, left to internal convenience.

3.3. Order sensitivity made concrete: a two-element worked example. The abstract claim "carrier order changes downstream artifacts" can be made completely concrete at the smallest nontrivial size, and the exercise is worth a page because it shows the change reaching all three artifact classes — equality, matrices, and canonical forms — from a single transposition.

Let the element set be $\{u, v\}$ and consider the two enumerations: $X = (\texttt{Pair}, (u, v))$ and $X' = (\texttt{Pair}, (v, u))$. These are unequal objects (same name, different carrier order), and the intended map "send everything to u" is a different tuple over each: over X it is the arrow c with mapping $(0, 0)$ (both positions to index 0, which is u); over X' it is the arrow c' with mapping $(1, 1)$ (both positions to index 1, which is u there).

Equality: c and c' are extensionally unequal — different sources, different tuples — although both realize "constantly u" on the same element set. Any digest over denotation keys distinguishes them; a system that conflated them would be conflating the coordinate systems in which every other arrow over these objects is written.

Matrices: the lift of c over X has matrix rows $(1, 1)$ and $(0, 0)$ — column j has its 1 in row 0 because both elements map to index 0 — while the lift of c' over X' has rows $(0, 0)$ and $(1, 1)$. The two matrices are permutation-conjugate (Lemma 2.44 territory: same spectrum-relevant structure, same norm $\sqrt{2}$), but they are *different matrices*, and any artifact that serializes matrix entries — a test expectation, a digest, a Lean fixture — sees the difference. Spectral invariants survive reordering; byte-level evidence does not.

Canonical forms: put both arrows in generator catalogs. Over X, a catalog declaring generators in the order (c, id_X) ranks c as letter 0; the same semantic content delivered in the other declared order ranks it as letter 1, and every shortlex tiebreak between words of equal length can flip (Example 2.55 showed the flip at alphabet level). Carrier order and catalog order are thus two independent dials, each of which the kernel forces into explicit, versioned data.

None of this is pathology; it is coordinates behaving as coordinates. The pathology would be letting a hash table set the dials.

INVARIANT 3.8 (Replayability of constructed values). Every value constructible through the kernel's public constructors is a pure function of its constructor arguments read as ordered data. Two calls with equal ordered arguments, in any two processes on any two machines, construct equal values with equal serializations. No kernel output depends on hash seeds, iteration order of unordered containers, object identity (memory addresses), wall-clock time, or any other ambient interpreter state.

Invariant 3.8 is the chapter's central promise, and Rule 3.7 is most of its enforcement. The remaining exposure — the caller might *produce* the ordered argument nondeterministically before the kernel ever sees it — cannot be closed by a library and is addressed honestly: the kernel's constructors make the ordering decision the caller's explicit, visible act, which is the most a constructor can do. Section 4 explains why so much weight is placed here.

4. Hash-order nondeterminism as an evidence problem

This section develops, at mechanism level, the failure that Rule 3.7 exists to exclude. Although the concrete details are Python's, the problem is general, and we state it generally before and after the specifics.

4.1. The mechanism. Hash-based containers — sets and hash maps — store elements in an internal table at positions derived from the elements' hash values. Iteration walks the table, so iteration order is a function of (i) the hash values of the elements, (ii) the table's size history, which depends on insertion order and interim deletions, and (iii) implementation details of collision resolution. None of these is part of the container's semantic contract; the contract is membership.

In CPython specifically, the hash of a string is additionally *randomized per process*: since Python 3.3, string and bytes hashing incorporates a random key generated at interpreter startup, as a hardening measure against algorithmic-complexity denial-of-service attacks on hash tables (the modern implementations use keyed hashes of the SipHash family). The environment variable `PYTHONHASHSEED` controls the key: unset, the key is random per process; set to an integer, the key is that integer (fixed across processes); set to `0`, randomization is disabled entirely. Consequently:

- the set {`"calm"`, `"alert"`, `"overload"`} iterates in an order that varies from process to process under default settings;
- small integers hash to themselves, so sets of small integers typically iterate in a stable-looking order — which makes int-keyed prototypes appear deterministic and lets the defect ship, to detonate later when the element type changes to strings, tuples, or enums;
- `dict` preserves insertion order as a language guarantee in modern Python, so a mapping built by deterministic insertions does iterate deterministically — but a mapping built from set-derived data inherits the upstream nondeterminism, and the

abstract `Mapping` contract promises nothing; the kernel rejects the whole class rather than adjudicating pedigrees it cannot see.

4.2. How the nondeterminism reaches digests. The leak path is short and typical. Order-sensitive data is assembled by iterating a hash-ordered container — `tuple(states)`, `list(mapping)`, a comprehension over a set, a `join` over `dict.keys()` of a set-fed dict. The assembled sequence is then serialized — into a JSON document, a repr string, a pickle, a canonical byte encoding — and the serialization is digested with a cryptographic hash (in this project, SHA-256) whose output is recorded as evidence: in a manifest, a test log, a signed receipt, a changelog entry. Every step after the first is deterministic; the digest is nonetheless a random variable, because its input embedded an iteration order.

The scenario of Section 5.2 traced one such path end to end: a three-element set becomes a carrier, the carrier order fixes standard indices, an intended-constant endomap serializes as $(1, 1, 1)$ in one process and $(2, 2, 2)$ in another, and the recorded digest fails to reproduce under the auditor's replay. Note the exact epistemic damage: the digest does not become *weaker* evidence — it becomes *no* evidence, because a verifier that fails on honest replays cannot distinguish honest divergence from tampering or corruption. And the failure is maximally confusing to diagnose, because every individual function in the pipeline is correct in isolation; the defect lives in a container choice that type-checks, lints clean, and passes every single-process test.

Threat 3.9 (Hash-order leakage). *Asset*: replayability of every digest, canonical form, and equality judgment produced by the kernel and recorded in its manifests. *Vector*: any construction of order-sensitive data by iteration over a hash-ordered or otherwise contract-free container, anywhere upstream of a recorded artifact — including in caller code, test fixtures, and data preparation scripts. *Amplifiers*: element types whose hashes are process-seeded (strings, bytes, composites containing them); deterministic-looking prototypes over small integers; mappings of unverifiable pedigree; iterator consumption that silently yields different data to validation and storage. *Kernel mitigation*: Rule 3.7, enforced at all seven order-sensitive constructor positions, converting the vector into an immediate typed error. *Residual exposure*: nondeterministic assembly in caller code prior to the constructor boundary; addressed by making the ordering an explicit argument and by the manifest practice of digesting inputs as well as outputs, so a nondeterministic caller fails to reproduce its own inputs and is caught at the evidence layer.

4.3. A minimal reproduction experiment. The mechanism can be exhibited in two interpreter invocations, and we record the experiment in a form any reader can replay, because the failure's invisibility in source code is best dispelled by watching it happen. Run the same one-liner twice as separate processes:

```
python -c "print(tuple({'calm', 'alert', 'overload'}))"
```

Under default settings (hash randomization on), the two processes will, with high probability across a handful of repetitions, print the three strings in different orders — same program text, same input data, different output. Fixing the seed makes the accident repeatable without making it principled:

```
PYTHONHASHSEED=0 python -c "print(tuple({'calm', 'alert', 'overload'}))"
```

now prints the same order on every invocation — *some* order, chosen by the interpreter's hash function at that version, promised by nothing, and subject to change when the interpreter changes. Repeating the experiment with the integer set `{1, 2, 3}` typically prints `(1, 2, 3)` stably under any seed — the deceptive stability of small-integer hashing noted

above, and the reason prototypes over integers ship the defect. The corresponding kernel experiment is the negative fixture: constructing a `FiniteObject` from any of these set literals raises `ValidationError` under every seed, which is the only outcome of the four that is both deterministic and honest.

For contrast, the deterministic path through the kernel:

```
obj = FiniteObject(name="States", elements=("alert", "calm", "overload"))
arrow = FiniteArrow(name="to-calm", source=obj, target=obj, mapping=(1,
1, 1))
```

constructs, in every process on every machine, an object whose element `"calm"` has standard index 1 and an arrow whose serialized denotation key is byte-identical across runs — the property the digest discipline below consumes.

4.4. Digest discipline at the evidence layer. The kernel's outputs become evidence when they are serialized and digested, so the discipline extends one level beyond the constructors. Three rules govern the project's digest practice; they are stated here because they complete the determinism story, though their enforcement lives in the manifest tooling rather than in `category.py`.

Digest denotations, not presentations. Semantic digests are computed over denotation keys — source object (name and ordered carrier), target object, mapping tuple — and never over presentation names, memory addresses, or repr strings of container types whose formatting is version-dependent. Two arrows that are extensionally equal must digest identically, or the digest would distinguish what the mathematics identifies.

Serialize with a fixed, versioned encoding. The byte encoding of a denotation key (field order, integer encoding, string encoding, delimiter convention) is itself part of the evidence contract: it is fixed, documented, and versioned, so that a digest names one byte string under one encoding version. An "obvious" serialization left to a library default is a hash-order problem one level up — defaults change across library versions exactly as iteration orders change across interpreter versions.

Digest inputs alongside outputs. A manifest records digests of the inputs to a computation as well as its outputs, so that a nondeterministic producer upstream of the kernel is caught by failing to reproduce its own inputs, rather than remaining invisible behind a reproducible kernel. This is the residual-exposure mitigation named in Threat 3.9: the kernel cannot make caller code deterministic, but the evidence layer can make caller nondeterminism *visible*.

4.5. The general software-evidence problem. Nothing above is specific to Python, to this kernel, or even to hashing. The general form: *a computation is offered as evidence exactly to the extent that an independent party can replay it to the same bytes; any dependence on ambient state — hash seeds, iteration order, thread interleavings, locale, filesystem enumeration order, clock, address-space layout — silently converts a replayable computation into an unreplayable one, and the conversion is invisible at the point of use.* The reproducible-builds discipline in software distribution fights precisely this class (embedded timestamps, archive member order, parallel link order); numerical computing fights its floating-point cousin (reduction order under parallelism); and every attestation scheme that hashes an artifact inherits every nondeterminism of the artifact's producer. The principled responses available to a system designer are three: (a) *canonicalize at the boundary* — sort, normalize, and fix an order before anything order-sensitive is derived; (b) *refuse at the boundary* — reject inputs whose order is not part of their contract, and force the caller to decide; (c) *pin*

the ambient state — fix seeds and environments so that the accident is at least a repeatable accident.

The kernel chooses (b), and it is worth recording why the two alternatives were declined. Canonicalization (a) requires a canonical order on arbitrary hashable elements, which does not exist in general (elements need not be mutually comparable) and, where it does exist, encodes a semantic choice — *which* order — that belongs to the modeler, not the library; a silently sorted carrier is exactly the kind of unreviewed decision the kernel refuses to make. Pinning (c) — running everything under a fixed `PYTHONHASHSEED` — makes runs repeatable but leaves the semantics coupled to an environment variable: the "canonical" form becomes a function of a deployment setting, unreproducible by any party who does not know it, and fragile under interpreter upgrades that change hash functions. Refusal (b) is the only option under which the constructed value is a pure function of reviewable arguments (Invariant 3.8); its cost — callers must write `sorted(...)` or an explicit literal where they previously wrote a set — is paid once, visibly, at the point where the decision belongs.

Claim Boundary 3.10. The kernel's determinism guarantees are claims about the kernel's own constructors and outputs (Invariant 3.8), validated by the negative fixtures of Section 7 on the tested platforms. They are not a claim that programs *using* the kernel are deterministic, nor a general theorem about Python semantics, nor a certification of any digest produced outside the kernel's constructor discipline.

5. Arrow judgments and extensional equality

5.1. The arrow judgment.

Definition 3.11 (Arrow). An *arrow judgment* $f : X \to Y$ consists of a declared source object X, a declared target object Y, and a total map from the standard indices of X to the standard indices of Y, represented as the *mapping tuple* $(f(0), f(1), \ldots, f(|X|-1))$ with each entry in $\{0, \ldots, |Y|-1\}$.

The implementation is the frozen dataclass `FiniteArrow` with fields `name`, `source`, `target`, and `mapping`. Construction validates: the name is a non-blank string; source and target are `FiniteObject` values; the mapping passes the ordered-input rule and has length exactly $|X|$ (totality is syntactic: one image per source index, no more, no fewer); and every entry is an integer within range for the target. Two validation details are easy to miss and are load-bearing:

Booleans are rejected as indices. In Python, `bool` is a subclass of `int`, so `True` would silently function as index 1. The constructor tests for `bool` explicitly and rejects it: an index that arrived as a truth value is almost certainly a logic error upstream, and the coincidence `True` $= 1$ is exactly the kind of accidental meaning the kernel exists to refuse.

Range checking is against the declared target, not against use. An out-of-range entry fails at construction, not at first application; there is no constructible arrow whose application can index outside its target's carrier.

Application is then defined elementwise: for a carrier element x of X, the arrow returns the target carrier element at position mapping[index(x)] — element in, element out, with the index representation confined to the interior.

5.2. Element-level application and the index bridge. Application deserves a worked micro-example, because it is where the two vocabularies of the kernel — elements, which callers care about, and indices, which the mathematics runs on — meet, and because the

meeting point is where a lesser design would leak coordinates to the caller or elements into the algebra.

Take $X = (\texttt{Signal}, (\texttt{low}, \texttt{high}))$ and $Y = (\texttt{Action}, (\texttt{hold}, \texttt{raise}, \texttt{drop}))$, and the arrow $\texttt{policy} : X \to Y$ with mapping $(1, 2)$. Applying `policy` to the element `"low"` proceeds in three steps, each total and each validated: the source's `index` method resolves `"low"` to standard index 0 (raising `ValidationError` for any non-element, so application cannot be fed a stray value); the mapping tuple sends 0 to 1; the target's carrier resolves index 1 to the element `"raise"`, which is returned. Elements in, elements out; the integer 1 existed only in the interior. The caller never handles indices, so no caller can develop a dependency on them; the algebra never handles elements, so no element's incidental properties (its hash, its string form, its comparability) can influence a computation. Every structure of Chapters 4 through 8 — composition, powers, lifts, words — operates purely on the index side, with `index` and carrier lookup as the only crossings, both owned by the object that defines the coordinates.

The design also fixes where partiality is allowed to appear: *application to a non-element* is a caller error and raises; *application of a validated arrow to a carrier element* cannot fail, by the range validation at construction. There is no code path on which a constructed arrow returns an out-of-carrier value, and the test suite's positive families exercise exactly this closure property across every arrow they enumerate.

5.3. Extensional equality.

Definition 3.12 (Extensional equality of arrows). Arrows f and g are *extensionally equal*, written $f = g$, iff their sources are equal objects, their targets are equal objects, and their mapping tuples are equal. The triple (source, target, mapping) is the arrow's *denotation key*; extensional equality is equality of denotation keys.

Proposition 3.13 (Equality is a congruence). *Extensional equality is an equivalence relation on arrows, and composition respects it: if $f = f'$ and $g = g'$ and $g \circ f$ is defined, then $g' \circ f'$ is defined and $g \circ f = g' \circ f'$. Likewise application respects it: $f = f'$ implies $f(x) = f'(x)$ for every carrier element x of the common source.*

Proof. Equivalence is immediate: equality of triples of (name-inclusive) objects and tuples is reflexive, symmetric, and transitive componentwise. For the congruence: $f = f'$ gives equal targets and $g = g'$ equal sources, so definedness of $g \circ f$ — target of f equals source of g as objects — transfers to $g' \circ f'$. The composite's denotation key is determined by the operands' keys: source of f, target of g, and the tuple whose i-th entry is $\text{mapping}_g[\text{mapping}_f[i]]$; equal operand keys therefore force equal composite keys. Application reads only the source, mapping, and target, all fixed by the key. □

Corollary 3.14 (Derived operations respect equality). *Extensional equality is respected by every derived operation of the kernel: if $f = f'$ then $f^n = f'^n$ for every $n \geq 0$ (when the endomap constructions are defined, which they are for both or neither); if two paths have componentwise equal arrow sequences, their composites are equal; and equal arrows produce equal lifts, equal closure-table entries, and equal normal-form assignments in Chapters 7 and 8.*

Proof. Powers: induction on n from Proposition 3.13, with base case $f^0 = \text{id}_X = f'^0$ (equal sources give equal identity arrows). Definedness transfers because `is_endomap` is a predicate on the denotation key (source equals target as objects). Path composites: induction on path length using the congruence at each fold step. The lift and closure statements are

established in their own chapters; each construction there reads only denotation keys, which is all the present corollary requires of them. □

Proposition 3.13 and its corollary are short but foundational: they say that every construction of the kernel factors through denotation keys, so that reasoning may be conducted entirely on the extensional data. In the implementation, `denotation_key` is exposed as a property precisely so that higher layers — the deduplication table of the normal-form closure in Chapter 8 above all — can key on the mathematical identity of an arrow rather than on any incidental attribute. Which brings us to the one incidental attribute an arrow carries.

6. Presentation names are not identity

Every arrow has a `name`: a non-blank string, required at construction, and used in every diagnostic the kernel emits — composition mismatch errors print `f:X->Y` then `g:Z->W` in terms of these names, and the error messages of Chapters 5 through 8 do likewise. Names are also synthesized by the kernel itself: the identity constructor names its result `id[X]` after the object; composition names its result `(g o f)` after the operands; the power operation names f^n with a compact exponent label; the path composite is named by its length. For very large exponents the synthesized name uses the exponent's bit length plus a short SHA-256 label rather than full decimal rendering, so that name construction is logarithmic in the exponent and immune to the host language's integer-to-decimal conversion limits (Chapter 5 details this; regression tests cover exponents 2^{30} and 10^{5000}).

Rule 3.15 (Presentation names never enter equality). The `name` field of an arrow is presentation data: it participates in no equality comparison, no hash, no denotation key, no digest of semantic content, and no decision of any kernel algorithm. Two arrows with equal denotation keys are equal whatever their names; two arrows with distinct keys are unequal whatever their names. In the implementation the field is declared `compare=False`, excluding it from the generated equality and hash of the frozen dataclass.

The rule resolves a tension that Section 2 created deliberately. *Object* names are semantic — they carry modeling intent that the carrier underdetermines, and so they participate in identity. *Arrow* names are not — an arrow's full semantic content is already present in its denotation key (source object including its name, target object including its name, mapping tuple), and a name over and above that content can only describe *how the arrow was obtained*: as a composite, as a power, as somebody's labeled generator. History of construction is exactly what extensional mathematics discards — f^6 computed by binary powering and the same map entered as a literal table are the same arrow — and letting names into equality would make the kernel intensional: normal forms would distinguish denotations by their derivations, the closure's deduplication would fail to merge equal maps, and the count of "distinct" arrows would depend on the syntactic route taken to each. The two rules are therefore two sides of one principle: *identity includes exactly the data that composition legality and denotation need — all of it, and nothing else.* For objects that includes the name; for arrows it excludes it.

One consequence is worth flagging for users: since names do not participate in equality, they are also not *protected* by it — nothing stops two extensionally different arrows from bearing the same name, and diagnostics that print names can therefore be ambiguous where keys are not. The kernel's diagnostics accept this, always printing source and target names alongside, and the evidence layer never relies on arrow names for anything.

7. Negative-fixture testing of the constructors

7.1. Rejection behavior is specified behavior. A kernel whose safety story is "invalid things are unwritable" owes its users evidence that invalid things are, in fact, unwritable. Positive tests cannot supply this: a suite that only exercises valid constructions would pass unchanged against an implementation that accepted everything. The kernel's suite therefore treats *rejection* as first-class specified behavior, tested by *negative fixtures*: minimal invalid inputs, each asserted to raise the exact typed error — `ValidationError` for malformed data, `CompositionTypeError` for type mismatches — at the exact boundary, with no partial construction surviving. This is the executable content of ledger items PO-TKS8-CORE-02 (reject every source/target mismatch) and PO-TKS8-CORE-03 (keep one-carrier powers distinct from heterogeneous paths), both of status EXECUTABLE VALIDATION.

CHECK CLASS 3.16 (Negative fixtures for object construction). Each of the following must raise `ValidationError`: a blank or non-string object name; carrier elements supplied as a `set`, a `dict`, a generator expression, or a string; carrier elements containing a duplicate; carrier elements containing an unhashable value (e.g. a list). Additionally, `index` on a non-element must raise `ValidationError` naming the object.

CHECK CLASS 3.17 (Negative fixtures for arrow construction). Each of the following must raise `ValidationError`: a blank arrow name; a source or target that is not a `FiniteObject`; a mapping supplied as a set, mapping, or one-shot iterable; a mapping tuple whose length differs from $|X|$ (both too short — a partial function — and too long); a mapping entry that is a `bool`; a mapping entry that is not an integer; a mapping entry outside $\{0, \ldots, |Y| - 1\}$ (both negative and $\geq |Y|$).

CHECK CLASS 3.18 (Negative fixtures for typed operations). Each of the following must raise `CompositionTypeError`: composing $g \circ f$ where the target of f and source of g differ in name only (equal carriers); differ in carrier only (equal names); differ in carrier order only (equal names, equal element sets); constructing an `Endomap` from an arrow whose source and target are distinct objects — including the same-carrier different-name case; constructing a `SemigroupoidPath` containing one adjacent mismatch of any of the above kinds. Each of the following must raise `ValidationError`: an empty path; a path containing a non-arrow; a negative or non-integer or boolean endomap exponent.

The three same-carrier variants inside Checkclass 3.18 are the anti-coercion argument of Section 2 turned into executable assertions: they pin exactly the coercions — by name, by carrier, by order — that a weaker identity discipline would silently perform, and they fail against any implementation that relaxes Rule 3.5 in any of the three directions. Similarly, the set-and-generator fixtures of Checkclasses 3.16 and 3.17 pin Rule 3.7 at every constructor that Section 3 enumerated, and the boolean-index fixture pins the subclass subtlety of Section 5.

7.2. The validation surface in one table. Table 1 summarizes the constructor validation surface that the fixtures pin: each order-sensitive or type-sensitive input position, what it rejects, and with which typed error. The table is descriptive of the Gate 1 prototype sources; the authoritative statement is the code bound by the manifest, and the fixtures are what tie the two together.

7.3. What negative fixtures establish, and what they do not. A negative fixture establishes that one representative of an invalid class is rejected on the tested platform. It does not establish that every member of the class is rejected — that would require the

Constructor	Input	Rejected forms	Error
`FiniteObject`	`name`	blank, non-string	`ValidationError`
`FiniteObject`	`elements`	set, mapping, iterator, string	`ValidationError`
`FiniteObject`	`elements`	duplicates, unhashables	`ValidationError`
`FiniteArrow`	`name`	blank, non-string	`ValidationError`
`FiniteArrow`	`source`, `target`	non-objects	`ValidationError`
`FiniteArrow`	`mapping`	unordered containers	`ValidationError`
`FiniteArrow`	`mapping`	wrong length (totality)	`ValidationError`
`FiniteArrow`	`mapping`	bool, non-int, out of range	`ValidationError`
`compose`	middle objects	any object inequality	`CompositionTypeError`
`Endomap`	`arrow`	source $\neq$ target	`CompositionTypeError`
`Endomap.power`	exponent	negative, bool, non-int	`ValidationError`
`SemigroupoidPath`	`arrows`	empty, non-arrows	`ValidationError`
`SemigroupoidPath`	`arrows`	adjacent mismatch	`CompositionTypeError`

TABLE 1. Constructor validation surface of `category.py`. "Object inequality" includes all three coercion variants: name-only, carrier-only, and order-only differences.

exhaustive or formal treatment that positive properties receive in Chapters 10 and 11 — and the suite's design acknowledges this by choosing representatives at the *boundary* of each class (length off by exactly one; index exactly $|Y|$; the same-carrier coercion cases), where an implementation bug is most likely to live. The honest summary, maintained throughout the monograph's evidence chapters: positive behavior on the bounded families is checked exhaustively; rejection behavior is checked by boundary representatives; neither is proof-assistant verification of the implementation, and the Lean seed's verified scope (Chapter 11) covers the mathematical laws, not the Python constructors. Within those limits, the fixtures make the chapter's three rules — name-inclusive identity, ordered input, names-out-of-equality — into properties that cannot regress silently: any future edit that weakens a rule turns at least one fixture red.

8. Summary

The chapter's content in one table of commitments:

- An object is a (name, ordered carrier) pair; equality includes both (Definition 3.1, Rule 3.5); the argument is anti-coercion, with the register, refactor, and units examples (Examples 3.2–3.4) as the canonical failure patterns.
- Order-sensitive constructor inputs — seven positions, enumerated — must be replayable ordered sequences; sets, mappings, one-shot iterables, and strings are rejected at construction (Rule 3.7); the constructed values are then pure functions of their arguments (Invariant 3.8).
- Hash-order iteration is a general software-evidence threat (Threat 3.9): it converts digests from evidence into noise, and the kernel's refusal-at-the-boundary is the only response that keeps canonical forms independent of ambient interpreter state.
- An arrow is its judgment — declared source, declared target, mapping tuple; equality is extensional over exactly this data and is a congruence for composition and application (Definitions 3.11–3.12, Proposition 3.13); presentation names are diagnostics only and never enter equality (Rule 3.15).

- Every rule above is pinned by negative fixtures asserting exact typed errors at exact boundaries (Checkclasses 3.16–3.18), with the evidential status EXECUTABLE VALIDATION and the limits of that status stated.

With objects and arrows fixed and the determinism discipline in force, Chapter 4 can state and prove the category laws for the kernel exactly as Chapter 2 proved them for bare finite functions — the typing adds hypotheses, never obstacles — and Chapters 5 and 6 can erect the endomap/path separation on ground that no longer shifts.

CHAPTER 4

Typed Composition and the Category Laws

The previous chapter fixed the object layer: a named finite carrier with an explicitly ordered element sequence, whose identity is the pair of name and ordered carrier. This chapter builds the arrow layer on top of it and proves the central structural fact of the kernel: the objects and arrows form a category, with identities that are neutral and a composition that is associative wherever it is defined. The qualification "wherever it is defined" is not a formality. Composition in the kernel is a *partial* operation, deliberately so, and the precise condition under which it is defined — declared target equals declared source, as objects, meaning both name and ordered carrier — is the single most consequential design decision in the entire system. Everything downstream, from the endomap/path separation of Chapters 5 and 6 to the lift functor and the normal-form machinery, inherits its soundness from the typing discipline established here.

The chapter has four movements. First, we state the definitions of identity and composition exactly as the kernel implements them, and we spell out what the definedness condition does and does not permit (Section 1). Second, we prove the category-laws theorem in full, pointwise, with no appeal to imported category-theoretic machinery (Section 2); the proof is elementary, and its elementarity is part of the point, because an executable kernel should rest on arguments a reviewer can check by hand. Third, we state the mismatch-rejection contract — the behavioral specification of what the implementation must do when the definedness condition fails — and we catalog the negative tests that exercise it (Section 3). Fourth, we give the verification story in depth (Sections 4 and 5): what the bounded exhaustive suites actually enumerate, including the exact counting behind the 19,683 associativity triples and the 499 typed maps; what bounded exhaustive testing does and does not establish, situated against the small-scope hypothesis of Jackson [**15**]; and what the pinned Lean formal seed kernel-checks, including the axiom-audit discipline and the honest statement of the open gap between the Python implementation and the Lean model.

Throughout, claim status is stated inline. The category laws carry a complete informal proof (this chapter), bounded exhaustive executable evidence, and exact Lean lemmas for the Lean model of the definitions. None of that is proof-assistant verification of the running Python code; the equivalence of the two artifacts is an open obligation, recorded as such in the ledger of Appendix B, and we will not blur that line.

1. Identity and Composition

1.1. Arrows recalled. We briefly recall the arrow definition from Chapter **??** in the form we need here, since every proof in this chapter manipulates it directly.

Definition 4.1 (Arrow judgment). An *arrow judgment* $f : X \to Y$ consists of three data: a declared source object X, a declared target object Y, and a total map on standard indices, represented as the tuple

$$\big(f(0), f(1), \ldots, f(|X| - 1)\big),$$

where each $f(i)$ is a standard index of Y, that is, an integer with $0 \le f(i) < |Y|$. The tuple has exactly $|X|$ entries: one image for each source element, no more and no fewer. Totality is enforced at construction; there is no notion of a partially defined arrow in the kernel.

DEFINITION 4.2 (Extensional arrow equality). Two arrows are *extensionally equal* if and only if their declared sources are equal as objects, their declared targets are equal as objects, and their mapping tuples are equal as tuples. The presentation name attached to an arrow for display and debugging purposes participates in neither equality nor hashing.

In the implementation (`category.py`), extensional equality is realized by excluding the name field from the generated comparison of the frozen dataclass `FiniteArrow`; the triple of source, target, and mapping tuple is exposed as the arrow's *denotation key*. We return to the consequences of name-exclusion in Chapter 5, where presentation names for large powers are deliberately compressed; the compression is sound precisely because names are extensionally inert.

NOTATION 4.3. We write $\mathrm{Hom}(X, Y)$ for the set of arrow judgments from X to Y. Since an arrow is determined by its mapping tuple once X and Y are fixed, and each of the $|X|$ tuple positions ranges over $|Y|$ values,

$$|\mathrm{Hom}(X, Y)| = |Y|^{|X|},$$

with the convention $|Y|^0 = 1$ (the unique empty map out of an empty source) and $0^{|X|} = 0$ for $|X| \ge 1$ (no total map from a nonempty source into an empty target). These conventions are not stylistic: the bounded enumeration counts of Section 4 depend on them, and the executable suite includes the empty-carrier cases explicitly.

1.2. Identity.

DEFINITION 4.4 (Identity arrow). For each object X, the *identity arrow* $\mathrm{id}_X : X \to X$ is the arrow whose declared source and declared target are both X and whose mapping tuple is $(0, 1, \ldots, |X| - 1)$, that is, $\mathrm{id}_X(i) = i$ for every standard index i.

Note what the definition quantifies over: there is one identity per *object*, not one identity per carrier. Two objects with the same ordered carrier but different names — say $X = (\texttt{register_a}, (0, 1))$ and $Y = (\texttt{register_b}, (0, 1))$ — have distinct identity arrows id_X and id_Y, because the arrows' declared sources differ as objects. This is the arrow-level echo of name-inclusive object identity, and it is what makes the definedness condition below strict enough to be useful.

1.3. Composition and its definedness condition.

DEFINITION 4.5 (Composition). Let $f : X \to Y$ and $g : Y' \to Z$ be arrows. The composite $g \circ f$ is *defined if and only if* $Y = Y'$ *as objects* — that is, the declared target of f and the declared source of g agree in both name and ordered carrier. When defined, $g \circ f : X \to Z$ is the arrow with declared source X, declared target Z, and mapping tuple

$$(g \circ f)(i) = g\big(f(i)\big) \qquad \text{for every standard index } i \text{ of } X.$$

When $Y \neq Y'$, the composite does not exist, and the implementation raises a typed error (Section 3). No coercion of any kind is performed.

The phrase "no coercion of any kind" compresses three distinct refusals, each of which blocks a distinct failure mode. It is worth separating them, because informal operator notations typically permit at least one of the three, and each permission reintroduces a bug class the kernel exists to exclude.

RULE 4.6 (No cardinality coercion). If $|Y| = |Y'|$ but $Y \neq Y'$, the composite $g \circ f$ is undefined. Equal cardinality means only that a bijection *exists* between the carriers; it does not select one, and any silent selection (by position, by element equality, or by hash order) would smuggle an unrecorded arrow into the composite. If the caller intends a particular bijection, the caller must construct it as an explicit arrow $b : Y \to Y'$ and write $g \circ b \circ f$, at which point the bijection is part of the auditable term.

RULE 4.7 (No carrier-only coercion). If Y and Y' have equal ordered carriers but different names, the composite is undefined. Carrier equality without name equality means the two objects are extensionally alike but were declared with distinct intent — two registers that both carry $(0, 1)$, two stages of a workflow that both carry the same label set. Composing across them silently would erase the declared distinction, and the entire point of name-inclusive identity (Chapter **??**) is that declared distinctions are load-bearing. The sanctioned bridge is again explicit: a caller who asserts that the two objects should communicate constructs a named transport arrow between them and includes it in the term.

RULE 4.8 (No name-only matching). Symmetrically, if Y and Y' share a name but differ in carrier — an error state that can arise when two modules independently declare an object called `state` with different element orders — the composite is undefined. Object equality is the conjunction of name equality and carrier equality; neither conjunct alone suffices. This refusal is what prevents a renamed or reordered carrier from silently changing the meaning of every downstream index.

REMARK 4.9 (Partiality is the feature). It is tempting to regard partial composition as an inconvenience to be engineered away — for instance, by embedding all carriers into one large universal carrier and making every composite defined. The kernel refuses this on principle. A total composition over a universal carrier is exactly the untyped setting whose failure modes motivated the system: every expression composes, including the meaningless ones, and mistypings surface (if at all) as wrong answers rather than as errors. Partiality localizes the failure to the construction site of the illegal term, with the two object identities in hand, which is the earliest and cheapest place to fail.

EXAMPLE 4.10 (A composite that exists). Let $X = (\texttt{stage_in}, (a, b, c))$, $Y = (\texttt{stage_mid}, (u, v))$, and $Z = (\texttt{stage_out}, (p, q, r))$. Take $f : X \to Y$ with mapping $(0, 0, 1)$ — so $a \mapsto u$, $b \mapsto u$, $c \mapsto v$ — and $g : Y \to Z$ with mapping $(2, 0)$ — so $u \mapsto r$, $v \mapsto p$. The middle objects agree exactly, so $g \circ f : X \to Z$ is defined, and its mapping tuple is

$$\big(g(f(0)), g(f(1)), g(f(2))\big) = \big(g(0), g(0), g(1)\big) = (2, 2, 0),$$

that is, $a \mapsto r$, $b \mapsto r$, $c \mapsto p$.

EXAMPLE 4.11 (A composite that is refused). With X, Y, Z as above, let $Y'' = (\texttt{stage_mid_v2}, (u, v))$ — same ordered carrier as Y, different name — and let $g'' : Y'' \to Z$ have the same mapping tuple $(2, 0)$ as g. The expression $g'' \circ f$ is undefined: f declares target Y, g'' declares source Y'', and $Y \neq Y''$ because their names differ. The mapping tuples of g and g'' are identical; the refusal is purely a matter of declared type. This is Rule 4.7 in action, and it is a genuine negative test in the executable suite, not a hypothetical.

1.4. Implementation fidelity: the tuple formula. Definition 4.5 is phrased in terms of the abstract function application $g(f(i))$. The implementation works with mapping tuples, and it is worth writing down — and proving — the tuple-level formula it computes, both because the executable tests compare tuples and because the Lean model's composition is stated at exactly this level.

PROPOSITION 4.12 (Tuple form of composition). *Let $f : X \to Y$ have mapping tuple $(a_0, \ldots, a_{|X|-1})$ and let $g : Y \to Z$ have mapping tuple $(b_0, \ldots, b_{|Y|-1})$. Then the composite $g \circ f$ of Definition 4.5 has mapping tuple*

$$\left(b_{a_0},\, b_{a_1},\, \ldots,\, b_{a_{|X|-1}}\right),$$

that is, the tuple obtained by indexing g's tuple at each entry of f's tuple, in order. Every index operation involved is in range.

PROOF. By Definition 4.1, $f(i) = a_i$ and $g(j) = b_j$ for standard indices i of X and j of Y. Hence $(g \circ f)(i) = g(f(i)) = g(a_i) = b_{a_i}$ for each i, which is the displayed tuple read entrywise. Well-formedness of f gives $0 \le a_i < |Y|$, so b_{a_i} is a legal lookup into a tuple of length $|Y|$; well-formedness of g gives $0 \le b_{a_i} < |Z|$, so every entry of the result is a standard index of Z. □

The formula makes the computational cost visible — composition is a single pass over the source tuple, $\Theta(|X|)$ time with no search and no allocation beyond the result — and it makes the direction convention unambiguous: the *outer* arrow's tuple is indexed by the *inner* arrow's entries. Transposing the two tuples is the classic implementation bug for this operation; it happens to produce a well-formed arrow whenever $|X| = |Z|$ (so no validation catches it), and it is exactly the kind of defect the exhaustive associativity sweep of Section 4.1 would expose, since the transposed formula is not associative against the correct one. A worked instance of the formula on Example 4.10: f's tuple is $(0, 0, 1)$, g's tuple is $(2, 0)$, and indexing g's tuple at $0, 0, 1$ yields $(2, 2, 0)$, as computed there.

REMARK 4.13 (Where the definedness check sits). In the implementation, the object-equality comparison between the declared target of the inner arrow and the declared source of the outer arrow is the first statement of the composition function, executed before the tuple of Proposition 4.12 is built. The ordering matters for the error contract of Section 3: the check must not be reachable only through some later validation failure (an out-of-range index, a length mismatch) that would report the wrong cause. Because carrier sizes can coincide while objects differ, a composition across mismatched middles could produce an *apparently* well-formed tuple; only the up-front object comparison rejects it for the right reason.

2. The Category-Laws Theorem

We now prove that the definitions of Section 1 satisfy the category axioms. The statement is deliberately phrased in terms of the concrete data — declared sources, declared targets, mapping tuples — rather than by invoking the abstract definition and checking it off, because the concrete phrasing is what the executable tests and the Lean lemmas actually verify, clause by clause.

THEOREM 4.14 (Category laws). *Under Definitions 4.1, 4.4, and 4.5:*

(1) (Typing of identities) *For every object X, id_X is an arrow from X to X.*
(2) (Typing of composites) *Whenever $f : X \to Y$ and $g : Y \to Z$, the composite $g \circ f$ is defined and is an arrow from X to Z.*
(3) (Right neutrality) *For every $f : X \to Y$, the composite $f \circ \mathrm{id}_X$ is defined and $f \circ \mathrm{id}_X = f$.*
(4) (Left neutrality) *For every $f : X \to Y$, the composite $\mathrm{id}_Y \circ f$ is defined and $\mathrm{id}_Y \circ f = f$.*

(5) (Associativity) *Whenever* $f : X \to Y$, $g : Y \to Z$, *and* $h : Z \to W$, *all four composites* $g \circ f$, $h \circ g$, $(h \circ g) \circ f$, *and* $h \circ (g \circ f)$ *are defined, and*

$$(h \circ g) \circ f = h \circ (g \circ f).$$

Consequently the finite objects and arrow judgments form a category, which we denote $\mathcal{C}$ *in this chapter.*

Status: complete informal proof below; bounded exhaustive executable evidence (Section 4); exact Lean lemmas for the Lean model (Section 5); Python-to-Lean equivalence open.

Proof. We verify each clause against the definitions, pointwise on standard indices. Recall that by Definition 4.2, two arrows are equal exactly when their declared sources, declared targets, and mapping tuples agree; so each equality claim below reduces to three componentwise checks, of which the first two are bookkeeping and the third is the pointwise computation.

Clause 1. By Definition 4.4, id_X has declared source X, declared target X, and mapping tuple $(0, \ldots, |X| - 1)$. Every entry i of the tuple satisfies $0 \le i < |X|$, so the tuple is a valid total map into the target X, and the tuple length equals $|X|$, the size of the source. Hence id_X is a well-formed arrow $X \to X$.

Clause 2. Suppose $f : X \to Y$ and $g : Y \to Z$; the definedness condition of Definition 4.5 holds because the declared target of f and the declared source of g are literally the same object Y. We must check that the composite's mapping tuple is well-formed. Its length is $|X|$, one entry per standard index of X. For each such index i: $f(i)$ is a standard index of Y because f is a well-formed arrow into Y, so $0 \le f(i) < |Y|$; therefore $f(i)$ is a legal argument position for the mapping tuple of g, whose length is $|Y|$; and $g(f(i))$ is then a standard index of Z because g is a well-formed arrow into Z. Hence every entry of the composite tuple lies in $\{0, \ldots, |Z| - 1\}$, and $g \circ f$ is a well-formed arrow $X \to Z$. Note that this clause is where totality earns its keep: because f is total and lands inside Y's index range, and g is total on exactly that range, the composite is total by construction, with no possibility of an out-of-range lookup.

Clause 3. The composite $f \circ \mathrm{id}_X$ is defined because the declared target of id_X is X, which equals the declared source of f. Its declared source is that of id_X, namely X; its declared target is that of f, namely Y; both agree with the corresponding data of f. For the mapping tuple, fix any standard index i of X:

$$(f \circ \mathrm{id}_X)(i) = f\big(\mathrm{id}_X(i)\big) = f(i),$$

using $\mathrm{id}_X(i) = i$. Since i was arbitrary, the mapping tuples agree entrywise, hence as tuples. All three components of extensional equality hold, so $f \circ \mathrm{id}_X = f$.

Clause 4. The composite $\mathrm{id}_Y \circ f$ is defined because the declared target of f is Y, the declared source of id_Y. Its declared source is X and its declared target is Y, agreeing with f. Pointwise, for each standard index i of X:

$$(\mathrm{id}_Y \circ f)(i) = \mathrm{id}_Y\big(f(i)\big) = f(i),$$

using $\mathrm{id}_Y(j) = j$ at $j = f(i)$, which is legitimate because Clause 2's argument shows $f(i)$ is a standard index of Y. Hence $\mathrm{id}_Y \circ f = f$.

Clause 5. First, definedness of all four composites. $g \circ f$ is defined (target of f is Y, source of g is Y) and has type $X \to Z$ by Clause 2. $h \circ g$ is defined (target of g is Z, source of h is Z) and has type $Y \to W$. Then $(h \circ g) \circ f$ is defined because the target of f is Y and the source of $h \circ g$ is Y; and $h \circ (g \circ f)$ is defined because the target of $g \circ f$ is Z and the

source of h is Z. Both outer composites have declared source X and declared target W, so the first two components of extensional equality agree.

For the mapping tuples, fix a standard index i of X. Unfolding Definition 4.5 twice on each side:

$$\big((h \circ g) \circ f\big)(i) = (h \circ g)\big(f(i)\big) = h\Big(g\big(f(i)\big)\Big),$$
$$\big(h \circ (g \circ f)\big)(i) = h\Big((g \circ f)(i)\Big) = h\Big(g\big(f(i)\big)\Big).$$

The two right-hand sides are the same expression — three nested total lookups $h(g(f(i)))$ — so the tuples agree at every index, hence as tuples. Therefore $(h \circ g) \circ f = h \circ (g \circ f)$.

All five clauses hold, and the data $(\text{objects}, \text{Hom}, \text{id}, \circ)$ satisfy the axioms of a category in the standard sense [2]. □

Remark 4.15 (Why the proof is this short). The proof is five paragraphs of index-chasing, and a reader familiar with **Set** will recognize it as the standard argument that finite sets and total functions form a category, decorated with bookkeeping about declared names. That is exactly right, and it is worth being explicit about what the decoration buys. The mathematical content — functions compose associatively, identities are neutral — is classical and utterly standard [2]. The kernel's contribution is not the theorem but the *definedness discipline* under which the theorem is stated: composition is partial, the partiality tracks declared intent (names) and not merely extension (carriers), and the associativity clause is proved only for triples that are composable under that strict discipline. The theorem's hypotheses are the contract; the proof is routine once the contract holds. The system's engineering claim is that the contract is enforced at construction time, every time, which is a statement about the implementation, verified by the evidence of Sections 4 and 5 to the bounded extents stated there.

Corollary 4.16 (Uniqueness of identities). *For each object X, id_X is the unique arrow $e : X \to X$ such that $f \circ e = f$ for every f with source X and $e \circ g = g$ for every g with target X.*

Proof. Existence is Theorem 4.14, Clauses 3 and 4. For uniqueness, suppose $e : X \to X$ satisfies the right-neutrality property. Instantiate it at $f = \text{id}_X$: then $\text{id}_X \circ e = \text{id}_X$. But by left neutrality of id_X (Clause 4 applied to e), $\text{id}_X \circ e = e$. Hence $e = \text{id}_X$. □

Corollary 4.17 (Endomorphism monoids). *For each object X, the set $\text{Hom}(X, X)$ of endo-arrows on X is a monoid under composition, with identity element id_X, of cardinality $|X|^{|X|}$.*

Proof. Composition of two endo-arrows on X is always defined (the middle object is X on both sides) and yields an endo-arrow on X (Theorem 4.14, Clause 2), so $\text{Hom}(X, X)$ is closed under $\circ$. Associativity is Clause 5 restricted to endo-arrows; neutrality of id_X is Clauses 3 and 4. The cardinality is Notation 4.3 with $Y = X$. □

Corollary 4.17 is the doorway to Chapter 5: the monoid structure on $\text{Hom}(X, X)$ is what makes powers of a single endomap meaningful, and its *absence* for arrows whose source and target differ is what makes powers of heterogeneous data meaningless. The kernel promotes this mathematical asymmetry into a type asymmetry, and Chapters 5 and 6 develop the two sides.

Remark 4.18 (What the category is not). $\mathcal{C}$ is not **Set**, and not even the category of finite sets, in one substantive respect: its objects are *named* carriers, so the same finite carrier

appears as the underlying data of many non-isomorphic-by-fiat objects — or more precisely, many distinct objects that happen to be isomorphic via the evident positional bijection but are not *equal*, and between which no arrow exists until one is explicitly constructed. The skeleton of $\mathcal{C}$ is equivalent to the category of finite sets in the usual way, and nothing in this monograph depends on any exotic property; the naming layer is an engineering surface, not new mathematics. We flag this to preempt a possible over-reading: no claim is made that name-inclusive identity changes the categorical content. It changes which composites are *writable*, which is a statement about the term language, not about the equivalence class of the category.

3. The Mismatch-Rejection Contract

Theorem 4.14 governs the composites that exist. An executable kernel must be equally precise about the composites that do not, because in practice the value of a type discipline is measured at its boundary: what happens when a caller writes the illegal term. This section states the contract and catalogs the negative tests.

3.1. The contract.

Invariant 4.19 (Mismatch rejection). For all arrows f and g, the implementation of $g \circ f$ either

(1) returns the arrow of Definition 4.5, in the case that the declared target of f equals the declared source of g as objects (name and carrier both); or
(2) raises the dedicated typed error `CompositionTypeError`, carrying a message that names both offending arrows and their declared types, in every other case.

There is no third behavior: no silent coercion, no warning-and-proceed, no best-effort composite over a common sub-carrier, no reordering of either carrier to force a match.

Three properties of the contract deserve comment.

The error is typed. `CompositionTypeError` is a distinct exception class (a subclass of the host language's `TypeError`), not a generic failure. Callers and test harnesses can therefore assert the *reason* for a failure, not merely its occurrence. A negative test that catches a bare exception would pass even if the implementation failed for the wrong reason — an off-by-one in tuple validation, say — and such a test would be evidence of nothing. Every negative fixture in the suite asserts the specific exception class.

The error is early. The check runs before any part of the composite tuple is constructed. There is no partially built arrow to leak, and the failure site is the construction expression itself, with both declared types in the error message. In a system whose purpose is auditability, the distance between the mistake and its detection is a first-class quality metric, and here that distance is zero.

The error is symmetric in the three refusals. Rules 4.6, 4.7, and 4.8 all funnel into the same check — object equality of the middle objects — because object equality is the conjunction of name and carrier equality. The implementation does not special-case "close" mismatches; a near-miss (equal carriers, different names) and a gross mismatch (disjoint carriers of different sizes) are rejected by the same comparison at the same site. This uniformity is deliberate: any special-casing would be a coercion pathway waiting to be widened.

3.2. The negative-test catalog. The executable suite exercises the contract with a family of negative fixtures, each targeting one way the definedness condition can fail. We catalog them as checkclasses; Chapter **??** gives the mapping to concrete test names.

Check Class 4.20 (Carrier-equal, name-distinct middle). Arrows $f : X \to Y$ and $g : Y'' \to Z$ with Y and Y'' carrying equal ordered carriers under different names, as in Example 4.11. Expected: `CompositionTypeError`. This is the sharpest fixture in the family, because every coercion-friendly design passes it wrongly: any implementation that compares carriers, cardinalities, or element sets — anything short of full object identity — will accept the composite. The fixture therefore discriminates the kernel's discipline from its most plausible near-neighbors.

Check Class 4.21 (Cardinality-equal, carrier-distinct middle). Middle objects of equal size with different element sequences (different elements, or the same elements in a different order). Expected: `CompositionTypeError`. The reordered-carrier subcase is the determinism guard: two declarations of "the same" set in different orders are different objects, and composing across them would silently permute every downstream index.

Check Class 4.22 (Gross type mismatch). Middle objects differing in both name and size. Expected: `CompositionTypeError`. This is the base case, included so that the family covers the full lattice of agreement patterns between name, carrier, and cardinality.

Check Class 4.23 (Malformed arrow construction). Not a composition test but a prerequisite for the others: attempts to construct arrows whose mapping tuple has the wrong length, contains an out-of-range index, contains a non-integer (including the host language's boolean values, which the implementation rejects explicitly even though the host treats them as integers), or is supplied as an unordered container. Expected: the construction-time `ValidationError`. These fixtures ensure that every arrow reaching a composition site is well-formed, which is the standing hypothesis of Theorem 4.14.

Check Class 4.24 (Path-adjacency mismatch). The path constructor of Chapter 6 performs the same middle-object check between each adjacent pair of a sequence of arrows; fixtures supply sequences with one ill-typed adjacency and assert `CompositionTypeError` naming the offending segment. We list it here because it is the same contract, enforced at a second surface.

3.3. The agreement lattice. The negative fixtures are not an ad hoc list; they cover a small, complete lattice of ways the middle objects can agree or disagree, and completeness of the coverage is checkable by inspection of that lattice. The middle objects Y (declared target of the inner arrow) and Y' (declared source of the outer arrow) can agree or disagree in name and in ordered carrier; carrier agreement further refines by whether the carriers at least have equal cardinality. Since carrier equality implies cardinality equality, the meaningful combinations are five:

Names	Carriers	Cardinalities	Composite	Exercised by
equal	equal	equal	*defined*	positive suites (Sections 4.1, 4.2)
distinct	equal	equal	refused	Checkclass 4.20
equal	distinct	equal	refused	Checkclass 4.21
distinct	distinct	equal	refused	Checkclass 4.21
any	distinct	distinct	refused	Checkclass 4.22

Only the top row composes; the implementation reaches all five rows through the single object-equality comparison, and the fixture families place at least one test in each refused row. The third row — equal names, distinct carriers — deserves the extra emphasis given to it by Rule 4.8: it models the situation where two modules independently declare an object under the same name with different element orders, and it is the row where a name-keyed

registry design (looking objects up by name and trusting the hit) would silently compose across genuinely different types. The kernel has no registry; objects are values, and equality is structural over the full pair.

Claim Boundary 4.25. The negative-test catalog establishes that the implementation rejects the enumerated fixture families. It does not establish that the implementation rejects *every* ill-typed composite expressible in the host language: that universal statement is a property of the code paths, argued informally by inspection (there is exactly one composition function, and its first action is the middle-object equality check) and not verified by proof assistant. The distinction matters and is recorded in the ledger (PO-TKS8-CORE-02, status executable validation).

4. The Verification Story I: Bounded Exhaustive Testing

The kernel's executable evidence for Theorem 4.14 consists of bounded exhaustive enumeration: not sampled property tests but complete sweeps of every instance within a stated finite scope. This section explains exactly what is enumerated, reproduces the counting arithmetic behind the recorded case counts, and then confronts the epistemological question of what such testing establishes.

4.1. The associativity sweep: counting the 19,683 triples. Fix one three-element object X_3 — a single named carrier of size three. By Corollary 4.17, the endo-arrows on X_3 form the full transformation monoid on three points, of cardinality

$$|\mathrm{Hom}(X_3, X_3)| = 3^3 = 27 :$$

each of the three source indices independently maps to one of three target indices. Because every arrow in this family has source and target X_3, *every ordered triple* (f, g, h) of them is composable in both associations — the definedness condition holds automatically at both middle objects — and so the associativity clause can be tested on all triples without any composability filtering. The number of ordered triples is

$$27^3 = \left(3^3\right)^3 = 3^9 = 19{,}683.$$

The suite constructs, for each of the 19,683 triples, both $(h \circ g) \circ f$ and $h \circ (g \circ f)$, and asserts their extensional equality — source, target, and full mapping tuple. This is the recorded figure of 19,683 associativity triples: it is the complete associativity table of the transformation monoid T_3, checked in extension.

Two remarks on the choice of scope. First, why endo-arrows on one carrier: because that is the unique way to make every triple composable, which turns the sweep into a clean product count with no reachability analysis; the typed (multi-carrier) sweep below covers the heterogeneous cases. Second, why size three: size two gives only $4^3 = 64$ triples, which is thin, and size four gives $256^3 = 16{,}777{,}216$ triples, which is affordable but was not the recorded snapshot's scope; the recorded suite pinned 19,683. The count is exact and the enumeration is complete within its scope; the scope is what it is, and Section 4.4 addresses what that means.

4.2. The typed sweep: counting the 499 maps. The second family exercises the typed discipline across *distinct* carriers: identity neutrality, typed composition, and mismatch rejection for maps whose source and target sizes range through four, including the empty carrier. The recorded count is 499 typed maps, and the arithmetic behind it is the

complete enumeration of Notation 4.3 over one representative source-and-target pair per size combination:

$$\sum_{m=0}^{4}\sum_{n=0}^{4} n^m = 499,$$

with the conventions $n^0 = 1$ (including $0^0 = 1$: the empty map from the empty object to the empty object) and $0^m = 0$ for $m \geq 1$. The row-by-row expansion makes the conventions visible:

Source size m	n=0	n=1	n=2	n=3	n=4	Row total
$m = 0$	1	1	1	1	1	5
$m = 1$	0	1	2	3	4	10
$m = 2$	0	1	4	9	16	30
$m = 3$	0	1	8	27	64	100
$m = 4$	0	1	16	81	256	354
Total						499

The presence of the $m = 0$ row and the $n = 0$ column is not pedantry. Empty carriers are where off-by-one and vacuous-quantification bugs live: the unique arrow out of the empty object composes with everything typed-compatibly, its mapping tuple is empty, and an implementation that mishandles zero-length tuples would fail exactly here and nowhere else. The suite's inclusion of the degenerate sizes is part of its value, and the lift functor's own sweep (Chapter **??**) makes the same choice for the same reason.

For every enumerated map the suite checks the identity laws ($f \circ \mathrm{id} = f$ and $\mathrm{id} \circ f = f$, with the appropriate identities), and for composable pairs among the enumerated maps it checks the typed composition of Definition 4.5; the negative fixtures of Section 3 run against representatives of the mismatched combinations.

4.3. Design alternative: sampled property testing. A reviewer familiar with property-based testing frameworks will ask why the suite enumerates rather than samples: the standard tooling generates random arrows, checks the law, shrinks counterexamples, and scales to carriers of size fifty as easily as size four. The kernel's choice of bounded exhaustion over sampling is deliberate, and the reasoning is worth recording because it recurs in every later chapter's evidence design.

First, coverage claims compose better than confidence claims. "Every triple through the stated bound passes" is a statement another artifact can cite and build on — the Lean equivalence obligation, for instance, could discharge part of its burden by replaying exactly the enumerated family against a verified oracle. "No counterexample found in 10^5 random triples" supports no such composition; its evidentiary content depends on the generator's distribution, which is itself unverified code, and distributional subtleties in random generation of structured data are a classic source of vacuous test suites (generators that never produce empty carriers, or never produce equal-carrier-distinct-name pairs, silently test nothing about those cases).

Second, the failure modes plausible for this code are small-instance failure modes. The composition path branches on nothing but tuple lengths and index values; a defect that manifests only above size four would need to be size-dependent in a way the code has no means to express. This is the small-scope hypothesis [**15**] applied with knowledge of the implementation, which is the strongest form in which it can be applied.

Third, determinism of the evidence artifact. The exhaustive suite is a pure function of the code under test: two runs, on two machines, in two decades, enumerate the identical case list in the identical order. A sampled suite is a function of the code and a seed, and its transcript is evidence only relative to that seed's provenance. In a system whose entire object-layer design exists to eliminate interpreter-state nondeterminism (Chapter **??**), reintroducing randomness in the verification layer would be a discordant note.

None of this is an argument against sampling in general — for spaces that explode combinatorially, later chapters accept different trade-offs — but for the category laws the exhaustive design dominates: the space is small, the enumeration is cheap, and the resulting claim is exact.

4.4. What bounded exhaustive testing establishes. We now state, carefully, the epistemic status of the numbers just derived.

Bounded exhaustive testing occupies a specific point between random testing and proof. Unlike sampled property-based testing, it leaves no gaps *within its scope*: every instance up to the bound is checked, so any counterexample of size at most the bound is found with certainty, not with probability. Unlike proof, it says nothing beyond the bound: a defect that first manifests on carriers of size five, or on a thirty-first arrow, or on a triple whose middle carrier is empty in a configuration the sweep did not include, is invisible to it.

The methodological warrant for taking small scopes seriously is the *small-scope hypothesis*, articulated by Jackson in the context of the Alloy analyzer [**15**]: a large fraction of the defects in structural specifications can be exhibited by counterexamples of small size, so exhaustive checking within a small bound catches a disproportionate share of real bugs. For the properties at issue here the hypothesis is unusually plausible, because the properties are *uniform in the carrier*: the associativity computation $h(g(f(i)))$ does not branch on carrier size, and an implementation error in index composition — composing tuples in the wrong order, off-by-one in an index bound, mishandling the empty tuple — would almost certainly manifest already at sizes zero through four. The sweeps are designed around exactly this: they saturate the small sizes completely rather than sampling large ones sparsely.

But plausibility is not proof, and the hypothesis is a hypothesis. The honest statement is the following, and the monograph will repeat it wherever the numbers are cited:

Claim Boundary 4.26. The 19,683-triple and 499-map sweeps establish that the implementation satisfies the category laws and the typed-composition contract *on every instance within their stated bounds*, and nothing more. They are not proof-assistant verification of the implementation; they are not evidence about carriers of size five or beyond except via the small-scope heuristic, which is an empirical regularity about bug distributions [**15**], not a theorem; and passing them closes no open obligation in the ledger. The mathematical theorem (Theorem 4.14) is proved for all sizes by the pointwise argument; the *testing* speaks only to whether the code implements the definitions, and only within scope.

It is worth dwelling on the division of labor in that boundary statement, because it is easy to garble in summary. There are three artifacts: the mathematical proof, the implementation, and the test suite. The proof is complete and covers all finite sizes, but it is about the *definitions*, not the code. The tests connect the code to the definitions, but only within bounds. The missing third leg — a machine- checked connection between code and definitions at all sizes — is exactly what the Lean seed begins and does not finish, and it is to the seed that we now turn.

5. The Verification Story II: The Lean Formal Seed

5.1. What kernel-checking means. The kernel's formal artifact is a pinned Lean 4.32.1 development [16] containing, among other modules, `TypedCategory`: a Lean model of the definitions of Section 1 together with machine-checked proofs of the identity and associativity lemmas for that model.

"Machine-checked" has a precise meaning that is stronger than "a tool accepted the file," and the distinction is the entire value proposition. Lean is architected around a small trusted kernel: the elaborator, the tactic framework, and the standard automation are large and unverified, but every proof they produce is re-checked, term by term, by the kernel, which implements only the type theory's inference rules. A theorem "kernel-checks" when the fully elaborated proof term passes this final validation. Trust in a kernel-checked theorem therefore reduces to trust in (i) the kernel implementation, a deliberately small and heavily scrutinized artifact, (ii) the consistency of the axioms the proof depends on, and (iii) the fidelity of the *statement* — whether the formalized proposition says what the informal one says. Points (i) and (ii) are standard and shared with every Lean development; point (iii) is where all the residual risk concentrates, and Section 5.3 takes it up.

5.2. The axiom audit. Lean proofs can depend on axioms: the standard classical trio (`propext`, `Classical.choice`, `Quot.sound`), or worse, project-declared axioms, which are simply unproved assumptions, and `sorry`, which is an explicit hole. A development that quietly depends on a project axiom proves nothing about anything; a development audit must therefore inspect the transitive axiom dependencies of every theorem it advertises.

The verifier script for the formal seed (`scripts/verify_psr8_lean_gate1.py`) mechanizes this audit. It pins the toolchain (Lean 4.32.1, exact commit), checks source hashes so the audited files are the shipped files, builds the development through Lake, and then extracts the exact axiom-dependency report of each exported theorem. The policy it enforces is maximally strict for this development: the *sole* permitted axiom is `propext` — propositional extensionality, the principle that logically equivalent propositions are equal, one of Lean's standard axioms and among the least controversial — and the verifier *rejects* any occurrence of a project-declared axiom and any occurrence of `sorry`. The `TypedCategory` identity and associativity lemmas pass under this policy: their proofs are constructive index-chasing of exactly the shape of the proof of Theorem 4.14, and need nothing beyond `propext` (used, where at all, in the ambient library plumbing rather than the combinatorial core).

Adverse Finding 4.27. The Lean model of Definitions 4.1–4.5 satisfies the identity and associativity laws, as kernel-checked theorems depending on no axiom beyond `propext`, with project axioms and `sorry` mechanically excluded by the audit script, under the pinned Lean 4.32.1 toolchain. Status: verified within the stated scope; this finding is about the Lean model, not about the Python implementation.

5.3. The open gap: Python-to-Lean equivalence. The final clause of Finding 4.27 is the load-bearing caveat of this chapter, and we state it as its own obligation rather than a footnote.

The Lean development formalizes a *model* of the kernel's definitions: Lean types for named finite objects and total index maps, a Lean composition operation with the same definedness condition, Lean statements of the laws. The Python implementation realizes the *same definitions by intent* — the informal specification and this monograph's prose are the bridge — but no formal artifact connects the two. Nothing machine-checked establishes that `compose` in `category.py` and the composition of the Lean model compute the same

function of their arguments, or that the validation predicates coincide, or even that the two data representations are in the evident bijection. The equivalence claim currently rests on side-by-side inspection of two short pieces of code by human readers.

This is a familiar situation in verification practice — it is the standard "verified model versus running artifact" gap — and there are standard ways to close it, each with costs: re-implement the kernel in Lean and extract or transpile executable code from the proved artifact; or state the Python semantics formally and prove refinement; or generate exhaustive conformance vectors from the Lean model and replay them against Python, which would still be testing, but testing against a verified oracle. None of these has been done. The ledger records the gap as open (PO-TKS8-CORE-01 carries PARTIAL FORMAL VERIFICATION PRESENT precisely because the Lean side alone is verified; PO-TKS8-CORE-07 records the formalization seed as partial and present), and no artifact of this monograph should be summarized as "the kernel is formally verified." The kernel's category layer has: a complete informal proof about its definitions; bounded exhaustive tests of its implementation; and a kernel-checked formalization of its model under a one-axiom policy. Those are three genuinely strong legs. The fourth — proved equivalence of implementation and model — is open, and stating so precisely is a result of this chapter, not a deficiency in it.

6. Exercises

The following exercises are checkable entirely within the material of this chapter; the first two are computations, the rest are short proofs or design analyses. They exist because the chapter's claims are of the kind best internalized by attempting to break them.

EXERCISE 4.28. Let X carry four elements and let $f, g : X \to X$ have mapping tuples $(1, 2, 3, 0)$ and $(0, 0, 2, 2)$ respectively. Compute the tuples of $g \circ f$ and $f \circ g$ via Proposition 4.12, and confirm they differ — endomorphism monoids are not commutative in general, which is why Corollary 5.6 (powers of a *single* arrow commute) has content.

EXERCISE 4.29. Redo the counting of Section 4.2 with the bound raised from four to five and verify the total is $\sum_{m,n \le 5} n^m = 4{,}425$. (This is the size the typed sweep would have at scope five; the recorded suite pins scope four.)

EXERCISE 4.30. Define the *transposed* composition of tuples by indexing the inner tuple at the outer tuple's entries (the bug of Section 1.4). Exhibit three endo-arrows on a two-element carrier for which the transposed operation fails associativity against extensional equality, confirming that the 19,683-triple sweep would catch the bug at size three or below.

EXERCISE 4.31. Show that Corollary 4.16 still holds if the right-neutrality hypothesis is weakened to hold for the single arrow $f = \mathrm{id}_X$. Conclude that the identity is determined by its behavior against itself and one side of neutrality.

EXERCISE 4.32. Suppose a hypothetical variant of the kernel added carrier-only coercion (Rule 4.7 dropped): composition is accepted whenever the middle *carriers* agree, names ignored. Show first that the variant still satisfies all five clauses of Theorem 4.14 — tuple composition is associative and unital regardless of names — so no law-level test, exhaustive or not, can distinguish the variant from the kernel. Then explain why the variant nevertheless fails Checkclass 4.20, and conclude that the negative fixtures, not the law sweeps, are the artifacts that pin the kernel's typing discipline. (This is the precise sense in which the mismatch-rejection contract is an independent specification rather than a consequence of the category laws.)

7. Summary of Claims and Their Status

Claim	Status
Category laws for the definitions (Theorem 4.14)	Complete informal pointwise proof, this chapter; standard mathematics [2].
Category laws in the implementation	Bounded exhaustive evidence: 19,683 associativity triples (27^3 over the 3^3 endo-arrows on a three-carrier), 499 typed maps ($\sum_{m,n\leq 4} n^m$, empty carriers included); not verification [15].
Mismatch rejection (Invariant 4.19)	Executable validation: typed error class, negative fixture families of Section 3; universal rejection argued by code inspection only.
Category laws in the Lean model	Kernel-checked under pinned Lean 4.32.1, `propext`-only axiom audit, `sorry` rejected [16].
Python-to-Lean equivalence	Open. No machine-checked connection between implementation and model; recorded in the ledger.

The next two chapters put the category to work. Chapter 5 develops the one place where iteration is licensed — the endomorphism monoid of Corollary 4.17 — through to the finite idempotence theorem and its verification. Chapter 6 develops the heterogeneous side — semigroupoid paths — and analyzes, in three extended case studies, the invalid inference that the separation between the two is engineered to make unwritable.

CHAPTER 5

Endomaps, Powers, and Finite Idempotence

Chapter 4 established that the kernel's objects and arrows form a category, and observed (Corollary 4.17) that for each object X the endo-arrows $\mathrm{Hom}(X, X)$ form a monoid under composition. That monoid is the subject of this chapter, and it is the only place in the entire kernel where the expression f^n — "apply f, n times" — is granted a meaning. The grant is deliberate and narrow: powers are defined for a single arrow whose declared source and declared target are the same object, and for nothing else. Heterogeneous sequences of arrows, however composable, receive a different construction (the semigroupoid path of Chapter 6) with deliberately no power operation, and the type system keeps the two apart.

Within the licensed territory, the mathematics is classical and rich. A map from a finite set to itself, iterated, exhibits a completely understood long-run structure: every orbit runs down a finite tail and then cycles forever, the powers of the map form an eventually periodic sequence, and the monogenic subsemigroup generated by the map contains a unique idempotent. The chapter develops this structure theory in full — functional graphs, orbit tails and cycles, the index and period of the power sequence, the cyclic-group kernel of the monogenic subsemigroup — because it is the correct setting in which to state and prove the result the kernel actually ships: for every endomap f on a carrier of size N, the power $f^{N!}$ is idempotent,

$$f^{N!} \circ f^{N!} = f^{N!}.$$

We prove this completely (Theorem 5.23), and then — because intellectual honesty about bounds is part of the claim discipline — we present the sharper statements that the structure theory makes available: the exact characterization of which powers are idempotent, the minimal idempotent power in terms of index and period, and the smaller uniform exponent $\mathrm{lcm}(1, \ldots, N)$ that also works. All of this is standard mathematics, long predating this project [**13**, **14**], and the chapter presents it as such; the kernel's contribution is the disciplined implementation, and the kernel implements and tests exactly the factorial bound.

The chapter closes with the computational layer: exponentiation by squaring, whose correctness we prove against the monoid power laws, and the bounded presentation-name construction that keeps the *names* of astronomically large powers small — bit length plus a short SHA-256 label — so that computing $f^{10^{5000}}$ neither materializes a five-thousand- digit decimal string nor trips the host language's integer-to-string conversion limit. A short section explains why none of this naming machinery can affect correctness: names are extensionally inert by construction, excluded from arrow equality and hashing, and the regression suite pins the behavior at 2^{30} and 10^{5000}.

1. The Endomap Construction

1.1. Definition and the certification reading.

Definition 5.1 (Endomap). An *endomap* is an arrow $f : X \to X$ whose declared source and declared target are the same object — equal in name and in ordered carrier, per the

object identity of Chapter **??**. The kernel represents the property by a distinct wrapper type, Endomap(f), whose constructor *verifies* f's source-target equality and refuses construction otherwise, raising the same typed error class used for composition mismatches. The wrapped arrow's underlying object is called the *carrier* of the endomap.

The wrapper deserves a word, because it is easy to misread as bureaucracy. Endomap(f) adds no data to f: it carries exactly the arrow it was given. What it adds is a *certificate*, checked once at construction, that the arrow inhabits an endomorphism monoid — and therefore that the operations defined only on such arrows (the powers of Section 2) are meaningful for it. Code that receives an Endomap value may take powers without re-checking anything; code that holds a bare arrow may not, and the host language's type distinction enforces the may-not. In the terminology of typed API design, the wrapper is a *parse-don't-validate* boundary: the fact "this arrow is endo" is established at one site and thereafter carried by the type rather than re-derived or, worse, assumed.

Two refusals of the constructor are worth making explicit, because both correspond to real confusions that arise in informal operator notation.

RULE 5.2 (Same carrier is not enough). An arrow between two distinct objects with equal ordered carriers — say $f : X \to X''$ where X and X'' both carry $(0, 1)$ under different names — is *not* an endomap, and Endomap(f) is refused. This is the arrow-level consequence of name-inclusive object identity: the kernel has no basis for asserting that a trip from `register_a` to `register_b` can be repeated, because the second application would need an arrow *out of* `register_b`, and f is not one. If the caller means the two registers to be identified, the identification must be an explicit arrow, and the composite with it will be an endomap on whichever object the caller chooses.

RULE 5.3 (Same cardinality is nothing at all). A fortiori, an arrow between distinct objects of merely equal size is not an endomap, and no positional re-indexing is performed to make it one. The untyped habit this blocks is pervasive in matrix-flavored notation, where any square matrix can be squared; in the kernel, squareness of the index range does not confer iterability — only sameness of the object does.

1.2. The endomorphism monoid, concretely. By Corollary 4.17, the endo-arrows on a fixed object X with $|X| = N$ form a monoid of cardinality N^N: the full transformation monoid on N points, in its standard-index presentation. It contains the symmetric group (the $N!$ arrows whose mapping tuples are permutations of $(0, \ldots, N-1)$), the constant maps (the N arrows whose tuples are constant), the identity id_X, and everything between. Composition of endomaps is always defined — the middle object is X on both sides — which is precisely why every ordered triple participated in the associativity sweep of Section 4.1.

For the record, and because the number recurs in the verification story (Section 6), the count of *all* endomaps on carriers of size at most four, one representative object per size, is

$$\sum_{k=0}^{4} k^k = 1 + 1 + 4 + 27 + 256 = 289,$$

with the convention $0^0 = 1$: the empty object carries exactly one endomap, the empty identity. These 289 endomaps are the complete population of the bounded exhaustive idempotence test.

2. Monoid Power Laws

Definition 5.4 (Powers). Let $f : X \to X$ be an endomap. Define $f^0 = \mathrm{id}_X$ and, for $n \geq 0$, $f^{n+1} = f \circ f^n$. Every power is again an endomap on X, since composition of endo-arrows on X stays in $\mathrm{Hom}(X, X)$.

The definition is the usual recursive one; the familiar exponent laws follow from associativity and neutrality, and we prove them because the correctness of the fast exponentiation algorithm (Section 8) reduces to them.

Proposition 5.5 (Additivity of exponents). *For all $m, n \geq 0$: $f^{m+n} = f^m \circ f^n$.*

Proof. Induction on m, with n fixed. Base case $m = 0$: $f^{0+n} = f^n$ and $f^0 \circ f^n = \mathrm{id}_X \circ f^n = f^n$ by left neutrality (Theorem 4.14, Clause 4). Inductive step: assume $f^{m+n} = f^m \circ f^n$. Then

$$f^{(m+1)+n} = f^{(m+n)+1} = f \circ f^{m+n} = f \circ (f^m \circ f^n) = (f \circ f^m) \circ f^n = f^{m+1} \circ f^n,$$

using the definition, the inductive hypothesis, and associativity (Clause 5). □

Corollary 5.6 (Powers commute with each other). *For all $m, n \geq 0$: $f^m \circ f^n = f^n \circ f^m$.*

Proof. Both sides equal f^{m+n} by Proposition 5.5 and the commutativity of addition on natural numbers. □

Proposition 5.7 (Multiplicativity of exponents). *For all $m, n \geq 0$: $(f^m)^n = f^{mn}$.*

Proof. Induction on n. Base case: $(f^m)^0 = \mathrm{id}_X = f^0 = f^{m \cdot 0}$. Inductive step: assuming $(f^m)^n = f^{mn}$,

$$(f^m)^{n+1} = f^m \circ (f^m)^n = f^m \circ f^{mn} = f^{m+mn} = f^{m(n+1)},$$

using Proposition 5.5 in the third step. □

Remark 5.8 (The monogenic submonoid). Propositions 5.5 and 5.7 say that the map $n \mapsto f^n$ is a monoid homomorphism from $(\mathbb{N}, +, 0)$ to $(\mathrm{Hom}(X, X), \circ, \mathrm{id}_X)$. Its image $\{\mathrm{id}_X, f, f^2, f^3, \dots\}$ is the *monogenic* (single- generator) submonoid generated by f; discarding the identity gives the monogenic subsemigroup $\langle f \rangle = \{f, f^2, f^3, \dots\}$ in the sense standard in semigroup theory [14]. Because $\mathrm{Hom}(X, X)$ is finite, this image is finite, so the sequence of powers must repeat — and the precise pattern of repetition is the structure theory of the next section.

3. Functional-Graph Structure Theory

Everything provable about iteration of a single finite endomap flows from one picture: the *functional graph*. This section develops the picture and its consequences with full proofs. The material is classical — it is the elementary structure theory of finite transformation semigroups and of monogenic semigroups, for which we cite Howie [14] and Pin [13] as standard references — and we present it as background, with the kernel-relevant statements extracted as numbered results.

3.1. The functional graph and its shape.

Definition 5.9 (Functional graph). Let $f : X \to X$ be an endomap with $|X| = N$, acting on standard indices. The *functional graph* of f is the directed graph G_f on vertex set $\{0, \dots, N-1\}$ with an edge $x \to f(x)$ for every vertex x. Every vertex has out-degree exactly one; in-degree is unconstrained (it is the fiber size $|f^{-1}(x)|$).

DEFINITION 5.10 (Orbit, tail, cycle). The *orbit* of x under f is the sequence $x, f(x), f^2(x), f^3(x), \ldots$. A vertex x is *cyclic* (or periodic) if $f^k(x) = x$ for some $k \geq 1$; the set of cyclic vertices is denoted $\mathrm{Cyc}(f)$. The *tail length* of x, written $t(x)$, is the least $j \geq 0$ such that $f^j(x)$ is cyclic (we show in Lemma 5.12 that such j exists). For a cyclic vertex c, its *cycle length* is the least $k \geq 1$ with $f^k(c) = c$.

LEMMA 5.11 (Every orbit repeats within N steps). *For every x, there exist $0 \leq a < b \leq N$ with $f^a(x) = f^b(x)$.*

PROOF. The $N + 1$ values $f^0(x), f^1(x), \ldots, f^N(x)$ all lie in the N-element vertex set, so by the pigeonhole principle two of them coincide; take $a < b$ to be the positions of a coincidence. □

LEMMA 5.12 (Every orbit enters a cycle; tail and cycle bounds). *For every x, the tail length $t(x)$ is well defined, and if $\ell(x)$ denotes the cycle length of the cyclic vertex $f^{t(x)}(x)$, then*

$$t(x) + \ell(x) \leq N, \qquad \textit{hence} \qquad t(x) \leq N - 1 \quad \textit{and} \quad \ell(x) \leq N.$$

PROOF. Take $a < b$ as in Lemma 5.11 with $b - a$ minimal among coincidences with minimal a; write $y = f^a(x)$ and $k = b - a \geq 1$. Then $f^k(y) = f^b(x) = f^a(x) = y$, so y is cyclic and $t(x) \leq a$ is well defined. Moreover the vertices $f^0(x), f^1(x), \ldots, f^{a+k-1}(x)$ are pairwise distinct: a coincidence among them would contradict either the minimality of a (if it involved an index below a) or the minimality of k (if both indices were at least a; two equal values on the cycle at distance less than k would shorten the cycle). Hence $a + k \leq N$. Since $t(x) \leq a$ and the cycle containing $f^{t(x)}(x)$ is the same cycle of length $\ell(x) = k$ (once the orbit meets a cyclic vertex it stays on that vertex's cycle forever, as f maps each cycle to itself), we get $t(x) + \ell(x) \leq a + k \leq N$. The two displayed consequences follow from $\ell(x) \geq 1$ and $t(x) \geq 0$. □

PROPOSITION 5.13 (Shape of the functional graph). *Each weakly connected component of G_f contains exactly one cycle, and the component consists of that cycle together with disjoint trees rooted on the cycle's vertices, all tree edges oriented toward the cycle. The cyclic vertices are exactly the vertices of these cycles, and $\mathrm{Cyc}(f) = \mathrm{im}(f^N)$: the cyclic vertices are precisely the eventual image of f.*

PROOF. Existence of at least one cycle per component: pick any vertex of the component and follow out-edges; by Lemma 5.12 the walk enters a cycle, which lies in the same component. Uniqueness: distinct cycles are vertex-disjoint (a shared vertex would have two distinct out-edges along the two cycles, contradicting out-degree one), and if a component contained two disjoint cycles, consider any undirected path between them; some vertex on that path would need out-degree two or an out-edge leaving both cycles' basins consistently — more carefully, every vertex's forward orbit terminates on exactly one cycle (out-degree one makes the forward walk deterministic), so "which cycle does the forward orbit reach" is a well-defined function on vertices, constant along every edge in both directions (x and $f(x)$ reach the same cycle); hence it is constant on each weakly connected component, so a component sees only one cycle. Removing the unique cycle's edges leaves each remaining vertex with its single out-edge pointing one step closer (in tail length) to the cycle, which is exactly a forest of trees rooted on cycle vertices, oriented rootward.

For the last claim: if c is cyclic with cycle length ℓ, then $c = f^{\ell m}(c)$ for all m, and choosing $\ell m \geq N$ exhibits $c \in \mathrm{im}(f^N)$. Conversely if $y = f^N(x)$ for some x, then since

$t(x) \le N-1 < N$ by Lemma 5.12, the orbit of x has already entered its cycle by step N, so y is cyclic. □

REMARK 5.14. On the eventual image $\mathrm{Cyc}(f)$, the restriction of f is a bijection: it permutes each cycle by rotation. All the "loss" a finite endomap can inflict happens in finitely many steps, after which the dynamics is a permutation of a (possibly much smaller) invariant set. This two-phase structure — transient contraction, then permutation — is the conceptual content behind every bound in this chapter.

3.2. Index and period of the power sequence. We now shift attention from orbits of points to the sequence of arrows $f, f^2, f^3, \dots$ itself, which lives in the finite monoid $\mathrm{Hom}(X,X)$ and is therefore eventually periodic. The right vocabulary is the index and period of an eventually periodic sequence, standard in the theory of monogenic semigroups [**14**, **13**].

DEFINITION 5.15 (Index and period). The *index* $i(f)$ is the least $i \ge 0$ such that $f^i = f^{i+k}$ for some $k \ge 1$. The *period* $p(f)$ is the least $k \ge 1$ such that $f^{i(f)} = f^{i(f)+k}$.

(With the monoid convention $f^0 = \mathrm{id}_X$, a permutation has index 0; readers of semigroup-theoretic sources, where exponents start at 1, should note the harmless offset.)

PROPOSITION 5.16 (Structure of the power sequence). *Write $i = i(f)$ and $p = p(f)$. Then:*

1. *The arrows $f^0, f^1, \dots, f^{i+p-1}$ are pairwise distinct, and for all $n \ge i$, $f^{n+p} = f^n$.*
2. *For $m, n \ge i$: $f^m = f^n$ if and only if $m \equiv n \pmod p$.*
3. *For $m < i$ and any $n \ne m$: $f^m \ne f^n$.*
4. *$i(f) = \max_x t(x)$ and $p(f) = \mathrm{lcm}\{\, \ell(c) : c \in \mathrm{Cyc}(f) \,\}$, the least common multiple of the cycle lengths.*
5. *$i(f) \le N-1$ when $N \ge 1$ (and $i(f) = 0$ when $N = 0$), and $p(f) \le$ the maximal order of a permutation of at most N points; in particular every cycle length divides $p(f)$ and $p(f)$ divides $\mathrm{lcm}(1, 2, \dots, N)$ for $N \ge 1$.*

PROOF. (1) By definition of i there is $k \ge 1$ with $f^i = f^{i+k}$, and p is the least such k. Composing $f^i = f^{i+p}$ with f^{n-i} on the appropriate side (Proposition 5.5) gives $f^n = f^{n+p}$ for all $n \ge i$. Distinctness: a coincidence $f^a = f^b$ with $a < b \le i+p-1$ would either have $a < i$, contradicting minimality of i (since then f^a recurs), or $a \ge i$ with $b - a < p$, contradicting minimality of p after translating down to exponent i (compose both sides with f^{i-a}... more precisely, from $f^a = f^b$ and (1)'s recurrence one derives $f^i = f^{i+(b-a)}$ by adding f^p-multiples until both exponents exceed i and reducing; the least- period property then forces $p \mid b-a$, impossible for $0 < b-a < p$).

(2) The "if" direction is (1) iterated. For "only if," suppose $m, n \ge i$, $m < n$, $f^m = f^n$, and let $d = n - m$. Adding multiples of p to both exponents using (1), we may arrange comparisons at exponent exactly i: $f^i = f^{i+d'}$ where $d' \equiv d \pmod p$ and $0 \le d' < p$; minimality of p forces $d' = 0$, i.e. $p \mid d$.

(3) If $m < i$ and $f^m = f^n$ with $n > m$, then f^m recurs in the sequence, and composing the relation forward shows $f^m = f^{m+(n-m)s}$ for all $s \ge 1$, exhibiting an index at most $m < i$ — contradiction.

(4) For the index: unwind what $f^a = f^b$ means pointwise. For every x, once $a \ge t(x)$, the value $f^a(x)$ is on x's cycle and thereafter the sequence $a \mapsto f^a(x)$ is purely periodic with period $\ell(x)$. If $a \ge \max_x t(x)$ then *every* point is on-cycle, and taking $k = \mathrm{lcm}$ of all cycle lengths gives $f^{a+k} = f^a$; hence $i(f) \le \max_x t(x)$. Conversely, if $a < t(x_0)$ for some x_0, then $f^a(x_0)$ is a tail vertex; tail vertices are never revisited (their in-orbit occurrence is

unique since the orbit is injective until it meets the cycle, by the distinctness argument in Lemma 5.12), so $f^b(x_0) \neq f^a(x_0)$ for all $b > a$, and f^a never recurs; hence $i(f) \geq t(x_0)$ for the maximizing x_0. For the period: with all points on-cycle at exponent i, $f^{i+k} = f^i$ holds iff k is a multiple of every cycle length — rotation of a cycle of length ℓ by k steps is the identity on that cycle iff $\ell \mid k$ — so the least valid k is the lcm.

(5) The index bound is Lemma 5.12 ($t(x) \leq N - 1$) combined with (4). For the period: by (4), $p(f)$ is the lcm of the cycle lengths, which are the cycle lengths of the permutation f restricted to $\mathrm{Cyc}(f)$, a set of at most N points; the lcm of the lengths of disjoint cycles fitting in N points is at most the maximal order of a permutation on N points (Landau's function), and since each individual cycle length is at most N, the lcm divides $\mathrm{lcm}(1, \ldots, N)$. □

3.3. The monogenic subsemigroup and its unique idempotent.

Proposition 5.17 (Structure of $\langle f \rangle$; standard [14, 13]). *Let $i = i(f)$, $p = p(f)$, and let $\langle f \rangle = \{f, f^2, f^3, \ldots\}$ be the monogenic subsemigroup. Write $i' = \max(i, 1)$. Then:*

1. *$\langle f \rangle = \{f, f^2, \ldots, f^{i'+p-1}\}$, with these elements pairwise distinct; in particular $|\langle f \rangle| = i' + p - 1$.*
2. *The subset $K_f = \{f^{i'}, f^{i'+1}, \ldots, f^{i'+p-1}\}$ is closed under composition and is a cyclic group of order p.*
3. *The identity element of K_f is f^{k^*}, where k^* is the unique integer with $i' \leq k^* \leq i' + p - 1$ and $p \mid k^*$; equivalently, k^* is the least positive multiple of p that is at least i.*
4. *f^{k^*} is the unique idempotent element of $\langle f \rangle$.*

Proof. (1) is Proposition 5.16(1)–(3) restricted to positive exponents. (2) Closure: for $a, b \geq i'$, $f^a \circ f^b = f^{a+b}$ with $a + b \geq i'$, and by Proposition 5.16(2) this equals the unique representative f^c with $i' \leq c \leq i' + p - 1$ and $c \equiv a + b \pmod{p}$. Thus K_f with composition is isomorphic to the additive group $\mathbb{Z}/p\mathbb{Z}$ via $f^a \mapsto a \bmod p$ (a bijection on K_f by (1)-distinctness and (2)); a semigroup isomorphic to a group is a group, cyclic of order p, generated by $f^{i'}$'s appropriate power. (3) The identity of K_f corresponds to residue 0, i.e. the unique k^* in the stated window divisible by p; the window $[\, i', i' + p - 1 \,]$ contains exactly one multiple of p, and it is the least positive multiple of p that is $\geq i$ (when $i = 0$ this is p itself, matching $i' = 1$). (4) An idempotent f^k satisfies $f^{2k} = f^k$. If $k < i$ this contradicts Proposition 5.16(3); so $k \geq \max(i, 1) = i'$, and by (2), $2k \equiv k \pmod{p}$, i.e. $p \mid k$. Hence $f^k \in K_f$ with residue 0, so $f^k = f^{k^*}$. Conversely f^{k^*} is idempotent since $k^* \geq i$ and $p \mid k^*$ give $f^{2k^*} = f^{k^*}$ by Proposition 5.16(2). □

Remark 5.18 (Terminology across the literature). The vocabulary of this section varies across sources, and since the monograph cites the semigroup literature as background, a small concordance prevents misreadings. What we call the index and period of f are the standard parameters of a monogenic (equivalently: cyclic) semigroup, often written m and r, with the pair called the *type* of the semigroup [14]; the cyclic group K_f of Proposition 5.17 is the *kernel* of the monogenic semigroup in the semigroup-theoretic sense — an unfortunate but entrenched collision with the linear-algebraic "kernel" and with this monograph's use of "kernel" for the software artifact, which is why we consistently write K_f and never overload the word. The functional-graph picture, with its rho-shaped orbits, is the standard combinatorial companion of these parameters and underlies both the cycle-detection algorithms of computational practice and the finite- semigroup structure theory surveyed by Pin [13]. Nothing in this section is original; the presentation is arranged so that each numbered statement is exactly what a later chapter or fixture cites.

This completes the background. Every finite endomap generates a finite monogenic semigroup whose tail has length governed by the longest orbit tail and whose head is a cyclic group whose order is the lcm of the cycle lengths; the unique idempotent sits at the first exponent that is simultaneously past the tail and divisible by the period. The theorem the kernel ships is now a corollary of choosing one crude, uniform exponent that provably satisfies both conditions.

3.4. Worked examples on four points. Three concrete endomaps on a four-element carrier $X = (\texttt{state}, (0, 1, 2, 3))$ exercise every feature of the structure theory, and we work each one completely; the reader is encouraged to draw the three functional graphs, which make every claim visible at a glance.

Example 5.19 (Mixed tail and cycle). Let f have mapping tuple $(1, 2, 1, 0)$: so $0 \mapsto 1$, $1 \mapsto 2$, $2 \mapsto 1$, $3 \mapsto 0$. The orbit of 1 is $1 \to 2 \to 1$: a 2-cycle, so $\mathrm{Cyc}(f) = \{1, 2\}$ and both cyclic points have $t = 0$, $\ell = 2$. The orbit of 0 is $0 \to 1 \to 2 \to 1 \cdots$: tail length $t(0) = 1$. The orbit of 3 is $3 \to 0 \to 1 \to \cdots$: tail length $t(3) = 2$. Hence by Proposition 5.16(4),

$$i(f) = \max(0, 0, 1, 2) = 2, \qquad p(f) = \mathrm{lcm}\{2\} = 2,$$

and by Proposition 5.26 the minimal idempotent power is $k^* = 2$ (the least positive multiple of 2 that is at least 2). Check directly: f^2 has tuple $\big(f(1), f(2), f(1), f(0)\big) = (2, 1, 2, 1)$, and composing f^2 with itself indexes $(2, 1, 2, 1)$ at its own entries: $\big(f^2(2), f^2(1), f^2(2), f^2(1)\big) = (2, 1, 2, 1)$ — idempotent, as predicted. The safe exponent of Theorem 5.23 is $4! = 24$, and indeed $f^{24} = f^2$ because $24 \geq i(f) = 2$ and $24 \equiv 2 \pmod 2$ (Proposition 5.16(2)), confirming Corollary 5.25: the factorial power lands exactly on the monogenic semigroup's unique idempotent, here reached already at exponent 2. The gap between 24 and 2 is the price of uniformity discussed in Remark 5.24.

Example 5.20 (Pure permutation). Let σ have tuple $(1, 2, 3, 0)$: a single 4-cycle. Every point is cyclic, so $i(\sigma) = 0$ and $p(\sigma) = 4$; the minimal idempotent power is $k^* = p = 4$, and $\sigma^4 = \mathrm{id}_X$ — for a permutation, the unique idempotent in the monogenic semigroup is the identity, reached at the permutation's order. The factorial bound gives $\sigma^{24} = (\sigma^4)^6 = \mathrm{id}_X$ by Proposition 5.7, idempotent trivially. Note how the two proof obligations of Remark 5.24 split here: the index condition is vacuous ($i = 0$) and the entire content is divisibility of the exponent by the cycle length.

Example 5.21 (Constant map). Let c have tuple $(2, 2, 2, 2)$. The point 2 is a fixed point (1-cycle); every other point has tail length 1. So $i(c) = 1$, $p(c) = 1$, $k^* = 1$: the map is already idempotent, $c \circ c = c$, as direct computation confirms $(c(c(x)) = c(2) = 2 = c(x))$. Here the divisibility condition is vacuous and the entire content is the index. The monogenic subsemigroup is the singleton $\{c\}$, the degenerate case $i' = 1$, $p = 1$ of Proposition 5.17.

3.5. Idempotents as retractions. The idempotent that the factorial power produces has a clean pointwise characterization worth recording, both because it explains *what* the stabilized map is and because it connects the algebra back to the functional-graph picture.

Proposition 5.22 (Idempotent endomaps are retractions onto their fixed points). *Let $e : X \to X$ be an endomap. The following are equivalent:*

(1) *e is idempotent, $e \circ e = e$;*
(2) *every point of $\mathrm{im}(e)$ is a fixed point of e;*
(3) *e acts as the identity on its image.*

In that case $\mathrm{im}(e) = \mathrm{Fix}(e)$*, and* e *is a retraction of* X *onto its fixed-point set: it maps every point somewhere, and points already in the image stay put.*

PROOF. (1) $\Rightarrow$ (2): let $y \in \mathrm{im}(e)$, say $y = e(x)$; then $e(y) = e(e(x)) = e(x) = y$. (2) $\Leftrightarrow$ (3) is a rephrasing. (2) $\Rightarrow$ (1): for any x, $e(x) \in \mathrm{im}(e)$, so $e(e(x)) = e(x)$; pointwise agreement at every index plus shared declared types gives $e \circ e = e$. For the final claim: (2) gives $\mathrm{im}(e) \subseteq \mathrm{Fix}(e)$, and any fixed point y satisfies $y = e(y) \in \mathrm{im}(e)$, so the two sets coincide. □

For the factorial power specifically, the fixed-point set of $f^{N!}$ is exactly the cyclic vertex set $\mathrm{Cyc}(f)$: every cyclic vertex is returned to itself by an exponent divisible by its cycle length, and every tail vertex has left the tail by exponent $N!$ and cannot be its own image. So Theorem 5.23 has a picture: *the factorial power is the retraction of the functional graph onto its cycles*, collapsing each tree to the cycle vertex its flow reaches, and freezing the cycles pointwise. In Example 5.19, f^2 sends $0, 2 \mapsto 2$ and $1, 3 \mapsto 1$: the four-point graph retracts onto the 2-cycle $\{1, 2\}$, exactly as the picture demands.

4. The Factorial Idempotence Theorem

THEOREM 5.23 (Safe finite power). *Let* $f : X \to X$ *be an endomap on a carrier of size* $N = |X|$*. Then* $f^{N!}$ *is idempotent:*

$$f^{N!} \circ f^{N!} = f^{N!}.$$

Status: standard result [**13, 14**]; complete informal proof below; verified exhaustively by the executable suite for all 289 endomaps on carriers of size at most four; Lean formalization open (ledger PO-TKS8-CORE-04).

We give two proofs: a self-contained pointwise proof, which is the argument the seed paper records and which requires nothing beyond Lemma 5.12, and a one-line derivation from the structure theory, which situates the theorem correctly as the crude-bound corollary of Proposition 5.17.

FIRST PROOF (SELF-CONTAINED, POINTWISE). The cases $N = 0$ and $N = 1$ are immediate: for $N = 0$ the only endomap is the empty identity and every power equals it; for $N = 1$ the only endomap is id_X and likewise. Assume $N \geq 2$, and fix an arbitrary point x. By Lemma 5.12, the tail length satisfies $t(x) \leq N - 1$. Since $N \geq 2$ gives $N! \geq N > N - 1 \geq t(x)$, the point $f^{N!}(x)$ lies on x's cycle. Let $\ell \leq N$ be that cycle's length (Lemma 5.12 again). Because $\ell \leq N$, the integer ℓ divides $N!$ — it is one of the factors $1, 2, \ldots, N$ of the product — so a further $N!$ applications of f traverse the cycle a whole number $N!/\ell$ of times and return to the same vertex:

$$f^{2N!}(x) = f^{N!}\big(f^{N!}(x)\big) = f^{N!}(x).$$

The point x was arbitrary, so the arrows $f^{2N!}$ and $f^{N!}$ agree at every standard index; they share declared source and target X; hence $f^{2N!} = f^{N!}$ as arrows, and by Proposition 5.5 this is exactly $f^{N!} \circ f^{N!} = f^{N!}$. □

SECOND PROOF (VIA STRUCTURE THEORY). By Proposition 5.16(4,5), $i(f) \leq \max(N - 1, 0) \leq N!$ and $p(f)$ is the lcm of cycle lengths, each at most N, so $p(f) \mid N!$. Thus $N! \geq i(f)$ and $p(f) \mid N!$, and Proposition 5.16(2) applied to the exponents $N!$ and $2N!$ (both $\geq i(f)$, congruent mod $p(f)$) gives $f^{2N!} = f^{N!}$. (For $N \in \{0, 1\}$, $N! = 1$ and every endomap on at most one point is the identity, idempotent trivially — note the $N = 1$ case needs the separate sentence because $1! = 1$ and $f^1 = f$, which is idempotent only because $f = \mathrm{id}_X$ there.) □

REMARK 5.24 (Why the factorial). The exponent $N!$ is doing two jobs at once, and the proof shows them cleanly separated: it must be *large enough* to outrun every tail ($N! \geq N-1$, with room to spare), and it must be *divisible enough* to be a multiple of every possible cycle length ($\ell \mid N!$ for all $\ell \leq N$). Any exponent with both properties works; $N!$ is the crudest natural choice with a one-line proof of each property. Its virtue is not sharpness but *uniformity and checkability*: it depends only on the carrier size, not on any structural analysis of f, so the implementation can compute the safe exponent without first computing the functional graph. The next section quantifies what sharper choices exist and what they would cost.

COROLLARY 5.25 (The factorial power is the idempotent of $\langle f \rangle$ — when $N \geq 2$). *For $N \geq 2$, $f^{N!}$ equals f^{k^*}, the unique idempotent of the monogenic subsemigroup of Proposition 5.17.*

PROOF. $f^{N!}$ is an idempotent element of $\langle f \rangle$ (it is a positive power of f, since $N! \geq 1$, and idempotent by Theorem 5.23); uniqueness in Proposition 5.17(4) forces $f^{N!} = f^{k^*}$. □

5. Sharper Variants, Stated Honestly

The factorial bound invites two immediate refinements, both classical, and this section presents them with the same care as the theorem itself — including the explicit statement of what the kernel does and does not implement.

5.1. Exact characterization of idempotent powers. The structure theory already delivered the exact answer, and we restate it in bound-free form:

PROPOSITION 5.26 (Which powers are idempotent). *For $k \geq 1$, the power f^k is idempotent if and only if $k \geq i(f)$ and $p(f) \mid k$. The minimal such k is*

$$k^*(f) = \begin{cases} p(f) \cdot \lceil i(f)/p(f) \rceil, & i(f) \geq 1, \\ p(f), & i(f) = 0, \end{cases}$$

that is, the least positive multiple of the period that is at least the index.

PROOF. Contained in the proof of Proposition 5.17(3,4). □

The minimal exponent $k^*(f)$ can be dramatically smaller than $N!$: for a permutation consisting of a single N-cycle, $i = 0$ and $p = N$, so $k^* = N$ versus $N!$; for a map with one long tail into a fixed point, $p = 1$ and $k^* = i \leq N - 1$. But computing $k^*(f)$ requires knowing $i(f)$ and $p(f)$, i.e. analyzing the functional graph of the specific f — a linear-time computation, but a computation about f rather than about N alone.

5.2. The lcm bound for the uniform exponent. Between the exact per-map answer and the factorial there is a natural intermediate: keep the exponent uniform in N, but replace $N!$ by the least common multiple of 1 through N.

PROPOSITION 5.27 (Uniform lcm exponent). *Let $L(N) = \mathrm{lcm}(1, 2, \ldots, N)$ for $N \geq 1$. For every endomap f on a carrier of size $N \geq 1$, the power $f^{L(N)}$ is idempotent.*

PROOF. By Proposition 5.16(5), $p(f) \mid L(N)$. For the index: $i(f) \leq N - 1$, and $L(N) \geq N > N - 1$ for $N \geq 1$ (indeed $N \mid L(N)$ so $L(N) \geq N$). Hence $L(N) \geq i(f)$ and $p(f) \mid L(N)$, and Proposition 5.26 applies. (For $N = 1$, $L(1) = 1$ and the unique endomap is the identity.) □

The gap between the two uniform exponents is enormous: $L(N)$ grows like $e^{(1+o(1))N}$ (by the prime number theorem, since $\log L(N)$ is the Chebyshev function $\psi(N)$), while $N!$ grows like $e^{N\log N-N}$ by Stirling. Already at $N = 10$: $10! = 3{,}628{,}800$ versus $L(10) = 2{,}520$. Both are standard facts of elementary number theory combined with the semigroup structure theory above [13, 14]; the maximal possible *period* over all endomaps on N points is Landau's function (the maximal order of a permutation of at most N points), which is smaller still, but the exponent must also clear the index, so the clean uniform statements are the two just given.

The divergence is visible immediately in a small table:

N	$N!$	$L(N) = \mathrm{lcm}(1,\dots,N)$
1	1	1
2	2	2
3	6	6
4	24	12
5	120	60
6	720	60
7	5,040	420
8	40,320	840

The two agree through $N = 3$ and separate permanently at $N = 4$, where the factorial first multiplies in a factor (4) whose prime content (2^2) is only partially new. Neither column is remotely tight for a *typical* endomap — Example 5.19's map on four points stabilizes at exponent 2 against the factorial's 24 — but tightness is not the design goal; a uniform, structure-blind, provably safe exponent is, and for that role either column serves, with $N!$ chosen for the shorter proof.

5.3. What the kernel implements.

Adverse Finding 5.28. The kernel implements and tests *the factorial bound and only the factorial bound*. The endomap type exposes an operation returning $f^{|X|!}$ (computed by binary exponentiation, Section 8), together with a predicate testing idempotence of an arbitrary endomap by direct comparison of $f\circ f$ with f. It does not compute $i(f)$, $p(f)$, $k^*(f)$, or $L(N)$, and no kernel API claims minimality of the factorial exponent. The sharper variants of this section are presented as standard mathematics [13, 14] for the reader's orientation and as candidate future obligations; they carry no executable or formal artifact in this kernel, and nothing downstream may cite them as implemented.

The design rationale for shipping only the crude bound is the same throughout the kernel: prefer the statement whose proof is short, whose implementation is a one-liner over an already-tested primitive, and whose correctness does not depend on a per-instance structural analysis that would itself need tests. A minimal-exponent API would be genuinely useful — and genuinely riskier, since an off-by-one in tail computation would produce non-idempotent "idempotents" silently. The kernel takes the safe exponent; the analysis layer can be built later, on top, under its own obligations.

6. Verification of the Idempotence Theorem

6.1. The exhaustive sweep and its count. The executable suite verifies Theorem 5.23 by complete enumeration over small carriers: for every endomap f on a representative object

of each size $N \in \{0, 1, 2, 3, 4\}$, it computes $g = f^{N!}$ via the implemented exponentiation and asserts $g \circ g = g$ by extensional arrow equality. The population, as computed in Section 1, is

$$\sum_{k=0}^{4} k^k = 0^0 + 1^1 + 2^2 + 3^3 + 4^4 = 1 + 1 + 4 + 27 + 256 = 289$$

endomaps, matching the recorded suite count. The factorial exponents involved are $0! = 1! = 1$, $2! = 2$, $3! = 6$, and $4! = 24$; the sweep therefore exercises the exponentiation path with genuinely composite binary expansions (e.g. $24 = 11000_2$) as well as the trivial ones.

Size four is a meaningful boundary for this particular theorem, not merely an affordable one: carriers of size at most two cannot exhibit a simultaneous nontrivial tail and nontrivial cycle, and size three barely can; at size four the sweep includes maps with tail length up to three, cycle lengths 1 through 4, mixed components, and every combination of index and period realizable on four points. In small-scope terms [**15**], the structural features the proof relies on — tails, cycles, divisibility of the exponent by the cycle length — are all present in force within the swept scope.

Claim Boundary 5.29. The 289-endomap sweep establishes the idempotence identity for every endomap on carriers of size at most four, under the implemented exponentiation, and nothing more. Theorem 5.23 itself holds for all finite sizes by the proofs above, which are standard mathematics [**13, 14**]; but the connection between the theorem and the running code is tested only within the stated bound, and the theorem has no Lean formalization: ledger obligation PO-TKS8-CORE-04 stands at STANDARD RESULT / FORMALIZATION OPEN. In particular, no artifact should describe the factorial idempotence theorem as machine-verified.

6.2. What closing the formalization obligation would involve. Since PO-TKS8-CORE-04 is the most prominent open formalization in the kernel's ledger, it is worth being concrete about its shape, so the obligation reads as a bounded engineering task rather than an indefinite aspiration. The mathematical content of Theorem 5.23 is elementary: the proof uses the pigeonhole principle (Lemma 5.11), induction along orbits, and the divisibility facts $\ell \leq N \Rightarrow \ell \mid N!$ and $N! \geq N-1$. None of this strains a proof assistant; comparable statements about iterates of functions on finite types are routine in mature libraries. The cost driver is the seed's own discipline: the formal development is deliberately dependency-minimal and axiom-audited (`propext` only, no external mathematics library), so the formalization must either build the required elementary number theory and finite-iteration lemmas in-house under the same audit, or extend the audit policy to admit a vetted library dependency — a policy decision with consequences beyond this one theorem, since it would also move the open norm-theorem and shortlex obligations. Either way, the theorem statement to be formalized is exactly the extensional identity $f^{N!} \circ f^{N!} = f^{N!}$ over the Lean model's arrows, stated for every carrier size — the model-level analogue of what the 289-case sweep checks for the Python artifact at sizes through four. Until that is done, the theorem's status line remains what the ledger says: standard result, formalization open.

6.3. Why exhaustive rather than sampled. For this theorem the exhaustive design is strictly superior to sampling at equal cost, and it is worth recording why, since the reasoning generalizes. The failure modes worth fearing are not uniform over the space of endomaps: an exponentiation bug (say, a wrong squaring order) might affect only exponents with a particular bit pattern; an off-by-one in the safe exponent would spare permutations (whose idempotent power tolerates slack in the index condition trivially when $i = 0$) and bite only maps whose index is maximal. Such structured defects occupy thin slices of the space, exactly

where uniform random sampling is weakest. Complete enumeration through size four costs a few hundred cases — trivially affordable — and guarantees that every realizable small configuration of tail, cycle, and exponent bit pattern is hit. The suite's marginal design question is never "how many random cases"; it is "what is the largest scope we can afford to saturate," which is the small-scope methodology applied deliberately [**15**].

7. The Power API Contract

Before descending into the exponentiation algorithm, we state the behavioral contract of the power operation as an invariant, parallel to the mismatch-rejection contract of Chapter 4. The contract is what the negative fixtures pin; the algorithm of the next section is one implementation satisfying it.

Invariant 5.30 (Power contract). The power operation is defined on certified endomaps and non-negative integer exponents, and satisfies:

(1) (Domain) The operation exists only on the endomap wrapper type; no arrow, path, or other value has it. The exponent must be a non-negative integer; negative integers, non-integers, and the host language's boolean values (which the host treats as integers, a coercion the kernel explicitly refuses) are rejected with a validation error.
(2) (Semantics) The result is the certified endomap whose arrow is f^n in the sense of Definition 5.4: extensionally equal to the n-fold composite, with $f^0 = \mathrm{id}_X$ on the endomap's carrier.
(3) (Closure) The result is again a certified endomap on the same carrier, so powers can be taken of powers, and Proposition 5.7 governs the iteration.
(4) (Name discipline) The result's presentation name embeds the bounded label of Construction 5.37; the name has no effect on the result's identity (Invariant 5.38).
(5) (Resource bound) Time is $O(|X| \log n)$ and space $O(|X|)$, independent of n's decimal size; in particular no operation in the power path converts n to decimal when n exceeds the naming threshold.

Check Class 5.31 (Power fixtures). The suite pins the contract with: rejection fixtures for exponent -1, for a floating-point exponent, and for boolean exponents (both values); the semantic fixtures $f^0 = \mathrm{id}_X$, $f^1 = f$, and agreement of f^n with the naive n-fold composite for all n up to a small bound on representative endomaps; the exhaustive idempotence sweep of Section 6, which exercises exponents $1, 2, 6, 24$ across all 289 small endomaps; and the large-exponent regressions at 2^{30} and 10^{5000}, which pin clauses (4) and (5). The boolean-rejection fixture earns its place by a real hazard: in the host language, `True` silently behaves as 1 in arithmetic, so an accidental boolean argument would otherwise compute f^1 without complaint — a plausible bug (a comparison result passed where a count was meant) producing a plausible-looking wrong answer, which is the worst combination.

Remark 5.32 (The idempotence predicate). Alongside `power`, the endomap type exposes a predicate testing whether the map is idempotent, implemented as the direct comparison $f \circ f = f$ — one composition and one tuple equality, $O(|X|)$. The predicate makes no structural analysis and returns no certificate beyond its boolean; it exists so that the idempotence assertions of the test suite are stated against the public API rather than against internals, and so that downstream code can *check* claimed idempotents cheaply instead of trusting them. The pairing is characteristic of the kernel's posture: the theorem (Theorem 5.23) supplies an exponent that provably works, and the predicate supplies an independent, trivially auditable confirmation on each concrete instance.

8. Binary Exponentiation

8.1. The algorithm. Computing f^n by $n-1$ successive compositions is linear in n, which is unusable at the exponents the kernel supports (the regression suite goes to 10^{5000}). The implementation uses the standard square-and-multiply scheme, iterating over the binary expansion of the exponent.

Algorithm 5.33 (Exponentiation by squaring). Input: an endomap f on X, an integer $n \geq 0$ (the implementation validates that n is a non-negative integer and explicitly rejects booleans). Output: f^n.

(1) Set $r \leftarrow \mathrm{id}_X$ (the accumulator), $b \leftarrow f$ (the current square), $m \leftarrow n$ (the remaining exponent).
(2) While $m > 0$:
  (a) if m is odd, set $r \leftarrow b \circ r$;
  (b) set $m \leftarrow \lfloor m/2 \rfloor$;
  (c) if $m > 0$, set $b \leftarrow b \circ b$.
(3) Return the arrow with the accumulator's mapping tuple, re-wrapped as an endomap on X with a bounded presentation name (Construction 5.37).

Proposition 5.34 (Correctness of Algorithm 5.33). *Algorithm 5.33 returns f^n.*

Proof. We claim the loop maintains the invariant $r\circ$-combined with the remaining work equals the answer; precisely:

$$b^m \circ r = f^n$$

at every loop-condition evaluation, where b^m denotes the m-th power of the current square. Initialization: $b = f$, $m = n$, $r = \mathrm{id}_X$, and $f^n \circ \mathrm{id}_X = f^n$ by right neutrality. Preservation: write $m = 2q + \epsilon$ with $\epsilon \in \{0, 1\}$. If $\epsilon = 1$ the body performs $r' = b \circ r$, and

$$(b \circ b)^q \circ r' = b^{2q} \circ (b \circ r) = b^{2q+1} \circ r = b^m \circ r = f^n,$$

using Proposition 5.7 for $(b \circ b)^q = b^{2q}$, associativity, and Proposition 5.5. If $\epsilon = 0$ the same computation without the extra factor gives $(b \circ b)^q \circ r = b^m \circ r = f^n$. (When $q = 0$ the final squaring is skipped; the invariant holds vacuously with $b^0 = \mathrm{id}_X$.) Termination: m strictly decreases toward 0. At exit $m = 0$, so $f^n = b^0 \circ r = \mathrm{id}_X \circ r = r$ by left neutrality. □

Example 5.35 (Trace at $n = 24$). The safe exponent for a four-point carrier is $4! = 24 = 11000_2$, and the algorithm's run at $n = 24$ is short enough to trace in full. Write the state as (m, r, b) at each loop test, with r and b given as powers of f:

iteration	m	r	b
start	24	f^0	f^1
1	24 even; halve, square	f^0	f^2
2	12 even; halve, square	f^0	f^4
3	6 even; halve, square	f^0	f^8
4	3 odd; multiply, halve, square	f^8	f^{16}
5	1 odd; multiply, halve; final squaring skipped	f^{24}	–

Five iterations, six compositions (three squarings before the first multiply, one more squaring, two multiplies), against twenty-three compositions for the naive method. The invariant $b^m \circ r = f^{24}$ of Proposition 5.34 can be checked at each row: $f^{1\cdot 24} \circ f^0$, $f^{2\cdot 12} \circ f^0$, $f^{4\cdot 6} \circ f^0$, $f^{8\cdot 3} \circ f^0$, $f^{16\cdot 1} \circ f^8$, and finally $f^0 \circ f^{24}$ — all equal to f^{24} by Proposition 5.5. Applied to the endomap of Example 5.19, the returned arrow's tuple is $(2, 1, 2, 1)$, the retraction onto the 2-cycle, and its label is the plain decimal `24`, far below the 256-bit naming threshold of Construction 5.37.

REMARK 5.36 (Cost). The loop executes $\lceil \log_2(n+1) \rceil$ iterations, each performing at most two compositions, and each composition of endo-arrows on X is $\Theta(|X|)$ tuple indexing. Total: $O(|X| \log n)$ time and $O(|X|)$ space per power. For $n = 10^{5000}$ this is roughly 1.7×10^4 iterations — the exponent's bit length is about 16,610 — times a four-entry tuple map in the regression's configuration: microseconds, not megayears. The point of the 10^{5000} regression is not performance bragging; it is that *nothing* in the power path scales with the exponent's decimal magnitude except the loop count, and the next construction is what makes that true for the name as well.

9. Bounded Presentation Names

9.1. The problem. Every arrow carries a presentation name — a human-readable string used in error messages, logs, and debugging output. The natural naming scheme for a power, something like `f^(24)`, embeds the exponent in decimal. For $n = 10^{5000}$ that name would be five thousand and one characters of digits; worse, converting truly enormous integers to decimal is expensive (quadratic in the classical algorithm), and the host language, as a denial-of-service mitigation, enforces a hard limit on integer-to-decimal conversion size, so a naive `str(n)` on a big-enough exponent does not merely produce an unwieldy name — it raises an exception and takes the power operation down with it. A correctness-irrelevant string must never be able to do that.

9.2. The construction.

CONSTRUCTION 5.37 (Bounded integer labels). The name of a computed power f^n embeds a *bounded label* for n, defined as follows:

(1) if the bit length of n is at most 256, the label is the ordinary decimal rendering of n;
(2) otherwise, the label is the string

$$\texttt{bits=}b\texttt{,sha256=}d,$$

where b is the bit length of n (a small integer) and d is the first sixteen hexadecimal characters of the SHA-256 digest of n's big-endian byte representation.

In addition, the *intermediate* arrows created inside Algorithm 5.33 — the accumulator and the running square — carry fixed constant names, not compounded ones, so that name length cannot grow with the iteration count; only the final returned arrow receives the labeled name.

The threshold and the digest length are engineering choices with simple rationales. A 256-bit threshold keeps every humanly-plausible exponent (anything up to about 77 decimal digits) fully readable, while guaranteeing that decimal conversion is only ever invoked on integers far below the host's conversion limit. Above threshold, the label's cost is $O(\log n)$ — dominated by the byte serialization of the exponent — and its size is constant. The sixteen-hex-character (64-bit) digest prefix is not a cryptographic commitment; it is a collision-*unlikely* mnemonic that lets a human distinguish two large exponents in a log with high probability. Nothing anywhere depends on the label being collision-free, which is the subject of the next section.

The two regression pins bracket the construction's branches: $n = 2^{30}$ has bit length $31 \le 256$ and must render in full decimal (`1073741824`); $n = 10^{5000}$ has bit length about 16,610 and must render in the `bits=...,sha256=...` form, with the whole power computation completing without touching large-integer decimal conversion. Both cases assert, additionally,

the mathematical correctness of the resulting arrow — the label machinery is exercised *and* the underlying mapping tuple is checked.

9.3. Why names can never affect correctness.

INVARIANT 5.38 (Extensional inertness of names). Presentation names participate in no semantic operation of the kernel. Arrow equality and hashing exclude the name field entirely (Definition 4.2); the denotation key of an arrow is the triple of source, target, and mapping tuple; composition, power, idempotence testing, path composition, lift construction, and normal-form computation read only denotational data. Consequently: two arrows with colliding labels but different mapping data are unequal; two arrows with different labels but equal denotational data are equal; and any name whatsoever — truncated, hashed, or constant — yields the same results in every kernel computation.

The invariant is enforced structurally rather than by policy: the equality and hash implementations are generated from the dataclass declaration with the name field marked non-comparing, so there is no code path in which a name could be consulted, rather than a convention that no code path should. The negative-style test for this is the pleasant one: construct the same power twice under different naming (e.g. once through Algorithm 5.33 with its labeled name, once by repeated composition with default composite names) and assert the results *equal*; construct two different arrows and force their names equal, and assert them *unequal*. Both directions are in the suite.

This is also why the truncated digest is harmless. A collision between two labels — possible in principle, vanishingly unlikely at 64 bits within any log a human reads — could at worst cause a moment of confusion in a debugging session, at which point the denotation key, which is exact, resolves the question. No proof, no test oracle, and no canonical form consumes a label.

CLAIM BOUNDARY 5.39. The naming construction makes a bounded-cost, bounded-size claim about label *construction*, verified by the two regression pins; it makes no uniqueness or injectivity claim about labels, and no kernel artifact may treat presentation names as identifiers. The unique identifier of an arrow is its denotation key.

10. Exercises

EXERCISE 5.40. Analyze the endomap on five points with tuple $(1, 2, 3, 1, 0)$: draw the functional graph, compute every tail length and cycle length, determine i, p, and k^* via Proposition 5.26, and verify that $f^{5!} = f^{k^*}$ as Corollary 5.25 predicts.

EXERCISE 5.41. (i) Show that on N points ($N \geq 2$) the index bound $i(f) \leq N - 1$ is attained exactly by the maps whose functional graph is a single path of $N - 1$ tail edges ending in a fixed point, and that those maps have $p = 1$: maximal index forces trivial period. (ii) Show that the *pointwise* bound $t(x) + \ell(x) \leq N$ of Lemma 5.12 does *not* globalize: the inequality $i(f) + p(f) \leq N$ fails for the permutation of five points with cycle type $(2, 3)$, which has $i = 0$ and $p = \operatorname{lcm}(2, 3) = 6 > 5$. Identify exactly where the globalization argument breaks: the period is an lcm across components, while each point sees only its own cycle length.

EXERCISE 5.42. Enumerate the squares among the 27 endomaps on three points: compute $\{g^2 : g \in \operatorname{Hom}(X, X)\}$ for $|X| = 3$ and verify that the 3-cycles are squares (each 3-cycle is the square of the other) while the transpositions are not. Compare with the two-point enumeration in Chapter 6, where no nonidentity permutation is a square.

EXERCISE 5.43. Compute $N!$, $L(N) = \mathrm{lcm}(1, \ldots, N)$, and the maximal $k^*(f)$ over all endomaps on N points, for $N = 1, \ldots, 6$, and confirm that $L(N) < N!$ strictly from $N = 4$ onward (they agree through $N = 3$). (For the maximal k^*, combine a long tail with a cycle structure of large lcm and note the competition for points.)

EXERCISE 5.44. Using Invariant 5.38, argue that if two distinct exponents $n_1 \neq n_2$ had colliding bounded labels (Construction 5.37) *and* $f^{n_1} = f^{n_2}$ as arrows, no kernel observation could distinguish the two computations — and explain why this is correct behavior rather than a defect, by identifying which datum (the denotation key) is the arrow's identity and which (the label) is commentary.

11. Summary of Claims and Their Status

Claim	Status
Power laws (Propositions 5.5, 5.7)	Complete informal proofs from the category laws; standard.
Functional-graph structure; index/period; monogenic structure (Propositions 5.13, 5.16, 5.17)	Standard mathematics [**14**, **13**]; complete informal proofs given as background; no kernel API computes index or period.
Factorial idempotence (Theorem 5.23)	Standard result [**13**, **14**]; two complete informal proofs; exhaustive executable verification for all $289 = \sum_{k \leq 4} k^k$ endomaps through carrier size four; Lean formalization OPEN (POTKS8-CORE-04).
Sharper bounds ($k^*(f)$; $\mathrm{lcm}(1..N)$; Propositions 5.26, 5.27)	Standard mathematics, presented for orientation; *not implemented, not tested, no artifact* (Finding 5.28).
Binary exponentiation correctness (Proposition 5.34)	Complete informal invariant proof; exercised by the idempotence sweep and the large-exponent regressions.
Bounded presentation names (Construction 5.37)	Executable validation: regression pins at 2^{30} and 10^{5000}; names extensionally inert by construction (Invariant 5.38).

The endomap is the licensed half of the kernel's iteration story: one arrow, one object, a genuine monoid, and a theorem with teeth. The next chapter is the unlicensed half — sequences of arrows across different objects, which compose but do not iterate — and the analysis of exactly what goes wrong when the two halves are confused.

CHAPTER 6

Semigroupoid Paths and the Invalid-Inference Analysis

Chapter 5 developed the licensed side of iteration: a single endomap on a single object, living in a genuine monoid, with powers and a finite idempotence theorem. This chapter develops its deliberately crippled sibling. A *semigroupoid path* is a nonempty sequence of arrows whose adjacent objects match; it has a typed composite — the category laws guarantee that composing along the sequence is well defined and association-independent — and it has, by construction, *no power operation*. There is no p^2, no p^n, no "run the path again." The omission is not an unimplemented feature; it is the central design statement of the kernel, and the reason the two constructions exist as distinct runtime types rather than as one type with a flag.

Why spend a chapter on an absence? Because the absence encodes a theorem- shaped mistake that recurs, in the wild, wherever operator formalisms meet informal reasoning about repetition. The mistake has a fixed anatomy: take a *sequence* of operations — typed, heterogeneous, possibly time-varying — and reason about it as if it were the *iteration of one* operation, thereby importing conclusions (fixed points, idempotence, convergence, periodicity) that are theorems only about single elements of finite monoids. Chapter 5 proved those theorems in their legitimate habitat; this chapter shows, in three extended case studies, exactly how each step of the illegitimate transfer fails in the kernel — not rhetorically but mechanically, with the precise construction site at which a typed error is raised, or the precise operation that does not exist in the API.

The chapter also situates the path construction against its standard mathematical background — the free category on a directed graph, in the sense of Mac Lane [**2**] — and closes with the boundary statement that paths carry no dynamics and no semantics: a path is a bookkeeping object for typed composition, and nothing about workflows, processes, or cognition follows from one.

1. The Path Construction

1.1. Definition.

Definition 6.1 (Semigroupoid path). A *semigroupoid path* is a nonempty finite sequence of arrows

$$X_0 \xrightarrow{f_1} X_1 \xrightarrow{f_2} X_2 \longrightarrow \cdots \xrightarrow{f_n} X_n, \qquad n \geq 1,$$

such that for each adjacent pair, the declared target of f_k equals the declared source of f_{k+1} *as objects* — name and ordered carrier both, exactly the definedness condition of Definition 4.5. The path's *source* is X_0 and its *target* is X_n; the objects $X_1, \ldots, X_{n-1}$ are its *intermediate objects*. The constructor validates every adjacency at construction time and raises the typed composition error, naming the offending segment, if any adjacency fails; it also rejects the empty sequence and any unordered or one-shot container of arrows, per the ordered-input discipline of Chapter **??**.

Definition 6.2 (Path composite). The *composite* of the path in Definition 6.1 is the arrow

$$f_n \circ f_{n-1} \circ \cdots \circ f_1 \ : \ X_0 \to X_n,$$

computed by left-to-right folding of the kernel's composition. By the associativity clause of Theorem 4.14, every bracketing of the fold yields the same arrow, so the composite is well defined independently of evaluation order.

Rule 6.3 (No power operation). The path type exposes construction, source, target, the composite, and appending a type-compatible arrow. It exposes no exponentiation of any kind: no operation takes a path and a natural number and returns "the path repeated." This is an API absence enforced by the type system of the host language: the power operation is defined only on the endomap wrapper (Definition 5.1), the path type is not that type and is not convertible to it, and the endomap constructor accepts only a single arrow whose source and target are equal as objects.

Remark 6.4 (Why "semigroupoid"). The name records the algebraic habitat precisely. A semigroupoid (also called a semicategory) has objects and typed composition, associative where defined, but is not required to have identities at each object, and its composable sequences do not form monoids unless source and target coincide. The kernel's ambient structure $\mathcal{C}$ is a full category — identities exist — but a *path* is an element of the free composable-sequence structure over $\mathcal{C}$'s arrows, and the operations the kernel grants to paths are exactly the semigroupoid operations: composition along matching types, and nothing that presupposes an enclosing monoid. Powers presuppose an enclosing monoid. Hence their absence.

1.2. Relation to free categories on graphs. The path construction is a disciplined instance of a completely standard object. Given a directed graph G — vertices, edges, each edge with a source and target vertex — the *free category* $\mathcal{F}(G)$ on G has the vertices as objects and, as morphisms, the finite composable strings of edges, including one empty string at each vertex serving as its identity; composition is concatenation [**2**]. The construction is left adjoint to the forgetful functor from small categories to directed graphs, and its universal property is the practical content of that adjunction: for any category $\mathcal{D}$ and any graph morphism ϕ from G into the underlying graph of $\mathcal{D}$ — an assignment of objects to vertices and morphisms to edges, respecting sources and targets — there is a *unique* functor $\hat{\phi} : \mathcal{F}(G) \to \mathcal{D}$ extending ϕ, namely the one sending each composable edge string to the composite of its images,

$$\hat{\phi}\big(e_1 e_2 \cdots e_n\big) = \phi(e_n) \circ \cdots \circ \phi(e_2) \circ \phi(e_1),$$

and the empty string at a vertex v to $\mathrm{id}_{\phi(v)}$. Freeness means the strings satisfy no equations beyond associativity and identity bookkeeping: distinct strings remain distinct morphisms of $\mathcal{F}(G)$ even when their images under some $\hat{\phi}$ coincide — which is the categorical statement of the forgetting phenomenon exhibited concretely in Example 6.10.

Two comparisons locate the kernel's paths in this picture.

First: the kernel's paths are, up to the excluded empty string, the morphisms of the free category on the underlying graph of $\mathcal{C}$ (vertices: the finite objects; edges: the arrows). The adjacency condition of Definition 6.1 is exactly composability of edge strings. The kernel excludes the empty path deliberately: an empty path would need a distinguished object to serve as its source and target — data not derivable from an empty arrow sequence — and the identity arrows of $\mathcal{C}$ already serve every legitimate use of "do nothing at X." Nonemptiness keeps the path type's source and target total functions of its data, with no special case.

Second, and more importantly: the kernel's path is a *formal string that remembers its factorization*, whereas its composite is a single arrow that forgets it. The evaluation functor from the free category to $\mathcal{C}$ — the unique functor extending the identity assignment on vertices and edges, per the universal property [2] — is exactly the kernel's `composite` operation. Distinct paths can have equal composites (the fold collapses the string), and the kernel keeps both artifacts available because they answer different questions: the path answers "what steps, in what order, through what intermediate types," which is the auditability question; the composite answers "what total function resulted," which is the denotational question. Nothing is ever inferred about the path from properties of its composite — that direction of inference is precisely where the invalid inferences of Section 4 live.

NOTATION 6.5. For a path p with arrows $f_1, \ldots, f_n$ we write $[\![p]\!] = f_n \circ \cdots \circ f_1$ for its composite, mirroring the denotation brackets used for typed words in the normal-form chapter; both are evaluation maps from syntax-remembering objects to arrows.

2. Path Algebra

Although paths have no powers, they are not algebraically inert: they concatenate, they extend one arrow at a time, and their evaluation respects both operations. This section proves the three small facts the rest of the monograph uses, and exhibits the forgetting phenomenon — distinct paths with equal composites — that motivates keeping paths and composites as separate artifacts.

2.1. Concatenation.

DEFINITION 6.6 (Concatenation). Let p be a path with arrows $f_1, \ldots, f_m$ from X_0 to X_m, and q a path with arrows $g_1, \ldots, g_n$ from Y_0 to Y_n. The *concatenation* $p \cdot q$ is defined if and only if $X_m = Y_0$ as objects, in which case it is the path with arrows $f_1, \ldots, f_m, g_1, \ldots, g_n$ from X_0 to Y_n.

Definedness of the concatenation is a single adjacency check — the same comparison as arrow composition, at the junction — and the result is well formed because all its other adjacencies are inherited from p and q. In the implementation, concatenation is available as iterated `append`; the definition above is its batch form.

PROPOSITION 6.7 (Paths form a semigroupoid). *Concatenation is associative wherever defined: if $p \cdot q$ and $q \cdot r$ are defined, then so are $(p \cdot q) \cdot r$ and $p \cdot (q \cdot r)$, and they are equal.*

PROOF. Definedness: the junctions of $(p \cdot q) \cdot r$ are those of $p \cdot q$ (the p/q junction) and the q/r junction, since concatenation changes neither the source nor the target of the q segment; symmetrically for the other association. Equality: both associations have the same underlying arrow sequence — concatenation of sequences is associative — and a path is determined by its arrow sequence (the objects are read off the arrows). Hence the two paths are equal as values. □

PROPOSITION 6.8 (Evaluation respects concatenation). *If $p \cdot q$ is defined, then $[\![p \cdot q]\!] = [\![q]\!] \circ [\![p]\!]$.*

PROOF. Write the arrows as in Definition 6.6. Both sides are composites of the sequence $f_1, \ldots, f_m, g_1, \ldots, g_n$ in order: the left side by the fold of Definition 6.2 over the whole sequence, the right side by folding each half and then composing the results. By the associativity clause of Theorem 4.14, every bracketing of a fixed composable sequence yields the same arrow (formally: by induction on n, moving one arrow at a time across the central composition

using associativity), so the two sides agree. All intermediate composites are defined because every adjacency in the combined sequence holds. □

LEMMA 6.9 (Append form). *For a path p and an arrow g with source equal to p's target, $[\![p \cdot g]\!] = g \circ [\![p]\!]$, where $p \cdot g$ abbreviates appending g to p.*

PROOF. The case $n = 1$ of Proposition 6.8, with q the one-arrow path (g), whose composite is g itself. □

In categorical language, Propositions 6.7 and 6.8 say that nonempty paths form a semicategory (identities excluded by the nonemptiness convention of Section 1.2) and that evaluation $p \mapsto [\![p]\!]$ is a semifunctor into $\mathcal{C}$ — the restriction to nonempty strings of the evaluation functor from the free category [**2**]. Nothing deeper is claimed; the propositions exist so that downstream code may reassociate path-building operations freely, knowing the denotation cannot shift.

2.2. Forgetting: distinct paths, equal composites.

EXAMPLE 6.10 (Two factorizations of one function). Let $X = (\texttt{pair}, (0, 1))$ and let $Y = (\texttt{triple}, (a, b, c))$ and $Z = (\texttt{single}, (*))$ be auxiliary objects. Consider two paths from X to X:

(1) p_1: through Y, with $u_1 : X \to Y$ mapping $(0, 0)$ and $d_1 : Y \to X$ mapping $(1, 0, 0)$; composite $[\![p_1]\!]$ has tuple $\big(d_1(0), d_1(0)\big) = (1, 1)$.
(2) p_2: through Z, with $u_2 : X \to Z$ mapping $(0, 0)$ and $d_2 : Z \to X$ mapping (1); composite $[\![p_2]\!]$ has tuple $(1, 1)$.

The two paths are unequal as paths — different lengths would already suffice, but here even the lengths agree and the arrow sequences differ — while their composites are extensionally equal: both denote the constant map at index 1. Every question about "what happened along the way" (which intermediate object, what fan-out at the middle stage) distinguishes p_1 from p_2; no question about the resulting total function does.

The example is elementary, but its consequence for system design is not: any pipeline audit that stores only composites has irreversibly discarded the factorization, and any tool that needs the factorization — tracing, staging analysis, per-step attribution — must consume paths. The kernel therefore treats "compute the composite" as a lossy, explicit, caller-initiated operation, never an implicit normalization. This is the same design posture as the name-inclusive object identity of Chapter **??**: information declared by the caller is never silently collapsed, and every collapse is an auditable call site.

REMARK 6.11 (Paths and the normal-form machinery). The typed words of the normal-form chapter are close cousins of paths — sequences of generator labels with matching types — and the closure algorithm there is, in these terms, a systematic exploration of all paths over a declared generator catalog, quotiented by extensional equality of composites and represented by shortlex-least witnesses. The forgetting phenomenon of Example 6.10 is exactly what the closure's denotation table records: many words, one denotation, one canonical representative. The present chapter's machinery is the untabulated substrate of that construction.

3. The Invalid Inference

We now name the mistake the separation exists to block, in its general form, before dissecting three concrete instances.

Adverse Finding 6.12 (The sequence-to-power fallacy). Let $g_1, g_2, \ldots, g_n$ be a sequence of operations applied in temporal order. The *sequence-to-power fallacy* is any inference of the form: "the system after n steps is g^n for some single operation g; therefore conclusions about powers of a single finite endomap — eventual periodicity, the factorial idempotence of Theorem 5.23, existence of a unique idempotent in the generated subsemigroup (Proposition 5.17) — apply to the n-step system." The inference requires, and the informal notation silently assumes, three separate facts: (i) all g_t act on one and the same object; (ii) all g_t are equal to a single g; and (iii) that g is an endomap. Each can fail independently, and the kernel is engineered so that whichever one fails first does so as a construction- time type error or as an operation that does not exist, rather than as a false theorem silently applied.

The taxonomy is worth fixing in tabular form before the case studies, because the three conditions fail at three different kernel surfaces, and keeping the correspondence straight is what makes the studies transferable to new instances:

Condition assumed	Fails in study	Kernel surface that objects	Failure form
(i) one object throughout	(a)	composition / endomap certification	typed error
(ii) one generator repeated	(b)	no root-extraction operation exists	API absence
(iii) a certified endomap	(c)	endomap constructor; no path power	typed error / absence

Note the two distinct failure forms. Conditions that assert a false *equality of objects* fail loudly, as typed errors at a construction site — there is a place in the program where the identification is attempted, and that place raises. The condition that asserts a false *existence* (of a single generator g with the right power) fails silently in a stronger sense: the kernel simply provides no operation whose use would presuppose it, so the assumption has nowhere to be written down. Both forms serve the same audit goal by different means, and the case studies illustrate the difference in texture.

The three case studies below realize the three failure points in order: (a) fails condition (i) — the state spaces change over time; (b) fails condition (ii) — the objects agree and the composite is even an endomap, but no single generator reproduces it; (c) attempts to import Theorem 5.23 wholesale into a pathful setting and founders on (iii) at the endomap certification boundary. In each study we give the setup, the tempting informal claim, and then — the load-bearing part — the exact kernel step at which the formalization of the claim becomes unwritable, with the specific error and the specific construction that raises it.

4. Three Case Studies

4.1. Case study (a): a time-varying workflow across changing state spaces.

Example 6.13 (Setup). A three-stage pipeline processes items whose representation changes at each stage. Declare three objects:

$$S_0 = (\texttt{intake}, (\texttt{raw_a}, \texttt{raw_b}, \texttt{raw_c})),$$
$$S_1 = (\texttt{screened}, (\texttt{keep}, \texttt{drop})),$$
$$S_2 = (\texttt{archived}, (\texttt{bin_0}, \texttt{bin_1}, \texttt{bin_2})),$$

and two processing arrows: $g_1 : S_0 \to S_1$ with mapping $(0, 0, 1)$ (items `raw_a` and `raw_b` are kept, `raw_c` dropped), and $g_2 : S_1 \to S_2$ with mapping $(2, 0)$ (kept items to `bin_2`, dropped items to `bin_0`). The path $p = (g_1, g_2)$ is well formed — the single adjacency check compares S_1 against S_1 and passes — and its composite is $[\![p]\!] = g_2 \circ g_1 : S_0 \to S_2$ with mapping $(2, 2, 0)$.

The tempting claim. In an untyped operator notation, one writes the pipeline as a single operator T and reasons: "running the pipeline repeatedly, the system must stabilize; indeed T^k is eventually idempotent by finiteness" — an appeal to exactly the mathematics of Chapter 5, with T playing the role of f.

Where the kernel stops it. The formalization must produce the object on which T iterates, and there is none. Concretely, each of the three available moves fails at a stated site:

(1) *Iterate the path.* The expression "p^2" does not typecheck against any operation: the path type has no power (Rule 6.3). This failure is an absence, not an exception — the program cannot be written.

(2) *Certify the composite as an endomap.* The expression Endomap($[\![p]\!]$) raises the typed composition error at the certification check of Definition 5.1: the composite has declared source $S_0 = \texttt{intake}$ and declared target $S_2 = \texttt{archived}$, which are unequal as objects (they differ in name *and* carrier — sizes 3 and 3, but different element sequences; and even equal carriers would not save it, by Rule 5.2). The error message carries both declared types, so the audit trail shows exactly which identification the informal argument assumed and never justified.

(3) *Compose the pipeline with itself.* The expression $[\![p]\!] \circ [\![p]\!]$ — or equivalently, appending g_1 to the path p to begin a second lap — raises the composition error of Invariant 4.19: the would-be middle objects are S_2 (target of the first lap) and S_0 (source of the second), and $S_2 \neq S_0$. The failed adjacency is the precise formal location of the informal argument's gap: "run it again" presupposes an arrow $S_2 \to S_0$ returning archived items to the intake representation, and no such arrow was ever declared.

The repair, if intended. If the modeler genuinely intends re-intake, the kernel does not forbid it — it demands that the intent be an artifact. Declare a re-intake arrow $r : S_2 \to S_0$ (choosing, and thereby documenting, what each bin becomes on re-entry); then $r \circ [\![p]\!]$ is an endomap on S_0, Endomap($r \circ [\![p]\!]$) certifies, and Theorem 5.23 applies to *that* arrow, with the conclusion depending — as it must — on the declared content of r. The fallacy was never "iteration is impossible"; it was "iteration for free, with the return map left implicit and unexamined."

4.2. Case study (b): a heterogeneous composite that happens to be endo, confused with a generator's power. The second failure point is subtler: the types all line up, the composite genuinely is an endomap, and the fallacy is the identification of that composite with a *power of a single generator* — an equality claim between arrows, false in general, whose falsity the kernel makes checkable rather than assumable.

Example 6.14 (Setup). Declare $X = (\texttt{register}, (0, 1))$ and $Y = (\texttt{buffer}, (\texttt{lo}, \texttt{mid}, \texttt{hi}))$, with arrows $h_1 : X \to Y$, mapping $(2, 0)$ (register 0 to `hi`, register 1 to `lo`), and $h_2 : Y \to X$, mapping $(0, 1, 1)$ (`lo` to register 0; `mid` and `hi` to register 1). The path $q = (h_1, h_2)$ is well formed, and its composite $c = [\![q]\!] = h_2 \circ h_1 : X \to X$ computes to

$$c(0) = h_2\big(h_1(0)\big) = h_2(2) = 1, \qquad c(1) = h_2\big(h_1(1)\big) = h_2(0) = 0 :$$

the composite is the *transposition* $\tau : X \to X$ with mapping $(1, 0)$. Since source and target are both X, Endomap(τ) certifies without complaint. Everything so far is legitimate.

The tempting claim. "The round trip through the buffer is a two-step process; so the system state after the round trip is g^2 for the underlying step operation g, and therefore" —

here the fallacy cashes out — "it inherits the structural properties of squares: for instance, its cycle structure is constrained to that of a square of an endomap on two points."

Why the claim is false, concretely. On the two-point carrier there are exactly $2^2 = 4$ endomaps: the identity id (mapping $(0,1)$), the transposition τ (mapping $(1,0)$), and the two constants c_0 (mapping $(0,0)$) and c_1 (mapping $(1,1)$). Their squares are

$$\mathrm{id}^2 = \mathrm{id}, \qquad \tau^2 = \mathrm{id}, \qquad c_0^2 = c_0, \qquad c_1^2 = c_1.$$

The set of squares is therefore $\{\mathrm{id}, c_0, c_1\}$, and τ — the actual round-trip composite of Example 6.14 — *is not the square of any endomap on* X. The two-step round trip through a three-point buffer realizes a map that no doubled one-step endo-process on the register can realize. The informal claim is not merely unproved; it is refuted by a four-case enumeration.

Where the kernel stops it. The fallacy here is an arrow equality — $c = g^2$ for some endomap g — and equalities between arrows are exactly what the kernel makes decidable and cheap: arrow equality is extensional tuple comparison (Definition 4.2). The claim "c is a square" becomes a finite search over the $|X|^{|X|}$ candidate generators, each tested by one composition and one comparison, and it returns false. What the kernel refuses to supply is any construct that would let the claim be *assumed*: the path q is not an endomap and has no power; the certified endomap Endomap(c) has powers, but its powers are powers *of* c, the audited composite — the API affords c^2, and nothing affords "the g such that $g^2 = c$" because no such g need exist. In the type-theoretic reading: the kernel provides introduction forms for powers (build a certified endomap, then exponentiate) but no elimination form that extracts roots, since root extraction is a nontrivial, often unsatisfiable search problem, not a structural operation.

The general lesson. A factorization through foreign objects can realize endomaps unreachable as powers of any single same-object generator. The composite operation deliberately *loses* the factorization (Section 1.2); the path retains it; and no kernel operation manufactures a fictitious single-generator history for a composite. When downstream analysis needs the history, it must consume the path, which is the artifact that actually has one.

4.3. Case study (c): importing the idempotence theorem into a pathful setting. The third study takes the fallacy at its most seductive: a direct citation of Theorem 5.23 against a pipeline, with the carrier sizes even agreeing, so that every cardinality-based sanity check passes and only the typed discipline objects.

Example 6.15 (Setup). A processing loop alternates between two representations of a three-element state space that are, by design, carried by *equal* ordered carriers under different declared names:

$$W = (\texttt{working}, (s_0, s_1, s_2)),$$
$$W' = (\texttt{checkpoint}, (s_0, s_1, s_2)),$$

with a checkpoint arrow $u : W \to W'$, mapping $(0,2,2)$, and a resume arrow $v : W' \to W$, mapping $(1,1,0)$. The four-arrow path $m = (u, v, u, v)$ is well formed (adjacencies alternate W' against W' and W against W), and its composite $[\![m]\!] : W \to W$ is a certified endomap after wrapping. So far, again, everything is legitimate — and note that this loop, unlike case (a), *does* close.

The tempting claim. "Each pass of the loop is one step; by the factorial idempotence theorem, after $3! = 6$ steps the process is idempotent — so the six-step behavior is a stable projection, and we may reason about 'the eventual state' of the pipeline." The citation then

gets applied not to $[\![m]\!]$ but to the *stepwise* description: the reasoner takes "one step" to mean "one arrow of the path" and computes with u and v as if each were an iterate of a single f.

Where the kernel stops it — three distinct sites. Formalizing the claim forces a choice of which arrow the theorem is about, and every choice that matches the informal wording fails at a typed boundary:

(1) *The theorem applied to u.* Endomap(u) raises the typed error: u has declared source `working` and declared target `checkpoint`, unequal as objects *despite equal ordered carriers* — this is Rule 5.2, and it is the exact refusal that a carrier-only or cardinality-only identity discipline would have waved through. Under such a laxer discipline, u would certify as an "endomap" on the shared carrier, u^6 would compute, and the resulting arrow would be an artifact of an identification nobody declared: the checkpoint-indexed values would be silently re-read as working-indexed values on every second application. The kernel's name-inclusive identity makes this precise failure unconstructible. The same applies symmetrically to v.

(2) *The theorem applied to the path.* "After six steps" invites $m^?$ or a six-arrow truncation reasoning; the path has no power (Rule 6.3), and building the six-step object explicitly means writing the six arrows down, at which point the alternation — u, v, u, v, u, v — is visible and the question "six steps of *what single map*?" answers itself: of none.

(3) *The theorem applied to the composite.* This is the one legitimate reading, and the kernel supports it exactly: $e = \text{Endomap}([\![m]\!])$ certifies (the composite is $W \to W$), the safe power $e^{3!} = e^6$ computes, and Theorem 5.23 guarantees its idempotence. But now the claim has changed meaning, and the change is the entire point: the theorem speaks about iterating *the whole closed loop* $[\![m]\!]$, i.e. about the 24-arrow process "four-arrow loop, six times," not about six arrows of the original path, and not about any property of u or v singly. The idempotence exponent is governed by the carrier of the *composite's* object ($|W| = 3$, so $3! = 6$ loop iterations), a quantity with no relation to the path's length. The kernel forces the reasoner to land on this reading because it is the only one that constructs.

The residue. What survives formalization is a true, precise, and noticeably weaker statement than the informal one: the closed loop's composite, iterated 6 times, is idempotent as a map on `working`. What died in formalization is everything the informal statement smuggled: that individual passes are iterates of one operation, that the number "six steps" refers to path arrows, and that stabilization happens *within* a single traversal. The kernel's contribution is that the smuggling has to clear customs: each assumption either becomes a declared arrow and a certified endomap, or it becomes a typed error with a named construction site.

5. Type-Level Enforcement

The case studies leaned on two enforcement mechanisms; this section states them as contracts, parallel to the mismatch-rejection contract of Chapter 4.

Invariant 6.16 (Distinct runtime types, no coercion). The endomap wrapper and the path are distinct runtime types with disjoint capabilities: the endomap type carries `power` and the idempotence predicate and wraps exactly one arrow with certified equal source and target; the path type carries adjacency-checked construction, `composite`, and `append`, and wraps a nonempty arrow sequence. Neither type is a subtype of the other; no implicit

conversion exists in either direction; and each constructor validates its own invariant and raises on violation:

(1) passing a path (or any non-arrow value) to the endomap constructor fails its arrow-type validation;
(2) passing an arrow with unequal declared source and target to the endomap constructor raises the typed composition error naming the arrow and its declared types;
(3) passing an arrow sequence with any failed adjacency to the path constructor raises the typed composition error naming the offending segment;
(4) the sanctioned bridge between the types is explicit and one-way: take the path's `composite` (an ordinary arrow), then certify it with the endomap constructor, which re-checks source-target equality from scratch. There is no bridge that bypasses the check.

Check Class 6.17 (Negative fixtures for the separation). The executable suite pins each clause of Invariant 6.16: construction of an endomap from a non-endo arrow (expected: typed composition error); construction of an endomap from a source-target pair with equal carriers and distinct names (expected: typed composition error — the case-study (c) fixture); construction of a path with one broken adjacency (expected: typed composition error identifying the segment); construction of a path from an empty sequence and from unordered containers (expected: validation error); and the positive control, certifying a closed path's composite and taking its factorial power (expected: success, with the idempotence assertion of Chapter 5 holding). The positive control matters: a separation that blocked the legitimate bridge as well as the illegitimate shortcuts would be mere obstruction, and the suite demonstrates that the sanctioned route stays open.

Remark 6.18 (Why not one type with a flag). A recurring design alternative deserves an explicit rejection note: a single "process" type carrying a sequence of arrows and a boolean `is_closed`, with `power` raising at runtime when the flag is false. This is behaviorally similar and structurally worse. The flagged design makes "path with a power method that sometimes works" the ambient shape, so every call site inherits a latent partiality, and the error surfaces at the *use* site (possibly deep in a computation, possibly only on certain inputs) rather than at the *construction* site where the modeling decision was made. The two-type design moves the failure to the earliest possible moment, makes the capability difference legible in signatures — a function that requires iterability asks for an endomap in its type, and cannot be handed a path at all — and lets the host language's checker do statically what the flag design defers to runtime. The kernel consistently prefers unrepresentable states to checked-at-use states, and this is the flagship instance.

5.1. Stronger and weaker enforcement points on the design spectrum. The two-type design sits at a particular point on a spectrum of enforcement strength, and honesty about the spectrum clarifies both what the kernel achieves and what it delegates.

Stronger: statically indexed types. In a host language with dependent or parametrically indexed types, the arrow type itself can carry its source and target as type indices — an arrow is a value of type Arrow $X\,Y$ — so that an ill-typed composition or an endomap claim about a non-endo arrow is rejected *at compile time*, before any program runs. The Lean model of the kernel lives at exactly this strength: its composition operation demands a definitional match of the middle object, and the invalid inferences of Section 4 are not runtime errors there but terms that fail to elaborate. The Python implementation cannot reach this point

— the host's type system does not track value-level indices — so it realizes the discipline at the next point down.

The kernel's point: certified wrappers with construction-time checks. Every constructor validates its invariant once, eagerly, and the resulting value is immutable, so possession of a value of the type is possession of a checked certificate. The residual gap versus static indexing is timing (violations surface when the offending construction executes, not when it is written) — the gap is not soundness: no sequence of kernel API calls yields an uncertified endomap or an unvalidated path, because the constructors are the only way in and the values are frozen.

Weaker, rejected: conventions and documentation. One step further down, the discipline lives in docstrings ("callers should not iterate heterogeneous pipelines") and code review. The failure data motivating this kernel — the red-team findings summarized in Chapter **??** — came from systems at this level, where the sequence-to-power fallacy is not blocked but merely deprecated. The kernel's position is that a discipline worth stating is worth enforcing at the strongest point the host allows, and that the distance between "deprecated" and "unconstructible" is where silent wrongness lives.

The ledger records the enforcement claim at its actual strength: obligation PO-TKS8-CORE-03 ("keep one-carrier powers distinct from heterogeneous paths") carries status EXECUTABLE VALIDATION — separate runtime types plus the negative fixtures of Checkclass 6.17 — not a formal-verification status, because the statement "no API sequence defeats the separation" is an informal argument about the module's construction surface, checked by inspection and by tests, not by proof. The Lean model's static enforcement is verified in Lean's terms, but transferring that assurance to the Python artifact is part of the open equivalence obligation discussed in Chapter 4.

6. Boundary: What Paths Do Not Claim

CLAIM BOUNDARY 6.19. A semigroupoid path is a bookkeeping object: a validated, ordered record of typed arrows with a computable composite. It carries *no* dynamics claim — nothing about time, causation, convergence, stability, or the behavior of any process the path might be used to describe — and *no* semantics claim: the kernel assigns no CAS, Noetic, cognitive, or workflow meaning to any object, arrow, or path, and the "workflow" and "pipeline" vocabulary of this chapter's case studies is expository narration about hypothetical modelers, not an asserted interpretation. Mapping any canonical TKS/CAS construct onto kernel arrows or paths remains an open, unlicensed obligation (ledger PO-TKS8-CORE-08, OPEN / NOT LICENSED); nothing in this chapter advances it. What this chapter establishes is strictly structural: which compositions exist, which iterations are licensed, and precisely where the unlicensed inferences fail to construct.

CLAIM BOUNDARY 6.20. The case-study analysis establishes that the three formalization attempts fail at the stated kernel boundaries, as pinned by the negative fixtures of Checkclass 6.17. It does not establish that the kernel blocks every conceivable encoding of the sequence-to-power fallacy: a determined user can always model around a type system — for example, by coercing everything into one giant declared object, at which point the kernel's theorems apply honestly to that declared model and the modeling choice itself is the auditable artifact. The claim is not that fallacies are impossible; it is that this fallacy cannot be committed *silently* within the kernel's term language: every route to it requires constructing an explicit identification (a return arrow, a declared shared object) that then appears in the audit trail.

7. Exercises

EXERCISE 6.21. For a path of n arrows over objects of size at most s, count the object-equality comparisons the constructor performs and the tuple operations the composite performs, and conclude that path validation is $O(n)$ object comparisons and evaluation is $O(ns)$ index operations. Explain why validation cannot be deferred to evaluation time without weakening the error contract (hint: a path may be constructed, stored, and audited without ever being evaluated).

EXERCISE 6.22. Call a path *closed* if its source equals its target. Over the single object X with $|X| = 2$ and the full arrow set $\mathrm{Hom}(X, X)$ as available arrows, count the closed paths of length exactly 3 and the distinct composites they denote, and verify that the denotation count is bounded by $|X|^{|X|} = 4$ while the path count is $4^3 = 64$ — the forgetting ratio of Example 6.10 made quantitative.

EXERCISE 6.23. Complete the enumeration sketched in Section 4.2: for $|X| = 2$, verify by the four-case table that the squares in $\mathrm{Hom}(X, X)$ are exactly $\{\mathrm{id}, c_0, c_1\}$, and then show that *every* endomap on two points arises as the composite of some two-arrow path through a three-point object. Conclude that on two points, factorization through a larger foreign object is strictly more expressive than squaring, which is the mathematical core of case study (b).

EXERCISE 6.24. In case study (a), suppose the modeler declares the re-intake arrow $r : S_2 \to S_0$ with mapping $(0, 1, 2)$ — bins map back to raw items positionally. Compute the endomap $e = r \circ [\![p]\!]$ on S_0, analyze its functional graph (tails, cycles, index, period) using the tools of Chapter 5, and find its minimal idempotent power. Then change r's mapping to $(2, 2, 2)$ and observe how the conclusion changes — demonstrating concretely that the iterated behavior depends on the declared content of the return map, which is exactly the datum the informal argument left implicit.

EXERCISE 6.25. The kernel excludes empty paths; the free category $\mathcal{F}(G)$ includes an empty path at each vertex [**2**]. Design the API of an empty-path-admitting variant: what extra datum must the constructor take, which operations change type, and where does Proposition 6.8 pick up side conditions? Weigh the gained uniformity (paths form a category, not a semicategory) against the cost (a partial function `source` becomes entangled with the extra datum), and compare your conclusion with the kernel's choice in Section 1.2.

8. Summary

Item	Status
Path construction and composite (Definitions 6.1, 6.2)	Well-definedness from the category laws (Theorem 4.14); constructor-validated adjacencies; executable validation.
Relation to free categories [**2**]	Standard background; paths are the nonempty morphisms of the free category on $\mathcal{C}$'s underlying graph; composite is the evaluation functor.
Absence of path powers (Rule 6.3)	API absence, enforced by distinct runtime types (Invariant 6.16); negative fixtures (Check-class 6.17); ledger PO-TKS8-CORE-03, EXECUTABLE VALIDATION.
Case studies (a)–(c) (Sections 4.1–4.3)	Each ill-typed step identified with its exact kernel error site; the two-point square enumeration in (b) is a complete four-case check.
Dynamics and semantics	None claimed (Boundary 6.19); semantic transfer remains OPEN / NOT LICENSED (PO-TKS8-CORE-08).

With the composition layer (Chapter 4), the licensed iteration theory (Chapter 5), and the unlicensed side's enforcement (this chapter) in place, the kernel's finite-function core is complete. The next chapter changes register entirely: it embeds the arrows into finite-dimensional Hilbert spaces via the deterministic standard-basis lift and proves exact operator-norm structure for the result.

CHAPTER 7

The Deterministic Standard-Basis Lift

1. Overview and evidential status

The preceding chapters built the typed finite-function category: named objects with explicitly ordered carriers, total arrows with declared source and target, partial type-checked composition, and the hard separation between endomaps (which carry monoid powers) and semigroupoid paths (which deliberately do not). This chapter develops the kernel's one piece of genuine linear algebra: the *deterministic standard-basis lift*, which assigns to every typed arrow $f : X \to Y$ a linear map A_f between finite-dimensional Hilbert spaces, acting on basis vectors by

$$A_f e_x = e_{f(x)}.$$

The lift is deliberately elementary. It introduces no probability, no dynamics, and no physical interpretation. Its purpose inside the kernel is threefold. First, it provides an *exactly analyzable* numerical shadow of every arrow: the matrix of A_f is a 0/1 matrix with exactly one 1 per column, every spectral quantity of interest is an integer or the square root of an integer, and the implementation in `src/tks_v8_core/deterministic_lift.py` computes all of them in exact integer arithmetic, converting to floating point only at the single property `operator_norm`. Second, it supplies a family of *permanent counterexamples* that block two tempting but false general claims about operator lifts of functions—claims that, if imported carelessly into downstream CAS reasoning, would license contractivity and projection arguments that are simply wrong. Third, its functoriality ($A_{\mathrm{id}_X} = I$, $A_{g\circ f} = A_g A_f$) gives an independent, exactly checkable consistency test of the categorical composition of Chapter 4: the executable suite verifies the functor laws by exact integer matrix equality on 1,678 composition pairs.

The evidential status of every claim in this chapter is stated where the claim is made, following the ledger discipline of the governing sources. In summary:

- The definition of the lift and its elementary structural facts (Section 3) are definitions and immediate computations.
- The exact operator-norm formula $\|A_f\|^2 = \max_{y\in Y} |f^{-1}(y)|$ (Theorem 7.25, ledger record THM-LIFT-NORM-FIBER-FORMULA) carries an *informal proof*, reproduced here in fully expanded form, together with bounded exhaustive structural tests. Its universal formalization in a proof assistant is *open* (obligation PO-TKS8-CORE-05): the pinned Lean seed kernel-checks only the exact two-state expansion of Section 8, not the general theorem.
- The corollaries (trace, rank, isometry) inherit the same status; they are short consequences of the norm analysis and of the fiber decomposition.
- The permanent counterexample (ledger record THM-LIFT-OCCUPATION-COUNTEREXAMPLE) is a finite, fully expanded computation; the Lean seed kernel-checks this exact two-state expansion and its noncontractivity.

- Functoriality (Theorem 7.46, obligation PO-TKS8-CORE-11) carries *partial formal verification*: the Lean seed kernel-checks the identity lift entrywise and on basis vectors, typed composition on basis vectors, and relational-product support composition; the finite-sum numerical matrix bridge and arbitrary-vector linearity remain open.
- No stochastic, quantum-channel, contractivity, dynamical, or empirical claim is made anywhere in this chapter; Section 11 is devoted to stating these non-claims precisely.

This chapter, like the whole monograph, is a Ring 4 draft artifact: nothing here is peer-reviewed, nothing is licensed for semantic transfer onto canonical TKS/CAS constructs, and the drafting of the chapter promotes no claim beyond what the underlying Gate 1 sources already record.

2. Finite-dimensional preliminaries

We collect the small amount of linear algebra the chapter needs, in the exact form in which the proofs use it. Everything in this section is standard; the canonical reference for the spectral background is Horn and Johnson [12]. We restate the facts with proofs where the proofs are short enough to keep the chapter self-contained, because the norm theorem below is a registered ledger item and we want its complete logical dependency chain visible on the page.

NOTATION 7.1 (Spaces and bases). For a finite object X with ordered carrier $(x_0, \dots, x_{|X|-1})$, let H_X denote the complex inner-product space $\mathbb{C}^{|X|}$ with the standard Hermitian inner product $\langle u, v\rangle = \sum_i \overline{u_i}\, v_i$ and with orthonormal basis $(e_{x_0}, \dots, e_{x_{|X|-1}})$ indexed by the ordered carrier; e_{x_i} is the i-th standard coordinate vector. The ordered-input discipline of Chapter 3 is what makes this indexing well defined: because the carrier is an explicitly ordered sequence, the correspondence between carrier elements and basis indices is a fixed, replayable bijection, not an artifact of interpreter hash order. When X is empty, H_X is the zero space.

DEFINITION 7.2 (Adjoint). For a linear map $A : H_X \to H_Y$, the *adjoint* $A^* : H_Y \to H_X$ is the unique linear map satisfying $\langle A^* w, u\rangle = \langle w, Au\rangle$ for all $u \in H_X$, $w \in H_Y$. In matrix terms, A^* is the conjugate transpose of A; for real (in particular 0/1) matrices, the plain transpose.

DEFINITION 7.3 (Operator norm). The *operator norm* of $A : H_X \to H_Y$ is

$$\|A\| = \sup\{\|Au\| : u \in H_X,\ \|u\| \le 1\} = \max\{\|Au\| : \|u\| = 1\}$$

when H_X is nonzero (the supremum is attained by compactness of the unit sphere in finite dimension), and $\|A\| = 0$ when H_X is the zero space. The second form is the one we use; the empty-domain convention matches the implementation, whose `max_fiber_size` takes `max(..., default=0)`.

LEMMA 7.4 (Norm via the Gram operator). *For any linear $A : H_X \to H_Y$ with H_X nonzero, $\|A\|^2 = \lambda_{\max}(A^*A)$, the largest eigenvalue of the positive semidefinite Hermitian operator A^*A.*

PROOF. For every unit vector u, $\|Au\|^2 = \langle Au, Au\rangle = \langle A^*Au, u\rangle$, so $\|A\|^2 = \max_{\|u\|=1}\langle A^*Au, u\rangle$. The operator $G = A^*A$ is Hermitian ($G^* = (A^*A)^* = A^*A = G$) and positive semidefinite ($\langle Gu, u\rangle = \|Au\|^2 \ge 0$). By the spectral theorem for Hermitian matrices [12], H_X admits an

orthonormal basis $(v_1, \dots, v_n)$ of eigenvectors of G with real eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$. Writing a unit vector as $u = \sum_k c_k v_k$ with $\sum_k |c_k|^2 = 1$,

$$\langle Gu, u \rangle = \sum_k \lambda_k |c_k|^2 \leq \lambda_1 \sum_k |c_k|^2 = \lambda_1,$$

with equality at $u = v_1$. Hence the maximum of the quadratic form on the unit sphere is exactly $\lambda_1 = \lambda_{\max}(G)$. □

LEMMA 7.5 (Spectrum is a similarity invariant). *If P is a permutation matrix and G any square matrix of matching size, then PGP^{-1} has the same characteristic polynomial, hence the same eigenvalues with the same algebraic multiplicities, as G. Moreover, when G is Hermitian, so is PGP^{-1}, because a real permutation matrix is unitary ($P^{-1} = P^T = P^*$).*

PROOF. $\det(tI - PGP^{-1}) = \det\big(P(tI - G)P^{-1}\big) = \det(P)\det(tI - G)\det(P^{-1}) = \det(tI - G)$. For the Hermitian statement, $(PGP^*)^* = PG^*P^* = PGP^*$. □

LEMMA 7.6 (Spectra of direct sums). *If $G = \bigoplus_j G_j$ is block diagonal, the spectrum of G is the multiset union of the spectra of the blocks G_j.*

PROOF. The characteristic polynomial of a block-diagonal matrix factors as the product of the characteristic polynomials of the blocks: $\det(tI - G) = \prod_j \det(tI_j - G_j)$. Eigenvalues and algebraic multiplicities therefore add up across blocks. □

These three lemmas are the entire spectral toolkit the norm theorem needs. One further standard notion appears in the counterexample analysis:

DEFINITION 7.7 (Projector, partial isometry). A linear operator Q on a Hilbert space is an *orthogonal projector* when $Q^2 = Q$ and $Q^* = Q$. A linear map $A : H_X \to H_Y$ is a *partial isometry* when $AA^*A = A$; equivalently, when A^*A is an orthogonal projector, equivalently when A restricts to an isometry on the orthogonal complement of its kernel [12]. An *isometry* is a map with $A^*A = I$, i.e. $\|Au\| = \|u\|$ for all u.

Note the direction of the traps already visible here: idempotence ($Q^2 = Q$) alone does *not* make an orthogonal projector—self-adjointness is a separate condition—and having 0/1 entries does not make a partial isometry. The counterexample of Section 8 lives exactly in this gap.

3. The lift: definition, matrix form, and running examples

3.1. Definition.

DEFINITION 7.8 (Deterministic standard-basis lift). Let $f : X \to Y$ be an arrow of the typed finite-function category: a total map sending each standard index of the ordered carrier of X to a standard index of the ordered carrier of Y. The *deterministic standard-basis lift* of f is the linear map

$$A_f : H_X \longrightarrow H_Y, \qquad A_f e_x = e_{f(x)} \quad \text{for every carrier element } x \text{ of } X,$$

extended to all of H_X by linearity: $A_f\big(\sum_x c_x e_x\big) = \sum_x c_x\, e_{f(x)}$.

The definition is well posed for exactly the reasons the kernel's object discipline was designed to secure. Because f is *total*, every basis vector of H_X has an image; because f lands in the declared target's standard indices, that image is a basis vector of H_Y; because carriers are *explicitly ordered*, the phrase "the i-th basis vector" has a fixed meaning independent of interpreter state; and because a linear map is determined by its values on a basis, A_f is unique.

Nothing else about f is used: in particular no injectivity, surjectivity, or endomorphism assumption. Every typed arrow lifts.

PROPOSITION 7.9 (Matrix form). *With respect to the standard bases, the matrix of A_f is the $|Y| \times |X|$ matrix with entries*

$$(A_f)_{yx} = \begin{cases} 1 & \text{if } f(x) = y, \\ 0 & \text{otherwise.} \end{cases}$$

Each column contains exactly one 1 *(in the row indexed by the image of that column's carrier element) and each column therefore sums to* 1*. Conversely, every $|Y| \times |X|$ matrix of* 0*s and* 1*s with exactly one* 1 *per column is the lift matrix of exactly one arrow $f : X \to Y$.*

PROOF. Column x of the matrix of A_f is the coordinate vector of $A_f e_x = e_{f(x)}$, which has a single 1 in row $f(x)$. For the converse, given such a matrix M, define $f(x)$ to be the unique row index with $M_{f(x),x} = 1$; this reads off a total map, and its lift matrix is M by the first part. The correspondence $f \mapsto A_f$ is therefore a bijection between arrows $X \to Y$ and one-1-per-column 0/1 matrices of shape $|Y| \times |X|$; in particular the lift is *faithful*: distinct arrows with the same source and target have distinct lifts. □

The implementation realizes Definition 7.8 as the frozen dataclass `DeterministicLift` wrapping a `FiniteArrow`; its method `matrix()` returns the tuple-of-tuples integer matrix of Proposition 7.9, its method `apply(vector)` computes A_f on an explicit ordered coordinate sequence by accumulating c_x into output slot $f(x)$, and `adjoint_apply(vector)` computes A_f^* on target coordinates by reading coordinate $f(x)$ back into slot x. Both methods enforce the ordered-input discipline: a set, mapping, string, or one-shot iterable in coordinate position raises `ValidationError`, for the same reason carriers do—coordinate order is basis order, and basis order is meaning.

REMARK 7.10 (The adjoint as pullback). The formula implemented by `adjoint_apply` deserves a second look, because it identifies the adjoint with something combinatorially natural. On basis vectors,

$$A_f^* e_y = \sum_{x \,:\, f(x) = y} e_x,$$

the indicator sum of the fiber $f^{-1}(y)$. Indeed $\langle A_f^* e_y, e_x \rangle = \langle e_y, A_f e_x \rangle = \langle e_y, e_{f(x)} \rangle$, which is 1 iff $f(x) = y$. On general coordinates, $(A_f^* w)_x = w_{f(x)}$: the adjoint is precomposition with f, i.e. the *pullback* of coordinate functions, while A_f itself is the *pushforward* of coordinates. The pushforward merges coordinates that share a fiber (adding them); the pullback duplicates a coordinate across its fiber. All four derived operators of Section 4 are compositions of one merge and one duplication in the two possible orders.

EXAMPLE 7.11 (Coordinates through the lift, numerically). To fix the mechanics of `apply` and `adjoint_apply`, push one concrete vector each way through the arrow $f : X_4 \to Y_3$ of the running quartet below ($f(p) = f(q) = a$, $f(r) = f(s) = b$). Take source coordinates (2, 3, 5, 7) in carrier order (p, q, r, s). Then

$$A_f(2, 3, 5, 7)^T = (2 + 3,\ 5 + 7,\ 0)^T = (5, 12, 0)^T :$$

the coordinates of p and q *merge additively* into slot a, those of r and s into slot b, and slot c — an empty fiber — receives nothing. In the other direction, take target coordinates $(10, 20, 30)$ in carrier order (a, b, c). Then

$$A_f^*(10, 20, 30)^T = (10,\ 10,\ 20,\ 20)^T :$$

each source slot *reads back* the coordinate of its image, so the value at a is duplicated across the fiber $\{p, q\}$ and the value at c is simply never consulted. Merge-by-addition and duplicate-by-fiber are the two primitive motions; every operator in this chapter is a composite of them, and every collision cost the norm theorem quantifies is visible already in the first computation, where the input mass $2+3$ arrived at a constructively. (Replacing $(2, 3)$ by $(1, 1)/\sqrt{2}$ exhibits the extremal vector of the norm theorem for this fiber.)

3.2. Running examples. Four arrows accompany us through the whole chapter. They are chosen to exhaust the qualitative possibilities: non-injective and non-surjective, injective and non-surjective, bijective, and maximally collapsing.

Example 7.12 (The running quartet). Fix three objects with the indicated ordered carriers:

$$X_4 = (\texttt{State4},\ (p, q, r, s)), \qquad X_2 = (\texttt{Pair},\ (u, v)), \qquad Y_3 = (\texttt{Obs3},\ (a, b, c)).$$

Define four arrows:

(R1) $f : X_4 \to Y_3$ with $f(p) = a$, $f(q) = a$, $f(r) = b$, $f(s) = b$. Fibers: $f^{-1}(a) = \{p, q\}$, $f^{-1}(b) = \{r, s\}$, $f^{-1}(c) = \emptyset$. Neither injective nor surjective.

(R2) $g : X_2 \to Y_3$ with $g(u) = c$, $g(v) = a$. Injective, not surjective.

(R3) $\pi : X_3 \to X_3$ on a three-element object $X_3 = (\texttt{Cycle3}, (0, 1, 2))$ with $\pi(0) = 1$, $\pi(1) = 2$, $\pi(2) = 0$. Bijective (a 3-cycle).

(R4) $c_0 : X_2' \to X_2'$ the constant map on a two-element object $X_2' = (\texttt{Bit}, (0, 1))$ with $c_0(0) = c_0(1) = 0$. Maximally collapsing; this is the permanent counterexample of Section 8.

Their lift matrices, rows indexed by target carrier order and columns by source carrier order, are

$$A_f = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad A_g = \begin{pmatrix} 0 & 1 \\ 0 & 0 \\ 1 & 0 \end{pmatrix}, \qquad A_\pi = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \qquad A_{c_0} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}.$$

Each matrix has exactly one 1 per column; the reader should check one entry of each against the defining rule $(A)_{yx} = 1$ iff the arrow sends x to y. Note how the empty fiber of f at c appears as an all-zero *row* (rows are free to be zero; columns are not), and how the bijectivity of π appears as "exactly one 1 per row" *in addition to* one per column—i.e. a permutation matrix.

Remark 7.13 (What the lift forgets and what it keeps). The lift keeps everything: by Proposition 7.9 it is a bijection onto its matrix class, so f is recoverable from A_f by reading each column. What changes is the *ambient theory*. As a function, f supports composition, powers (if endo), and fiber counting. As a matrix, A_f additionally supports addition, scalar action, adjoints, spectra, and norms—structure that has no preimage in the category of functions. The lift is thus an embedding of a small combinatorial world into a larger linear one, and every use the kernel makes of it takes the form: compute a linear invariant, then translate the answer back into fiber combinatorics. The norm theorem below is the paradigm case.

3.3. Degenerate shapes: empty carriers. The kernel admits empty carriers, and the lift theory must therefore say something coherent about them rather than excluding them by fiat; the executable evidence deliberately includes empty carriers in its 1,678 functor-law pairs, so the conventions here are load-bearing.

Empty source, nonempty target. If $|X| = 0$ there is exactly one arrow $f : X \to Y$ for each Y (the empty map: totality quantifies over an empty carrier and is vacuously satisfied). Its lift is the unique linear map from the zero space to H_Y, namely the zero map; its matrix has shape $|Y| \times 0$ — $|Y|$ rows and no columns — and the one-1-per-column rule is vacuously true. Every fiber is empty, the fiber maximum is 0 under the stated convention, and $\|A_f\| = 0$ consistently with Definition 7.3. The input Gram is the 0×0 matrix; the output Gram and the support projector are the $|Y| \times |Y|$ zero matrix; the trace-Gram is $0 = |X|$; the rank is $0 = |\mathrm{im} f|$. Every corollary of this chapter holds degenerately, which is exactly why the convention was chosen as it was.

Empty target. An arrow into an empty object exists only when the source is also empty: a total map must send each of the $|X|$ carrier elements somewhere, and there is nowhere. The typed constructor enforces this (there is no index to map to), so the case $|Y| = 0 < |X|$ simply does not arise, and no convention is needed for it. The one arrow $\emptyset \to \emptyset$ lifts to the identity of the zero space, which is simultaneously the zero map — the unique point where those two agree.

Is the empty-source lift injective? The implementation answers yes: `is_injective` checks that all fiber sizes are at most 1, which holds vacuously, and correspondingly `is_isometry` reports true. This is the mathematically standard reading (the empty map is injective; a zero-space map preserves all norms there are), but it produces the one place where a slogan of this chapter needs a footnote: "isometries have norm 1" holds only for nonempty sources, since the empty lift is an isometry of norm 0. Corollary 7.33 is stated with the nonemptiness hypothesis for precisely this reason. Degenerate cases do not threaten theorems; unexamined degenerate cases threaten slogans.

4. Three derived operators, kept distinct

From A_f the implementation derives three operators. Informal usage tends to conflate them—all three are "the projection-like thing associated to f"—and part of the kernel's discipline is to keep them apart as three distinct methods with three distinct semantics. We define each, compute all of them on the running examples, and record exactly when any two coincide.

4.1. The input-collision Gram matrix.

Definition 7.14 (Input Gram). The *input-collision Gram matrix* of f is $G_f = A_f^* A_f : H_X \to H_X$.

Proposition 7.15 (Gram entries). $(G_f)_{xx'} = 1$ *if* $f(x) = f(x')$ *and* 0 *otherwise. In particular every diagonal entry is* 1*, and* G_f *is the block-indicator matrix of the partition of the source carrier into fibers of* f*.*

Proof. $(G_f)_{xx'} = \langle e_x, A_f^* A_f e_{x'} \rangle = \langle A_f e_x, A_f e_{x'} \rangle = \langle e_{f(x)}, e_{f(x')} \rangle$, which is 1 iff $f(x) = f(x')$ by orthonormality of the target basis. Diagonal entries take $x = x'$. The relation "$f(x) = f(x')$" is an equivalence relation on the source carrier whose classes are exactly the nonempty fibers, so G_f is the 0/1 indicator of that partition. □

The name is chosen with care: G_f records *input collisions*. It is an operator on the *source* space, it sees only which inputs are merged, and it is insensitive to which target element they are merged into and to whether other target elements are hit at all. The implementation method is `input_gram_matrix()`.

Example 7.16 (Input Grams of the quartet).

$$G_f = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}, \qquad G_g = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad G_\pi = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad G_{c_0} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

For f, the two 2×2 all-ones blocks are the fibers $\{p, q\}$ and $\{r, s\}$; the carrier order happens to list each fiber contiguously, so the block structure is visible without reordering (in general it is not; the norm proof below handles the general case by an explicit permutation). For injective g and bijective π the Gram is the identity: no collisions. For the constant map the Gram is the all-ones matrix J_2: total collision.

4.2. The output Gram: fiber sizes on the diagonal.

Definition 7.17 (Output Gram). The *output Gram matrix* of f is $\widehat{G}_f = A_f A_f^* : H_Y \to H_Y$.

Proposition 7.18 (Output Gram entries). *$\widehat{G}_f$ is diagonal, with $(\widehat{G}_f)_{yy} = |f^{-1}(y)|$, the size of the fiber over y.*

Proof. $(\widehat{G}_f)_{yy'} = \langle e_y, A_f A_f^* e_{y'} \rangle = \langle A_f^* e_y, A_f^* e_{y'} \rangle$. By Remark 7.10, $A_f^* e_y$ is the indicator vector of the fiber $f^{-1}(y)$. Two distinct fibers are disjoint, so their indicator vectors are orthogonal, giving 0 off the diagonal; and the inner product of a fiber indicator with itself is the fiber's cardinality. □

So $\widehat{G}_f$ lives on the *target* space and counts *how many* inputs land on each output. Its diagonal is exactly the tuple computed by the implementation property `fiber_sizes` (an integer tuple of length $|Y|$, obtained by a single counting pass over the mapping), and the method `output_gram_matrix()` places those counts on a diagonal.

Example 7.19 (Output Grams of the quartet).

$$\widehat{G}_f = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \qquad \widehat{G}_g = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad \widehat{G}_\pi = I_3, \qquad \widehat{G}_{c_0} = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}.$$

The diagonals read off the fiber-size tuples $(2, 2, 0)$, $(1, 0, 1)$, $(1, 1, 1)$, and $(2, 0)$ respectively.

4.3. The codomain support projector.

Definition 7.20 (Support projector). The *codomain support projector* of f is the diagonal 0/1 matrix $S_f : H_Y \to H_Y$ with $(S_f)_{yy} = 1$ if $f^{-1}(y) \neq \emptyset$ and 0 otherwise.

Proposition 7.21. *S_f is an orthogonal projector, and its range is* $\mathrm{im} A_f = \mathrm{span}\{e_y : y \in \mathrm{im} f\}$.

Proof. A real diagonal matrix is self-adjoint, and a diagonal 0/1 matrix squares to itself, so S_f is an orthogonal projector. Its range is spanned by the basis vectors at which its diagonal is 1, i.e. $\{e_y : y \in \mathrm{im} f\}$. The columns of A_f are exactly the vectors $e_{f(x)}$, so $\mathrm{im} A_f$ is the span of $\{e_y : y \in \mathrm{im} f\}$ as well. □

The implementation method is `output_support_projector()`, computed by thresholding the fiber-size tuple at zero. Its docstring calls it "the orthogonal projector onto $\mathrm{im}(A_f)$ in the codomain," which Proposition 7.21 verifies.

EXAMPLE 7.22 (Support projectors of the quartet).

$$S_f = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \qquad S_g = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \qquad S_\pi = I_3, \qquad S_{c_0} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

4.4. When the operators coincide, and why the distinction is load-bearing. Comparing Examples 7.19 and 7.22: for g and π the output Gram and the support projector are *equal*; for f and c_0 they differ (the former has a 2 where the latter has a 1). This is the general pattern:

PROPOSITION 7.23 (Coincidence criterion). *$\widehat{G}_f = S_f$ if and only if every fiber of f has size at most 1, i.e. iff f is injective. Likewise $G_f = I_{H_X}$ iff f is injective, and G_f is an orthogonal projector iff its all-ones fiber blocks all have size 1, again iff f is injective. For non-injective f, all three derived operators are pairwise distinct objects: G_f is generally non-diagonal; $\widehat{G}_f$ is diagonal but not 0/1; S_f is diagonal and 0/1.*

PROOF. $\widehat{G}_f$ and S_f are both diagonal; they agree exactly when $|f^{-1}(y)| = \mathbf{1}[f^{-1}(y) \neq \emptyset]$ for every y, i.e. when no fiber has two or more elements. $G_f = I$ iff there are no off-diagonal collisions, i.e. no two distinct inputs share a fiber. For the projector claim about G_f: G_f is always self-adjoint (Proposition 7.15 makes it a real symmetric matrix), and by the block description G_f^2 has (x, x') entry equal to the size of the common fiber when $f(x) = f(x')$ and 0 otherwise (this is a special case of the computation $J_m^2 = mJ_m$ proved in Lemma 7.27 below); so $G_f^2 = G_f$ iff every nonempty fiber has size 1. □

CLAIM BOUNDARY 7.24 (Why three methods, not one). The conflation risk is concrete. In informal operator talk, all three of G_f, $\widehat{G}_f$, S_f get called "the projector associated with f," and for injective f the talk is harmless because $G_f = I$ and $\widehat{G}_f = S_f$. The moment a single collision appears, the three objects diverge, only S_f remains a projector, and any argument that silently substitutes one for another acquires a factor equal to a fiber size — which is precisely the quantity the norm theorem says governs the operator norm. The implementation therefore exposes them as three separately named methods (`input_gram_matrix`, `output_gram_matrix`, `output_support_projector`) and the test suite pins each to its own closed form. No claim of interchangeability is made or licensed anywhere in the kernel.

5. The exact norm theorem

We now prove the chapter's central result. Ledger status, restated for discipline: THM-LIFT-NORM-FIBER-FORMULA; informal proof present (this section *is* that proof, fully expanded); bounded exhaustive structural tests in the executable suite; universal formalization *open* — the pinned Lean seed kernel-checks only the two-state expansion of Section 8, not this theorem.

THEOREM 7.25 (Exact operator norm of a deterministic lift). *Let $f : X \to Y$ be a typed arrow with X nonempty. Then*

$$\|A_f\|^2 = \max_{y \in Y} \big|f^{-1}(y)\big|, \qquad \textit{hence} \qquad \|A_f\| = \sqrt{\max_{y \in Y} \big|f^{-1}(y)\big|}.$$

For empty X the norm is 0 by the convention of Definition 7.3, matching the formula if the empty maximum is read as 0.

The proof proceeds in four steps, each isolated as a lemma so that its logical role is visible: (1) reduce the norm to the spectrum of the input Gram; (2) reorder the source basis by a permutation similarity that groups fibers into contiguous blocks; (3) analyze the all-ones block J_m from scratch; (4) assemble the spectrum and take the maximum.

5.1. Step 1: reduction to the Gram spectrum. By Lemma 7.4, $\|A_f\|^2 = \lambda_{\max}(G_f)$ where $G_f = A_f^* A_f$ is the input-collision Gram matrix, whose entries Proposition 7.15 computed exactly: $(G_f)_{xx'} = \mathbf{1}[f(x) = f(x')]$. The problem is now purely spectral: find the largest eigenvalue of the 0/1 indicator matrix of the fiber partition.

5.2. Step 2: permutation similarity groups the fibers.

LEMMA 7.26 (Fiber-block form). *Let the nonempty fibers of f have sizes $m_1, \dots, m_r$ (so $r = |\mathrm{im} f|$ and $m_1 + \cdots + m_r = |X|$). There is a permutation matrix P on H_X such that*

$$P\, G_f\, P^* = J_{m_1} \oplus J_{m_2} \oplus \cdots \oplus J_{m_r},$$

a block-diagonal direct sum of all-ones blocks, where J_m denotes the $m \times m$ matrix every entry of which is 1.

PROOF. Choose any enumeration of the source carrier that lists the elements of the first nonempty fiber first (in any order), then the second fiber, and so on; formally, a bijection σ from $\{0, \dots, |X|-1\}$ to itself such that the σ-preimages of each fiber form a contiguous interval of indices. Let P be the permutation matrix of σ, i.e. $P e_{x_i} = e_{x_{\sigma(i)}}$ reindexed appropriately; P is real orthogonal, so $P^* = P^{-1}$. The (i, j) entry of PG_fP^* is the $(\sigma^{-1}(i), \sigma^{-1}(j))$ entry of G_f, which is 1 iff the carrier elements at positions $\sigma^{-1}(i)$ and $\sigma^{-1}(j)$ share a fiber. Under the chosen enumeration, positions share a fiber iff they lie in the same contiguous interval. Hence the conjugated matrix is 1 exactly on the r diagonal square blocks whose sizes are the fiber sizes, and 0 elsewhere: the stated direct sum. □

By Lemma 7.5, this conjugation changes neither the eigenvalues nor their multiplicities, so we may compute the spectrum of the direct sum instead of the spectrum of G_f.

It is worth pausing on why an explicit permutation is needed at all, when Example 7.16 already displayed a block-diagonal Gram. There, the declared carrier order happened to list each fiber contiguously. The kernel guarantees no such thing in general: carrier order is declared by the caller for identity and replay reasons (Chapter 3), not for the convenience of any particular arrow, and a single object is typically the source of many arrows whose fiber partitions are mutually incompatible with any one ordering. The permutation-similarity step is the precise bookkeeping that makes the "group the fibers" intuition into an argument valid for every arrow simultaneously, at the cost of one application of Lemma 7.5.

5.3. Step 3: the all-ones block, from scratch.

LEMMA 7.27 (Spectral analysis of J_m). *For $m \geq 1$, the $m \times m$ all-ones matrix J_m satisfies:*

1. *$J_m = \mathbf{1}\mathbf{1}^*$ where $\mathbf{1} = (1, \dots, 1)^T$; consequently $\mathrm{rank} J_m = 1$.*
2. *$J_m^2 = m J_m$.*
3. *The spectrum of J_m is: eigenvalue m with multiplicity 1 (eigenvector $\mathbf{1}$), and eigenvalue 0 with multiplicity $m - 1$ (eigenspace the hyperplane $\{v : \sum_i v_i = 0\}$).*

PROOF. (1) The outer product $\mathbf{1}\mathbf{1}^*$ has (i, j) entry $1 \cdot \overline{1} = 1$, which is J_m. Every column of J_m equals $\mathbf{1}$, so the column space is the line spanned by $\mathbf{1}$ and the rank is exactly 1 (it is not 0 since $J_m \neq 0$ for $m \geq 1$).

(2) Using (1) and associativity, $J_m^2 = \mathbf{1}(\mathbf{1}^*\mathbf{1})\mathbf{1}^* = \mathbf{1}\,(m)\,\mathbf{1}^* = mJ_m$, since $\mathbf{1}^*\mathbf{1} = \sum_{i=1}^{m} 1 = m$. (Entrywise, the (i,j) entry of J_m^2 is $\sum_{k=1}^{m} 1 \cdot 1 = m$.)

(3) First, $J_m\mathbf{1}$ has every entry $\sum_j 1 = m$, so $J_m\mathbf{1} = m\mathbf{1}$: the all-ones vector is an eigenvector with eigenvalue m. Second, if $\sum_i v_i = 0$ then every entry of $J_m v$ is $\sum_i v_i = 0$, so the sum-zero hyperplane, which has dimension $m-1$, lies in the kernel: eigenvalue 0 with geometric multiplicity at least $m-1$. These two eigenspaces are orthogonal ($\langle \mathbf{1}, v\rangle = \sum_i \overline{1}\, v_i = 0$ for sum-zero v) and their dimensions add to m, so together they exhaust $\mathbb{C}^m$; since J_m is Hermitian, algebraic and geometric multiplicities agree, and the spectrum is exactly $\{m\} \cup \{0 \text{ with multiplicity } m-1\}$. Alternatively and purely algebraically: by (2), every eigenvalue λ satisfies $\lambda^2 = m\lambda$, so $\lambda \in \{0, m\}$; the trace of J_m is m, and the trace is the eigenvalue sum, so m occurs exactly once and 0 fills the remaining $m-1$ slots. □

Note the two independent routes to the multiplicity count in part (3): the geometric route (explicit eigenvectors, orthogonal complement) and the algebraic route (minimal-polynomial constraint from $J_m^2 = mJ_m$ plus a trace count). The redundancy is intentional. The lemma is the computational heart of a registered ledger theorem, and the monograph's policy for such load-bearing steps is to give two proofs whenever two short proofs exist.

5.4. Step 4: assembling the spectrum.

Proof of Theorem 7.25. Let the nonempty fibers of f have sizes $m_1, \ldots, m_r$. By Lemma 7.4, $\|A_f\|^2 = \lambda_{\max}(G_f)$. By Lemma 7.26 and Lemma 7.5, G_f has the same spectrum as $J_{m_1} \oplus \cdots \oplus J_{m_r}$. By Lemma 7.6 and Lemma 7.27, that spectrum is the multiset

$$\{\, m_1, \ldots, m_r \,\} \;\cup\; \{\, 0 \text{ with multiplicity } \textstyle\sum_{j=1}^{r}(m_j - 1) = |X| - r \,\}.$$

Its maximum is $\max_j m_j = \max_{y \in Y} |f^{-1}(y)|$ (empty fibers contribute 0 to the maximum on the right and nothing to the block decomposition, so including them changes nothing when some fiber is nonempty, which holds because X is nonempty and f total). Hence $\|A_f\|^2 = \max_y |f^{-1}(y)|$, and taking square roots gives the norm itself. The empty-domain case: H_X is the zero space, every $A_f u$ with u in the zero space is 0, $\|A_f\| = 0$ by convention, and the fiber maximum over an empty source is 0 under the stated reading. □

Remark 7.28 (Full singular-value data). The proof delivers more than the norm: the complete singular-value multiset of A_f is $\{\sqrt{m_1}, \ldots, \sqrt{m_r}\}$ together with $|X| - r$ zeros, since the singular values are the square roots of the Gram eigenvalues [12]. Every spectral invariant of A_f that depends only on singular values is therefore an exact function of the fiber-size multiset. This is an expository consequence of the proof, not a separately registered ledger item; we state it because it explains *why* the implementation exposes `fiber_sizes` as the primitive property from which `max_fiber_size`, `operator_norm_squared`, `rank`, and the injectivity tests are all derived: the fiber-size tuple *is* the spectral data.

Example 7.29 (The theorem on the quartet). For f (fibers $2, 2, 0$): $\|A_f\|^2 = 2$, Gram spectrum $\{2, 2, 0, 0\}$ — and indeed G_f in Example 7.16 is $J_2 \oplus J_2$ already in block form, each block contributing eigenvalues $\{2, 0\}$. For g (fibers $1, 0, 1$): $\|A_g\|^2 = 1$; $G_g = I_2$ with spectrum $\{1, 1\}$; g is an isometry, as Corollary 7.33 will confirm from injectivity. For π: $\|A_\pi\|^2 = 1$; permutation matrices are unitary. For c_0 (fibers $2, 0$): $\|A_{c_0}\|^2 = 2$, so $\|A_{c_0}\| = \sqrt{2} > 1$ — the noncontractivity that Section 8 turns into a permanent fixture.

5.5. A second proof, avoiding the spectral theorem. The block-Gram proof above is the registered one, and its route through the spectral machinery is what makes the corollaries fall out for free. But the norm theorem itself admits a shorter, wholly elementary proof — two inequalities, one application of Cauchy–Schwarz, one witness vector — and recording it serves two purposes: it double-derives a load-bearing result, and it identifies the natural candidate route for the open Lean formalization, since it needs neither eigenvalues, nor permutation similarity, nor direct sums.

Second proof of Theorem 7.25. Write $M = \max_y |f^{-1}(y)|$ and let $u = \sum_x c_x e_x$ be arbitrary. Grouping the image coordinates by fiber,

$$\|A_f u\|^2 = \sum_{y \in Y} \Big| \sum_{x \in f^{-1}(y)} c_x \Big|^2 .$$

For each y with fiber size m_y, Cauchy–Schwarz applied to the m_y-term sum against the all-ones weight gives $\big|\sum_{x \in f^{-1}(y)} c_x\big|^2 \le m_y \sum_{x \in f^{-1}(y)} |c_x|^2$. Summing over y and using $m_y \le M$ together with the disjointness of fibers,

$$\|A_f u\|^2 \le \sum_y m_y \sum_{x \in f^{-1}(y)} |c_x|^2 \le M \sum_x |c_x|^2 = M \, \|u\|^2,$$

so $\|A_f\|^2 \le M$. For the reverse inequality, pick y^* with $|f^{-1}(y^*)| = M$ (possible since X is nonempty and f total) and let $u^* = \frac{1}{\sqrt{M}} \sum_{x \in f^{-1}(y^*)} e_x$, a unit vector. Then $A_f u^* = \frac{M}{\sqrt{M}} e_{y^*} = \sqrt{M}\, e_{y^*}$, of norm $\sqrt{M}$. Hence $\|A_f\|^2 \ge M$, and equality holds. □

Note where the equality conditions live: Cauchy–Schwarz is tight exactly on constant coefficient patterns within a fiber, which is why the extremal vector is the uniform fiber indicator — the same vector the spectral proof produces as the top eigenvector. The two proofs are not merely consistent; they locate the same extremal geometry by different means. A third, weaker route deserves a warning label: bounding $\lambda_{\max}(G_f)$ by the maximum row sum of G_f (a Gershgorin-type bound) also yields $\le M$, since each row of G_f contains exactly $m_{f(x)}$ ones — but a row-sum bound alone proves only the upper half, and quoting it as the theorem silently drops the attainment argument. The counterexample culture of this kernel applies to proofs too: half a proof with a confident name attached is how false lemmas enter ledgers.

Example 7.30 (The spectral decomposition, made explicit). For the running arrow f it is worth writing the eigenvectors down rather than merely counting them, because they carry the fiber combinatorics on their sleeves. The Gram $G_f = J_2 \oplus J_2$ (blocks on $\{p, q\}$ and $\{r, s\}$) has the orthogonal eigenbasis

$$v_1 = \tfrac{1}{\sqrt{2}}(1,1,0,0)^T, \quad v_2 = \tfrac{1}{\sqrt{2}}(0,0,1,1)^T, \quad v_3 = \tfrac{1}{\sqrt{2}}(1,-1,0,0)^T, \quad v_4 = \tfrac{1}{\sqrt{2}}(0,0,1,-1)^T,$$

with $G_f v_1 = 2v_1$, $G_f v_2 = 2v_2$, $G_f v_3 = 0$, $G_f v_4 = 0$. The eigenvalue-2 vectors are the *normalized fiber indicators*: uniform mass across a fiber is what the pushforward amplifies maximally ($A_f v_1 = \frac{1}{\sqrt{2}}(e_a + e_a) = \frac{2}{\sqrt{2}}\, e_a = \sqrt{2}\, e_a$, of norm $\sqrt{2}$). The kernel vectors are the *within-fiber differences*: mass patterns that cancel under merging ($A_f v_3 = \frac{1}{\sqrt{2}}(e_a - e_a) = 0$). The $|X| - r = 2$ dimensions of cancellation are exactly the information the non-injective f destroys, and the singular values $\sqrt{2}, \sqrt{2}$ on the surviving directions are exactly the amplification the norm theorem reports. Nothing about this picture is specific to f: for any arrow, the normalized indicator of each nonempty fiber is a singular vector with singular

value the square root of the fiber size, and the orthogonal complement inside each fiber block is killed.

The implementation computes `operator_norm_squared` as `max_fiber_size` — an exact integer, no linear algebra performed at runtime — and this identification is *justified* by Theorem 7.25; the executable suite's structural tests check the integer pipeline (fibers, maximum, squared norm) and cross-check derived quantities on bounded exhaustive families, but the identification of "maximum fiber size" with "operator norm squared" rests on the informal proof above until the open universal formalization closes it.

6. Corollaries with proofs

Three consequences are implemented as first-class properties of `DeterministicLift` and registered as implemented consequences of the norm analysis. We prove each from the structure already established.

COROLLARY 7.31 (Trace of the input Gram). $\mathrm{tr}(A_f^* A_f) = |X|$.

PROOF. Two proofs, agreeing as they must. *Diagonal count:* by Proposition 7.15 every diagonal entry of G_f is 1 (each input collides with itself), and there are $|X|$ of them. *Spectral count:* by the proof of Theorem 7.25 the eigenvalues of G_f are the fiber sizes $m_1, \ldots, m_r$ and zeros, and the trace is their sum $\sum_j m_j$, which is $|X|$ because f is total: the nonempty fibers partition the source. □

The implementation property `trace_input_gram` returns `len(arrow.source)` directly — the corollary is what entitles it to do so without forming any matrix.

COROLLARY 7.32 (Rank). $\mathrm{rank}(A_f) = |\mathrm{im}(f)|$, *the number of nonempty fibers.*

PROOF. The columns of A_f are the vectors $e_{f(x)}$ for x in the source carrier; the set of distinct columns is $\{e_y : y \in \mathrm{im} f\}$, which is orthonormal, hence linearly independent, and spans the column space. So the column space has dimension $|\mathrm{im} f|$. (Spectrally: the rank of A_f equals the number of nonzero singular values, which by Remark 7.28 is the number r of nonempty fibers, and $r = |\mathrm{im} f|$.) □

The implementation counts nonzero entries of `fiber_sizes`; Corollary 7.32 is the license for that shortcut.

COROLLARY 7.33 (Isometry criterion). *A_f is an isometry ($A_f^* A_f = I$) if and only if f is injective. Consequently $\|A_f\| = 1$ iff f is injective (for nonempty X), and $\|A_f\| > 1$ iff f has a collision.*

PROOF. By Proposition 7.15, $G_f = I$ iff all off-diagonal entries vanish (diagonal entries are automatically 1), iff no two distinct inputs share a fiber, iff f is injective. For the norm statement: if f is injective all fibers have size ≤ 1 and some fiber has size exactly 1 (nonempty source, total map), so the fiber maximum is 1 and $\|A_f\| = 1$ by Theorem 7.25; if f is not injective some fiber has size ≥ 2 and $\|A_f\| \geq \sqrt{2}$. □

COROLLARY 7.34 (No deterministic lift is a strict contraction). *For nonempty X, $\|A_f\| \geq 1$ always, with equality iff f is injective. There is no arrow whose lift has operator norm strictly below* 1.

PROOF. Immediate from Corollary 7.33: the fiber maximum of a total map on a nonempty source is at least 1. □

COROLLARY 7.35 (Partial-isometry criterion). *A_f is a partial isometry ($A_f A_f^* A_f = A_f$) if and only if f is injective — i.e., for total deterministic lifts, partial isometry and isometry coincide.*

PROOF. Compute on basis vectors, using Remark 7.10 and Proposition 7.18:

$$A_f A_f^* A_f\, e_x = A_f A_f^*\, e_{f(x)} = A_f \sum_{x' \,:\, f(x')=f(x)} e_{x'} = \left|f^{-1}(f(x))\right|\, e_{f(x)}.$$

This equals $A_f e_x = e_{f(x)}$ for all x iff every *inhabited* fiber has size exactly 1, iff f is injective. (Empty fibers impose no condition; but a total map's inhabited fibers cover the source.) The implementation's `is_partial_isometry` therefore returns the same Boolean as `is_injective`, and its docstring says exactly this: "for a total standard lift, equivalently whether f is injective." □

COROLLARY 7.36 (Frobenius comparison). *The Frobenius (Hilbert–Schmidt) norm of a lift satisfies $\|A_f\|_F^2 = |X|$, and the chain*

$$\|A_f\| \;\le\; \|A_f\|_F \;\le\; \sqrt{\operatorname{rank}(A_f)}\,\|A_f\|$$

holds with: left equality iff f has exactly one nonempty fiber (i.e. f is constant, or X is empty); right equality iff all nonempty fibers have equal size.

PROOF. $\|A_f\|_F^2$ is the sum of squared entries; the entries are 0 and 1 with exactly one 1 per column, so the sum is the number of columns, $|X|$. (Equivalently $\|A_f\|_F^2 = \operatorname{tr}(A_f^* A_f) = |X|$ by Corollary 7.31.) In fiber terms the chain reads

$$\max_j m_j \;\le\; \sum_j m_j \;\le\; r\, \max_j m_j$$

after squaring, with $m_1, \dots, m_r$ the nonempty fiber sizes: the middle quantity is $|X|$ and the outer ones are $\|A_f\|^2$ and $\operatorname{rank}(A_f)\|A_f\|^2$ by Theorem 7.25 and Corollary 7.32. A maximum equals the sum of positive terms iff there is one term; the sum equals r times the maximum iff all terms equal the maximum. □

The corollary is a useful sanity clamp in exact arithmetic: the integer identities $\|A_f\|_F^2 = |X|$ and $\|A_f\|^2 = \max_j m_j$ can be cross-checked against each other and against the rank without any floating point, and the equality cases are recognizable combinatorial conditions (constant map; equal-fiber map) rather than numerical coincidences.

REMARK 7.37 (Why Corollary 7.35 is not vacuous). For general matrices, "partial isometry" is strictly weaker than "isometry" (any orthogonal projector is a partial isometry but not an isometry unless it is the identity). The corollary says the deterministic lift class is too rigid to see the difference: totality forces every input to occupy some fiber, so the only way $A_f^* A_f$ can be a projector is to be the identity. This collapse is a genuine fact about the lift class, not a definitional triviality, and the counterexample below shows the other side: an idempotent lift that fails to be a partial isometry at all.

7. Composition, fibers, and submultiplicativity

The norm theorem concerns one arrow; this short section records how the norm interacts with composition, because the interaction is where the fiber picture earns its keep a second time. Everything here is an expository consequence of Theorem 7.25 and elementary fiber combinatorics; nothing is a separately registered ledger item.

PROPOSITION 7.38 (Fibers of a composite). *Let $f : X \to Y$ and $g : Y \to Z$ be composable typed arrows. For every z in the carrier of Z,*

$$(g\circ f)^{-1}(z) \;=\; \bigsqcup_{y\in g^{-1}(z)} f^{-1}(y) \qquad \textit{(a disjoint union), hence} \qquad \big|(g\circ f)^{-1}(z)\big| = \sum_{y\in g^{-1}(z)} \big|f^{-1}(y)\big|.$$

Consequently

$$\max_z \big|(g\circ f)^{-1}(z)\big| \;\le\; \Big(\max_y \big|f^{-1}(y)\big|\Big)\cdot\Big(\max_z \big|g^{-1}(z)\big|\Big).$$

PROOF. An input x satisfies $g(f(x)) = z$ iff $f(x)$ lies in $g^{-1}(z)$, which partitions the composite fiber by the intermediate value $f(x)$; the union is disjoint because f is a function. For the inequality: each summand is at most $\max_y |f^{-1}(y)|$, and the number of summands is $|g^{-1}(z)| \le \max_z |g^{-1}(z)|$. □

COROLLARY 7.39 (Submultiplicativity, with exact witnesses). $\|A_{g\circ f}\| \le \|A_g\|\,\|A_f\|$.

PROOF. Combine Proposition 7.38 with Theorem 7.25 on all three arrows; alternatively, apply the standard submultiplicativity of the operator norm [**12**] to $A_{g\circ f} = A_g A_f$ (Theorem 7.46). That the two routes agree is itself a small consistency check between the combinatorial and the operator-theoretic sides of the chapter. □

EXAMPLE 7.40 (Strictness). The inequality is frequently strict, and the fiber formula says exactly when: strictness occurs unless some single z simultaneously has a maximum-size g-fiber *all of whose* elements carry maximum-size f-fibers. Take $X_3' = (\texttt{Tri}, (t_0, t_1, t_2))$, $X_2' = (\texttt{Bit}, (0, 1))$, $X_1 = (\texttt{Pt}, (*))$; let $f : X_3' \to X_2'$ send $t_0, t_1 \mapsto 0$ and $t_2 \mapsto 1$ (max fiber 2), and let $g : X_2' \to X_1$ be the unique map (max fiber 2). The composite is the unique map $X_3' \to X_1$ with one fiber of size 3, and indeed

$$\sqrt{3} = \|A_{g\circ f}\| < \|A_g\|\,\|A_f\| = \sqrt{2}\cdot\sqrt{2} = 2 :$$

the bound $2 \cdot 2 = 4$ on the squared norm overcounts because g's big fiber contains one small f-fiber ($|f^{-1}(1)| = 1$), and the exact composite fiber size is the *sum* $2 + 1 = 3$, not the product. The sum formula of Proposition 7.38 is sharper than the product bound in every mixed case, and the implementation can afford the sharp value always: it simply computes the composite arrow and reads its fibers, never estimating.

PROPOSITION 7.41 (Normal lifts are exactly permutation lifts). *Let $f : X \to X$ be an endomap, so that A_f is a square matrix and normality ($A_f^* A_f = A_f A_f^*$) is meaningful. Then A_f is normal iff f is a bijection, in which case A_f is unitary.*

PROOF. By Propositions 7.15 and 7.18, $A_f^* A_f$ has unit diagonal while $A_f A_f^*$ is diagonal with the fiber sizes. If the two agree then $A_f^* A_f$ is a diagonal matrix with unit diagonal, i.e. the identity, so f is injective (Corollary 7.33); an injective endomap of a finite carrier is a bijection; and then A_f is a permutation matrix, which is unitary, hence normal. The converse inclusion (unitary implies normal) is immediate. □

REMARK 7.42 (Spectral radius versus norm). For any square matrix the spectral radius satisfies $\rho(A) \le \|A\|$, with equality for normal matrices [**12**]. The lift class realizes the gap concretely: the counterexample operator A_{c_0} of the next section is idempotent, so its eigenvalues lie in $\{0, 1\}$ and $\rho(A_{c_0}) = 1$, while $\|A_{c_0}\| = \sqrt{2}$. By Proposition 7.41 this gap is the *generic* situation for endomap lifts — any collision destroys normality — and it is one more reason no iteration-convergence story attaches to lifts: the "eigenvalues bounded by one" fact that powers such stories controls ρ, not $\|\cdot\|$, and controls nothing about transient

amplification, which is exactly what the norm measures and exactly what fiber collisions produce.

8. The permanent counterexample

Ledger record THM-LIFT-OCCUPATION-COUNTEREXAMPLE. This is the single most quoted fixture in the kernel's test suite, and the Lean seed kernel-checks this exact two-state expansion and its noncontractivity. Its role is prophylactic: it permanently blocks two general claims that are individually plausible, jointly tempting, and both false.

8.1. The fixture, verified property by property. Take the running arrow (R4): the constant map c_0 on the two-element object $X_2' = (\texttt{Bit}, (0,1))$, $c_0(0) = c_0(1) = 0$. Write $A = A_{c_0}$:

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \qquad A^* = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}.$$

We verify each claimed property by explicit computation.

(a) A is idempotent: $A^2 = A$.

$$A^2 = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1\cdot 1 + 1\cdot 0 & 1\cdot 1 + 1\cdot 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = A.$$

Categorically this mirrors $c_0 \circ c_0 = c_0$: a constant map composed with itself is itself, and by functoriality (Theorem 7.46 below) $A^2 = A_{c_0 \circ c_0} = A_{c_0} = A$. The implementation's `is_idempotent` checks the combinatorial form of this condition — the arrow is an endomap and every image point is a fixed point (`mapping[image] == image` for each image) — which is exactly $f \circ f = f$.

(b) $\|A\| = \sqrt{2}$. By Theorem 7.25: the fiber sizes are $(2, 0)$, the maximum is 2, the norm is $\sqrt{2}$. Directly, without the theorem: $A^*A = J_2$ (computed below), and J_2 has largest eigenvalue 2 by Lemma 7.27, so $\|A\|^2 = 2$ by Lemma 7.4. Still more directly: the unit vector $u = \frac{1}{\sqrt{2}}(1,1)^T$ has $Au = \frac{1}{\sqrt{2}}(2,0)^T$ with $\|Au\| = \sqrt{2}$, and no unit vector can do better because $\|A(u_1,u_2)^T\| = |u_1 + u_2| \le \sqrt{2}\,\|u\|$ by Cauchy–Schwarz. Three routes, one answer.

(c) $\mathrm{tr}(A^*A) = 2$.

$$A^*A = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = J_2, \qquad \mathrm{tr}(J_2) = 2 = |X_2'|,$$

confirming Corollary 7.31 on the fixture.

(d) $\mathrm{rank} A = 1$. The two columns of A are equal (both e_0), so the column space is the line through e_0: rank $1 = |\mathrm{im} c_0|$, confirming Corollary 7.32.

(e) A is not self-adjoint, hence not an orthogonal projector. $A \neq A^*$ by inspection: the $(1,2)$ entry of A is 1 while the $(1,2)$ entry of A^* is 0. Since an orthogonal projector must satisfy *both* $Q^2 = Q$ and $Q^* = Q$ (Definition 7.7), and A satisfies the first but not the second, A is idempotent without being a projector. (The implementation's `is_orthogonal_projector` returns true only for identity endomaps, which Proposition 7.23 and this computation jointly justify: a total deterministic lift on one declared carrier that is both idempotent and self-adjoint must have symmetric 0/1 matrix with one 1 per column fixing all image points — forcing the identity.)

(f) A is not a partial isometry.

$$AA^* = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, \qquad AA^*A = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 0 & 0 \end{pmatrix} = 2A \neq A.$$

So $AA^*A = 2A$, off by exactly the fiber size 2, as Corollary 7.35's basis computation predicts. Equivalently, $A^*A = J_2$ is not a projector ($J_2^2 = 2J_2 \neq J_2$).

For completeness, the three derived operators of Section 4 on the fixture: input Gram $G_{c_0} = J_2$; output Gram $\widehat{G}_{c_0} = \mathrm{diag}(2, 0)$; support projector $S_{c_0} = \mathrm{diag}(1, 0)$. All three are distinct, and only the third is a projector — the fixture separates the trio maximally.

8.2. The two false general claims the fixture blocks.

Claim Boundary 7.43 (Blocked claim 1: "deterministic lifts are contractive"). The claim would assert $\|A_f\| \leq 1$ for every arrow f. It is plausible because the matrix of A_f is a 0/1 matrix with unit column sums, and column-stochastic matrices are famously "non-expanding" in the folklore sense. It is false: $\|A_{c_0}\| = \sqrt{2}$, verified three ways in (b) above. The folklore intuition belongs to the ℓ^1 geometry of probability vectors, not to the ℓ^2 geometry of Hilbert space (Section 11.1 dissects this precisely). Any downstream argument that needs $\|A_f\| \leq 1$ must therefore prove injectivity of f first (Corollary 7.33); by Corollary 7.34 nothing better than equality with 1 is ever available.

Claim Boundary 7.44 (Blocked claim 2: "idempotent lifts are projectors"). The claim would assert that $A_f^2 = A_f$ implies A_f is an orthogonal projector. It is plausible because in many familiar operator classes (self-adjoint operators, normal operators) idempotence does imply projectorhood. It is false for lifts: A_{c_0} is idempotent, not self-adjoint, not a projector, and not even a partial isometry, by (a), (e), (f) above. Structurally: idempotent *arrows* $f \circ f = f$ abound (every constant map, every retraction onto a fixed-point set), and their lifts are idempotent by functoriality, but self-adjointness of a lift is an extremely rigid condition (symmetric 0/1 matrix, one 1 per column) that combinatorially forces the identity among endomaps. The implementation encodes the asymmetry as two distinct predicates — `is_idempotent` and `is_orthogonal_projector` — whose truth values differ on this fixture, permanently, in the regression suite.

Remark 7.45 (Why "permanent fixture" is the right category). The fixture is registered in the theorem ledger (thm-lift-occupation-counterexample) and kernel-checked in Lean, which is an unusual amount of ceremony for a 2×2 computation. The ceremony is the point. The two blocked claims are exactly the kind of plausible-sounding lemma that gets silently assumed in the middle of a long argument about "projection layers" or "contraction mappings" in cognitive-architecture prose. A counterexample that is registered, hashed, tested on every run, and formally verified cannot be argued away, forgotten, or weakened in a refactor without tripping an alarm. It converts a mathematical fact into an engineering invariant.

9. Functoriality

9.1. The theorem.

Theorem 7.46 (Lift functor laws). *For every object X, $A_{\mathrm{id}_X} = I_{H_X}$; and whenever the composite $g \circ f$ is defined in the typed category (declared target of f equals declared source of g, as objects with name and carrier),*

$$A_{g \circ f} = A_g \, A_f.$$

Consequently $A_{(-)}$ is a functor from the typed finite-function category to the category of finite-dimensional Hilbert spaces and linear maps, in the sense of [2]*: it assigns H_X to X and A_f to f, preserving identities and composition.*

PROOF. On basis vectors: $A_{\mathrm{id}_X} e_x = e_{\mathrm{id}_X(x)} = e_x = I_{H_X} e_x$ for every carrier element x, and

$$A_g A_f \, e_x = A_g \, e_{f(x)} = e_{g(f(x))} = e_{(g \circ f)(x)} = A_{g \circ f} \, e_x.$$

Two linear maps that agree on a basis agree everywhere, by linearity; and both sides are linear by construction. The typing side condition matters: $A_g e_{f(x)}$ is only meaningful because $f(x)$ is a standard index of the object that is *simultaneously* the declared target of f and the declared source of g — the same object, with the same name and the same ordered carrier, hence the same basis indexed the same way. If the kernel permitted cardinality-only coercion between distinct objects of equal carrier size, the displayed computation would silently identify two different bases and the middle equality would be unjustified. Functor preservation of longer composites follows by induction from the binary case together with associativity (Chapter 4). □

EXAMPLE 7.47 (Functoriality on the running examples). Let $h : X_2 \to X_4$ send $u \mapsto q$ and $v \mapsto s$ (injective), and recall $f : X_4 \to Y_3$ from (R1). The composite $f \circ h : X_2 \to Y_3$ sends $u \mapsto f(q) = a$ and $v \mapsto f(s) = b$. The three matrices:

$$A_h = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}, \qquad A_f = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad A_{f \circ h} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

Multiplying: row a of A_f dotted with column u of A_h is $1{\cdot}0+1{\cdot}1+0{\cdot}0+0{\cdot}0 = 1$; with column v: $1{\cdot}0+1{\cdot}0+0{\cdot}0+0{\cdot}1 = 0$. Row b: with u, 0; with v, 1. Row c: zeros. So $A_f A_h = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{smallmatrix}\right) = A_{f \circ h}$, an exact integer identity — the kind of identity the executable evidence checks wholesale.

REMARK 7.48 (Powers of endomaps). For an endomap $f : X \to X$, functoriality gives $A_{f^n} = A_f^n$ for all $n \geq 0$ by induction ($A_{f^0} = A_{\mathrm{id}_X} = I$). In particular the factorial-power idempotence theorem of Chapter 5 transports: $A_f^{|X|!}$ is an idempotent matrix, being the lift of the idempotent endomap $f^{|X|!}$. Note carefully what does *not* transport: by Boundary 7.44, that idempotent matrix is generally not a projector, and no convergence, mixing, or dynamical statement accompanies it. Semigroupoid paths, having no power operation by type (Chapter 6), induce no power identity here either: the lift of a path composite is the product of the lifts, and that is all.

REMARK 7.49 (Faithful, not full). The lift functor is *faithful* (Proposition 7.9: distinct parallel arrows have distinct matrices) but very far from *full*: almost no linear map $H_X \to H_Y$ is a lift. Any matrix with an entry outside $\{0, 1\}$ (say $2I$, or the averaging map $\frac{1}{2}J_2$), and any 0/1 matrix with a zero column or a two-1 column, lies outside the image. Fullness failure is a feature, not a defect: it is the formal statement that the linear world strictly extends the functional world, and it marks exactly where extension proposals — stochastic columns, superposed columns — would leave the licensed kernel and enter open research territory (Section 11). In the terminology of [**2**], the kernel uses the lift as a faithful comparison functor and never claims an equivalence of categories; no downstream argument may pull a linear-algebraic construction back along $A_{(-)}$ unless the construction demonstrably lands in the one-1-per-column image.

9.2. Executable evidence: the 1,678 pairs. The functor laws are the most heavily instrumented claim in the lift module. The executable suite verifies them by *exact matrix equality* — integer entries, tuple-of-tuples comparison, no floating point anywhere — for every

valid typed-map pair on carriers through size three: 1,678 composition pairs, including pairs involving empty carriers. Concretely, for each such pair (f, g) with matching intermediate object, the suite forms the integer matrix product of `matrix()` outputs and compares it entrywise with the `matrix()` of the composed arrow; identity lifts are compared against exact identity matrices.

Claim Boundary 7.50 (What the 1,678 pairs do and do not establish). Passing this family establishes the functor laws *on the covered finite family, for the running implementation, on the run in question.* It is bounded exhaustive testing in the small-scope tradition [15]: within the size bound, no case is sampled away. It is not a proof of the universal statement, not proof-assistant verification of the Python code, and it closes no ledger obligation. The universal statement's formal status is recorded next.

10. Formal verification scope: verified versus open

The monograph's policy is to state Lean scope exactly, per claim, with the same care used for the claims themselves. The formal seed is a pinned, isolated Lean 4.32.1 toolchain [16] whose verifier hardcodes the sole allowed standard axiom (`propext`), rejects project axiom declarations and `sorry`, checks source hashes, and audits exact theorem dependency reports (Chapter 11 describes the harness). For the material of this chapter:

Kernel-checked (verified scope).

- The identity lift, *entrywise*: the matrix of A_{id_X} equals the identity matrix entry by entry, and A_{id_X} fixes every basis vector.
- Typed composition *on basis vectors*: for composable f, g in the Lean model, $A_g A_f$ and $A_{g \circ f}$ agree on every basis vector.
- Relational-product *support composition*: the 0/1 support pattern of a composite behaves as the relational product of the supports.
- The exact two-state expansion of the permanent counterexample (Section 8) and its noncontractivity.

Open (not verified).

- The universal norm theorem (Theorem 7.25); obligation PO-TKS8-CORE-05 remains INFORMAL PROOF PRESENT / UNIVERSAL FORMALIZATION OPEN.
- The generic matrix bridge: the connection, in full generality, between the basis-vector definition of the lift and the numerical matrix, Gram, trace, and rank definitions — including finite-sum numerical matrix multiplication.
- Arbitrary-vector linearity: the extension of the verified basis-vector identities to all vectors of the space, in the Lean model.

Remark 7.51 (On the shape of the open norm formalization). It is worth recording, as an engineering observation rather than a commitment, why the universal norm theorem is the hard item on the open list and what the path of least resistance looks like. The registered block-Gram proof consumes three pieces of nontrivial formal infrastructure: basis permutation as a similarity, spectra of direct sums, and the spectral theorem for Hermitian matrices — each routine on paper, each a substantial formal dependency. The second proof of Section 5.5 consumes only finite sums, Cauchy–Schwarz, and one witness computation, all of which sit close to the finite-sum matrix bridge that is already an open item for the functor laws. A formalization that lands the bridge would therefore likely land the norm theorem by the two-inequality route in the same effort, while the spectral route would remain the source

of the corollaries' *spectral* phrasing only. None of this closes anything: PO-TKS8-CORE-05 remains open, and this remark is a map of the terrain, not a claim about it.

Obligation PO-TKS8-CORE-11 (lift preserves identities and composition) therefore carries the compound status PARTIAL FORMAL VERIFICATION PRESENT / FINITE-SUM MATRIX BRIDGE OPEN. Two remarks keep this scope honest. First, "basis vectors plus linearity" is a complete informal argument (Theorem 7.46) but the linearity step is exactly what is not yet formalized, so the Lean artifact verifies a strictly weaker statement than the theorem. Second, the equivalence between the Lean model and the Python implementation is itself an open obligation; a kernel-checked Lean lemma constrains the Lean model, not the running code.

11. What the lift is not

The lift's matrix shape invites three importations from neighboring theories — Markov chains, quantum information, and dynamical systems. All three are declined, explicitly and permanently. This section explains each refusal with enough mathematical detail that the boundary can be checked rather than merely asserted.

11.1. Not stochastic matrices. Every lift matrix has nonnegative entries and unit column sums, so as a *matrix* it satisfies the definition of a column-stochastic matrix — in fact it is the extreme, deterministic kind, in which each column is a point mass. The refusal is therefore not about the matrix class; it is about which *theory* of that matrix class is in force, and three separations do the work.

Different norm, different geometry. Stochastic-matrix theory lives on the probability simplex with the ℓ^1 norm, where every column-stochastic matrix M is nonexpanding on nonnegative vectors: $\|Mv\|_1 = \sum_y |\sum_x M_{yx} v_x| = \sum_x v_x \sum_y M_{yx} = \|v\|_1$ for $v \geq 0$ — mass is conserved. The lift theory of this chapter lives on Hilbert space with the ℓ^2 norm, where Theorem 7.25 gives $\|A_f\|_2 = \sqrt{\max_y m_y}$, which exceeds 1 the moment any fiber has two elements. Both statements are true of the same matrix; they answer different questions. The blocked claim of Boundary 7.43 is precisely the illegitimate transport of the first statement's conclusion into the second statement's setting.

Different morphism class. The deterministic lifts form a vanishingly thin subclass of stochastic matrices: a column such as $(\frac{1}{2}, \frac{1}{2}, 0)^T$ is column-stochastic but is not $e_{f(x)}$ for any f, so no genuinely randomizing kernel is a lift. Conversely, the kernel's category has no arrows whose lift would need fractional entries. The functor $A_{(-)}$ lands in stochastic matrices but is not surjective onto them, and no completion of the image to general stochastic matrices is defined, admitted, or licensed here; "stochastic kernels" appear on the excluded list of the governing specification and remain OPEN.

No probabilistic semantics. Even where the arithmetic coincides — A_f applied to a probability vector does compute the pushforward distribution of f — the kernel asserts no semantics of uncertainty, belief, frequency, or sampling. The coordinates that `apply` acts on are coordinates, full stop. Reading them as probabilities is a semantic transfer, and semantic transfers are exactly what the Ring 4 discipline gates behind separate review.

11.2. Not quantum channels. A quantum channel is a completely positive, trace-preserving (CPTP) map on density operators. There is a standard way to try to manufacture one from A_f: the conjugation map $\Phi_f(\rho) = A_f \, \rho \, A_f^*$. The map Φ_f is completely positive (it

has the single Kraus operator A_f), but it is trace-preserving only in the degenerate case:

$$\mathrm{tr}\big(A_f \rho A_f^*\big) = \mathrm{tr}\big(\rho\, A_f^* A_f\big) = \mathrm{tr}(\rho\, G_f),$$

which equals $\mathrm{tr}(\rho)$ for all ρ iff $G_f = I$ iff f is injective (Corollary 7.33). On the permanent fixture the failure is quantitative: with $\rho = \frac{1}{2}J_2$ (the density matrix of the uniform superposition state on two basis states),

$$\mathrm{tr}(\rho\, G_{c_0}) = \mathrm{tr}\big(\tfrac{1}{2}J_2 \cdot J_2\big) = \tfrac{1}{2}\mathrm{tr}(J_2^2) = \tfrac{1}{2}\mathrm{tr}(2J_2) = \tfrac{1}{2}\cdot 4 = 2 \neq 1.$$

A "channel" that doubles the trace of a state is no channel. One could repair trace preservation by renormalizing or by adding Kraus operators, but any such repair is a new mathematical object with new semantics, not the lift, and constructing it is not undertaken here: quantum channels sit on the specification's excluded list alongside stochastic kernels, and the noncontractivity fixture exists partly to keep the "lifts are like isometries, isometries are like channels" slide from ever starting. The only honest statement is the narrow one: for *injective* f, A_f is an isometry and Φ_f is a legitimate isometric embedding of state spaces; for anything else, no channel reading exists without modification.

11.3. No dynamics. Nothing in this chapter has a time parameter. The lift assigns one linear map to one arrow; it does not assert that anything *evolves* by repeated application of that map. Three specific non-claims:

No trajectory semantics. Iterating A_f for an endomap f computes lifts of iterates ($A_f^n = A_{f^n}$, Remark 7.48); it does not model a system's state at time n, because no system, state, or clock has been defined. The factorial-power idempotence result is a statement about the monoid structure of finite endomaps, not about equilibration.

No fixed-point or convergence claims. Spectral radius arguments ("iterate and converge to the dominant eigenspace") require normality or at least diagonalizability assumptions plus a semantics for convergence; the fixture A_{c_0} already shows lifts can be non-normal ($A^*A = J_2 \neq \mathrm{diag}(2,0) = AA^*$), and no convergence semantics is defined.

No cognitive-dynamics reading. The governing sources are explicit that mapping canonical Noetic or CAS constructs onto kernel arrows is an open, unlicensed obligation (PO-TKS8-CORE-08). A fortiori, reading A_f-iteration as "cognitive state evolution" is doubly out of scope: it would require both the semantic mapping and a dynamical interpretation, neither of which exists. The lift is bookkeeping — an exact, faithful, spectrally transparent ledger of which inputs an arrow merges and which outputs it reaches — and the entire content of this chapter is what can be proved about that ledger.

12. Summary and status table

The chapter's results, with their exact statuses:

Result	Where	Status
Lift well-defined; matrix bijection	Prop. 7.9	definition-level; immediate
Gram entries; fiber partition	Prop. 7.15	immediate computation
Output Gram diagonal, fiber sizes	Prop. 7.18	immediate computation
Support projector onto imA_f	Prop. 7.21	immediate computation
Trio coincidence iff injective	Prop. 7.23	immediate computation
Exact norm formula	Thm. 7.25	informal proof present; universal formalization open (PO-05)
Trace, rank, isometry corollaries	Cor. 7.31–7.33	consequences of the norm analysis
No strict contraction; partial-isometry collapse	Cor. 7.34, 7.35	consequences; expository registration
Permanent counterexample	Sec. 8	finite computation; Lean kernel-checked (exact two-state expansion)
Functor laws	Thm. 7.46	partial formal verification; matrix bridge and linearity open (PO-11); 1,678 exact-matrix pairs
Non-claims (stochastic, quantum, dynamics)	Sec. 11	permanent boundaries

EXERCISE 7.52. Let $h : X_5 \to Y_2$ be the arrow from a five-element carrier to a two-element carrier with fibers of sizes 4 and 1. Without forming any matrix, write down: $\|A_h\|$; the full spectrum of $A_h^* A_h$ with multiplicities; $\mathrm{tr}(A_h^* A_h)$; $\mathrm{rank}(A_h)$; the diagonal of $A_h A_h^*$; the support projector; and the truth values of `is_injective`, `is_isometry`, `is_partial_isometry`. Then check Corollary 7.36's chain numerically: $\sqrt{4} \le \sqrt{5} \le \sqrt{2} \cdot \sqrt{4}$, and identify which equality case fails and why.

EXERCISE 7.53. Prove that the matrix of A_f is doubly stochastic (all row sums and all column sums equal to 1) iff f is a bijection between carriers of equal size, and conclude via Proposition 7.41 that within the lift class "doubly stochastic," "unitary," "normal," and "permutation matrix" all coincide — a collapse of four generally distinct matrix classes into one, forced by the one-1-per-column shape.

EXERCISE 7.54. The counterexample A_{c_0} satisfies $A_{c_0} A_{c_0}^* A_{c_0} = 2A_{c_0}$. Show that in general, for an *idempotent* arrow $f = f \circ f$ whose image points all have fiber size k, one has $A_f A_f^* A_f = kA_f$ on the image summand, and use the constant map on an m-element carrier to exhibit $\|A_f\| = \sqrt{m}$ growing without bound over idempotent lifts. (Moral: not only are idempotent lifts not projectors; they are not even uniformly bounded.)

Everything above is Ring 4, draft, not licensed, with independent specialist review open (PO-TKS8-CORE-10). The next chapter turns from the linear shadow of single arrows to the combinatorics of arrow *words*: which compositions of a declared generator set produce which denotations, and how to name each denotation by a canonical representative that is stable, replayable, and honest about its resource limits.

CHAPTER 8

Denotational Normal Forms

1. The canonicalization problem

Chapters 4 and 5 equipped the kernel with typed composition and with powers where powers are legitimate. This chapter solves the remaining Gate 1 problem: *canonical names*. Given a finite set of declared generator arrows, many different composition words produce the same arrow. If two runs of an audit pipeline describe the same denotation by two different words — or worse, if the *same* run does so depending on interpreter hash order — then downstream digests, caches, and comparisons silently diverge. What is needed is a function NF_G that assigns to every reachable denotation one distinguished word, such that:

(1) the assignment is *deterministic*: it depends only on the declared catalog (objects, generators, and their declared orders), never on interpreter state;
(2) the assignment is *denotational*: two words receive the same normal form exactly when they denote the same typed arrow, with equality of arrows being the extensional equality of Chapter 4 (same source object, same target object, same mapping tuple);
(3) the assignment is *total or absent*: either the complete table of reachable denotations is computed, or a resource exception is raised; a partial table is never presented as canonical.

The solution adopted by the kernel, implemented in `src/tks_v8_core/normal_forms.py` and specified in the governing Gate 1 sources, is *shortlex minimization over a completed finite closure*: explore all well-typed generator words in shortlex order, record the first word achieving each denotation, and prove that first-discovery under this exploration order coincides with shortlex-leastness. The mathematical content of the chapter is exactly that proof, together with the order theory it needs, the finiteness and complexity analysis, the resource semantics, and two permanent boundaries that fence off what the construction does *not* claim.

Evidential status, stated up front and repeated where relevant: the correctness results of this chapter (shortlex well-ordering, pruning soundness, closure completeness) carry the ledger status of obligation PO-TKS8-CORE-06 — INFORMAL CORRECTNESS ARGUMENT / FORMALIZATION OPEN. The arguments below are those informal arguments, in fully expanded form. No Lean formalization of shortlex leastness exists; the pinned formal seed's verified scope does not include this chapter's results [16]. The executable suite exercises normal-form correctness and resource regressions on bounded families in the small-scope tradition [15]; passing them establishes behavior on the covered cases only. Everything is Ring 4, draft, not licensed.

2. Generator catalogs

DEFINITION 8.1 (Generator). A *generator* is a pair $g = (\ell_g, f_g)$ of a non-blank string label ℓ_g and a typed arrow f_g of the finite-function category. The implementation type is the frozen dataclass `Generator`, which validates both components at construction.

DEFINITION 8.2 (Catalog). A *generator catalog* is a triple

$$\mathcal{C} = (\,\mathcal{O},\ G,\ \text{declared orders}\,)$$

where $\mathcal{O} = (O_0, \ldots, O_{m-1})$ is a finite, explicitly ordered sequence of objects and $G = (g_0, \ldots, g_{k-1})$ is a finite, explicitly ordered sequence of generators, subject to the validation rules of Invariant 8.3. The position of a generator in the declared sequence is its *rank*: $\mathrm{rank}(g_j) = j$. The position of an object is its *object rank*. Both orders are part of the catalog's identity.

INVARIANT 8.3 (Catalog validation, as implemented). The constructor of `NormalFormCatalog` enforces, raising `ValidationError` on violation:

(1) the object sequence is nonempty, and both sequences pass the ordered-input discipline (sets, mappings, and one-shot iterables are rejected, for the reasons of Chapter 3: order is meaning);
(2) every declared object is a `FiniteObject` and every declared generator is a `Generator`;
(3) the objects are pairwise distinct, *and* their names are pairwise distinct (name-inclusive identity would otherwise allow two objects that collide in error messages and reports);
(4) the generator labels are pairwise distinct (labels are the alphabet of words; a duplicated label would make rank ill-defined);
(5) every generator's arrow has both its source and its target among the declared objects (no arrow may smuggle in an undeclared object);
(6) the resource caps `max_arrows` and `max_frontier` are positive integers (Booleans are explicitly rejected), with `max_frontier` defaulting to `max_arrows` and `max_arrows` defaulting to 100,000.

Note what is *not* required: generator arrows need not be distinct. Two generators may carry extensionally equal arrows under different labels; the theory below handles this case correctly (the two length-one words then denote the same arrow, and shortlex picks the lower-ranked label).

3. Typed words and denotations

DEFINITION 8.4 (Typed word). A *typed word* over a catalog $\mathcal{C}$ is a pair $w = (S_w, \lambda_w)$ of a *start object* $S_w \in \mathcal{O}$ and a finite (possibly empty) sequence of generator labels $\lambda_w = (\ell_1, \ldots, \ell_n)$. The word is *well-typed* when the arrows compose in sequence from the start: setting $X_0 = S_w$, we require, for each i from 1 to n, that the generator g_{ℓ_i} named by ℓ_i exists in the catalog and that its declared source equals X_{i-1}; then X_i is its declared target. The implementation type `TypedWord` validates *syntax* at construction (a genuine `FiniteObject` start, an ordered tuple of non-blank string labels); well-typedness is relative to a catalog and is certified by `NormalFormCatalog.evaluate`, which raises `ValidationError` on an unknown label or an undeclared start object and raises the composition type error of Chapter 4 on a junction mismatch.

DEFINITION 8.5 (Denotation). The *denotation* of a well-typed word $w = (S_w, (\ell_1, \ldots, \ell_n))$ is the typed arrow

$$[\![w]\!] \;=\; f_{\ell_n} \circ \cdots \circ f_{\ell_1} \circ \mathrm{id}_{S_w} \;:\; S_w \longrightarrow X_n,$$

i.e. the composite of the generator arrows applied *in reading order*: the first label acts first. The empty word at S denotes id_S. Equality of denotations is extensional equality of

arrows: equal source object, equal target object, equal mapping tuple. The implementation computes $[\![w]\!]$ by a left fold, starting from `identity(word.start)` and composing each successive generator arrow on the left; the result is repackaged under a presentation name (`word-evaluation[`n`]`) which, per the extensional-equality discipline, never affects arrow equality. The *denotation key* of an arrow — the triple (source object, target object, mapping tuple) — is the dictionary key under which the closure stores it.

Two small but load-bearing observations about Definition 8.5:

REMARK 8.6 (Words determine their type path). A nonempty well-typed word's start object is redundant data: it must equal the declared source of its first generator. The start object earns its place in the data type for the sake of the *empty* words, one per declared object, which are the only words whose denotation cannot be recovered from labels alone. This is why identities are always in the closure — they are seeded as empty words — even when no product of generators equals an identity. It is also why, as we will use repeatedly below, *two words with the same denotation have the same start object*: the denotation's source is the word's start.

REMARK 8.7 (Reading order versus application order). The word (ℓ_1, ℓ_2) denotes $f_{\ell_2} \circ f_{\ell_1}$: reading order is application order, which is the *reverse* of the conventional right-to-left composition notation. The convention is chosen so that extending a word on the right corresponds to composing on the left with a new generator — exactly the step the closure algorithm performs — and so that a word reads like an execution trace. Nothing depends on the convention except consistency, which the implementation enforces in one place (`evaluate`).

4. The shortlex order

DEFINITION 8.8 (Shortlex). Fix a catalog $\mathcal{C}$. For a typed word w, let the *rank sequence* of w be the sequence of generator ranks of its labels, and let its *key* be the triple

$$\kappa(w) = \big(\, |w|,\ \text{rank sequence of } w,\ \text{object rank of } S_w \,\big),$$

where $|w|$ is the number of labels. Keys are compared lexicographically: first by length; then by rank sequence in lexicographic order (compare position by position; the first differing position decides); then by start object rank. Write $u <_{\text{slex}} v$ iff $\kappa(u) < \kappa(v)$, and $u \leq_{\text{slex}} v$ iff $\kappa(u) \leq \kappa(v)$. This is precisely the key computed by the implementation method `shortlex_key`.

The third component deserves comment before the theory. For nonempty words the first two components already determine the word: the rank sequence determines the label sequence (ranks and labels are in bijection by Invariant 8.3(4)), and the label sequence determines the start object (Remark 8.6). Only the empty words at distinct objects share the key prefix $(0, ())$, and the object-rank component separates them. Thus:

LEMMA 8.9 (Keys are injective). *Distinct typed words over the same catalog have distinct keys. Consequently* $<_{\text{slex}}$ *is a strict total order on typed words: for any two distinct words, exactly one of* $u <_{\text{slex}} v$, $v <_{\text{slex}} u$ *holds.*

PROOF. Suppose $\kappa(u) = \kappa(v)$. Then $|u| = |v|$ and the rank sequences agree. If the common length is positive, the rank sequences determine equal label sequences, whose first labels name generators with equal declared sources, so the start objects agree and $u = v$. If the common length is zero, both words are empty and the equal third components force equal start objects (object ranks are positions in a sequence of *distinct* objects), so again

$u = v$. Totality and trichotomy now follow from totality of the lexicographic comparison of integer triples-of-sequences: lengths are integers; rank sequences of equal length are compared position by position over the total order of integers; object ranks are integers. □

LEMMA 8.10 (Well-foundedness; shortlex is a well-order). *Every nonempty set W of typed words over a catalog with k generators and m objects has a unique $<_{\text{slex}}$-least element. Equivalently, there is no infinite strictly $<_{\text{slex}}$-descending sequence of words.*

PROOF. Let $L = \min\{|w| : w \in W\}$, which exists because lengths are natural numbers. The set W_L of words in W of length L is nonempty and finite: there are at most k^L label sequences of length L and at most m start objects, so $|W_L| \le m\,k^L$ (for $L \ge 1$ even $\le k^L$, by Remark 8.6). A finite nonempty set under a strict total order has a unique least element; the least element of W_L is $<_{\text{slex}}$-below every other member of W (shorter beats longer; within W_L it is least by construction). For the equivalent formulation: an infinite strictly descending sequence would have nonincreasing lengths, hence eventually constant length L', and would then descend strictly and infinitely inside the finite set of words of length L' from at most m starts — impossible. □

LEMMA 8.11 (Right-append monotonicity). *Let u, u' be typed words with the same start object and $u <_{\text{slex}} u'$, and let s be a label sequence. Write us for the word $(S_u,\ \lambda_u{\cdot}s)$ and $u's$ for $(S_{u'},\ \lambda_{u'}{\cdot}s)$. Then $us <_{\text{slex}} u's$.*

PROOF. Case 1: $|u| < |u'|$. Then $|us| = |u| + |s| < |u'| + |s| = |u's|$, and shorter wins. Case 2: $|u| = |u'|$. Since the start objects agree, the key comparison was decided by the rank sequences: ranks(u) precedes ranks(u') lexicographically, so they differ at some first position $i \le |u|$, with the u-entry smaller. The rank sequences of us and $u's$ have equal length $|u|+|s|$ and agree with the originals on the first $|u|$ positions; their first difference is therefore still at position i, still in u's favor. Hence $us <_{\text{slex}} u's$. (The third key component is equal on both sides and never consulted.) □

REMARK 8.12 (Why shortlex and not plain lex). The length-first component is not a stylistic preference; it is what makes the order usable at all. Consider plain lexicographic order on words, with the standard convention that a proper prefix precedes its extensions. Under that order, minimization over an infinite language can fail to make sense algorithmically even when minima exist, and — decisive for us — exploration order and well-foundedness part company. Concretely, with two generators $a < b$, the set $\{b,\ ab,\ aab,\ aaab,\ \dots\}$ is an infinite strictly *descending* chain in plain lex ($aab < ab < b$, and each added a descends further), so plain lex is not well-founded on words and "the lex-least word denoting a" need not exist: a denotation reachable by all of $b, ab, aab, \dots$ (easily arranged when a's arrow acts as an identity on the relevant object) has no least representative at all. Shortlex repairs this with the length component, which caps every descending chain (Lemma 8.10) and simultaneously aligns the order with breadth-first exploration: all words of length n precede all words of length $n+1$, so a frontier processed in shortlex order is a breadth-first frontier with a deterministic within-level order. The alignment is what lets one algorithm serve as both an exhaustive explorer and a minimizer — the content of Theorem 8.20.

5. Normal forms

DEFINITION 8.13 (Reachable denotation; NF_G). An arrow a is a *reachable denotation* of the catalog $\mathcal{C}$ when $a = [\![w]\!]$ for some well-typed word w over $\mathcal{C}$ (the empty words make every

id_{O_i} reachable). For reachable a, the *normal form* of a is

$$\mathrm{NF}_G(a) \;=\; \min_{<_{\mathrm{slex}}} \big\{\, w \text{ well-typed over } \mathcal{C} \;\big|\; [\![w]\!] = a \,\big\}.$$

PROPOSITION 8.14 (NF_G is well defined). *For every reachable denotation a, the set of well-typed words denoting a is nonempty, and it has a unique shortlex-least element.*

PROOF. Nonemptiness is the definition of reachability; existence and uniqueness of the least element are Lemma 8.10 (uniqueness also directly from Lemma 8.9: two distinct least elements would be shortlex-comparable, contradicting joint minimality). □

The normal form is the object the rest of the kernel consumes: the method `normalize(word)` evaluates a word and returns $\mathrm{NF}_G([\![w]\!])$; the method `representative_for(arrow)` looks an arrow up by denotation key, raising `UnreachableDenotationError` when the arrow is not in the generated closure; the property `normal_forms` returns all representatives sorted by shortlex key; and `denotation_count` reports the closure size. What remains is to *compute* NF_G — and to prove that the computation is right.

6. The representative-first closure algorithm

ALGORITHM 8.15 (Representative-first closure). Input: a validated catalog $\mathcal{C}$ with caps `max_arrows`, `max_frontier`. State: a min-heap (priority queue) *frontier* of entries $(\kappa(w), \text{serial}, w, [\![w]\!])$ ordered by key κ with a monotone serial number as tiebreaker; a set *queued* of denotation keys currently represented in the frontier; a table *reps* from denotation keys to words (with a parallel table of the arrows themselves). Steps:

(1) **Seed.** For each declared object O in declared order: if |queued| $\geq$ `max_frontier`, raise `ClosureLimitExceeded`; otherwise push the empty word at O with its denotation id_O and add id_O's key to *queued*.
(2) **Loop.** While the frontier is nonempty:
   (a) Pop the entry with least key; let it be (w, a) with denotation key d. Remove d from *queued*.
   (b) **Duplicate check.** If $d \in$ reps, discard (w, a) and continue the loop (the word is recorded as a duplicate and *not expanded*).
   (c) **Cap check.** If |reps| $\geq$ `max_arrows`, raise `ClosureLimitExceeded`.
   (d) **Accept.** Record reps$[d] = w$.
   (e) **Expand.** For each generator g in declared (rank) order whose declared source equals the target of a: form the extension $w' = w{\cdot}\ell_g$ and its denotation $a' = f_g \circ a$ with key d'. If $d' \in$ reps or $d' \in$ queued, skip (denotation deduplication). Otherwise: if |queued| $\geq$ `max_frontier`, raise `ClosureLimitExceeded`; else push (w', a') and add d' to *queued*.
(3) **Return.** When the frontier empties without a cap being hit, the table *reps* is complete and the catalog object becomes usable.

This is a faithful transcription of `NormalFormCatalog._build`. The serial number exists so that heap entries are always comparable even if two keys were equal; by Lemma 8.9 keys of distinct words are never equal, so the serial is a belt-and-braces guard for the runtime, not a mathematical ingredient.

Three design points before the proofs. First, *representative-first* names the policy of step 2(b): only the first word to arrive at a denotation is ever expanded; all later arrivals are pruned unexpanded. The entire correctness question is whether this pruning can ever lose

a smaller representative or an entire denotation — Section 7 proves it cannot. Second, the *queued* set prunes even earlier: a candidate whose denotation is already awaiting processing in the frontier is not pushed at all. This is an optimization with teeth — it bounds the frontier by the number of distinct pending denotations — and it carries a proof obligation of its own (Lemma 8.18), because dropping a candidate is only sound if the word already queued for that denotation is at least as good. Third, the caps are checked *before* accepting or pushing, so the exception fires instead of the table silently saturating; the construction never returns a partial table (Section 9).

6.1. The closure as a graph search. A reader from combinatorial semigroup theory will recognize the skeleton of Algorithm 8.15. Define the *typed denotation graph* of a catalog: vertices are the reachable denotations; there is a labeled edge $a \xrightarrow{\ell} a'$ whenever the generator labeled ℓ has source equal to the target of a and $a' = f_\ell \circ a$. This is the typed, multi-object analogue of the (left) Cayley graph of a monoid with respect to a generating set [**13, 14**], with the identities id_O playing the role of the neutral vertex — one per declared object, since the "monoid" here is really the arrow family of a category. Under this reading:

- the closure is a graph search from the m seed vertices;
- a representative $\mathrm{NF}_G(a)$ is a *geodesic* label sequence from a seed to a — shortest first, by the length component — with ties broken deterministically by the rank-lexicographic component;
- the duplicate check of step 2(b) is the classic "visited" set of breadth-first search; the *queued* set of step 2(e) is the equally classic "discovered but not yet expanded" refinement that prevents the frontier from holding one vertex twice;
- Theorem 8.20 is then the typed, tie-broken sharpening of the standard fact that breadth-first search computes shortest paths: not merely *some* geodesic per vertex, but the lexicographically least one, which is what determinism requires.

The analogy also delimits itself. Cayley-graph methods in semigroup theory typically continue toward presentations, relations, and rewriting [**14**]; the kernel stops deliberately short of all three (Boundary 8.36). The graph view is offered as an aid to intuition and a bridge to the literature, not as an alternative formalization: the definitions that govern are those of Sections 2–6.

7. Soundness of pruning

Throughout this section, fix a validated catalog and consider a run of Algorithm 8.15 that terminates normally (no cap raised). "Popped" means returned by step 2(a); the popped words form a finite sequence $p_1, p_2, \ldots, p_T$ in temporal order.

The proof rests on the two invariants announced in the governing sources: the *dequeue-order* invariant (words leave the frontier in shortlex order) and the *prefix-domination* invariant (a better prefix stays better under any common suffix). We prove each, plus the auxiliary skip-domination lemma that the frontier deduplication requires.

Lemma 8.16 (Invariant I: dequeue monotonicity). *$\kappa(p_1) \le \kappa(p_2) \le \cdots \le \kappa(p_T)$; that is, words are popped in nondecreasing (by Lemma 8.9, strictly increasing) shortlex order.*

Proof. Every non-seed frontier entry is pushed during the expansion step of some pop, and an extension $p_t \cdot \ell$ satisfies $\kappa(p_t \cdot \ell) > \kappa(p_t)$ because its length is $|p_t| + 1$. Now fix t and compare p_{t+1} with p_t. At the moment p_t was popped, the entry that will later be popped as p_{t+1} either (i) was already in the frontier — in which case $\kappa(p_{t+1}) \ge \kappa(p_t)$ because

the heap returned p_t as the *minimum* of the then-current frontier — or (ii) had not yet been pushed, in which case it is pushed during the expansion of p_t itself (pushes happen only inside expansions, and the only expansion between the two pops is that of p_t), whence $\kappa(p_{t+1}) = \kappa(p_t \cdot \ell) > \kappa(p_t)$ for some label ℓ. Either way $\kappa(p_t) \le \kappa(p_{t+1})$, and induction over t finishes the claim. (Seeds are all pushed before the first pop, so case (ii) never involves a seed.) □

LEMMA 8.17 (Invariant II: prefix domination under common suffixes). *Let u, u' be well-typed words with $[\![u]\!] = [\![u']\!]$ and $u <_{\text{slex}} u'$. Then for every label sequence s such that $u's$ is well-typed: us is well-typed, $[\![us]\!] = [\![u's]\!]$, and $us <_{\text{slex}} u's$.*

PROOF. Equal denotations have equal sources and equal targets, so u and u' share a start object (Remark 8.6) and end at the same object Z. Well-typedness of an appended suffix depends only on the object at the junction and on the suffix's own internal typing: $u's$ well-typed means the first label of s names a generator with source Z and the remaining labels chain on from there; the identical condition makes us well-typed. For the denotation, evaluation is a fold, so $[\![us]\!] = F_s \circ [\![u]\!]$ where F_s denotes the composite of the suffix's generator arrows (an arrow $Z \to Z'$); likewise $[\![u's]\!] = F_s \circ [\![u']\!]$; the right-hand factors are equal by hypothesis, so the composites are extensionally equal. The order claim is Lemma 8.11, whose same-start hypothesis we just established. □

LEMMA 8.18 (Skip domination). *Suppose that during the expansion of a popped word p_t, the candidate $w' = p_t \cdot \ell$ with denotation key d' is* skipped *in step 2(e). Then at that moment there exists a well-typed word q with denotation key d' and $q <_{\text{slex}} w'$, where q is either the accepted representative of d' (if $d' \in$ reps) or the unique frontier word with denotation key d' (if $d' \in$ queued).*

PROOF. *Accepted case.* The representative $q = \text{reps}[d']$ was popped as some p_s with $s \le t$, so $\kappa(q) \le \kappa(p_t) < \kappa(w')$ by Lemma 8.16 and the strict length increase of extensions.

Queued case. The frontier word q with key d' was pushed either as a seed or during the expansion of some earlier pop p_s with $s \le t$. Since q and w' have the same denotation, they have the same start object (Remark 8.6); note this rules out the one scenario in which the object-rank tiebreaker could interfere, namely distinct empty-word seeds — two seeds have different denotations (identities of distinct objects), so if q is a seed then w', being nonempty, is longer and $q <_{\text{slex}} w'$ immediately. Otherwise $q = p_s \cdot \ell'$ for some $s \le t$ and label ℓ'. Compare:

- If $s < t$ and $|p_s| < |p_t|$: then $|q| < |w'|$ and $q <_{\text{slex}} w'$.
- If $s < t$ and $|p_s| = |p_t|$: by Lemma 8.16 and key injectivity, $\kappa(p_s) < \kappa(p_t)$ with equal lengths, so the rank sequences differ at some first position inside the common length, in p_s's favor; appending one label to each does not disturb that first difference, so $q <_{\text{slex}} w'$ (the argument of Lemma 8.11, applied across two different prefixes).
- If $s = t$: both q and w' extend the same popped word p_t, and generators are enumerated in declared rank order, so the earlier push has the smaller final rank: $\text{rank}(\ell') < \text{rank}(\ell)$, whence $q <_{\text{slex}} w'$ (equal lengths, equal prefix, smaller last entry).

In every case $q <_{\text{slex}} w'$, as claimed. □

With the three lemmas in hand, we can state and prove the main result. Call a word w *hereditarily minimal* when every prefix of w (including w itself, with its start object) is the shortlex-least word for that prefix's own denotation.

LEMMA 8.19 (Prefixes of normal forms are normal forms). *If $w = \mathrm{NF}_G(a)$ for some reachable a, then w is hereditarily minimal.*

PROOF. Let p be a proper prefix of w, say $w = ps$ with s nonempty, and suppose toward a contradiction that some well-typed r has $[\![r]\!] = [\![p]\!]$ and $r <_{\mathrm{slex}} p$. By Lemma 8.17 (with $u = r$, $u' = p$, suffix s), the word rs is well-typed, denotes $[\![ps]\!] = a$, and satisfies $rs <_{\mathrm{slex}} ps = w$ — contradicting $w = \mathrm{NF}_G(a)$. The case $p = w$ is the definition of NF_G. □

THEOREM 8.20 (Closure correctness; status: informal correctness argument, formalization open (PO-TKS8-CORE-06)). *Suppose Algorithm 8.15 terminates normally on a validated catalog. Then the computed table* reps *satisfies:*

(1) *(*Completeness*) the key set of* reps *is exactly the set of reachable denotations of the catalog;*
(2) *(*Minimality*) for every reachable a, reps$[a] = \mathrm{NF}_G(a)$.*

PROOF. We first prove the following claim by strong induction on the shortlex order (legitimate by Lemma 8.10):

Claim. For every reachable denotation a, the word $w^* = \mathrm{NF}_G(a)$ is pushed at some point of the run, and when it is popped, a is not yet in *reps*; consequently a enters *reps* with representative w^*.

Proof of claim. Fix a and $w^* = \mathrm{NF}_G(a)$, and assume the claim holds for every reachable denotation whose normal form is shortlex-smaller than w^*.

Getting w^* pushed. If w^* is empty, it is a seed and is pushed in step 1. Otherwise write $w^* = p \cdot \ell$ where p is the longest proper prefix. By Lemma 8.19, $p = \mathrm{NF}_G([\![p]\!])$, and $p <_{\mathrm{slex}} w^*$, so the induction hypothesis applies to $[\![p]\!]$: the word p is popped and accepted as the representative of its denotation. During the expansion of p, the candidate $p \cdot \ell = w^*$ is generated (the generator named ℓ has source equal to the target of $[\![p]\!]$ because w^* is well-typed). Could it be skipped? By Lemma 8.18, a skip would produce a word $q <_{\mathrm{slex}} w^*$ with $[\![q]\!] = a$ — contradicting minimality of w^*. So w^* is pushed.

Popping w^* first among a's words. The run terminates normally with an empty frontier, and every popped entry is either accepted or discarded as a duplicate, so every pushed entry is eventually popped; in particular w^* is popped, say as p_t. Suppose $a \in$ reps already at that moment, via an earlier accepted word $r = p_s$, $s < t$, with $[\![r]\!] = a$. By Lemma 8.16 and key injectivity, $r \neq w^*$ implies $\kappa(r) < \kappa(w^*)$ — again contradicting $w^* = \mathrm{NF}_G(a)$. Hence $a \notin$ reps when w^* is popped, the duplicate check passes, and (normal termination having excluded the cap branch) w^* is accepted as reps$[a]$. This proves the claim.

Completeness. Every key in *reps* is the denotation key of some evaluated word, hence reachable. Conversely, by the claim every reachable denotation is accepted. *Minimality.* A denotation enters *reps* exactly once (the duplicate check forbids overwriting), and by the claim the entering word is $\mathrm{NF}_G(a)$. □

COROLLARY 8.21 (Determinism and stability). *On normal termination, the table* reps — *keys, representatives, and the sorted listing* `normal_forms` — *is a pure function of the catalog identity: the object sequence (names, carriers, and order), the generator sequence (labels, arrows, and order), and nothing else. Two runs on equal catalogs produce equal tables, on any machine, under any* `PYTHONHASHSEED`.

PROOF. Algorithm 8.15 consults only: declared object order (seeding), declared generator order (expansion and ranks), arrow composition (deterministic by Chapter 4), and shortlex

keys (functions of the declared orders). The heap's behavior is determined by keys and the deterministic serial tiebreaker; no iteration over a hash-ordered container occurs on any code path (the ordered-input discipline of Invariant 8.3 exists to guarantee precisely this). Theorem 8.20 then pins the *output* to the mathematical object NF_G, which depends only on the catalog. □

8. Finiteness and complexity

PROPOSITION 8.22 (Finiteness of the closure). *Let the catalog declare objects $O_0, \dots, O_{m-1}$ with carrier sizes $n_0, \dots, n_{m-1}$. The number of reachable denotations N satisfies*

$$N \;\le\; \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} n_j^{\,n_i},$$

since for each ordered pair of objects (O_i, O_j) there are at most $n_j^{n_i}$ total maps $O_i \to O_j$, and every denotation is a total map between declared objects. (The convention $0^0 = 1$ counts the unique empty map, so empty carriers are covered.) Consequently Algorithm 8.15 terminates: each denotation is accepted at most once, each acceptance pushes at most k extensions, at most one frontier entry exists per pending denotation, and the frontier therefore empties after finitely many pops.

PROOF. The bound is the counting statement just made. For termination, observe that every pop either accepts a new denotation or discards a duplicate; there are at most N acceptances; total pushes are bounded by m seeds plus k per acceptance, i.e. $m + kN$; every pop removes one entry; so the loop performs at most $m + kN$ pops and halts with an empty frontier (or raises earlier). □

REMARK 8.23 (Cost accounting). Write N for the actual closure size, k for the generator count, m for the object count, $n = \max_i n_i$, and F for the peak frontier size. The run performs at most $m+kN$ pushes and pops, each costing $O(\log F)$ comparisons of keys; a key comparison costs $O(L)$ where L is the length of the longer word, and L is bounded by N itself in the worst case (each accepted representative extends an earlier one by one label, and representatives are pairwise distinct denotations, so a representative's length never exceeds the number of acceptances that preceded it — a chain of new denotations). Each expansion performs at most k compositions at $O(n)$ integer work apiece plus one denotation-key construction at $O(n)$. Total: $O\big((m+kN)\,(\,k\,n + L \log F\,)\big)$ in the worst case, with $L \log F$ typically far below its bound because representative lengths in practice grow like the diameter of the denotation graph, not like N.

REMARK 8.24 (Frontier growth, and what deduplication buys). Without the *queued* set, the frontier could hold many words per denotation — up to one per accepted predecessor — and its size could approach kN. With deduplication, the frontier holds *at most one entry per pending denotation*, so $F \le N$ always, and $F \le$ `max_frontier` is enforced besides. The price is the proof obligation discharged by Lemma 8.18: dropping a later word for an already-queued denotation is only sound because the queued word is strictly shortlex-smaller. The pattern — an optimization that is also a theorem — recurs throughout the kernel, and the monograph's policy is to state the theorem next to the optimization every time.

CLAIM BOUNDARY 8.25 (The bound is doubly exponential in carrier size, and that is fine). The worst-case closure bound $\sum_{i,j} n_j^{n_i}$ grows brutally: two objects of size 6 already admit $6^6 = 46{,}656$ maps per direction. This is not a defect to engineer away; it is the true size

of the function space, and any "canonical form" scheme that returned answers faster than the space's size on adversarial catalogs would be sampling, not closing. The kernel's answer to the growth is not cleverness but honesty: dual resource caps and a fail-closed exception, next.

9. Resource semantics

Construction 8.26 (Dual caps and fail-closed completion). The constructor accepts `max_arrows` (default 100,000; bounds the number of *accepted* denotations) and `max_frontier` (default: equal to `max_arrows`; bounds the number of *pending* denotations in the frontier at any moment). Three guard sites exist, all raising `ClosureLimitExceeded`: seeding a frontier already at the frontier cap; accepting a denotation when the table already holds `max_arrows` entries; and pushing an extension when the frontier is at the cap. The exception's message states, in both cap cases, that *no partial normal-form catalog was returned* — and none is: the exception escapes the constructor, so no `NormalFormCatalog` object exists afterward; there is no partially built catalog to misuse.

Rule 8.27 (Fail-closed canonicalization). A normal-form table is either *complete* — every reachable denotation present, every representative shortlex-least, per Theorem 8.20 — or it is *absent*. There is no third state. In particular the implementation never: returns the accepted portion of an interrupted closure; marks a table "best effort"; or degrades to a sampled or truncated table under resource pressure.

The rationale deserves spelling out, because the alternative is common engineering practice. A truncated "canonical" table is worse than none: a lookup that fails could mean *unreachable* or could mean *unexplored*, and a representative drawn from a truncated exploration may be beaten by an unexplored shorter word, so equality of representatives would no longer certify anything. Every downstream consumer of the catalog — digest computation, cross-run comparison, deduplication — relies on representatives being genuine minima. The kernel therefore converts resource exhaustion into a loud, typed failure at construction time, and the two caps let a caller choose the failure threshold rather than whether failure is loud. Separately, `representative_for` distinguishes the *unreachable* case with its own exception, `UnreachableDenotationError`: on a *completed* table, "not found" soundly means "not generated by the declared generators," and the exception name says so.

Example 8.28 (Tripping the cap, concretely). To see the guard fire on an honest workload rather than a contrived one, declare a single object $T = (\texttt{Three}, (0, 1, 2))$ and three generators on it: c with the 3-cycle $0 \mapsto 1 \mapsto 2 \mapsto 0$; t with the transposition swapping 0 and 1; and e with the collapse $0 \mapsto 0$, $1 \mapsto 0$, $2 \mapsto 2$. It is a standard fact of transformation-semigroup theory that a generating set of the symmetric group together with one idempotent of rank $n - 1$ generates the full transformation monoid on n letters [14]; for $n = 3$ the reachable denotations are therefore all $3^3 = 27$ endomaps of T (plus nothing else: one object, so every denotation is an endomap, and the empty word supplies the identity). A catalog constructed over these generators with `max_arrows` $= 27$ or more completes with `denotation_count` $= 27$; the same catalog with `max_arrows` $= 10$ raises `ClosureLimitExceeded` during the eleventh acceptance, and no catalog object exists afterward. The example makes the design point tangible: the cap is not an error in the catalog — the closure is finite and perfectly computable — but a caller-chosen budget, and the failure it produces is loud, typed, and total, never a 10-entry table masquerading as the canonical closure of a 27-element monoid.

10. Generator-order relativity

Canonical forms are relative to the declared generator order. This is not a bug to be normalized away but a definitional fact to be documented and enforced; this section proves the invariance that does hold, exhibits the variance that does not, and states the resulting comparison discipline.

PROPOSITION 8.29 (Denotations are order-invariant; representatives are not). *Let $\mathcal{C}$ and $\mathcal{C}'$ be catalogs with the same object set and the same generator* set *(same labels, same arrows) but different declared generator orders. Then:*

(1) *the reachable denotation sets of $\mathcal{C}$ and $\mathcal{C}'$ are equal;*
(2) *the closure tables have equal key sets, and* `denotation_count` *agrees;*
(3) *the representative* NF_G *of a given denotation may differ between $\mathcal{C}$ and $\mathcal{C}'$; when it differs, the two representatives denote the same arrow.*

PROOF. (1) Reachability quantifies over well-typed words, and well-typedness of a word (Definition 8.4) mentions labels, arrows, sources, and targets — never ranks. The same words are well-typed over both catalogs, with the same denotations. (2) Immediate from (1) and completeness (Theorem 8.20). (3) The normal form minimizes over the same set of words but under a different order, so the minimum may move; whatever word is selected still denotes the arrow being represented, by definition of the set minimized over. □

EXAMPLE 8.30 (Worked order-relativity: two commuting transpositions). Declare one object $Q = (\texttt{Quad}, (0,1,2,3))$ and two generators on it: a with arrow the transposition swapping positions 0 and 1 (fixing $2,3$), and b with arrow the transposition swapping positions 2 and 3 (fixing $0,1$). The two arrows commute: $a \circ b = b \circ a$ is the double transposition τ swapping both pairs. The reachable denotations are the four arrows of the Klein four-group: id_Q, a, b, τ (products of longer words fold back: $aa = bb = \mathrm{id}$, $ab = ba = \tau$, and so on).

Under declared order (a, b) — ranks $a \mapsto 0$, $b \mapsto 1$ — the closure accepts, in pop order: $()$ for id_Q; (a) for a; (b) for b; then among length-two words, (a,a) duplicates id, and (a,b) is popped before (b,a) because rank sequence $(0,1) < (1,0)$ lexicographically — so τ gets representative (a,b), and when (b,a) is generated it is skipped (denotation already present).

Under declared order (b, a) — ranks $b \mapsto 0$, $a \mapsto 1$ — the identical word set is explored under the mirrored order, and τ gets representative (b,a): rank sequence $(0,1)$ again, but $(0,1)$ now *spells* "b then a." The denotation table is identical in both runs — four keys, the same four arrows — while the representative of τ differs as a word. Both representatives evaluate to τ; neither is wrong; each is canonical *relative to its catalog*.

CLAIM BOUNDARY 8.31 (Catalog identity is required for cross-comparison). A normal form is meaningful only as a pair (catalog identity, word). Comparing representatives produced under different catalogs — different generator orders, different labels, different object declarations, or different generator sets — is meaningless: equal words may denote different arrows (if labels were rebound) and equal arrows may receive different words (Example 8.30). Any workflow that stores, transmits, or diffs normal forms must bind them to a versioned catalog identity — in the kernel's artifact discipline, the declared sequences and their hashes — and must treat a catalog change as a schema change that invalidates all stored representatives, even when the denotation sets coincide (Proposition 8.29 guarantees they often do, which makes the silent-mismatch failure mode *more* dangerous, not less: the keys line up while the names have moved).

11. Extended worked example: a two-object catalog, by hand

We now run Algorithm 8.15 by hand, round by round, on a catalog small enough to trace completely yet rich enough to exercise every branch: cross-object typing, non-injective collapse, frontier deduplication in both its forms (accepted and queued), and termination.

EXAMPLE 8.32 (The catalog). Objects, in declared order: $X = (\texttt{Left}, (x_0, x_1))$ and $Y = (\texttt{Right}, (y_0, y_1))$. Generators, in declared order:

- u (rank 0), arrow $u : X \to Y$ with $u(x_0) = y_1$, $u(x_1) = y_0$ (a "crossing" bijection into the other object);
- v (rank 1), arrow $v : Y \to X$ with $v(y_0) = x_0$, $v(y_1) = x_0$ (a collapse: everything to x_0).

Object ranks: $X \mapsto 0$, $Y \mapsto 1$. To keep the trace compact, name the arrows that will appear: $c_X : X \to X$ constant at x_0; $c_Y : Y \to Y$ constant at y_1; $c_{XY} : X \to Y$ constant at y_1; $c_{YX} : Y \to X$ constant at x_0. (The reader can verify as we go that these are exactly the composites that arise.)

Seeding. Push the empty word at X (key $(0, (), 0)$, denotation id_X) and the empty word at Y (key $(0, (), 1)$, denotation id_Y). Frontier: two entries; *queued* $= \{\mathrm{id}_X, \mathrm{id}_Y\}$; *reps* empty.

Round 1. Pop $()$ at X — least key. New denotation: accept id_X with representative $()$. Expand: generators with source X: only u. Extension (u), denotation u itself, key $(1, (0), 0)$; not accepted, not queued: push. Frontier: $\{() \mathbin{@} Y,\ (u)\}$.

Round 2. Pop $()$ at Y (key $(0, (), 1)$ beats $(1, (0), 0)$ on length). Accept id_Y as $()$. Expand: source-Y generators: only v. Push (v), denotation v, key $(1, (1), 1)$. Frontier: $\{(u),\ (v)\}$.

Round 3. Pop (u) (rank sequence $(0) < (1)$). Accept u. Its target is Y, so expand by v: word (u, v), denotation $v \circ u$. Compute: $x_0 \mapsto u \mapsto y_1 \mapsto v \mapsto x_0$; $x_1 \mapsto y_0 \mapsto x_0$. So $v \circ u = c_X$. Push (u, v), key $(2, (0, 1), 0)$. Frontier: $\{(v),\ (u, v)\}$.

Round 4. Pop (v). Accept v. Target X; expand by u: word (v, u), denotation $u \circ v$: $y_0 \mapsto x_0 \mapsto y_1$; $y_1 \mapsto x_0 \mapsto y_1$. So $u \circ v = c_Y$. Push (v, u), key $(2, (1, 0), 1)$. Frontier: $\{(u, v),\ (v, u)\}$.

State after Round 4

Accepted denotation	Representative	Key	
id_X	$()$ at X	$(0, (), 0)$	
id_Y	$()$ at Y	$(0, (), 1)$	
u	(u)	$(1, (0), 0)$	
v	(v)	$(1, (1), 1)$	
Frontier	$(u, v) \mapsto c_X$	$(2, (0, 1), 0)$	pending
	$(v, u) \mapsto c_Y$	$(2, (1, 0), 1)$	pending

Round 5. Pop (u, v) — rank sequence $(0, 1) < (1, 0)$. Accept c_X with representative (u, v): the first endomap in the table, and already a non-obvious fact made canonical — the least word producing the collapse of X routes through Y and back. Expand: target X, generator u: word (u, v, u), denotation $u \circ c_X$: both points $\mapsto x_0 \mapsto y_1$, i.e. c_{XY}. Push, key $(3, (0, 1, 0), 0)$.

Round 6. Pop (v, u). Accept c_Y as (v, u). Expand: target Y, generator v: word (v, u, v), denotation $v \circ c_Y$: both $\mapsto y_1 \mapsto x_0$, i.e. c_{YX}. Push, key $(3, (1, 0, 1), 1)$.

Round 7. Pop (u, v, u). Accept c_{XY} as (u, v, u). Expand: target Y, generator v: candidate (u, v, u, v), denotation $v \circ c_{XY}$: both points of $X \mapsto y_1 \mapsto x_0$ — that is c_X, whose key is *already*

accepted (Round 5). Skip: no push. This is the accepted-case prune of Lemma 8.18, and the lemma's guarantee is visible concretely: the retained representative (u, v) is shortlex-far below the skipped (u, v, u, v).

Round 8. Pop (v, u, v). Accept c_{YX} as (v, u, v). Expand: target X, generator u: candidate (v, u, v, u), denotation $u \circ c_{YX} = c_Y$ — accepted in Round 6. Skip. Frontier now empty: **terminate normally**.

Final table: 8 denotations (`denotation_count` = 8)

Denotation	Type	Representative
id_X	$X \to X$	() at X
id_Y	$Y \to Y$	() at Y
u	$X \to Y$	(u)
v	$Y \to X$	(v)
c_X	$X \to X$	(u, v)
c_Y	$Y \to Y$	(v, u)
c_{XY}	$X \to Y$	(u, v, u)
c_{YX}	$Y \to X$	(v, u, v)

Reconciliation with the theory: the closure size 8 sits comfortably below the Proposition 8.22 bound $\sum_{i,j} 2^2 = 4 \cdot 4 = 16$; every representative is visibly the shortest word denoting its arrow (and among equal-length candidates, none competed here — the catalog is too rigid); the peak frontier size was 2, illustrating Remark 8.24's bound $F \leq N$; and the two skips in Rounds 7–8 are exactly the closure of the "ping-pong" orbit $c_X \to c_{XY} \to c_X$ and $c_Y \to c_{YX} \to c_Y$: from length 3 onward, every longer word folds into one of the eight accepted denotations, which is *why* the algorithm halts. A reader wishing to internalize the machine is invited to rerun the trace under the reversed generator order (v, u) and confirm: the same eight denotations, with the length-1 and length-3 representatives unchanged as words but re-ranked, and the pop order of Rounds 3–8 reshuffled accordingly (Proposition 8.29).

Exercise 8.33. Extend the catalog of Example 8.32 with a third generator w (rank 2) carrying the arrow $w : Y \to Y$ that swaps y_0 and y_1. Before running the closure, predict: which new denotations become reachable, which existing representatives change (careful: none of the existing eight should — adding a higher-ranked generator can only add words that lose ties), and what `denotation_count` becomes. Then run the closure by hand and check. (The exercise rewards noticing that $w \circ u$ "uncrosses" u, making the inclusion-like bijection $X \to Y$ reachable at length 2.)

11.1. Lookups against the completed table. The completed catalog of Example 8.32 also serves to exercise the query surface and its error taxonomy, which is as much a part of the construction's semantics as the table itself.

Check Class 8.34 (Error taxonomy of the query surface). Three failure kinds are deliberately kept distinct, because they mean three different things:

(1) *Malformed or foreign input* — `ValidationError`. Raised by `evaluate` (hence by `normalize`) for an unknown generator label or an undeclared start object, and by the constructors for unordered or blank inputs. Meaning: the question itself is not a question about this catalog.

(2) *Ill-typed word* — the composition type error of Chapter 4, propagated unchanged through `evaluate`'s fold when a label's declared source fails to match the object reached so far. Meaning: the word is syntactically over this catalog's alphabet but

does not type-check; it *has no denotation*, so no normal form is defined for it, and none is invented.

(3) *Unreachable arrow* — `UnreachableDenotationError` from `representative_for`. Meaning: the arrow is a perfectly valid typed arrow between declared objects, but the declared generators do not generate it. On a *completed* table this verdict is sound and final (Theorem 8.20, completeness), which is exactly why the fail-closed rule matters: against a partial table the same lookup failure would be uninterpretable.

The third kind has a crisp instance in the worked catalog: the swap endomap $\sigma : X \to X$, $x_0 \leftrightarrow x_1$, is a legitimate arrow on a declared object, yet the final table's only $X \to X$ entries are id_X and c_X. So `representative_for(`σ`)` raises `UnreachableDenotationError` — and that is a *theorem* about the catalog, not a shrug: since v's collapse is the only way back into X, every $X \to X$ word of positive length factors through c_X and is constant; no word denotes σ. The completed closure turns "not found" into "provably not generated," which is the entire difference between a cache miss and a mathematical statement.

Exercise 8.35. (i) Show that for a catalog whose generators are all endomaps of a single declared object, Definition 8.8 collapses to the classical shortlex order on the free monoid over the ranked alphabet, the third key component never differing. (ii) Construct, for each $n \geq 1$, a single-object catalog with one generator whose closure has exactly n denotations and whose longest representative has length $n - 1$ (hint: a cycle of length n; compare the worst-case length bound in Remark 8.23). (iii) Verify that in your construction the frontier never holds more than one entry, so the peak $F = 1$: the cost bounds of Remark 8.23 are loose precisely when the denotation graph is thin.

12. Two permanent boundaries

Claim Boundary 8.36 (Not a rewrite-system claim). NF_G is *denotational minimization over a completed finite closure*. It is not — and must never be quoted as — a termination or confluence result for a rewrite system, because *no rewrite system is defined here*. The distinction matters because the surrounding literature creates the expectation: canonical forms for words modulo relations are classically obtained by completing a set of rewrite rules and proving termination and local confluence, in the tradition of string-rewriting presentations of monoids and semigroups [**14**, **13**]. The kernel deliberately takes the other road: it never manipulates words by rules at all; it *evaluates* every word into the finite semantic domain of typed total functions and minimizes over each semantic equivalence class directly. Consequently: no claim is made that any set of TKS-inherited equations, oriented in any way, terminates; no claim is made that any such system is confluent; no critical-pair analysis exists; and no such analysis is owed, because no result in this chapter depends on one. Should a future lane define a rewrite presentation for inherited equations, its termination and confluence obligations would be new ledger items, starting from zero evidence — nothing proved here transfers.

Claim Boundary 8.37 (Catalog identity, restated as a permanent rule). Restating Boundary 8.31 in its permanent, cross-artifact form: normal forms compose with catalog identity and with nothing else. A representative is a name *within* a catalog, not a name *of* an arrow in the abstract. Every artifact that records a normal form must record the catalog (or its digest) beside it; every comparison of normal forms must first establish catalog equality; and every catalog modification — reordering, relabeling, adding or removing a generator or object — creates a *new* catalog whose representatives are incomparable with the

old, even where Proposition 8.29 guarantees the denotation sets agree. The executable suite enforces the constructor-level half of this rule (validated, ordered, hash-bindable inputs); the workflow-level half is a discipline on consumers, recorded here as a permanent boundary so that no future integration can cite this chapter as license for cross-catalog comparison.

13. Status summary

Item	Where	Status
Shortlex is a total well-order; keys injective	Lemmas 8.9–8.10	informal proof, th
NF_G well defined	Prop. 8.14	informal proof, th
Dequeue monotonicity; skip domination; prefix domination	Lemmas 8.16, 8.18, 8.17	informal correctn ment (PO-TKS 06)
Closure completeness and minimality	Thm. 8.20	informal correctn ment; formalizat not in Lean verifi
Finiteness; termination; cost bounds	Prop. 8.22, Rem. 8.23	informal proof; bo
Fail-closed resource semantics	Constr. 8.26, Rule 8.27	executable valida source regressions
Order-relativity of representatives	Prop. 8.29, Ex. 8.30	informal proof; w ample
No rewrite-system claim; catalog identity required	Boundaries 8.36, 8.37	permanent bound

The chapter closes the mathematical development of the kernel's five capabilities. What remains for the monograph's final part is the engineering: how the implementation realizes these definitions (Chapter 9), how the bounded exhaustive suites and regression fixtures exercise them (Chapter 10), what the pinned Lean seed does and does not verify (Chapter 11), and where the kernel's deliberate boundaries leave the wider TKS/CAS program (Chapter 12). Nothing in this chapter licenses semantic transfer onto canonical constructs; obligation PO-TKS8-CORE-08 remains OPEN / NOT LICENSED, and the shortlex machinery, like the lift before it, is offered as substrate — exact, replayable, fail-closed — awaiting separately gated review.

CHAPTER 9

Implementation

The preceding chapters developed the mathematics of the typed finite-function kernel: objects with ordered carriers, total arrows with declared types, endomap monoids and their factorial idempotent powers, semigroupoid paths, the deterministic standard-basis lift with its exact norm theory, and shortlex denotational normal forms. This chapter documents the executable artifact that realizes those constructions: the Python package `src/tks_v8_core`, comprising three substantive modules — `category.py`, `deterministic_lift.py`, and `normal_forms.py` — together with a small `__init__.py` that re-exports the public names.

The documentary stance of this chapter is strict: we describe the code as it is at the manifested snapshot, not as an idealization of it. Every class, constructor precondition, rejection behavior, and error type reported below is read directly from the source files whose SHA-256 digests are bound in the prototype manifest (Section 5). Where a design decision is visible in the code — a field excluded from equality, a guard placed before rather than after an insertion, a name generated by hashing rather than by decimal expansion — we record the decision and the rationale that the code and its docstrings themselves supply. The artifact carries the status GATE 1 PROTOTYPE / IN REVIEW / NOT FROZEN, admission Ring 4, license status NOT LICENSED; nothing in this chapter promotes that status.

1. The package and its documentary boundaries

1.1. Module inventory. The package consists of four files:

File	Role (as in the manifest)	Bytes
`src/tks_v8_core/__init__.py`	package exports	1,018
`src/tks_v8_core/category.py`	typed finite-function category	9,850
`src/tks_v8_core/deterministic_lift.py`	deterministic standard-basis lift	5,483
`src/tks_v8_core/normal_forms.py`	denotational shortlex closure	9,889

The dependency structure is a chain: `deterministic_lift.py` imports `FiniteArrow` and `ValidationError` from `category.py`; `normal_forms.py` imports `CompositionTypeError`, `FiniteArrow`, `FiniteObject`, `ValidationError`, the module-internal helper `_ordered_tuple`, and the constructors `compose` and `identity` from `category.py`. Neither of the two downstream modules imports the other, and no module imports anything outside the Python standard library (`collections.abc`, `dataclasses`, `hashlib`, `math`, `numbers`, `heapq`, `itertools`, `typing`). There is no dependency on the production CAS notation compiler, on any numerical library, or on any network or filesystem resource. This is not incidental hygiene: the manifest's integration boundary asserts `production_runtime_dependency: false`, and the boundary regression test of Chapter 10 asserts the same field.

1.2. What the modules deliberately do not contain.

CLAIM BOUNDARY 9.1. The module docstring of `category.py` states the boundary in its second sentence: the module "does not assign CAS or Noetic meaning to an arrow." No

identifier anywhere in the three modules names a Noetic, a Domain, a Drive, a Register, an RPM pathway, or any other construct of the TKS/CAS ontology. The kernel supplies a reference semantic layer in which an arrow always carries an exact source and target and composition is rejected when those types do not match — nothing more. The mapping of canonical content onto this substrate is proof obligation PO-TKS8-CORE-08, which is OPEN / NOT LICENSED (Chapter 12).

2. `category.py`: the typed finite-function category

2.1. Purpose. `category.py` implements the category $\mathcal{C}$ of Chapters 3 and 4 as executable data: named finite objects, total finite-function arrows with declared source and target, partial type-checked composition, the endomap wrapper that alone carries monoid powers, and the semigroupoid path that alone carries heterogeneous typed composites. The module realizes, in fewer than ten kilobytes, the three disciplines that the red-team audit found missing from kind-level checking: typed composition, power discipline, and deterministic identity.

2.2. Error taxonomy. The module defines exactly two exception types, and their placement in Python's built-in exception hierarchy is itself a design statement:

Exception	Base class	Meaning
`ValidationError`	`ValueError`	malformed object, arrow, or path
`CompositionTypeError`	`TypeError`	source/target objects do not match

`ValidationError` subclasses `ValueError`: the caller supplied data of an acceptable Python type whose *value* violates a kernel precondition (a blank name, a duplicate carrier element, an out-of-range image index, a negative exponent). `CompositionTypeError` subclasses `TypeError`: the caller attempted an operation that is ill-typed *categorically* — composing arrows whose middle objects differ, or wrapping a non-endo arrow as an `Endomap`. The distinction mirrors the mathematical one between a malformed datum and a well-formed datum used at the wrong type, and it lets test fixtures assert the precise failure class rather than a generic exception. Both exception types carry human-readable messages that name the offending arrow or object by its presentation name and spell out both type signatures in the composition case, so a rejection is diagnosable from the message alone.

2.3. The ordered-input guard. The private helper `_ordered_tuple(values, field)` is the single chokepoint through which every ordered input in the package passes. Its docstring reads: "Reject containers whose iteration order is not part of their contract." The implementation is a single guard:

```
if (isinstance(values, (str, bytes, bytearray, Mapping, Set)) or
not isinstance(values, Sequence)):  raise ValidationError(...)
```

followed by `return tuple(values)`. The accepted inputs are exactly the objects registered as `collections.abc.Sequence` — lists, tuples, and other sequence types with a stable, replayable order — minus the string-like sequences `str`, `bytes`, and `bytearray`, which are technically sequences but almost always signal a caller error when passed where a sequence of elements is expected. Rejected are: Python `set` and `frozenset` values (whose iteration order depends on insertion history and on `PYTHONHASHSEED`), mappings (whose iteration yields keys, and whose acceptance would silently discard values), and one-shot iterables such as generators (which are not `Sequence` instances, and whose consumption would make a second validation pass observe different data).

RULE 9.2 (Ordered-input rejection). Carrier elements, arrow mappings, path arrow sequences, word label sequences, catalog object sequences, catalog generator sequences, and lift coordinate vectors are all validated through this guard (or, for the two lift vector entry points, through an inline copy of the same check). A set, mapping, or one-shot iterable at any of these positions raises `ValidationError` at construction, before any downstream computation can observe hash-order data.

The error message is deliberately uniform — "`<field> must be an ordered sequence; sets, mappings, and one-shot iterables are not accepted`" — with the field name interpolated, so every rejection site reports both what was wrong and where. The rationale, developed at length in Chapter 3, is compressed into the specification's one sentence: the order of these inputs changes basis indices, typed denotations, or shortlex rank, so silently accepting hash-order iteration would make results depend on interpreter state. The guard converts a reproducibility hazard into an immediate, attributable construction error.

2.4. Compact integer labels. The second private helper, `_compact_integer_label(value)`, renders a nonnegative integer for use inside a presentation name. For integers of at most 256 bits it returns the ordinary decimal string. For larger integers it encodes the value big-endian into bytes, takes the first sixteen hexadecimal characters of the SHA-256 digest of that encoding, and returns the string `bits=<bit_length>,sha256=<digest16>`.

Two facts make this helper load-bearing rather than cosmetic. First, CPython by default refuses to convert integers with more than a few thousand digits to decimal strings (the integer-string conversion length limit), so naive `str(exponent)` inside a power name would raise for very large exponents; the helper's cost is $O(\log n)$ in the exponent and cannot hit that limit. Second, the label appears only in presentation names, which Definition $4.x$ of the arrow discipline (and the field declaration `compare=False` documented below) exclude from extensional equality — so the truncated digest cannot introduce a collision into any equality, hashing, or denotation computation. The regression suite exercises exponents 2^{30} (below the threshold, decimal path) and 10^{5000} (above it, digest path).

2.5. `FiniteObject`.

CONSTRUCTION 9.3 (The object type). `FiniteObject` is a frozen dataclass with `slots`, with two fields: `name: str` and `elements: tuple[Hashable, ...]`. Construction runs a `__post_init__` validator that enforces, in order:

(1) `name` is a `str` whose `strip()` is nonempty ("a finite object needs a non-blank name");
(2) `elements` passes the ordered-input guard of Rule 9.2 and is frozen to a tuple;
(3) every element is hashable — checked by attempting to build a `set`; a `TypeError` from unhashable elements is caught and re-raised as `ValidationError` with the original exception chained as its cause;
(4) the elements are pairwise distinct — the set's cardinality must equal the tuple's length.

The public surface beyond the two fields is small: `index(element)` returns the standard-basis index of an element, converting the underlying `ValueError` of tuple search into a `ValidationError` whose message names both the element and the object; and `__len__` returns the carrier size, so $|X|$ is written `len(x)` in client code.

INVARIANT 9.4 (Name-inclusive object identity). Because `FiniteObject` is a frozen dataclass with both fields participating in equality and hashing, two objects are equal exactly when their names and their carrier tuples (with order) are equal. The class docstring states

the consequence: two carriers with the same elements but different declared object identities "are not silently interchangeable." Every downstream type check — composition legality, endomap admissibility, path segment matching, catalog membership — inherits this identity, so the name participates in typing everywhere, uniformly, through ordinary equality.

Freezing matters beyond immutability hygiene: frozen dataclasses are hashable, and `normal_forms.py` uses objects directly as dictionary keys (in the object-rank table and inside denotation keys). The `slots` declaration additionally prevents attribute injection — no caller can attach ad hoc metadata to an object and smuggle state past the declared fields.

2.6. `FiniteArrow`.

Construction 9.5 (The arrow type). `FiniteArrow` is a frozen, slotted dataclass with four fields: `name` (declared `field(compare=False)`), `source` and `target` (both `FiniteObject`), and `mapping`, a tuple of integers in which `mapping[i]` is the target-basis index of the image of source element i. Validation enforces, in order:

(1) a non-blank presentation name;
(2) `source` and `target` are `FiniteObject` instances ("an arrow source and target must be FiniteObject values");
(3) `mapping` passes the ordered-input guard;
(4) the mapping length equals the source carrier length — totality; the message counts the required images explicitly ("needs one image for each of the n source elements");
(5) every image index is an `int` that is *not* a `bool` (the guard `isinstance(image_index, bool)` is checked first, because Python's `bool` subclasses `int` and would otherwise pass silently as 0 or 1);
(6) every image index lies in the half-open range $[0, |Y|)$; the message names the offending index and the target object.

Invariant 9.6 (Extensional arrow equality). Because `name` is declared with `compare=False`, dataclass equality and hashing are computed from `(source, target, mapping)` only. Two arrows with different presentation names but identical typed mapping data are equal and hash identically; conversely, arrows differing in source, target, or any image index are unequal even under the same name. This is the executable form of the extensionality discipline of Chapter 4: names are for humans and diagnostics, denotations are for mathematics.

The asymmetry with `FiniteObject` — where the name *does* participate in equality — is deliberate and worth pausing on. An object's name is part of its type identity: two registers that both carry $\{0, 1\}$ are different objects, and conflating them would license exactly the silent coercions the kernel exists to reject. An arrow's name, by contrast, is a presentation artifact: the same function $f : X \to Y$ may be constructed under many names (as a composite, as a power accumulator, as a closure candidate), and all of those constructions must be recognized as the same denotation for the normal-form machinery to work at all. The code encodes the distinction in one keyword argument.

Three further members complete the arrow surface:

- `__call__(element)` applies the arrow to a carrier element: it resolves the element's source index via `source.index` (raising `ValidationError` for foreign elements) and returns the target carrier element at the mapped index. Arrows are thus directly usable as functions on carrier elements, not only on indices.

- `is_endomap` is a property testing `source == target` — name-inclusive object equality, per Invariant 9.4. An arrow between two distinct same-carrier objects is *not* an endomap.
- `denotation_key` is a property returning the triple `(source, target, mapping)` — "a name-independent key for the denoted finite function," in the docstring's words. This triple is precisely the equality-participating data, and `normal_forms.py` uses it as the dictionary key for the closure table.

2.7. Identity and composition. Two module-level constructors realize the categorical structure.

`identity(obj, *, name=None)` builds the identity arrow on an object: source and target both `obj`, mapping `tuple(range(len(obj)))`, default name `id[<object name>]`.

`compose(after, before, *, name=None)` returns the categorical composite "`after` after `before`." Its docstring is explicit about the guard: it returns the composite "after checking the middle object exactly." The check is

```
if before.target != after.source:  raise CompositionTypeError(...)
```

where the inequality is full object inequality — name and ordered carrier — and the error message prints both arrows with their complete type signatures (`f:X->Y then g:Z->W`), making the mismatch self-describing. When the check passes, the composite is a fresh `FiniteArrow` with source `before.source`, target `after.target`, mapping

```
tuple(after.mapping[index] for index in before.mapping)
```

and default name `(<after> o <before>)`. The mapping expression is the whole of composition: index chasing through two tuples, with no arithmetic, no floating point, and no reference to carrier elements. The argument order — `after` first — matches the mathematical notation $g \circ f$ read aloud, and the keyword-only `name` parameter keeps presentation naming an explicit, optional act.

Remark 9.7. Nothing in `compose` inspects carrier contents or cardinalities except through object equality. There is no code path that coerces a mismatch: no "compatible carrier" fallback, no implicit reindexing, no name-blind comparison. PO-TKS8-CORE-02 (reject every source/target mismatch) is discharged at the level of executable validation by this single guard plus the negative tests of Chapter 10 that drive foreign objects, same-carrier-different-name objects, and reversed-order arguments into it.

2.8. `Endomap`: certified membership in one monoid.

Construction 9.8 (The endomap wrapper). `Endomap` is a frozen, slotted dataclass wrapping a single field `arrow: FiniteArrow`. Its validator enforces two conditions: the wrapped value is a `FiniteArrow` (else `ValidationError`), and `arrow.is_endomap` holds (else `CompositionTypeError`, with a message that prints the arrow's actual type signature and states "so it is not an endomap"). The docstring calls the wrapper "an arrow certified to lie in one endomorphism monoid End(X)" — construction is the certificate.

The property `carrier` returns the (shared) source object. The substantive method is `power(exponent)`:

(1) *Exponent validation.* The exponent must be an `int`, not a `bool`, and nonnegative; otherwise `ValidationError` ("an endomap exponent must be a non-negative integer"). Zero is legal and yields the identity.

(2) *Binary powering.* The method initializes an accumulator to `identity(carrier)` under the internal name `power-accumulator`, sets the base to the wrapped arrow, and loops over the bits of the exponent: when the low bit is set, the accumulator becomes `compose(base, accumulator)`; the exponent is shifted right; and — only if bits remain — the base is squared via `compose(base, base)` under the internal name `power-square`. Guarding the final squaring on `remaining` being nonzero avoids one wasted composition per call, a small courtesy visible in the code.
(3) *Result packaging.* The method returns a fresh `Endomap` whose arrow carries the accumulator's mapping and the presentation name `<base name>^(<compact label>)`, with the exponent rendered by `_compact_integer_label` (Section 2.4).

Because each loop iteration performs at most two tuple-indexing compositions, the cost of `power(n)` is $O(\log n)$ compositions of $O(|X|)$ work each — so the factorial powers of the next paragraph are computable even where $|X|!$ is astronomically large as an integer, since only its bit pattern is traversed.

`factorial_idempotent_power()` returns `power(factorial(len(carrier)))` — the executable form of the safe finite power $f^{|X|!}$ of Chapter 5. `is_idempotent()` tests `compose(arrow, arrow) == arrow`, an extensional comparison in which the composite's fresh presentation name is irrelevant by Invariant 9.6.

REMARK 9.9 (The docstring as a claim boundary). The `power` docstring carries a sentence that belongs to the claim discipline as much as to the API documentation: "This operation is deliberately absent from `SemigroupoidPath`. It does not license powers of heterogeneous or time-varying paths." The prohibition that Chapter 6 develops mathematically is thus stated at the exact point in the code where a maintainer contemplating "adding `power` to paths for convenience" would read it.

2.9. `SemigroupoidPath`: heterogeneous composites without powers.

CONSTRUCTION 9.10 (The path type). `SemigroupoidPath` is a frozen, slotted dataclass wrapping `arrows: tuple[FiniteArrow, ...]`. Validation enforces, in order: the sequence passes the ordered-input guard; it is nonempty ("a semigroupoid path is nonempty"); every member is a `FiniteArrow`; and every adjacent pair matches, `before.target == after.source` under full object equality, with violations raising `CompositionTypeError` whose message prints both segments' type signatures under the prefix "ill-typed path segment."

The surface is: a classmethod `from_iterable(arrows)` that simply delegates to the constructor (its docstring — "Build from an explicitly ordered, replayable sequence" — restates the input contract); `source` and `target` properties projecting the first arrow's source and the last arrow's target; `composite()`, which folds `compose` left to right over the arrows, naming intermediate results `path-composite[k]` for the step count k, and returns a final arrow renamed `path-composite[n]` for the path length n; and `append(arrow)`, which returns a *new* path with the arrow adjoined (revalidating the junction through the constructor — there is no in-place mutation anywhere in the package).

INVARIANT 9.11 (No power operation on paths). `SemigroupoidPath` defines no `power` method, no `__pow__`, no `__mul__`, and no conversion to `Endomap`. A path whose endpoints happen to coincide still cannot be raised to a power without the caller explicitly extracting `composite()` and wrapping it in `Endomap` — at which point the endomap validator independently certifies that the composite is endo. The invalid inference from "sequence of generators" to "element of one finite monoid" is thereby not merely discouraged but unwritable in the type vocabulary the module exports, exactly as PO-TKS8-CORE-03 requires.

2.10. Design decisions visible in `category.py`. Gathering the threads, six decisions are legible in the module's source and worth recording as decisions rather than accidents:

(1) *Frozen, slotted dataclasses everywhere.* Immutability makes every value hashable and safely shareable; slots foreclose attribute injection; and both together mean an object or arrow, once validated, can never drift from its validated state.
(2) *Validation in `__post_init__`, never later.* All preconditions are checked at construction; no method needs to re-validate its own receiver. An invalid value is unrepresentable, not merely detected.
(3) *The equality asymmetry.* Object names are type-forming; arrow names are presentation-only. One `compare=False` keyword carries the whole extensionality discipline.
(4) *Two exceptions, mapped onto Python's own value/type split.* Malformed data is a `ValueError` subclass; ill-typed use of well-formed data is a `TypeError` subclass.
(5) *The `bool` guard.* Both the arrow mapping validator and the exponent validator explicitly reject `bool` before accepting `int`, closing the subclass loophole through which `True` would otherwise be read as the index 1.
(6) *Bounded names.* Presentation-name construction is $O(\log n)$ in any exponent via the compact label helper, so no name-building code path can dominate, overflow, or hit interpreter conversion limits — and since names are outside equality, the bounding is free of semantic cost.

3. `deterministic_lift.py`: the standard-basis lift

3.1. Purpose and representation policy. `deterministic_lift.py` realizes the lift functor of Chapter 7: for a typed arrow $f : X \to Y$, the linear map A_f on standard bases with $A_f e_x = e_{f(x)}$. The module docstring states both the norm fact — "the exact operator norm is $\sqrt{\max_y |f^{-1}(y)|}$ (zero for an empty domain)" — and the representation policy: "The implementation exposes exact integer-valued squared data wherever possible; only `operator_norm` converts to floating point."

That policy is the module's central design decision. Every matrix the module returns — the lift matrix itself, both Gram matrices, the support projector — is a tuple of tuples of Python `int`s. Every scalar invariant except one — squared norm, maximum fiber size, trace, rank — is an exact integer. The single floating-point value in the module is the property `operator_norm`, computed as `sqrt` of the exact integer `operator_norm_squared`; a caller who needs exactness compares squared norms and never calls it. This is what makes the 1,678-pair functoriality check of Chapter 10 an *exact* matrix comparison with no tolerance parameter: there is no rounding anywhere to tolerate.

3.2. Construction and the applied maps.

Construction 9.12 (The lift wrapper). `DeterministicLift` is a frozen, slotted dataclass wrapping `arrow: FiniteArrow`; its validator rejects non-`FiniteArrow` values with `ValidationError`. There is no other precondition — every typed arrow lifts, including arrows with empty source or empty target.

`matrix()` returns the $|Y| \times |X|$ zero-one matrix as a tuple of row tuples: entry (r, c) is `int(mapping[c] == r)`. Each column c therefore contains exactly one 1, in row `mapping[c]` — the column-selection form derived in Chapter 7.

`apply(vector)` applies A_f to source coordinates. Its guard repeats the ordered-sequence check inline (rejecting strings, bytes, mappings, sets, and non-sequences with "source coordinates must be an ordered sequence"), then checks the length against $|X|$ with a message reporting both expected and received counts. The computation is a scatter-add:

```
output[mapping[source_index]] += value
```

over an output list initialized to zeros of length $|Y|$. Coordinates on colliding inputs accumulate — this is precisely the summation $(A_f v)_y = \sum_{x \in f^{-1}(y)} v_x$, and it is why a constant map's lift is noncontractive: mass concentrates rather than spreads.

`adjoint_apply(vector)` applies the conjugate transpose A_f^* to target coordinates, with the mirrored guard and length check against $|Y|$. The computation is a gather:

```
tuple(vector[target_index] for target_index in mapping)
```

— entry x of the result is coordinate $f(x)$ of the input, which is $(A_f^* w)_x = w_{f(x)}$. Because all matrix entries are real integers, conjugation is trivially the transpose; the docstring nonetheless says "conjugate transpose," keeping the Hilbert-space reading in view. Note that the pair `apply`/`adjoint_apply` lets a test verify adjointness numerically — $\langle A_f v, w \rangle = \langle v, A_f^* w \rangle$ — without ever materializing a matrix.

3.3. Fibers and the derived operators. The property `fiber_sizes` computes, in one pass over the mapping, the tuple of fiber cardinalities $\big(|f^{-1}(y)|\big)_{y \in Y}$: a zero-initialized list of length $|Y|$, incremented at each image index. Everything else in the module is a pure function of this tuple and the mapping:

- `max_fiber_size` is `max(fiber_sizes, default=0)`; the `default` argument realizes the empty-target convention — and since an empty *source* yields an all-zero tuple, the empty-domain value is also 0, matching the docstring's parenthesis.
- `operator_norm_squared` returns `max_fiber_size` and is documented as "the exact value $\|A_f\|^2 = \max_y |f^{-1}(y)|$" — the norm theorem of Chapter 7 hard-coded as a property. Note the implementation does not *compute* a spectral radius; it computes the right-hand side of a theorem proved off-line. The structural tests of Chapter 10 are what tie this shortcut to matrix reality on the bounded family.
- `operator_norm` is the lone float: `sqrt(operator_norm_squared)`.
- `input_gram_matrix()` returns $A_f^* A_f$ directly from the collision structure: entry (x, x') is `int(mapping[x] == mapping[x'])`. No matrix multiplication is performed — the code implements the *lemma* that the Gram entry indicates fiber-mates, not the product definition.
- `output_gram_matrix()` returns $A_f A_f^*$ as the diagonal matrix of fiber sizes.
- `output_support_projector()` returns the diagonal zero-one matrix marking nonempty fibers — "the orthogonal projector onto $\mathrm{im}(A_f)$ in the codomain."

REMARK 9.13 (Three operators kept distinct). Chapter 7 warned that informal usage conflates $A_f^* A_f$, $A_f A_f^*$, and the support projector. The module's answer is architectural: they are three differently named methods with three different docstrings, and the second and third agree only when every fiber has size at most one. A caller cannot obtain one while asking for another, and the test suite pins each method to its own fixture.

3.4. Scalar invariants and predicates. The remaining properties read off the implemented consequences of the norm theory:

Property	Value	Justification in code
`trace_input_gram`	$\lvert X\rvert$	`len(arrow.source)`
`rank`	$\lvert\mathrm{im} f\rvert$	count of nonzero fiber sizes
`is_injective`	f injective	all fiber sizes ≤ 1
`is_isometry`	$A_f^* A_f = I$	delegates to `is_injective`
`is_partial_isometry`	—	delegates to `is_injective`
`is_idempotent`	$A_f^2 = A_f$	see below
`is_orthogonal_projector`	$A_f = A_f^* = A_f^2$	endomap and identity mapping

`trace_input_gram` does not sum the Gram diagonal; it returns the source size directly, implementing the identity $\mathrm{tr}(A_f^* A_f) = \lvert X\rvert$ rather than computing its left side — the same theorem-as-shortcut pattern as the norm, and tested the same way. `rank` counts nonempty fibers, implementing $\mathrm{rank}(A_f) = \lvert\mathrm{im} f\rvert$. `is_isometry`'s docstring records the equivalence it relies on ("whether $A_f^* A_f$ is the identity (equivalently, f injective)"), and `is_partial_isometry`'s docstring notes the special situation of total deterministic lifts: "for a total standard lift, equivalently whether f is injective." For general operators partial isometry is strictly weaker than isometry; for column-selection matrices the two coincide, and the code documents that it is using the coincidence rather than the general definition.

`is_idempotent` first returns `False` for non-endomaps — A_f^2 is not even typeable on one space otherwise — and then checks

```
all(mapping[image] == image for image in mapping)
```

which says exactly that f fixes every point of its image, the combinatorial characterization of $f \circ f = f$. No matrix is squared.

`is_orthogonal_projector` is the most carefully documented predicate in the module. Its docstring states the fact the implementation encodes: "A total deterministic lift on one declared carrier is an orthogonal projector only for the identity map. Non-endomaps are not operators on one Hilbert space and therefore return `False`." The check is accordingly `is_endomap` together with the mapping equaling `tuple(range(len(source)))`. The mathematics behind the docstring is Chapter 7's: an orthogonal projector must be self-adjoint, a zero-one column-selection matrix is symmetric only when it is a permutation matrix fixed by transposition on its support, and combining self-adjointness with idempotence forces the identity. The constant-map fixture — idempotent, norm $\sqrt{2}$, not self-adjoint — is exactly the witness that `is_idempotent` and `is_orthogonal_projector` must not be merged, and the permanent regression test holds them apart.

3.5. Error taxonomy of the lift module. The module introduces no new exception types. All rejections raise `ValidationError` imported from `category.py`: wrapping a non-arrow, passing an unordered container as a coordinate vector, or passing a vector of the wrong length. There is no `CompositionTypeError` here because the module performs no composition — functoriality is a property the *tests* check by composing arrows in `category.py` and comparing matrices, not an operation the lift class itself exposes. The lift module consequently has no way to combine two lifts incorrectly: it has no way to combine two lifts at all.

4. `normal_forms.py`: the shortlex closure engine

4.1. Purpose. `normal_forms.py` implements the denotational normal-form catalog of Chapter 8: given a finite, explicitly ordered set of typed generators over a finite object set,

compute for every reachable finite-function denotation the shortlex-least well-typed word denoting it. The module docstring compresses the claim discipline into three sentences: no rewrite system is assumed; finiteness of the carriers makes the denotation closure finite; and "construction either completes or raises a resource guard exception, so an incomplete table is never presented as canonical."

4.2. Error taxonomy. Two exception types are defined here, again with meaningful base classes:

Exception	Base class	Meaning
`ClosureLimitExceeded`	`RuntimeError`	a resource guard tripped; no partial catalog
`UnreachableDenotationError`	`LookupError`	arrow not generated by the declared generators

The docstring of `ClosureLimitExceeded` is a policy statement: "Raised instead of returning a partial normal-form catalog." `UnreachableDenotationError` subclasses `LookupError` because it is semantically a failed lookup — the caller asked for the representative of a denotation the closure does not contain — and distinguishing it from `ValidationError` lets callers separate "you asked a malformed question" from "you asked a well-formed question whose answer is that the arrow is outside the generated subcategory." Malformed inputs to this module (blank labels, undeclared objects, unknown generator labels, non-positive resource bounds) raise the imported `ValidationError`, keeping the package-wide taxonomy at four exception types total.

4.3. `Generator` and `TypedWord`. `Generator` is a frozen, slotted dataclass pairing a `label` (a `str` whose `strip()` must be nonempty) with an `arrow` (a `FiniteArrow`). The label is the unit of word spelling; the arrow is its denotation.

`TypedWord` is a frozen, slotted dataclass with fields `start: FiniteObject` and `labels: tuple[str, ...]` (defaulting to the empty tuple). Its validator checks that `start` is a `FiniteObject`, passes the labels through the ordered-input guard, and requires every label to be a nonempty string. The class docstring records a division of labor that the constructor cannot see past: "The constructor validates syntax. Categorical well-typedness is relative to a generator catalog and is certified by `NormalFormCatalog.evaluate`." A `TypedWord` standing alone is a spelling; only a catalog can say whether its labels resolve and compose. Carrying the start object explicitly is what makes the empty word meaningful — the empty word at X denotes id_X, and without a declared start there would be one empty word instead of one per object.

4.4. `NormalFormCatalog`: constructor validation. The catalog constructor takes an ordered sequence of objects, an ordered sequence of generators, and two keyword-only resource bounds: `max_arrows` (default 100,000) and `max_frontier` (default `None`, in which case it is set equal to `max_arrows`). Validation is extensive and worth enumerating completely, because each rule closes a distinct determinism or well-formedness hole:

(1) objects and generators each pass the ordered-input guard — their declared order is data (it fixes shortlex ranks), so unordered containers are rejected at the door;
(2) the object sequence is nonempty ("a normal-form catalog needs at least one object");
(3) every declared object is a `FiniteObject`; every declared generator is a `Generator`;
(4) the objects are pairwise distinct *as objects*, and separately their *names* are pairwise distinct — the second check is strictly stronger and exists so that error messages and external references can use names unambiguously;

(5) `max_arrows` and `max_frontier` are positive `int`s, with the `bool` subclass explicitly excluded, as in the arrow and exponent validators;
(6) generator labels are pairwise distinct — a duplicated label would make word spellings ambiguous;
(7) every generator's source and target appear among the declared objects ("generator ... references an undeclared object") — the catalog's object list is a closed world, and the closure can never escape it.

After validation the constructor builds four private tables — label to generator, label to rank (position in the declared generator order), object to rank (position in the declared object order), and the two closure dictionaries mapping denotation keys to representative words and to arrows — and then immediately runs `_build`. A catalog is therefore never observable in a half-constructed state: by the time the constructor returns, the closure is complete, or an exception has propagated and there is no catalog at all. This constructor-runs-closure design is the object-level form of the fail-closed policy.

4.5. Shortlex keys and evaluation. `shortlex_key(word)` maps a word to the triple

$$(|w|,\ \text{rank sequence of } w,\ \text{rank of start object })$$

realized as `(len(labels), ranks, start_rank)`. Unknown labels and undeclared start objects raise `ValidationError` with the offending name in the message. Python's lexicographic tuple comparison then implements exactly the shortlex order of Chapter 8 — length first, then generator ranks positionwise — extended by a final tie-break on the start-object rank, which orders the empty words of distinct objects and never separates two words of the same start and spelling. This start-rank component is a totalizing convenience for the catalog-wide sorted listing (`normal_forms` below), not a change to the per-denotation minimization: all words denoting one fixed arrow share the arrow's source, hence share the tie-break.

`evaluate(word)` certifies well-typedness by construction: it starts from `identity(word.start)` (rejecting undeclared starts), then left-folds `compose(generator.arrow, result)` over the labels, resolving each label through the catalog table (unknown labels raise `ValidationError`) and letting `compose` itself raise `CompositionTypeError` at the first ill-typed junction. The result is repackaged under the presentation name `word-evaluation[n]`. Evaluation is thus the one gate through which a word acquires a denotation, and it inherits every type check of Section 2.7 rather than re-implementing any.

Three query methods complete the public surface. `normalize(word)` evaluates the word and returns the stored representative for its denotation key — by completeness of the closure, any word that evaluates successfully over declared objects and generators has a representative, so `normalize` on well-formed input does not miss. `representative_for(arrow)` looks up an arbitrary arrow's denotation key directly and raises `UnreachableDenotationError` (naming the arrow) when the arrow is outside the generated closure — this is the method a downstream consumer uses to ask "is this function expressible in the declared generators, and if so, what is its canonical spelling?" The property `denotation_count` reports the closure size, and `normal_forms` returns all representatives sorted by `shortlex_key` — a deterministic, replayable listing of the entire generated subcategory.

4.6. The closure engine `_build`. The private method `_build` is the algorithmic heart of the module — the executable form of the representative-first closure of Chapter 8. Its machinery:

- a min-heap `frontier` of quadruples `(shortlex_key, serial, word, arrow)`;

- a monotone counter `serial` from `itertools.count`, whose value is the second heap component — it breaks exact key ties by insertion order and, decisively, prevents the heap from ever comparing the third and fourth components, since `TypedWord` and `FiniteArrow` admit equality but not ordering;
- a set `queued_denotations` of denotation keys currently in the frontier, used to deduplicate candidates before they are pushed.

The run has two phases. *Seeding:* for each declared object, in declared order, the empty word and identity arrow are pushed — each push preceded by a frontier-capacity check that raises `ClosureLimitExceeded` if the queued-denotation count has reached `max_frontier`. Identities are thus in the closure by fiat, with the empty word as their (trivially shortlex-least) spelling. *Exploration:* while the frontier is nonempty, pop the least quadruple; discard its key from the queued set; skip it if the key is already represented (a defensive guard — the push-side deduplication already prevents duplicate enqueueing, and the pop-side check makes the invariant locally evident); check the `max_arrows` guard *before* inserting, raising `ClosureLimitExceeded` if the accepted count has reached the cap; record the word as the representative and the arrow as the stored denotation; then attempt to extend by every generator in declared order, skipping generators whose source does not equal the current arrow's target, building the extended word (labels plus the generator's label) and the extended arrow (via `compose`, under the internal name `normal-form-closure-candidate`), and pushing the extension — with the same frontier-capacity check — only if its denotation key is in neither the representative table nor the queued set.

Invariant 9.14 (First insertion is shortlex-least). Words are popped in nondecreasing shortlex order (heap order on the key, with serial tie-break), and every word is pushed with the key `shortlex_key` assigns it. By the pruning-soundness argument of Chapter 8, the first word popped for a given denotation key is the shortlex-least well-typed word with that denotation, and pruning later words with the same key removes no denotation and no least representative. The dictionary insertion in `_build` is therefore the whole of the minimization: no post-processing compares or replaces representatives.

Construction 9.15 (Resource guards). Two independent caps bound the run. `max_arrows` bounds the number of accepted denotations: the guard fires on the pop side, before the insertion that would exceed the cap. `max_frontier` bounds the number of distinct denotations simultaneously queued: the guard fires on the push side, before the push that would exceed the cap — including during seeding, so a catalog declaring more objects than `max_frontier` fails immediately rather than after partial work. Both guards raise `ClosureLimitExceeded` with messages that name the tripped bound and repeat the policy: "no partial normal-form catalog was returned." Because the guards raise out of the constructor, the partially built dictionaries are unreachable — there is no API through which a caller could observe the truncated state.

Remark 9.16 (Why deduplicate the frontier by denotation). The push-side check — skip extensions whose key is already represented *or already queued* — is a completeness-preserving optimization justified by the same pruning argument: if some word for a denotation is already queued, the queued one has a shortlex key at most that of the candidate arriving later (candidates are generated in nondecreasing prefix order, and equal keys are resolved by the earlier serial), so discarding the newcomer never discards a least representative. Operationally it also makes `max_frontier` meaningful as a bound on *distinct* pending denotations rather than on raw queue length, tightening the resource semantics the guard enforces.

4.7. Design decisions visible in `normal_forms.py`. Four decisions deserve explicit record. First, *completion in the constructor*: there is no lazy or incremental mode, so every reachable catalog object is a completed catalog, and the fail-closed policy needs no cooperation from callers. Second, *denotation keys as the sole identity*: the closure tables are keyed by `arrow.denotation_key` — the extensional triple — so all the presentation names generated during search (`normal-form-closure-candidate`, `word-evaluation[n]`) are invisible to the mathematics, as they must be. Third, *ranks from declared order only*: generator rank and object rank are positions in the caller's sequences; nothing is sorted by name, hash, or carrier content, so the canonical forms are exactly as replayable as the caller's declared order — which Chapter 8's claim boundary makes versioned data. Fourth, *the serial tie-break*: a two-line idiom (`count()` plus a tuple slot) that simultaneously guarantees heap totality without ordering instances and makes exploration order fully deterministic given the declared orders — determinism again pushed into structure rather than left to accident.

5. Manifest binding of the implementation

5.1. The artifact manifest. The prototype is bound by the artifact manifest `TKS_V8_CORE_PROTOTYPE_MAN` (artifact id TKS-V8-TYPED-CORE-PROTOTYPE, artifact version 0.1.0, schema version 0.1). The manifest records the artifact's status block — GATE 1 PROTOTYPE / IN REVIEW / NOT FROZEN, admission ring 4, license status NOT LICENSED — its provenance (the governing mathematical red-team report, the theorem ledger, and the three theorem record ids THM-LIFT-NORM-FIBER-FORMULA, THM-LIFT-OCCUPATION-COUNTEREXAMPLE, and THM-FINITE-IDEMPOTENT-POWER), its integration boundary, its formal-verification and validation snapshots (documented in Chapters 10 and 11), its claim records, and — the subject of this section — its *artifact bindings*: SHA-256 digests and byte counts for every file the artifact comprises.

5.2. The eight code bindings. Four implementation files and four test modules are hash-bound with the following roles, quoted from the manifest:

Path	Role	Bytes
`src/tks_v8_core/__init__.py`	package exports	1,018
`src/tks_v8_core/category.py`	typed finite-function category	9,850
`src/tks_v8_core/deterministic_lift.py`	deterministic standard-basis lift	5,483
`src/tks_v8_core/normal_forms.py`	denotational shortlex closure	9,889
`tests/tks_v8_core/test_category.py`	category and power regressions	5,588
`tests/tks_v8_core/test_deterministic_lift.py`	lift structure and functor-law regressions	9,142
`tests/tks_v8_core/test_manifest_boundaries.py`	claim and integration boundary regression	1,965
`tests/tks_v8_core/test_normal_forms.py`	normal-form correctness and resource regressions	5,672

The SHA-256 digests, exactly as recorded:

- `__init__.py`: `6D7559758D5CB9A2CE4A9CD0EC152D0C8A7B9057708429571FEC1374639CDB77`
- `category.py`: `B69F494814E9D9CD54C80ADD8D6AF850C5A8F762B1A0C6DFC85435BA4458D947`
- `deterministic_lift.py`: `A4649F62B44ED0DE820316E930958D2E32E143406DC1F9BC169A8EC533095115`

- normal_forms.py:
  7C85D6BA77E39F953F59916C907E4D7C9C0564C7AB0E89CAE05087188BC596E5
- test_category.py:
  0D8592E1231FA02263E63496BE0774441610D3E11A17A6D18DF39DBBCBD789C2
- test_deterministic_lift.py:
  D14126D80A056ED222DB3DFC58BEA4DFFF0E6466D31C43BC8D7B2F78CE509D01
- test_manifest_boundaries.py:
  9D3D1DEA155A5E1F8A13638B915EEF28C4DF51A90226B5826BA862BF10F2E097
- test_normal_forms.py:
  A4DE6902CC4AD986AAAB1A448473C3D72AE528907577E6F6F1CE328E0C396F9B

Beyond the eight code files, the manifest binds four further artifacts: the formal specification document TKS_V8_CORE_FORMAL_SPEC_AND_PROOF_OBLIGATIONS_v0.1.md (role: formal specification and explicit obligations; SHA-256 F740BA05AB7ADB2DCDF55E271016D127E45073B015497DE8982D44
10,917 bytes), the integrated Lean receipt LEAN_PROOF_MANIFEST_v0.1.json (7C27E50DCD6E45177F279A3B57BC
5,959 bytes), the lift-functor formal subreceipt LEAN_DETERMINISTIC_LIFT_FUNCTOR_MANIFEST_v0.1.json (BEED66D60666D48EF33204779D8DCE0557AD43245E5D89CD63691CC97492C3A2, 4,509 bytes), and the verifier script scripts/verify_tks_v8_gate1.py (role: hash and executable test verifier; SHA-256 E4FF72C5980059BE9359509827CE4D96F39D12765A003CD25AD5063A0590BC28, 4,230 bytes). The specification of Chapters 1–8, the Lean receipts of Chapter 11, and the verifier of Chapter 10 are thus all inside the same binding envelope as the code they govern: any byte change to any of the twelve files is detectable against the manifest.

5.3. What the binding does and does not establish.

Protocol 9.17 (Hash-binding discipline). The binding envelope supports one operation, implemented by scripts/verify_tks_v8_gate1.py and detailed in Chapter 10: recompute the SHA-256 of every bound path, compare against the manifest, and fail on any mismatch or missing file before rerunning the executable suite. A successful run establishes that the bytes examined are the bytes the manifest describes and that those bytes pass their own tests — nothing further.

Claim Boundary 9.18 (The manifest does not hash itself). The manifest's own binding_boundary field states the limit of the scheme in its own words: "The manifest does not hash itself. A later reviewed commit or acceptance receipt must bind this manifest SHA-256. Until then this is an uncommitted Gate 1 candidate, not a freeze receipt." The point is structural, not procedural: a document cannot contain its own digest, so the chain of custody necessarily terminates outside the manifest — in a reviewed commit, a signed receipt, or some other artifact that records the manifest's hash under independent authority. Until such an outer binding exists, an adversary (or an accident) that rewrites the manifest *and* the files consistently would pass verification; the manifest therefore certifies internal consistency of the snapshot, and the snapshot's authority awaits the outer commitment. Recording this boundary here, in the manifest's own language, is part of documenting the artifact as it is.

6. Summary

The implementation is small by design: three modules, roughly twenty-five kilobytes, four exception types, no dependencies beyond the standard library. Its character is uniform across the modules. Validation is total and constructor-resident, so invalid values are unrepresentable rather than detected. Identity discipline is carried by ordinary equality on frozen data, with exactly one asymmetry — objects own their names, arrows do not — doing the

work of the extensionality theory. Determinism is enforced at the input boundary by a single ordered-sequence guard used everywhere order is semantic. Theorems proved in the mathematical chapters appear in the code as directly implemented right-hand sides — the norm as a maximum fiber size, the trace as a source cardinality, idempotence as an image-fixing check — with the bounded-exhaustive suites of Chapter 10 tying the shortcuts back to matrix reality. Prohibitions appear as absent methods (`power` on paths) and as predicates that refuse to conflate (`is_idempotent` against `is_orthogonal_projector`). Resource limits fail closed, out of constructors, leaving no observable partial state. And the whole — sources, tests, specification, Lean receipts, verifier — sits inside one hash-binding envelope whose sole acknowledged gap, self-binding, is documented in the artifact's own text and assigned to a later reviewed commit. Chapter 10 examines how this code is tested; Chapter 11, what a proof assistant has and has not checked about its mathematics; Chapter 12, what may and may not yet be built on it.

CHAPTER 10

Verification Methodology

A kernel whose entire purpose is to make invalid inferences unwritable must hold itself to a correspondingly explicit standard of evidence. This chapter documents how the TKS v8.0 prototype is tested: the bounded-exhaustive philosophy that governs the suite design, the four test modules and the exact coverage they achieve, the boundary regression that pins the artifact's claim discipline into the suite itself, the Gate 1 verifier that binds hashes to test runs, and — given equal weight throughout — a precise statement of what a green suite does *not* establish. The numbers in this chapter are the numbers of the validation snapshot of 2026-08-04 recorded in the prototype manifest: result PASS, 20 tests run, with bounded exhaustive coverage of 19,683 associativity triples, 289 endomaps, 499 typed maps, and 1,678 lift-functor composition pairs. No number below is a projection or an extrapolation; each is either quoted from the manifest or derived arithmetically and marked as a derivation.

1. The bounded-exhaustive philosophy

1.1. Small scope as a working hypothesis. The suite's design principle is *exhaustion within an explicit bound*: rather than sampling large instances at random, enumerate *every* instance up to a small size and check the property on all of them. The methodological warrant is the small-scope hypothesis articulated by Jackson in the context of the Alloy analyzer: a high proportion of bugs in declarative models and structured code can be exhibited by counterexamples of small size, so exhaustively checking all small instances is a disproportionately effective use of finite checking budget [15]. The hypothesis is empirical and defeasible — it is a claim about where bugs tend to live, not a theorem about where they must — and the suite treats it as exactly that: a rational allocation policy for executable evidence, never a substitute for proof.

For this kernel the hypothesis has unusually strong purchase, for a structural reason worth spelling out. The properties under test — associativity of index-chasing composition, neutrality of identities, idempotence of factorial powers, functoriality of the lift — are *uniform* in carrier size: the code that handles a three-element carrier is the same code, on the same control path, that handles a three-hundred-element carrier. There are no size-dependent branches, no buffering thresholds, no algorithmic mode switches anywhere in the three modules. A defect in the composition formula, the powering loop, or the matrix builder would therefore almost certainly manifest at the smallest size at which the relevant structure exists — collisions need two colliding elements, associativity needs three composable arrows, nontrivial cycles need two points. Exhausting small scopes covers every *shape* of instance even though it cannot cover every *size*, and for uniform code, shape is where the bugs are.

1.2. Contrast: sampled property-based testing. The natural alternative is property-based random testing in the QuickCheck tradition [24]: state the law as an executable predicate, draw pseudorandom instances from a generator distribution, and shrink any failure to a minimal counterexample. The two disciplines share the crucial first step — both force

the specification into executable form — and differ in their quantifier. A QuickCheck-style run establishes "the property held on n samples drawn from this distribution"; a bounded-exhaustive run establishes "the property holds on *all* instances within this bound." The former statement is probabilistic and distribution-relative: its evidential weight depends on how well the generator's measure covers the shapes where defects hide, and a generator that rarely produces, say, empty carriers or maximal-collision mappings can pass indefinitely against a defect it never exercises. The latter statement is categorical within its bound and requires no distributional argument at all — but it is silent beyond the bound.

The kernel's suite chooses exhaustion for three reasons. First, determinism: a random suite's failures depend on seeds, and this artifact's entire identity discipline is a war on interpreter-state dependence; a test suite whose coverage varied by seed would be tonally at odds with the code it checks, and a replayable suite is a precondition for the hash-bound verification protocol of Section 5. Second, countability: the instance spaces at small scope are so small (hundreds to tens of thousands, computed exactly below) that exhaustion is cheaper than any statistically meaningful sampling campaign — when the whole space costs less than a second, sampling from it is strictly dominated. Third, auditability: "all 1,678 composable pairs through size three" is a claim a reviewer can recount from first principles, as we do below, whereas a sampling claim can only be re-audited by rerunning the sampler. None of this is a criticism of the sampled discipline in its home territory — vast or infinite instance spaces where exhaustion is impossible — but that territory is precisely what the kernel's finite, small-scope design avoids.

Claim Boundary 10.1. Exhaustive-within-bounds is still bounded. Every coverage claim in this chapter carries an explicit scope — carriers through size three or four — and establishes nothing, even empirically, about behavior beyond that scope. The bridge from bounded evidence to universal statements is mathematical proof (Chapters 4–8) and, where present, kernel-checked formalization (Chapter 11); the suite's role is to tie the running code to the proved mathematics on the bounded family, not to extend the mathematics.

1.3. The oracle problem. Every test needs an oracle: an independent source of the expected answer. The suites resolve the oracle problem differently per family, and the differences are methodologically instructive.

For the *law families* — associativity, identity neutrality, functoriality — the oracle is internal to the law itself: the test computes both sides of an equation through the implementation and asserts their equality. This is the cheapest kind of oracle and also the weakest in one specific sense: a defect that breaks both sides *identically* — say, a composition routine that consistently reversed its arguments — can satisfy an equational assertion while computing the wrong function. The suites compensate by anchoring the equations to independently constructed values at their edges: identities are built by an independent constructor (`identity`), not by composing anything, so the unit laws compare a computed composite against a value the composition code never touched; and the lift functor law compares a matrix *product*, computed by explicit integer summation in the test, against a matrix built directly from the composed arrow — two genuinely different computational routes to the same object.

For the *theorem families* — factorial idempotence, the norm consequences — the oracle is the informal proof: the test asserts the proved right-hand side (idempotence of $f^{|X|!}$; trace equal to $|X|$; rank equal to image size) against implementation output. Here the mathematics is the independent authority, and the test would catch either an implementation defect or — more interestingly — a misremembered theorem, since the asserted value is written into the test from the proof, not read from the code.

For the *fixture families* — the two-state noncontractivity example, the hand-worked closure catalogs — the oracle is hand computation, of exactly the kind Appendix A performs in print. These are the strongest oracles in the suite (fully independent of both the implementation and any single theorem) and necessarily the narrowest.

1.4. Determinism of the suite itself. The suite contains no randomness: no seeds, no time-dependent values, no reliance on dictionary iteration order in any assertion. Enumeration of the exhaustive families is by nested iteration over mapping tuples in lexicographic order — itself a replayable sequence — so not only the verdict but the complete execution trace of the suite is identical on every conforming interpreter. This property is not merely aesthetic consistency with the kernel's determinism discipline. It is what makes the Gate 1 verifier's conjunction meaningful: "these bytes pass these tests" is a reproducible fact only if "these tests" denotes one fixed computation, and a suite with ambient nondeterminism would smuggle interpreter state back into the artifact's evidence base through the test channel after the implementation had so carefully evicted it.

2. The four suites

The test tree `tests/tks_v8_core` contains four modules, each hash-bound in the manifest with a declared role (Chapter 9, Section on manifest binding): `test_category.py` (category and power regressions, 5,588 bytes), `test_deterministic_lift.py` (lift structure and functor-law regressions, 9,142 bytes), `test_normal_forms.py` (normal-form correctness and resource regressions, 5,672 bytes), and `test_manifest_boundaries.py` (claim and integration boundary regression, 1,965 bytes). Together they comprise the 20 tests of the snapshot. The suites run under Python's standard `unittest` discovery — `python -m unittest discover -s tests/tks_v8_core -v` with `src` on `PYTHONPATH` — with no third-party test framework, no fixtures requiring network or filesystem state, and no randomness anywhere: every run of the suite is the same run.

2.1. Category and power regressions. The category suite carries the two largest exhaustive families.

Associativity on a three-point carrier: 19,683 *triples.* Fix one three-element carrier X. The endo-arrows on X are the total functions from a three-element set to itself, of which there are $3^3 = 27$; every ordered triple (f, g, h) of them is composable in both associations, and there are $27^3 = (3^3)^3 = 19{,}683$ such triples. The suite composes each triple both ways — $(h \circ g) \circ f$ and $h \circ (g \circ f)$ — and asserts extensional equality of the results. This family checks the associativity component of the category laws (Chapter 4) on every possible shape of collision, fixed point, and cycle structure that three points admit, which includes every way two composed collisions can interact. Identity neutrality is checked on typed families as well as endo families, so the unit laws are not confined to the single-carrier world.

Endomaps through size four: 289 *maps.* For each carrier size $k \in \{0, 1, 2, 3, 4\}$ the endomaps on a k-element carrier number k^k (with the empty case counting the one empty map, $0^0 = 1$):

$$\sum_{k=0}^{4} k^k \;=\; 1 + 1 + 4 + 27 + 256 \;=\; 289,$$

a derivation consistent with the manifest's recorded count. For every one of these 289 endomaps the suite computes the factorial power $f^{|X|!}$ through the `Endomap.power` pathway — exercising the binary powering loop, not a naive iteration — and asserts idempotence of

the result by extensional comparison, the executable face of the safe finite power theorem of Chapter 5. The family includes the degenerate sizes ($0! = 1! = 1$), all permutations (where $f^{|X|!} = \mathrm{id}$), all constants, and every mixed tail-and-cycle shape through four points.

The choice of bound four is not arbitrary, and it is worth recording what size four buys that size three does not. The proof of the factorial theorem (Chapter 5) has two moving parts — tail absorption and cycle divisibility — and the interesting stress cases are those where the least idempotent exponent is governed by the *least common multiple* of several cycle lengths rather than by any single cycle. On three points the possible cycle structures are limited (cycle lengths 1, 2, 3, never two nontrivial cycles at once); on four points the double transposition appears — two disjoint 2-cycles — along with the 4-cycle, the 3-cycle-plus-fixed-point, and every combination of a nontrivial cycle with a genuine tail. Size four is thus the first size at which the family contains maps whose cycle structure is plural, and $4! = 24$ is the first factorial that must simultaneously absorb a length-3 tail bound and divide out cycle lengths 1 through 4. The family also stresses the powering *implementation* distinctly at each size: the exponents actually exercised are $1, 1, 2, 6, 24$, whose binary expansions (1, 10, 110, 11000) drive different accumulate/square interleavings through the loop of Chapter 9.

Typed maps through size four: 499 *maps.* Across ordered pairs of carrier sizes (m, n) with $0 \le m, n \le 4$, the total maps from an m-element source to an n-element target number n^m — zero when $n = 0 < m$, one when $m = 0$. Summing:

$$\sum_{m=0}^{4} \sum_{n=0}^{4} n^m \;=\; 5 + 10 + 30 + 100 + 354 \;=\; 499,$$

again a derivation consistent with the recorded count (the five terms are the sums for $m = 0, 1, 2, 3, 4$ respectively). These typed families drive the constructor validation and the composition guard across *distinct* carriers: identity neutrality with typed identities on both sides, and rejection behavior — every source/target mismatch raising `CompositionTypeError` — are checked here, where same-carrier coincidences cannot mask a name-blind or cardinality-blind comparison.

The suite's regression (as opposed to exhaustive) component pins the powering machinery at scale: the exponents 2^{30} and 10^{5000} exercise the bounded presentation-name construction of Chapter 9 — the second exponent would overflow Python's decimal-conversion limit if names were built naively — and the negative fixtures drive sets, mappings, generators, blank names, out-of-range indices, boolean indices, and negative exponents into their respective `ValidationError` and `CompositionTypeError` rejections, one assertion per taxon of the error taxonomy.

2.2. Lift structure and functor-law regressions. The lift suite is the largest test module by byte count, and its central family is the exact functoriality sweep.

Lift-functor composition pairs through size three: 1,678 *pairs.* Consider all composable typed pairs $f : X \to Y$, $g : Y \to Z$ with $|X|, |Y|, |Z| \le 3$, including empty carriers. For fixed sizes (a, b, c) there are $b^a \cdot c^b$ such pairs, and summing over the middle size first,

$$\sum_{b=0}^{3} \Big(\sum_{a=0}^{3} b^a \Big) \Big(\sum_{c=0}^{3} c^b \Big) \;=\; 1 \cdot 4 + 4 \cdot 6 + 15 \cdot 14 + 40 \cdot 36 \;=\; 4 + 24 + 210 + 1{,}440 \;=\; 1{,}678,$$

a derivation consistent with the manifest count. For every pair the suite builds the three matrices A_g, A_f, and $A_{g \circ f}$ through `DeterministicLift.matrix()`, multiplies $A_g A_f$ in

exact integer arithmetic, and asserts *entrywise equality* — tuples of tuples of `int`s compared with `==`, no tolerance, no floating point anywhere in the comparison. Identity preservation $A_{\mathrm{id}_X} = I$ is checked for every carrier in scope by the same exact comparison. This is the executable half of the lift functor laws (Chapter 7); the kernel-checked half, and the gap between them, is Chapter 11's subject.

The structural regressions of the module tie the theorem-as-shortcut properties of `deterministic_lift.py` back to matrix reality on the bounded family: fiber sizes against explicitly counted preimages, the squared norm against the maximum fiber size, the trace shortcut against the actual Gram diagonal sum, rank against the count of nonempty fibers, the input Gram matrix against a directly computed $A_f^{\mathsf{T}} A_f$, the output Gram against the diagonal of fiber sizes, and the support projector against the zero-one support pattern — with fixtures where $A_f A_f^*$ and the support projector *differ* (any fiber of size two or more), so the three-operator distinction of Chapter 9 is enforced by test, not just by method naming. The permanent noncontractivity fixture — the constant map on two points with $A_f^2 = A_f$, norm $\sqrt{2}$, Gram trace 2, rank 1, neither a partial isometry nor an orthogonal projector — is asserted property by property, standing guard over the two tempting over-readings ("deterministic lifts are contractive," "idempotent lifts are projectors") that Chapter 7 identified and the theorem ledger records as THM-LIFT-OCCUPATION-COUNTEREXAMPLE.

2.3. Normal-form correctness and resource regressions. The normal-form suite verifies the closure engine of Chapter 9 on hand-checkable catalogs, in three bands. *Correctness:* for small generator catalogs whose closures can be enumerated by hand (of the kind worked in full in Appendix A), the suite asserts the exact denotation count, the exact representative word for each reachable denotation — shortlex-least, verified against independent enumeration — the identity of `normalize` on words with equal denotations, and the stability of the `normal_forms` listing order. *Rejection:* constructor-level negative fixtures cover duplicate generator labels, duplicate object names, generators referencing undeclared objects, unknown labels at evaluation, undeclared start objects, and unordered container inputs; `representative_for` on an arrow outside the generated closure is asserted to raise `UnreachableDenotationError` specifically. *Resource guards:* catalogs constructed with deliberately small `max_arrows` or `max_frontier` are asserted to raise `ClosureLimitExceeded` — and, decisively, to leave *no* accessible partial catalog: the exception propagates from the constructor, so there is no object on which a truncated table could be observed. The fail-closed policy is thereby a tested behavior, not a documentation promise.

2.4. The manifest-boundary regression. The fourth module is small — under two kilobytes — and unusual in kind: it tests the *claim discipline* rather than the code. It loads the prototype manifest and asserts the integration boundary fields on which every downstream reading of the artifact depends:

- `active_cas_compiler_modified` is `false` — the kernel has not touched the production notation compiler;
- `production_runtime_dependency` is `false` — no production system depends on the kernel;
- `canonical_tks_v8_status` is OPEN — the kernel is not canonical TKS v8;
- `cas_semantic_transfer_status` is OPEN / NOT LICENSED — no CAS semantic content has been mapped onto the kernel.

REMARK 10.2 (Why boundaries belong in the suite). A boundary that lives only in prose can drift silently: a manifest edit that flipped `cas_semantic_transfer_status` to a

permissive value would, without this module, leave the suite green while the artifact's most important commitment quietly inverted. Placing the assertions in the suite makes boundary drift a *test failure*: the 20-test green result certifies not only that the code behaves but that the artifact still claims exactly as little as it is licensed to claim. This is the same instinct as the permanent noncontractivity fixture — encode the prohibition where the machinery will trip over it — applied to the artifact's governance metadata.

3. Coverage summary

Bounded exhaustive family	Cases	Law exercised
Three-point associativity triples	19,683	associativity (Ch. 4)
Endomaps on carriers through size four	289	factorial idempotence (Ch. 5)
Typed maps, sizes through four	499	typing, identity, rejection (Ch. 4)
Lift composition pairs through size three	1,678	functor laws, exact (Ch. 7)

All four counts are recorded in the manifest's validation snapshot (`validated_at` 2026-08-04, `result` PASS, `tests_run` 20) and re-derived arithmetically above. The per-family scopes differ deliberately: associativity burns its budget on one carrier at depth three (the law is about composition structure, not carrier variety); the endomap and typed-map families spend theirs on carrier variety through size four (the laws are about map shapes); the functor sweep stops at size three because its cost is cubic in the three sizes jointly and the matrix shapes it must distinguish — collision, non-surjectivity, empty domain and codomain — all occur by then.

Two structural remarks on the families' composition. First, the empty cases are inside the bounds, not excluded from them: the typed-map family includes the $m = 0$ column (one empty map into each target, including the empty map on the empty object) and the $n = 0 < m$ cells (no maps at all, which the enumerator must produce as an empty family rather than an error), and the lift sweep explicitly includes empty carriers — zero-by-n and n-by-zero matrices whose products are degenerate in ways that off-by-one code notoriously mishandles. The empty cases carry disproportionate defect-finding weight for exactly that reason. Second, the four families overlap by construction — a three-point endomap appears in the associativity family, the endomap family, the typed-map family, and (lifted) in the functor sweep — and the overlap is a feature: the same mathematical object is exercised through four different code paths (composition, powering, construction and typing, matrix building), so an inconsistency *between* code paths, not just within one, has four chances to surface.

A note on cost. The largest family runs 19,683 triple compositions of three-element mappings; the functor sweep performs 1,678 exact 3×3-bounded integer matrix multiplications; the whole 20-test suite completes in seconds on commodity hardware. The bounded-exhaustive design's practical thesis is visible in these numbers: at kernel scale, exhaustion within the interesting scope is cheaper than the infrastructure of sampling would be. The suite spends its complexity budget on *which* families to exhaust — the design decisions recorded above — and none on test-data machinery.

4. Trusting the tests

A verification methodology that scrutinizes the code and exempts the tests has merely relocated its trust, not reduced it. Three mechanisms address the suite's own trustworthiness; none eliminates it, and the residue is stated at the end.

First, the tests are bound. All four test modules carry SHA-256 bindings in the manifest with declared roles, and the Gate 1 verifier hashes them before running them (Section 5). A weakened assertion, a deleted family, or a broadened exception match is a digest change, detectable against the manifest exactly as an implementation edit would be. Test code is artifact, not scaffolding.

Second, the coverage counts are externally derivable. Each exhaustive family's cardinality is a closed-form combinatorial quantity — $(3^3)^3$, $\sum_k k^k$, $\sum_{m,n} n^m$, $\sum_b (\sum_a b^a)(\sum_c c^b)$ — recomputed independently in this chapter and checkable by any reviewer with a pencil. A suite that silently enumerated less than it claimed would disagree with arithmetic, not merely with its own comments. This is a modest but real defense: it pins the *quantifier* of each exhaustive claim to something outside the code.

Third, the assertion style is adversarial to weakening. The negative fixtures assert specific exception types (`CompositionTypeError`, not bare `Exception`) so a refactor that degraded a typed rejection into a generic crash would fail the suite; the fixture tests assert exact values (norm squared *equals* 2), not inequalities; and the boundary regression asserts exact manifest field values, not field presence.

The residue is honestly statable: the suite cannot certify its own *sufficiency*. Nothing in the mechanisms above excludes the possibility that some behavior of the implementation is simply never exercised — a code path outside every family and every fixture — and no coverage measurement is recorded in the manifest snapshot. Sufficiency review is part of what independent specialist review (PO-TKS8-CORE-10) is for, and the suite's self-discipline is a floor under that review, not a substitute for it.

5. The Gate 1 verifier

5.1. Protocol. The script `scripts/verify_tks_v8_gate1.py` — itself hash-bound in the manifest (4,230 bytes, role "hash and executable test verifier") — composes the two evidence mechanisms into a single replayable check, invoked as `python scripts/verify_tks_v8_gate1.py` from the repository root.

Protocol 10.3 (Gate 1 verification). The verifier performs, in order:

(1) *Hash check.* For every artifact binding in the prototype manifest, recompute the SHA-256 digest of the file at the bound path and compare it, together with the byte count, against the recorded values. Any missing file or digest mismatch fails the run before any test executes.
(2) *Suite rerun.* Execute the four-module test suite and require the full pass.

A successful run therefore certifies a conjunction: *these exact bytes* — and not some edited neighbor of them — *pass these exact tests.*

The ordering matters. Running hashes first means a green verifier run can never be produced by testing modified code: the failure mode "the tests pass because someone weakened them" is caught at step one, since the test modules are themselves bound. Conversely, the failure mode "the hashes match but the code is broken" is caught at step two. The two steps are individually cheap and jointly close the obvious substitution attacks on either evidence type alone — with the single documented exception, the manifest's own unbound digest, discussed in Chapter 9: until a reviewed commit binds the manifest hash, a consistent rewrite of manifest plus files together would pass. The verifier inherits that boundary; it does not resolve it.

5.2. Relation to the Lean verifier. A second verifier, `scripts/verify_psr8_lean_gate1.py`, reproduces the formal seed under the pinned Lean toolchain; it is the subject of Chapter 11 and is deliberately separate: executable evidence and kernel-checked evidence have different trust bases, different failure modes, and different scopes, and conflating them in one green light would blur exactly the distinctions this monograph exists to keep sharp. Gate 1 reproduction means running both.

6. What a green suite does not establish

This section is normative for every consumer of the artifact, and its content is quoted or directly derived from the manifest's own boundary field: "A passing bounded executable suite is not proof-assistant verification or independent specialist review."

Claim Boundary 10.4 (Non-claims of the executable evidence). A full pass of the 20-test suite, including under the Gate 1 verifier, does *not* establish any of the following.

(1) *Universal correctness.* The exhaustive families are bounded at carrier sizes three and four. The suite proves nothing about size five — the universal statements are carried by the proofs of Chapters 4–8, and their formalization status is the open ledger of Chapter 12, not a consequence of testing.
(2) *Proof-assistant verification of the Python.* The Lean seed of Chapter 11 checks a Lean model of the definitions, not the Python source; equivalence between the two is itself an open obligation. A green suite plus a green Lean build still leaves the running code unverified in the proof-assistant sense.
(3) *Independent review.* All tests were authored within the project. PO-TKS8-CORE-10 (independent specialist review) has no receipt and remains OPEN; nothing a self-authored suite does can close it.
(4) *Closure of any open obligation.* The ledger rows marked OPEN in Chapter 12 — formalizations, compiler integration, semantic reconciliation, review — are untouched by any number of green runs; the suite's role in the ledger is confined to the rows explicitly marked EXECUTABLE VALIDATION.
(5) *Semantic adequacy for TKS/CAS.* The suite contains no Noetic, Domain, Drive, or Register content — necessarily, since the code contains none. Green tests say nothing about whether the kernel's types are the *right* substrate for the canonical ontology; that is PO-TKS8-CORE-08, OPEN / NOT LICENSED.
(6) *Empirical claims of any kind.* No cognitive-dynamics, channel-contractivity, or CAS-efficacy claim follows from any test in the suite, per the standing claim boundaries of the manifest.

Remark 10.5 (The suite as a floor, not a ceiling). The correct reading of the executable evidence is as a floor under the artifact: the definitions are implementable, the implementation agrees with the proved mathematics everywhere a bounded check can look, the rejection behavior matches the error taxonomy case by case, the fail-closed guards actually fail closed, and the claim boundaries are mechanically pinned. Everything above that floor — universality, formal verification of the running code, review, semantic transfer — is exactly as open as the ledger says it is. Keeping the floor and the ceiling distinct is the methodology.

CHAPTER 11

The Formal Seed

Chapter 10 documented the executable evidence: bounded-exhaustive suites that tie the running Python to the proved mathematics wherever a finite check can reach. This chapter documents the artifact's second, independent evidence channel: a small body of machine-checked mathematics — the *formal seed* — developed in Lean 4 under a pinned toolchain and audited by a dedicated verifier script. The seed is deliberately modest in scope and immodest in rigor: what it checks, it checks down to the proof kernel with a single standard axiom; what it does not check, this chapter itemizes without euphemism. The governing records are the manifest's `formal_verification` block (status PARTIAL / PRESENT), the integrated Lean receipt `LEAN_PROOF_MANIFEST_v0.1.json`, and the lift-functor subreceipt, all hash-bound alongside the code they describe.

1. Kernel-checking as a trust story

1.1. Why a proof assistant, and what its word is worth. An informal proof — even a careful one, even one checked by several readers — is a social artifact: its correctness claim rests on the attention of the humans who have read it. A proof developed in Lean 4 [16] rests on a different foundation. Lean is architected around a small trusted kernel: the elaborate machinery that makes proofs writable — tactics, automation, syntax extensions, metaprogramming — all runs *outside* the trust boundary, producing proof terms that the kernel independently re-checks against the rules of the underlying dependent type theory. To trust a Lean-checked theorem one must trust the kernel's implementation of a well-studied type theory, the statement as written, and the axioms the proof depends on — and nothing else. Not the tactics that found the proof, not the libraries that suggested it, not the human who wrote it.

This architecture is the modern form of what the proof-assistant literature calls the de Bruijn criterion: the system's trustworthiness should rest on a small, independently auditable checker, with all proof-*finding* machinery outside the trusted base. Lean 4 satisfies the criterion by construction — the kernel implements a typechecker for its dependent type theory and nothing else, and every theorem, however it was found, is ultimately a term the kernel has typechecked against its stated proposition. The practical consequence for this artifact is a sharp asymmetry of failure modes. If a tactic is buggy, the worst case is a proof that fails to build — annoying, visible, safe. For a false statement to be *accepted*, the defect must be in the kernel itself, in the type theory it implements, or in the statement of the theorem — and the first two are shared, heavily exercised infrastructure whose failure would be a community event, not a private one. The third — a theorem statement that does not say what its author believes it says — is the one failure mode kernel-checking cannot address at all, and it is why the verifier discipline below insists on hash-bound sources and named theorem inventories: the statements themselves must be reviewable text, pinned to digests, so that what was checked is exactly identifiable.

That last dependency — the axioms — is where formal verification claims most often quietly leak. Lean permits axiom declarations, and its standard library itself provides classical axioms; a development can also locally postulate whatever it likes, and a proof "checked by Lean" may silently rest on an unproved assumption three imports away. Similarly, the placeholder `sorry` lets a development compile with holes. A verification claim is therefore only as strong as its *axiom audit*: an explicit accounting, per theorem, of exactly which axioms the checked proof term depends on, together with a policy on which axioms are admissible. The seed's verifier makes this audit mechanical, as Section 3 details: the sole admissible standard axiom is `propext` — propositional extensionality, the principle that logically equivalent propositions are equal, a standard and deliberately minimal extension of the core theory — and any project axiom or `sorry` anywhere in the build fails the verification run outright.

1.2. Why `propext`, and why only `propext`. The single admitted axiom deserves its own paragraph, because the choice encodes a policy. Propositional extensionality states that two propositions which imply each other are equal as propositions — an identification of logical equivalence with equality at the level of `Prop`. It is among the mildest of the standard classical extensions: it does not decide any proposition (it is not excluded middle), it does not choose witnesses (it is not choice), and it is validated by the standard proof-irrelevant semantics of `Prop`. Lean's own core library depends on it pervasively, which makes it the practical floor for any development that uses the standard library at all; a seed that admitted nothing would have to rebuild elementary logic from scratch for no gain in trust. The policy, then, is: admit the one axiom that the ecosystem's floor requires, and nothing above that floor. In particular the seed does not admit choice, quotient soundness beyond the kernel's built-in support, or classical excluded middle — and because the verifier audits dependencies per theorem, the claim is not "we tried to avoid stronger axioms" but "the compiled proof terms of the named theorems depend on `propext` and nothing else, mechanically checked." For the finite, discrete, computation-adjacent statements in the verified scope, this is also mathematically comfortable: nothing in typed identities, associativity, basis compositions, or two-state expansions has any classical content to hide.

1.3. The seed's place in the evidence architecture. The formal seed occupies a precise position between the other two evidence channels. Against the informal proofs of Chapters 4–8, it trades coverage for certainty: only a fraction of the monograph's mathematics is formalized, but that fraction is checked by a mechanism whose failure modes are qualitatively narrower than human review. Against the executable suites of Chapter 10, it trades concreteness for universality: the suites exercise the actual Python on bounded instances, while the seed proves *universal* statements — for all mappings, all carriers of the modeled form — about a Lean model that is not the Python. Neither channel subsumes the other, and the artifact's claim discipline insists on reporting them separately: the manifest's formal-verification status is PARTIAL / PRESENT, its claim field stating that the seed "kernel-checks only the exact modules and theorem dependencies named in the linked proof manifest," with everything else OPEN.

The three channels can be tabulated by what each quantifies over and what each trusts:

Channel	Quantifies over	Object of evidence	Trusted base
Informal proof	all instances	the mathematics	human review of the argument
Executable suite	bounded instances	the running Python	interpreter, test oracles, bound choice
Lean seed	all instances (in scope)	a Lean model	Lean kernel, type theory, statement fidelity

The table makes the complementarity mechanical. The suite is the only channel that touches the shipped implementation; the seed is the only channel whose checking is not human; the informal proofs are the only channel with full coverage. Any two channels overlapping on a statement — as all three do on the noncontractivity fixture — cross-check each other's distinct trusted bases, which is the strongest evidential configuration the artifact attains and the reason the fixture is maintained in all three forms.

2. The pinned toolchain

Formal verification inherits the reproducibility obligations of any other evidence. A proof that checks under one compiler version and fails under another — because of changed elaboration, moved library lemmas, or a revised axiom inventory — is not a stable receipt. The seed therefore pins its entire toolchain, and the manifest's environment probe of 2026-08-04 records the pin exactly:

Component	Recorded state
Lean	4.32.1 / `f054605aea4b840552cca2e725580bffd1e1b704` / PRESENT
Lake	5 / PRESENT
Python (verifier host)	3.14.0
`coqc`	UNAVAILABLE
Agda	UNAVAILABLE

The Lean entry pins not only the release number (4.32.1) but the exact toolchain commit hash, so "the toolchain" is an identified artifact rather than a version-string family. The build goes through Lake, Lean's standard build system, so module dependency resolution is itself declarative and replayable. The toolchain is furthermore *isolated* — pinned for this project rather than taken from whatever ambient Lean installation the host happens to carry — so a developer machine with a different global Lean cannot silently substitute its own elaboration behavior into a verification run. The recorded UNAVAILABLE entries for Coq and Agda are honesty about the probe environment, with a methodological implication worth noting: the seed is a single-prover development, and cross-prover corroboration — re-checking the same statements in an independent system with an independent kernel — is one of the things the artifact does not have.

Enumerating the trusted computing base explicitly, in decreasing order of scrutiny received: the Lean kernel at the pinned commit; the type theory it implements; the Lake build tool's module resolution; the Python 3.14.0 interpreter running the verifier script; the verifier script itself — which is why the script is kept small, its policy constants hardcoded rather than configurable, and its own bytes hash-bound in the artifact envelope; and the host operating system underneath all of it. The pin does not shrink this base; it makes the base *identified*, which is what turns "it verified" from an anecdote about some machine into a replayable claim about a named configuration.

3. The verifier discipline

The script `scripts/verify_psr8_lean_gate1.py` is to the formal seed what the Gate 1 verifier is to the executable suite: a single replayable command that either reproduces the full evidence or fails. Its checks, as recorded in the specification and the research paper, are these:

PROTOCOL 11.1 (Lean seed verification). The verifier, run as `python scripts/verify_psr8_lean_gate1.py`

(1) *hardcodes the sole allowed standard axiom*, `propext` — the admissible-axiom set is not a configuration option but a constant in the verifier source;
(2) *rejects project axiom declarations* — any `axiom` introduced by the development itself fails the run, so the seed cannot smuggle assumptions in as local postulates;
(3) *rejects* `sorry` — no proof hole anywhere in the built modules is tolerated;
(4) *checks source hashes* — the Lean source files are compared by digest against the proof manifest, so the theorems checked are the theorems bound, not edited neighbors;
(5) *builds through Lake* under the pinned toolchain of Section 2; and
(6) *audits exact theorem dependency reports* — for each named theorem, the axioms its compiled proof term actually depends on are extracted and compared against the expected inventory, so the audit is per-theorem and mechanical, not a global grep.

The design mirrors, in the formal channel, the substitution-closure of the Gate 1 protocol: hash checks prevent verifying edited sources; the axiom and `sorry` rejections prevent weakening the logic; the dependency audit prevents an individually honest theorem from silently importing a dishonest helper. What survives all six checks is a set of named statements, in identified source files, proved to the kernel's satisfaction from `propext` alone under an identified compiler.

REMARK 11.2 (Scope of the receipt). The integrated receipt `LEAN_PROOF_MANIFEST_v0.1.json` (hash-bound in the prototype manifest, with the lift-functor subreceipt `LEAN_DETERMINISTIC_LIFT_FUNCTOR_MAN` bound beside it) names the exact modules and theorem identifiers in scope. The prototype manifest's verification claim defers to that receipt — "only the exact modules and theorem dependencies named in the linked proof manifest" — and this chapter's itemization follows the same inventory as summarized in the manifest's `verified_scope` and `open_scope` fields.

4. Verified scope

The seed's kernel-checked content falls in seven items, quoted here from the manifest's verified-scope inventory and expanded with the mathematical identity of each.

(1) *Typed identities and associativity, with endomap-only powers.* The Lean module `TypedCategory` models the kernel's typed total-function discipline and proves the identity laws and associativity of composition — the formal counterpart of the category laws of Chapter 4 — together with the powering discipline that admits powers only for endomaps, the formal counterpart of the endomap/path separation of Chapters 5 and 6. This is the seed's contribution to PO-TKS8-CORE-01, whose ledger status PARTIAL FORMAL VERIFICATION PRESENT rests on exactly these lemmas.
(2) *The exact two-state constant-lift expansion.* For the constant map on a two-point carrier — the permanent counterexample of Chapter 7, ledger record THM-LIFT-OCCUPATION-COUNTEREXAMPLE — the seed kernel-checks the exact vector expansion of the lift's action and its *noncontractivity*: the checked statements exhibit the concrete vector on which the constant lift strictly increases norm, blocking forever

the general claim that deterministic lifts are contractive. The fixture thus exists in all three evidence channels at once: informal proof, executable regression, and kernel-checked lemma.

(3) *Lift identity, entrywise and on basis vectors.* The statements $A_{\mathrm{id}_X} = I$ are checked in two forms: entrywise equality of the modeled matrix, and action on standard basis vectors.

(4) *Typed basis composition.* The composition half of the functor law is checked at basis level: $A_g(A_f e_x) = e_{g(f(x))} = A_{g \circ f} e_x$ for the modeled typed maps — the on-basis-vectors content of Chapter 7's functoriality theorem.

(5) *Relational-product support composition.* At the level of supports — which target coordinates can be nonzero — the seed checks that the support of a composite lift is the relational product of the component supports, tying the matrix picture to the underlying relation composition.

(6) *Four-atom finite-context lemmas.* The seed includes the exhaustiveness and witness-map lemmas for the four-atom ACBE/RPM finite context used by the companion Noetic Topos development — finite case analyses whose value is precisely that a kernel, not a human, confirmed the case split is total. Their presence in this seed is an artifact of shared toolchain and shared verifier; their mathematical home is the companion paper, and no TKS/CAS semantic claim travels with them (the boundary of Section 6 applies with full force).

Together with the endomap-only power discipline these items constitute the entirety of PO-TKS8-CORE-07's PARTIAL / PRESENT status and the formal fraction of PO-TKS8-CORE-11 (PARTIAL FORMAL VERIFICATION PRESENT / FINITE-SUM MATRIX BRIDGE OPEN).

5. Open scope

The seed's honesty is its itemized deficit. The following are *not* formalized; each entry names the obligation it blocks or partially blocks.

(1) *The universal norm theorem.* The exact operator-norm formula $\|A_f\|^2 = \max_y |f^{-1}(y)|$ (Chapter 7) is checked in Lean only for the two-state constant expansion. The universal statement — all carriers, all mappings — has an informal fiber-block Gram proof and bounded-exhaustive structural evidence, but no kernel-checked form. (PO-TKS8-CORE-05: INFORMAL PROOF PRESENT / UNIVERSAL FORMALIZATION OPEN.)

(2) *The generic matrix bridge.* The checked two-state lemmas speak of a concrete expansion, not of the general definitions. The bridge from the checked `constantTwoLift` object to generic matrix, Gram, trace, and rank definitions — the step that would make the counterexample a corollary of a general theory rather than a standalone computation — is open.

(3) *Finite-sum matrix multiplication.* The seed does not formalize numerical matrix multiplication as a finite sum, which is the missing link between the basis-level functor law it checks and the entrywise matrix functor law the Python tests check. This is the "finite-sum matrix bridge" of PO-TKS8-CORE-11.

(4) *Arbitrary-vector linearity.* The checked lift laws act on basis vectors; the linear extension to arbitrary vectors — routine on paper, but a genuine formalization step — is open.

(5) *The factorial idempotent-power theorem.* Theorem $f^{|X|!} \circ f^{|X|!} = f^{|X|!}$ (Chapter 5) is a standard result with a complete informal proof and a 289-case exhaustive check, and no Lean form. (PO-TKS8-CORE-04.)
(6) *Shortlex leastness.* The soundness of representative-first pruning and the leastness of the computed normal forms (Chapter 8) rest on the informal induction of the pruning lemma. (PO-TKS8-CORE-06.)
(7) *Presheaf, sheaf, topology-generation, and product-site layers.* The full site-theoretic program of the companion Noetic Topos development — everything beyond the four-atom finite context lemmas — is unformalized.

The pattern across the list is consistent: the seed formalizes *discrete, finite, definition-adjacent* statements — laws that are within short reach of the modeled definitions — and leaves open everything requiring real analytic or combinatorial infrastructure (spectral bounds, factorials and cycle structure, well-founded minimization arguments, sheaf theory). This is a defensible frontier for a seed, and the word "seed" is doing exact work: the artifact claims a viable, pinned, audited starting point, not a formalized theory.

5.1. Formalization routes for the open items. Without promising schedules — the ledger records obligations, not plans — the open items admit an informed difficulty ordering, and recording it here serves whoever next extends the seed.

The *finite-sum matrix bridge* and *arbitrary-vector linearity* are the mechanical end: both are finite summation arguments over Kronecker deltas, requiring only finite-sum manipulation lemmas (reindexing, splitting over a fiber decomposition), and both would immediately upgrade PO-01-adjacent and PO-11 evidence by connecting the checked basis-level laws to the entrywise matrix laws the Python suite tests. They are the natural first targets.

The *factorial power theorem* is middle difficulty: its proof needs a formalized functional-graph decomposition (every point's orbit is eventually periodic; tail length bounded by $|X|-1$) and the divisibility fact $\ell \le |X| \Rightarrow \ell \mid |X|!$. Both components are within reach of standard library support for `Nat` arithmetic and finite iteration, and an orbit-based proof avoiding explicit graph theory — iterate the pigeonhole argument on the sequence $x, f(x), f^2(x), \ldots$ — is likely the short path.

The *universal norm theorem* has two routes with sharply different formalization costs. The spectral route of the informal proof (Gram block decomposition, largest eigenvalue) imports spectral theory of symmetric matrices — heavy machinery for a finite statement. The elementary route bounds $\|A_f v\|^2 = \sum_y \left|\sum_{x \in f^{-1}(y)} v_x\right|^2$ by Cauchy–Schwarz within each fiber, obtaining $\le \max_y |f^{-1}(y)| \cdot \|v\|^2$, and exhibits the normalized indicator of a maximal fiber as the equality witness — finite sums and one inequality, no spectra. The elementary route depends only on the linearity item above and is plainly the formalization-friendly path.

Shortlex leastness is the most program-like: it requires an abstract model of the closure exploration (a well-founded induction over shortlex order with the pruning invariant) and is best attacked as algorithm verification over an idealized queue rather than as a lemma about the Python. The *presheaf and sheaf layers* are the far end — genuine site theory — and belong to the companion program's formalization lane rather than the kernel's.

6. What kernel-checking does not verify

Claim Boundary 11.3 (Model theorems are not program verification). Every theorem in the seed is a statement about *Lean objects*: a Lean model of typed finite functions, a Lean expansion of a two-state lift. No theorem in the seed mentions, models, or constrains

the Python source of `src/tks_v8_core`. The relationship between the Lean model and the running implementation is exactly the relationship the red-team methodology treats it as: an informal correspondence, argued by inspection — the definitions were written to match — and checked on bounded instances by the executable suite, but never itself proved. A reader who concludes "the kernel is formally verified" has crossed a boundary that the artifact's own manifest refuses to cross: the manifest's claim records keep "equivalence between the Python implementation and the Lean model" explicitly OPEN.

The gap is not pedantry; it decomposes into three concrete failure channels, none excluded by the seed. First, *modeling error*: the Lean definitions could mismatch the Python semantics — an off-by-one in the modeled indexing, a subtly different totality convention — in which case the checked theorems are true of the model and silent about the code. Second, *coverage error*: the Python contains behavior the model omits entirely (resource guards, error taxonomy, presentation naming, the closure engine), so even a perfect model of the composition core would verify a fraction of the artifact. Third, *compilation and runtime error*: the Python executes on an interpreter whose own correctness is assumed, not proved — the standard trusted-computing caveat that verified-compilation projects address and this artifact does not attempt. Programs proved correct in the strong sense require either verification-aware extraction (the program is generated from the checked development) or a verified semantics of the implementation language; the seed does neither, claims neither, and the monograph records the fact in the ledger rather than in a footnote.

What the seed *does* deliver, within these limits, is significant: the mathematical laws at the heart of the design — the category laws, the power discipline, the counterexample that anchors the norm theory's claim boundary — hold in at least one fully precise, kernel-audited model of the definitions, under a single standard axiom, reproducibly, on a pinned toolchain. Errors in the *mathematics* of Chapters 4–7, if any survived informal review, would have had to survive the kernel too wherever the scopes overlap. The seed thus hardens the theory; it leaves the implementation's verification status exactly where Chapter 10 put it.

7. The PO-07 partial status

Obligation PO-TKS8-CORE-07 — "independent theorem-prover formalization" — carries the ledger status PARTIAL / PRESENT, and this chapter is that status rendered in full. It is PRESENT: a pinned Lean 4.32.1 seed exists, builds through Lake, passes a hardcoded single-axiom audit with project axioms and `sorry` rejected, has its sources hash-bound, and checks the seven-item scope of Section 4. It is PARTIAL: the seven-item open scope of Section 5 remains, the Python-to-model equivalence is unproved, and no second prover corroborates the first. The word "independent" in the obligation's title is also only partially honored in a further sense worth recording: the seed was developed within the project, so while the *checking* is independent of human attention in the way only a kernel can be, the *statement selection* is not independent of the authors — a reviewer could ask whether the formalized statements are the load-bearing ones, and PO-TKS8-CORE-10 (independent specialist review, OPEN) is where that question lives. Closing PO-07 outright would require, at minimum, discharging the open-scope items above; the ordering constraints among those items and the rest of the ledger are taken up in Chapter 12.

CHAPTER 12

Boundaries, Downstream Roles, and the Research Program

A kernel this small earns its keep in two ways: by what it makes possible and by what it refuses to permit. The preceding chapters developed the first; this closing chapter develops the second, and then turns the refusals into a program. We itemize the excluded transfers — the pieces of the wider TKS/CAS research corpus that this artifact deliberately does not encode, every one of them Ring 4 / NOT LICENSED — examine in depth the three downstream consumers designed to use the kernel strictly through its types, walk the full eleven-row proof-obligation ledger with a substantive discussion of each row, situate the work among its intellectual neighbors, and conclude.

1. Excluded transfers

The specification's exclusion list is the artifact's constitution. Each item below is excluded from the kernel and carries the status OPEN; as research transfers, all remain Ring 4 / NOT LICENSED — meaning no artifact, document, or downstream system may treat the transfer as available, and any future admission requires its own review, not an appeal to this monograph.

1.1. Surface syntax. The kernel neither parses nor evaluates TKS v7.4 or v8 surface syntax. No tokenizer, grammar, or expression evaluator appears anywhere in the three modules. This is not a deferred convenience: the production notation compiler already owns surface syntax, and a second parser in the kernel would create exactly the dual-authority hazard — two implementations with divergent corner cases, each citable as "the" semantics — that the red-team methodology exists to prevent. The kernel's interface is typed data (objects, arrows, words over declared generators), constructed programmatically; text stays outside.

1.2. The Noetic and CAS mapping. No identifier in the kernel names a Noetic, a Domain, a Drive, a Register, or any of the 40 Cognitive States, and no table anywhere maps canonical constructs to kernel arrows. This is the deepest exclusion, and the one most tempting to violate informally — it takes one line of client code to name an object `"Mind"` and begin drawing conclusions. The prohibition is therefore worth stating with its reason: mapping the canon onto the substrate is a *semantic* act with its own correctness conditions — which canonical construct is an object, which an arrow, which a path; whether canonical composition is kernel composition — and none of those conditions have been reviewed. The mapping is PO-TKS8-CORE-08, OPEN / NOT LICENSED; until it closes, a kernel arrow named after a Noetic is a string, not a claim.

1.3. The RPM monad. The kernel contains no RPM data, no monad structure, and no prerequisite gate. The corrected v7.4 record is precise that RPM is a section-retraction pair — one-directional prerequisite satisfaction, not an adjunction — and formalizing that structure over kernel types would require design decisions (what object models a pathway state, what arrow a satisfaction step) that are exactly the deferred semantic mapping. Importing the

kernel's factorial-power theorem into RPM reasoning without that mapping would reproduce the invalid power inference the kernel was built to block.

1.4. Rewrite systems. The normal-form machinery of Chapter 8 is denotational: short-lex minimization over a completed closure of *semantic* objects. It is not a rewrite system for inherited TKS equations, and no termination or confluence claim for any such system follows from it. The distinction is easy to blur because both produce "canonical forms"; the claim boundary in the specification is explicit that changing the declared generator order may change representatives, which no confluent rewriting story would tolerate. A syntactic rewrite layer, if ever wanted, is a separate artifact with separate obligations.

1.5. Noetic Topos 2.0 coverage and modalities. The companion topos-theoretic development — sites, coverage, sheaf conditions, modal operators — shares a type discipline with the kernel (Section 2.3) but transfers nothing else. The kernel's four-atom context lemmas in the Lean seed (Chapter 11) are finite case analyses housed alongside for toolchain convenience; the presheaf and sheaf layers themselves are in the Lean open scope and outside the kernel's Python entirely.

1.6. Stochastic, quantum, and empirical dynamics. The deterministic lift is a bookkeeping functor for finite deterministic maps. Stochastic kernels, quantum channels, contractivity claims, and any empirical cognitive-dynamics content are excluded — the permanent noncontractivity fixture exists precisely to block the most tempting bridge, the reading of A_f as a channel with norm at most one. A future stochastic extension would change the mathematics (column- stochastic real matrices, a genuinely different norm theory) and the evidence obligations; nothing in this artifact licenses it.

1.7. The Ten Advances. No transfer from the Ten Advances manuscript is admitted: none of its constructions appear in kernel types, none of its claims inherit kernel evidence. The manuscript remains a research input to the wider program, Ring 4 / NOT LICENSED like every other transfer in this section.

CLAIM BOUNDARY 12.1. The exclusions are load-bearing, not provisional politeness. Every downstream consumer in Section 2 is designed so that its use of the kernel would survive an auditor checking each exclusion above; any design that requires breaching one is, by that fact, blocked pending the corresponding ledger row.

THREAT 12.2 (Semantic transfer by naming). The cheapest breach of the exclusion regime requires no code change at all: construct kernel objects and arrows whose *presentation names* are canonical terms — an object named after a Noetic, a generator labeled after a Drive — and then read kernel theorems as if they were theorems about the named constructs. The kernel's own design partially blunts this (names are excluded from arrow equality and carry no semantics anywhere in the code), but no type system can stop a human from reading meaning into a string. The defense is procedural and must be stated as such: under the artifact's claim discipline, a canonical name on a kernel value confers exactly nothing — any document deriving a canonical conclusion from a named kernel computation is making the PO-08 transfer without a license, and the boundary regression's `cas_semantic_transfer_status` assertion is the tripwire that keeps the manifest, at least, honest about it.

THREAT 12.3 (Evidence transfer by association). A subtler breach transfers not semantics but *credibility*: citing the kernel's proof-and-test discipline as if it attached to a downstream system that merely uses the kernel. The certified-adapter design of Section 2.2 shows the

correct form — certificates bind exact kernel bytes and explicitly carry "no automatic CAS semantic theorem" — and the incorrect form is any sentence of the shape "built on the formally verified TKS kernel," which is wrong twice (the kernel is not formally verified, per Chapter 11; and evidence does not compose across an unverified boundary). Downstream documents inherit the kernel's obligations, not its receipts.

2. The three downstream consumers

Three companion candidates are designed to consume the kernel strictly through its public types. None is part of the bound artifact; each is documented here because the kernel's design choices are partly explained by the loads these consumers will place on it.

2.1. CAS Core 8: the fractal bridge. CAS Core 8 confronts a legacy: fractal digit sequences (the X.Y.Z cognitive-fractal addresses of the wider system) whose composition semantics were historically implicit. Its bridge design uses the kernel as a type discipline for that legacy. The caller — not the bridge — supplies an explicit catalog of typed arrows, one per legacy digit or digit transition, declaring for each its source and target object. The bridge then interprets a digit sequence as a `SemigroupoidPath` over the supplied catalog: adjacent-object matching is checked by the path constructor at every junction, composition is the path composite, and — decisively — *missing digits or type mismatches fail closed*: an unmapped digit or an ill-typed junction raises rather than guesses.

Two properties of this design deserve emphasis. First, the bridge is honest about its authority: as the specification puts it, it "proves type-correct composition only, not the meaning of a legacy digit." The caller's catalog is an assertion, not a theorem; the bridge certifies that *given* the catalog, the sequence composes with exact types. Whether digit 3 *means* the arrow the caller mapped it to is a semantic question belonging to PO-08's lane. Second, the choice of paths rather than endomaps is itself an application of Chapter 6's discipline: a digit sequence is a heterogeneous generator sequence, so the bridge exposes composites and never powers — a legacy "iterate this address" idiom would have to produce a certified endomap first, through the same gate as everyone else.

The failure modes the design forecloses are worth naming, because they are the failure modes the legacy idiom invites. A digit sequence whose adjacent stages do not actually connect — stage output in one state space, next stage input in another — historically produced a computed *something*; under the bridge it produces `CompositionTypeError` at the exact junction, with both stage signatures in the message. A digit with no catalog entry historically defaulted; under the bridge it fails closed. And a fractal address treated as "the same operator at every scale" — the reading that would license powers — is exactly the invalid power inference of Chapter 1's audit findings, unwritable here because the bridge's output type has no power operation. What the bridge cannot do is equally sharp: it cannot tell a caller that their catalog is *right*. Two callers with conflicting catalogs for the same legacy corpus will each get flawlessly typed, mutually contradictory composites. Catalog review is semantic work, and the bridge's design honestly locates it outside itself.

2.2. CAS PCBA 8: the certified-transition adapter. CAS PCBA 8 (the printed-circuit-board-assembly engineering line) consumes the kernel from the opposite direction: not legacy addresses but live component models. Its adapter admits a component transition function into the kernel only under a certificate that the transition is *total, finite, deterministic, stateless, and non-stochastic* — the five conditions under which a transition genuinely is a finite-function arrow. A certified transition becomes a kernel arrow; transition chains

become semigroupoid paths; and a feedback loop — the case where iteration talk is wanted — requires a certified *endomap*, so the factorial-power theorem and the idempotence machinery become available exactly when the type system says they may.

The adapter's certificate discipline mirrors the manifest's: a certificate "binds the exact kernel bytes" — it names the hash of the kernel version against which the adaptation was certified, so a kernel change invalidates outstanding certificates rather than silently reinterpreting them. And the specification is explicit that the certificate "carries no automatic CAS semantic theorem": adapting a solder-paste dispenser's state machine into kernel arrows yields typed composition guarantees for that machine, not a cognitive claim about anything. The adapter is thus the template for how empirical engineering systems touch the kernel: through a narrow, certified, hash-bound gate, gaining type discipline and gaining nothing else.

Each of the five certificate conditions excludes a specific misadaptation, and enumerating them shows how much real-world texture the finite-function abstraction refuses to absorb silently. *Totality* excludes transitions with undefined regions — a component that jams on some input is not a total function until its jam behavior is modeled as an explicit state. *Finiteness* excludes continuous state (temperatures, analog positions) until it is discretized by an explicit, reviewed quantization. *Determinism* excludes transitions with tolerance bands or race outcomes — the single largest gap between datasheet behavior and function semantics. *Statelessness* excludes hidden memory: a component whose response depends on history is not one arrow but a family, and must be certified as a transition on an enlarged explicit state space. *Non-stochasticity* excludes failure rates and noise, which belong — if anywhere — to the excluded stochastic extension, not to this lift. A transition failing any condition does not get a degraded certificate; it gets no certificate, and its chain gets no kernel typing. The discipline's value to the engineering line is exactly this refusal: what passes the gate composes with mathematical guarantees, and what does not pass is visibly, recordedly outside them.

2.3. Noetic Topos 2.0: shared type discipline and PO-08. The relationship with Noetic Topos 2.0 is subtler: not consumption of kernel objects but *convergence on a discipline*. The topos program admits deterministic maps as morphisms and deliberately refuses to identify Markov kernels, channels, or enriched sheaves with functions — the same determinism boundary the kernel draws with its lift, arrived at independently from the sheaf-theoretic side. Both artifacts, that is, encode the decision that the deterministic/stochastic line is a *type* distinction, not a parameter. The shared discipline makes the eventual reconciliation — matching the kernel's objects and arrows with the topos development's sites and the corrected v7.4 theorem records — a well-posed problem, and that problem has a name: it *is* PO-TKS8-CORE-08, no more and no less. Until that row closes, the two artifacts cite each other's boundaries, share a Lean toolchain and verifier discipline (the four-atom context lemmas of Chapter 11 are the visible overlap), and transfer nothing semantic.

3. The obligation ledger, row by row

The eleven-row ledger is reproduced in tabular form in Appendix B; this section discusses each row substantively — what closing it would require, why it matters, and how it is ordered against the others.

3.1. PO-TKS8-CORE-01: identity and associativity. *Status: partial formal verification present.* The category laws carry a pointwise informal proof, bounded exhaustive tests (19,683 associativity triples, typed identity families), and exact Lean lemmas in the

`TypedCategory` module. What remains is not the mathematics — the laws are as standard as mathematics gets — but the *correspondence*: the Lean lemmas govern a model, and the equivalence of model and Python is open. Closing the row outright means either a verified extraction (generate the composition core from the checked development) or a formalized semantics of the Python fragment in use; both are heavyweight, and the row is best read as an anchor whose residual gap is shared with every row that cites Lean evidence. It matters because every other law in the artifact composes through this one: functoriality, powering, closure soundness all quote associativity. Ordering: its residual is subsumed by any serious attack on PO-07.

3.2. PO-TKS8-CORE-02: reject every source/target mismatch. *Status: executable validation.* The rejection behavior is a finite, directly observable property of four code sites (compose, the endomap wrapper, the path validator, catalog evaluation), and the negative suites drive each. The row could be strengthened — a Lean model of the *partiality* of composition, or an exhaustive mismatch sweep over bounded object families — but its practical force is already the strongest in the ledger: this is the row that makes untyped composition unwritable, the first of the three audit failures the kernel answers. It matters because it is the row downstream consumers lean on hardest: the fractal bridge's fail-closed junctions and the PCBA adapter's chain checks are this row wearing application clothes. No ordering constraints; it is closed at its evidence level and stays open only in the sense that all executable validation is bounded.

3.3. PO-TKS8-CORE-03: powers distinct from paths. *Status: executable validation.* The separation is enforced by the runtime type system (no `power` on paths, endomap certification at construction) and tested negatively. Its strengthening would be a formal statement that the exported API admits no derivation of a path power — an API-level metatheorem, which is unusual territory and probably not worth its cost given that the property is syntactically evident: the method does not exist. The row matters because it kills the invalid power inference — the second audit failure — and because its discipline is what makes the factorial theorem *safe* to publish: the theorem is dangerous precisely when applied to things that are not single endomaps. Ordering: none; conceptually prior to PO-04.

3.4. PO-TKS8-CORE-04: factorial idempotence. *Status: standard result; formalization open.* The mathematics (Chapter 5) is classical finite-semigroup material [13]: tails enter cycles within $|X| - 1$ steps, all cycle lengths divide $|X|!$. The exhaustive 289-endomap check covers carriers through size four. Closing the row means a Lean proof of the universal statement, which requires formalized cycle decomposition of functional graphs and divisibility of factorials — genuine but well-trodden formalization work, with likely reusable library support. It matters for two reasons: it is the theorem downstream feedback-loop analysis will quote (via the PCBA adapter's certified endomaps), and it is the simplest ledger row whose closure would demonstrate the seed can grow beyond definition-adjacent lemmas. Ordering: independent of the lift rows; a natural first target for seed expansion.

3.5. PO-TKS8-CORE-05: the exact norm formula. *Status: informal proof present; universal formalization open.* The fiber-block Gram proof (Chapter 7) is complete informally, the structural tests check the formula's consequences on the bounded family, and Lean checks only the two-state constant expansion. Closing the row requires formalizing spectral reasoning — largest eigenvalue of a PSD Gram matrix, permutation similarity, the all-ones block spectrum — or an alternative elementary route (directly bounding $\|A_f v\|^2$ by Cauchy–Schwarz

per fiber and exhibiting the maximizing vector), the latter probably being the formalization-friendly path since it avoids general spectral theory [12]. The row matters because the norm formula is the artifact's main quantitative theorem and the anchor of its noncontractivity boundary. Ordering: depends on the matrix bridge (part of PO-11) if done spectrally; the elementary route depends only on arbitrary-vector linearity.

3.6. PO-TKS8-CORE-06: shortlex soundness and leastness. *Status: informal correctness argument; formalization open.* The pruning lemma's induction (Chapter 8) and the finiteness argument are short but subtle — the suffix-transfer step is exactly the kind of argument where informal proofs hide gaps. Closing the row means formalizing the closure algorithm's invariants: heap order delivers shortlex order, first insertion is least, pruning preserves reachability. This is program-adjacent verification (an algorithm, not just a definition), so it likely proceeds by modeling the algorithm abstractly in Lean rather than verifying the Python. It matters because canonical forms are only as trustworthy as leastness: a wrong representative is not detectably wrong to a consumer, unlike a type error. Ordering: independent of the lift rows; benefits from whatever induction infrastructure PO-04 builds.

3.7. PO-TKS8-CORE-07: independent theorem-prover formalization. *Status: partial; present.* Chapter 11 is this row in full: the pinned seed exists with its axiom audit, and the open scope is itemized. Closing it means discharging the open-scope items — the universal norm, the matrix bridge, finite-sum multiplication, arbitrary-vector linearity, factorial power, shortlex leastness — at which point rows 01, 04, 05, 06, and 11 all upgrade simultaneously. The row matters as the aggregate index of formal depth, and its ordering is the reverse of the others': it closes last, when its constituents do. A second-prover corroboration (Coq or Agda, both UNAVAILABLE in the probe environment) would strengthen it further but is not, on the ledger's text, required.

3.8. PO-TKS8-CORE-08: reconcile kernel types with corrected v7.4 records. *Status: open; not licensed.* The deepest row, and the only one whose closure is a semantic act rather than a mathematical or engineering one. Closing it requires: a reviewed mapping proposal (which canonical constructs correspond to objects, arrows, endomaps, paths); verification that the corrected v7.4 theorem records — admissibility relations, the section-retraction RPM structure, the open-simplex state space — are representable, or explicitly not representable, in kernel terms; and a licensing decision by review, not by drafting. It matters more than any other row because it is the bridge over which the kernel would stop being "an independent reference model" and start carrying canonical content; every downstream consumer's semantic silence is provisional on this row. Ordering constraints are the strictest in the ledger: it should not close before PO-10 (review) has structure, and nothing in Ring 4 crosses until it does.

3.9. PO-TKS8-CORE-09: integrate with the active notation compiler. *Status: open (outside this lane).* The kernel and the production compiler currently share nothing but intent. Integration — whether as a semantic checking layer behind the compiler's kinds, as a validation oracle in compiler tests, or as a type-annotation target — requires a compatibility design: mapping compiler kinds onto kernel types, deciding failure behavior when the kernel rejects what the compiler accepts, and versioning the joint artifact. The manifest's integration boundary (`active_cas_compiler_modified: false`, `production_runtime_dependency: false`) is the row's current truth, and the boundary regression of Chapter 10 tests it. The row matters because the audit findings that motivated the kernel are findings about the production system; the kernel repays its cost only when

its discipline reaches production. Ordering: after PO-08 for any semantically loaded integration; a purely structural integration (type-checking composition of compiler programs) could in principle proceed earlier under its own review.

3.10. PO-TKS8-CORE-10: independent specialist review. *Status: open; no external review receipt.* Every evidence channel in the artifact — proofs, tests, seed — is project-authored. Closing this row means an external reviewer with category-theoretic and formal-methods competence examines the claims, the code, and the seed, and issues a receipt that the artifact then binds. It matters because it is the only row that addresses statement-selection risk (Chapter 11's closing observation): insiders choose what to prove and what to test, and only outside eyes audit the choosing. Ordering: it gates PO-08's closure and should precede any licensing decision; it can begin at any time — nothing in the ledger blocks review, and review would sharpen everything else.

3.11. PO-TKS8-CORE-11: lift preserves identities and composition. *Status: partial formal verification; finite-sum matrix bridge open.* The functor laws hold on 1,678 exact bounded pairs and in Lean at entry, basis, and support levels; what is missing is the finite-sum matrix multiplication bridge that would let the basis-level Lean statements imply the entrywise matrix statements the Python tests check. Closing it is the most mechanical of the open formalizations — finite sums over Kronecker deltas — and would knit the two evidence channels together at their closest point of approach. It matters because functoriality is what makes the lift a *functor* rather than a family of matrices, and because its closure is a prerequisite for the spectral route to PO-05. Ordering: before PO-05 (spectral route); independent otherwise.

3.12. Ordering constraints in one view. Collecting the per-row ordering remarks into a single dependency sketch:

Row	Depends on / naturally follows
PO-01 residual	subsumed by PO-07's open scope (model equivalence apart)
PO-02, PO-03	no dependencies; closed at their evidence level
PO-04	independent; natural first seed-expansion target
PO-05	elementary route: linearity item of PO-07 scope; spectral route: PO-11 bridge
PO-06	independent; shares induction infrastructure with PO-04
PO-07	aggregates: closes when 04, 05, 06, 11 formalizations land
PO-08	gated by PO-10; gates semantic use everywhere
PO-09	semantically loaded form gated by PO-08; structural form separately reviewable
PO-10	no dependencies; can begin immediately; gates PO-08
PO-11	bridge is mechanical; precedes spectral PO-05

The sketch exposes the program's critical path: PO-10 (review) is the only row that gates the semantic future and depends on nothing, which makes commissioning review the highest-leverage single action available; and the formalization rows form an almost-parallel bundle whose one internal edge (bridge before spectral norm) dissolves if the elementary norm route is taken. A program that started review and the two mechanical Lean items simultaneously would be attacking every hub at once.

3.13. Reading the ledger whole. Three structural observations. First, the ledger separates *evidence upgrades* (04, 05, 06, 11, and ultimately 07: open formalizations of proved mathematics) from *semantic gates* (08, 09: acts requiring review and licensing) from *process obligations* (10: review itself); the three kinds close by different means and no amount of progress in one kind substitutes for another. Second, the dependency structure is shallow — most rows are independent — with two hubs: PO-07 aggregates the formalization rows, and PO-10 gates the semantic rows. Third, the ledger is conserved under this monograph: drafting these chapters upgraded no row, and the artifact's status line — GATE 1 PROTOTYPE / RING 4 / NOT LICENSED — is exactly what it was when the manifest was written. The program is the ledger; the monograph is its documentation.

4. Related work

The mathematics deployed here is deliberately standard. The categorical framework — objects, arrows, functors, the discipline of typed composition — is textbook material [2], and the kernel's category is among the simplest nontrivial examples in that literature: a skeleton-free category of finite sets with named objects. The finite-semigroup facts underlying the power discipline, including the idempotent-power phenomenon in finite semigroups, belong to the tradition surveyed by Pin [13]. The spectral analysis behind the norm theorem — Gram matrices, permutation similarity, the all-ones block spectrum — is standard matrix analysis [12]. The artifact's registered position is that none of its mathematical content is new: the theorem ledger records standard results and definitions, and no novelty search has been commissioned.

The methodological neighbors are likewise explicit. Bounded-exhaustive checking as a design philosophy descends from the small-scope hypothesis and the Alloy tradition of Jackson [15]; Chapter 10 details both the debt and the contrast with sampled property-based testing in the QuickCheck lineage [24]. The formal seed follows standard kernel-checking practice in Lean 4 [16], with the axiom-audit discipline — single admissible standard axiom, mechanical per-theorem dependency reports — as its distinguishing procedural feature.

What the artifact offers as a contribution is consequently not mathematics but *engineering discipline*: name-inclusive object identity, ordered-input rejection of hash-order nondeterminism, the endomap/path type separation, fail-closed closure semantics, hash-bound artifacts with an explicit self-binding boundary, and a claim ledger that tracks evidence kind per statement. Each element is individually known practice somewhere; their joint application to a cognitive-architecture semantic substrate, with the claim discipline held as strictly as the proofs, is the design.

Three contrasts locate the design more precisely among its neighbors. Against *model finders* in the Alloy tradition [15], the kernel inverts the workflow: Alloy checks declarative models against small scopes to find counterexamples before implementation, whereas this artifact implements first and then exhausts small scopes to tie the implementation to prior proofs — the small-scope hypothesis serving as post-hoc validation warrant rather than design-time exploration. Against *property-based testing practice* [24], the kernel's suites give up generator generality (there is no combinator library, no shrinking) in exchange for countable, derivable, deterministic coverage — a trade sensible only because the instance spaces are tiny, and explicitly the wrong trade for the large-state systems where the QuickCheck tradition thrives. Against *verified-kernel developments* in the Lean ecosystem [16], the artifact is deliberately more modest — a seed with an audited boundary rather than a formalized theory — and correspondingly more explicit than is customary about the unformalized remainder,

which the ledger enumerates item by item rather than leaving implicit in what the repository happens not to contain. The standard mathematical content, meanwhile, is simply inherited with attribution — category theory [2], finite semigroups [13], matrix analysis [12] — with the ledger's STANDARD RESULT tag marking exactly where the artifact claims curation rather than creation.

5. Conclusion

TKS v8.0, as documented across this monograph, is a small object with sharp edges. Its positive content fits in a sentence each: typed composition that cannot silently mistype; powers that exist only where a monoid exists, with the factorial bound as the one safe general theorem; a deterministic lift whose exact norm is proved rather than assumed, permanently accompanied by the counterexample that blocks its over-reading; and canonical forms that are deterministic because their inputs are ordered and their closure completes or refuses. Its evidence is layered and labeled: informal proofs for everything, bounded-exhaustive executable validation (19,683 triples, 289 endomaps, 499 typed maps, 1,678 lift pairs, 20 tests green at the 2026-08-04 snapshot), and a pinned, axiom-audited Lean seed for the definition-adjacent core — with the gaps between layers itemized, not blurred.

Its negative content is the part most easily lost in summary and most important to keep: the kernel encodes no Noetic, no Drive, no Register; it parses nothing; it claims nothing empirical; its manifest is not yet outer-bound; its suite proves nothing beyond its bounds; its seed verifies a model, not the program; and every transfer that would make it mean something canonically remains Ring 4 / NOT LICENSED behind an eleven-row ledger whose open rows close by proof, by engineering, or by review — never by prose. The three downstream consumers show that this austerity is usable: legacy addresses, certified component transitions, and a companion topos program can each take exactly the discipline they need through the types, and nothing more, today.

The research program, then, is not a promise appended to a result; it is the ledger read forward. Formalize the factorial theorem and the shortlex invariants; build the finite-sum bridge and take the elementary route to the universal norm; commission the review that audits the choosing; and only then, with receipts in hand, open the question the whole design defers — whether and how the canon maps onto the substrate. The kernel's refusal to answer that question prematurely is not a limitation of the work. It is the work.

APPENDIX A

Worked Examples

This appendix computes, entirely by hand, three complete examples of the kernel's machinery: an endomap on a three-element carrier taken through its functional graph and its powers to idempotence; the deterministic lift of the same map with all of its derived operators and its exact norm; and a full closure run of the normal-form catalog on a two-element carrier, with the shortlex table shown at every step. Nothing here requires the software — every line is derivable from the definitions of Chapters 3–8 — and that is the point: the examples are small enough to audit the implementation against, and the bounded-exhaustive suites of Chapter 10 are, in essence, these computations repeated over every small instance.

1. A three-carrier endomap and its powers

1.1. The object and the map. Fix the object $X = (\texttt{"X"}, (a, b, c))$: name `"X"`, ordered carrier (a, b, c), standard indices $0, 1, 2$. Define the endomap $f : X \to X$ by the mapping tuple

$$f = (1, 2, 2), \qquad \text{i.e.} \qquad f(a) = b, \quad f(b) = c, \quad f(c) = c.$$

Construction succeeds: the name is non-blank, the carrier elements are distinct and hashable, the mapping has length $3 = |X|$, and every image index lies in $\{0, 1, 2\}$. Since source and target are the same object, `is_endomap` holds and the `Endomap` wrapper accepts f.

1.2. The functional graph. The functional graph of f has an edge $x \to f(x)$ from each index:

$$0 \longrightarrow 1 \longrightarrow 2 \circlearrowleft$$

Index 2 is a fixed point — a cycle of length $\ell = 1$ — and indices 0 and 1 form a tail feeding it. Tail lengths (distance to the cycle): $d(0) = 2$, $d(1) = 1$, $d(2) = 0$. The maximum tail length is $2 = |X| - 1$, the extreme the safe-power proof allows.

1.3. Iterates. Compute the iterates by composing mapping tuples, $(g \circ f)(i) = g(f(i))$:

$$\begin{aligned} f^1 &= (1, 2, 2) \\ f^2 &= (f(1), f(2), f(2)) = (2, 2, 2) \\ f^3 &= (f(2), f(2), f(2)) = (2, 2, 2) = f^2. \end{aligned}$$

So the iterates stabilize at the constant map to c from exponent 2 onward. Check idempotence of f^2 directly: $f^2 \circ f^2 = (2, 2, 2) \circ (2, 2, 2)$; entry i is $f^2(f^2(i)) = f^2(2) = 2$, so $f^2 \circ f^2 = (2, 2, 2) = f^2$. Idempotent. Note f itself is *not* idempotent: $f \circ f = (2, 2, 2) \neq (1, 2, 2) = f$.

1.4. The factorial power by binary powering. The safe exponent is $|X|! = 3! = 6$. The implementation computes f^6 by binary powering (Chapter 9); we trace it exactly. Write the exponent in binary: $6 = 110_2$. Initialize the accumulator $r = \mathrm{id}_X = (0, 1, 2)$ and the base $b = f = (1, 2, 2)$, with remaining $= 6$.

Step	remaining (binary)	low bit	action
1	110	0	no accumulation; square: $b \leftarrow b \circ b = (2, 2, 2)$
2	11	1	$r \leftarrow b \circ r = (2, 2, 2)$; square: $b \leftarrow (2, 2, 2) \circ (2, 2, 2) = (2, 2, 2)$
3	1	1	$r \leftarrow b \circ r = (2, 2, 2)$; remaining becomes 0, final squaring skipped

In step 2, $b \circ r$ with $r = \mathrm{id}$ gives $b = (2, 2, 2)$; in step 3, $(2, 2, 2) \circ (2, 2, 2) = (2, 2, 2)$ as computed above. The result is

$$f^6 = (2, 2, 2),$$

which agrees with $f^2 = f^3 = \cdots$ from the iterate table, and is idempotent as the safe-power theorem requires. The returned arrow carries the presentation name `f^(6)` — the exponent is well under the 256-bit threshold, so the compact label is plain decimal — and, by extensional equality, equals the arrow $(2, 2, 2)$ regardless of its name.

1.5. A contrasting endomap: a genuine cycle. For contrast, let $g = (1, 0, 2)$: the transposition of a, b with c fixed. Its functional graph is a 2-cycle plus a fixed point; there are no tails. Iterates: $g^2 = (g(1), g(0), g(2)) = (0, 1, 2) = \mathrm{id}$, so $g^3 = g$, $g^4 = \mathrm{id}$, and in general the iterates alternate. The safe power is $g^6 = (g^2)^3 = \mathrm{id}^3 = \mathrm{id}$, which is idempotent — and here the factorial exponent is genuinely needed in the sense that odd exponents never yield an idempotent: $g^n = g$ for odd n, and $g \circ g = \mathrm{id} \neq g$. The two examples bracket the theorem's content: for tail-dominated maps the iterates stabilize early and $|X|!$ is generous; for permutation-like maps stabilization never happens, but $|X|!$ is divisible by every cycle length (1 and 2 here, with $2 \mid 6$) and so lands every point back at the start of its cycle simultaneously.

2. The lift of f, its operators, and its norm

2.1. The matrix. Lift the endomap $f = (1, 2, 2)$ of the previous section. The matrix A_f is $|X| \times |X| = 3 \times 3$, with entry (r, c) equal to 1 iff $f(c) = r$; column c has its single 1 in row $f(c)$:

$$A_f = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

(columns $0, 1, 2$ carry their 1 in rows $1, 2, 2$ respectively).

2.2. Fibers. The fibers of f are $f^{-1}(0) = \emptyset$, $f^{-1}(1) = \{0\}$, $f^{-1}(2) = \{1, 2\}$, so

$$\texttt{fiber_sizes} = (0, 1, 2), \qquad \texttt{max_fiber_size} = 2.$$

2.3. The three derived operators. *Input Gram matrix* $A_f^* A_f$: entry (x, x') is 1 iff $f(x) = f(x')$. The image tuple is $(1, 2, 2)$, so indices 1 and 2 collide and index 0 is alone:

$$A_f^* A_f \;=\; \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

Output Gram matrix $A_f A_f^*$: diagonal of fiber sizes,

$$A_f A_f^* \;=\; \mathrm{diag}(0, 1, 2).$$

Support projector: diagonal 0/1 marking of nonempty fibers,

$$P_{\mathrm{supp}} \;=\; \mathrm{diag}(0, 1, 1).$$

Note $A_f A_f^* \neq P_{\mathrm{supp}}$ precisely because the fiber over index 2 has size two — this instance witnesses the three-operator distinction that Chapter 9 documents and the test suite enforces.

2.4. Scalar invariants. Reading off the implemented consequences:

$$\mathrm{tr}(A_f^* A_f) = 1 + 1 + 1 = 3 = |X|, \qquad \mathrm{rank}(A_f) = |\{1, 2\}| = 2 = |\mathrm{im} f|,$$

with the trace computable either as the Gram diagonal sum (three ones) or as the sum of fiber sizes $0 + 1 + 2 = 3$; both agree, as they must. f is not injective (fiber size 2 over index 2), so `is_injective`, `is_isometry`, and `is_partial_isometry` are all false. f is an endomap but `mapping`$[1] = 2 \neq 1$ for the image value 1, so `is_idempotent` is false — consistent with $f \circ f \neq f$ above — and `is_orthogonal_projector` is false since the mapping is not the identity tuple.

2.5. The exact norm, spectrally and by witness. *Spectral route.* Group the input basis by fibers: the singleton fiber $\{0\}$ contributes the block $J_1 = (1)$ with eigenvalue 1; the fiber $\{1, 2\}$ contributes the all-ones block $J_2 = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ with eigenvalues 2 and 0. (Here the basis is already fiber-grouped, so no permutation is needed.) The spectrum of $A_f^* A_f$ is $\{1, 2, 0\}$, whence

$$\|A_f\|^2 = \max\{1, 2, 0\} = 2 = \max_y |f^{-1}(y)|, \qquad \|A_f\| = \sqrt{2}.$$

Witness route. The unit vector $v = \left(0, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$, supported on the largest fiber, maps under the scatter-add rule to

$$A_f v = \left(0,\ 0,\ \tfrac{1}{\sqrt{2}} + \tfrac{1}{\sqrt{2}}\right) = (0, 0, \sqrt{2}),$$

of norm $\sqrt{2}$: the two colliding coordinates add coherently in the shared output slot. So $\|A_f\| \geq \sqrt{2}$, and the Gram spectrum shows equality. The adjoint, for comparison, acts by gathering: $A_f^*(w_0, w_1, w_2) = (w_1, w_2, w_2)$.

2.6. An idempotent lift of norm $\sqrt{3}$. Finally, lift $h = f^2 = (2, 2, 2)$, the constant map to c. Its matrix has a single nonzero row:

$$A_h = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}, \qquad \texttt{fiber_sizes} = (0, 0, 3).$$

Every image value is 2 and `mapping`$[2] = 2$, so `is_idempotent` is *true* — indeed $A_h^2 = A_h$, since row 2 of $A_h A_h$ is again all ones. Yet $\|A_h\|^2 = 3$, $\operatorname{rank}(A_h) = 1$, $\operatorname{tr}(A_h^* A_h) = 3$, and A_h is not symmetric, hence not an orthogonal projector, and `is_orthogonal_projector` is false. This is the two-state noncontractivity fixture of Chapter 7 scaled to three states: an idempotent deterministic lift of norm $\sqrt{3} > 1$, maximally far from a contraction, produced by two applications of a perfectly ordinary endomap.

3. A closure run with the shortlex table

3.1. The catalog. Take one object $B = (\texttt{"B"}, (0, 1))$ and two generators, in this declared order:

Rank	Label	Mapping	Denotation
0	`s`	$(1, 0)$	the swap
1	`k`	$(0, 0)$	the constant to carrier index 0

Both generators are endo on B, so every extension is well-typed and the closure explores freely. The four total maps on a two-element carrier are id $= (0, 1)$, the swap $(1, 0)$, and the two constants $(0, 0)$ and $(1, 1)$; a complete run must find all four (this catalog generates the full transformation monoid) and assign each its shortlex-least word. Recall the word convention: a word's labels apply left to right, so the word $(\texttt{k}, \texttt{s})$ denotes $s \circ k$ — first `k`, then `s`.

3.2. The run, step by step. *Seeding.* The empty word ε_B at B, denoting id $= (0, 1)$, is pushed with key $(0, (), 0)$ (length 0, empty rank sequence, object rank 0). Frontier: $\{\varepsilon_B\}$; queued denotations: $\{(0, 1)\}$.

Step 1: pop ε_B. Its denotation $(0, 1)$ is new; accept it with representative ε_B. Extend by each generator in rank order:

- by `s`: word $(\texttt{s})$, denotation $s \circ \mathrm{id} = (1, 0)$ — new; push with key $(1, (0), 0)$.
- by `k`: word $(\texttt{k})$, denotation $(0, 0)$ — new; push with key $(1, (1), 0)$.

Table so far: $(0, 1) \mapsto \varepsilon_B$. Queued: $(1, 0), (0, 0)$.

Step 2: pop $(\texttt{s})$ (key $(1, (0), 0)$, the least in the frontier). Accept $(1, 0) \mapsto (\texttt{s})$. Extend:

- by `s`: denotation $s \circ s = (0, 1)$ — already represented (it is id); skip, do not push.
- by `k`: denotation $k \circ s = (k(1), k(0)) = (0, 0)$ — not represented, but already *queued* (the word $(\texttt{k})$ carries it); skip. This is the frontier deduplication of Chapter 9 doing its work: the queued word $(\texttt{k})$ has key $(1, (1), 0)$, shortlex-smaller than the candidate $(\texttt{s}, \texttt{k})$ at $(2, (0, 1), 0)$, so discarding the newcomer loses nothing.

Step 3: pop $(\texttt{k})$ (key $(1, (1), 0)$). Accept $(0, 0) \mapsto (\texttt{k})$. Extend:

- by `s`: denotation $s \circ k = (s(0), s(0)) = (1, 1)$ — new; push word $(\texttt{k}, \texttt{s})$ with key $(2, (1, 0), 0)$.
- by `k`: denotation $k \circ k = (0, 0)$ — already represented; skip.

Step 4: pop $(\texttt{k}, \texttt{s})$. Accept $(1, 1) \mapsto (\texttt{k}, \texttt{s})$. Extend:

- by `s`: $s \circ (1,1) = (0,0)$ — represented; skip.
- by `k`: $k \circ (1,1) = (0,0)$ — represented; skip.

Frontier empty; the closure completes.

3.3. The completed table and its evolution.

After step	$(0,1)$	$(1,0)$	$(0,0)$	$(1,1)$
seeding	—	—	—	—
1	ε_B	—	—	—
2	ε_B	(`s`)	—	—
3	ε_B	(`s`)	(`k`)	—
4	ε_B	(`s`)	(`k`)	(`k`, `s`)

The final catalog has `denotation_count` = 4, and the `normal_forms` listing, sorted by shortlex key, is ε_B, (`s`), (`k`), (`k`, `s`).

3.4. Verifying leastness by enumeration. Because the instance is tiny we can confirm shortlex-leastness against brute force. The words of length ≤ 2 and their denotations:

Word	Key	Denotes	Least for its denotation?
ε_B	$(0,(),0)$	$(0,1)$	yes
(`s`)	$(1,(0),0)$	$(1,0)$	yes
(`k`)	$(1,(1),0)$	$(0,0)$	yes
(`s`, `s`)	$(2,(0,0),0)$	$(0,1)$	no: ε_B is shorter
(`s`, `k`)	$(2,(0,1),0)$	$(0,0)$	no: (`k`) is shorter
(`k`, `s`)	$(2,(1,0),0)$	$(1,1)$	yes
(`k`, `k`)	$(2,(1,1),0)$	$(0,0)$	no

Every denotation's least word is exactly the table's representative. In particular $(1,1)$ — the constant to carrier index 1 — is reachable only at length two, and among length-two words only (`k`, `s`) denotes it, so leastness there is by uniqueness. Queries now behave as Chapter 9 describes: `normalize`((`s`, `k`)) = (`k`), `normalize`((`s`, `s`)) = ε_B, and `representative_for` on the arrow $(1,1)$ returns (`k`, `s`), while on any arrow over an object other than B it raises `UnreachableDenotationError`.

3.5. Order-dependence and the resource guard, exhibited. Two boundary behaviors of Chapter 8 are visible even at this scale. First, *representatives depend on the declared generator order*: re-declaring the catalog with `k` at rank 0 and `s` at rank 1 changes no denotation and no reachability, but the length-one words swap keys, and the constant $(1,1)$ keeps the same spelling with different key $(2,(0,1),0)$ — while in a richer catalog a reordering can change which of two equal-length words wins. Canonical forms are relative to the versioned catalog identity, exactly as the claim boundary states. Second, *the guard fails closed*: rerunning this catalog with `max_arrows` = 3 proceeds through steps 1–3 and then, at step 4, finds the accepted count already at the cap *before* inserting $(1,1)$ and raises `ClosureLimitExceeded` out of the constructor — so no catalog object exists, and no caller can mistake the three-entry table for a completed closure. The reader who traces these four steps has, in miniature, executed every branch of the closure engine: acceptance, both skip paths, the push-side deduplication, completion, and (in the rerun) the fail-closed guard.

APPENDIX B

The Proof-Obligation Ledger

This appendix reproduces, as a reference, the complete proof and integration ledger of the TKS v8.0 typed core prototype, exactly as recorded in the governing specification (`TKS_V8_CORE_FORMAL_SPEC_AND_PROOF_OBLIGATIONS_v0.1.md`, Section 5) at the bound snapshot. The discussion chapters of this monograph cite these rows constantly; here they are gathered in one place, first as a compact table and then row by row with the current evidence and status text kept faithful to the source document. The ledger is normative for the artifact: a claim not supported by a row's recorded status is not a claim the artifact makes. Drafting this monograph changed no row.

1. Reference table

ID	Obligation	Status
PO-TKS8-CORE-01	Identity and associativity for finite-function composition	PARTIAL FORMAL VERIFICATION PRESENT
PO-TKS8-CORE-02	Reject every source/target mismatch	EXECUTABLE VALIDATION
PO-TKS8-CORE-03	Keep one-carrier powers distinct from heterogeneous paths	EXECUTABLE VALIDATION
PO-TKS8-CORE-04	$f^{\lvert X\rvert !}$ is idempotent for finite endomaps	STANDARD RESULT / FORMALIZATION OPEN
PO-TKS8-CORE-05	Exact deterministic-lift norm formula	INFORMAL PROOF PRESENT / UNIVERSAL FORMALIZATION OPEN
PO-TKS8-CORE-06	Soundness and leastness of shortlex catalog	INFORMAL CORRECTNESS ARGUMENT / FORMALIZATION OPEN
PO-TKS8-CORE-07	Independent theorem-prover formalization	PARTIAL / PRESENT
PO-TKS8-CORE-08	Reconcile kernel object/arrow types with corrected v7.4 theorem records	OPEN / NOT LICENSED
PO-TKS8-CORE-09	Integrate with the active notation compiler	OPEN
PO-TKS8-CORE-10	Independent specialist review	OPEN

ID	Obligation	Status
PO-TKS8-CORE-11	Standard-basis lift preserves identities and composition	PARTIAL FORMAL VERIFICATION PRESENT / FINITE-SUM MATRIX BRIDGE OPEN

2. Per-row status and evidence

PO-TKS8-CORE-01. *Obligation.* Identity and associativity for finite-function composition. *Current evidence.* Pointwise argument; unit and bounded exhaustive tests; exact Lean `TypedCategory` lemmas. *Status.* PARTIAL FORMAL VERIFICATION PRESENT. *Reading.* The mathematics is Chapter 4; the executable coverage is the 19,683 three-point associativity triples and the typed identity families of Chapter 10; the formal fraction is the `TypedCategory` module of Chapter 11. The residual is the Python-to-Lean model equivalence, which no row of the ledger claims.

PO-TKS8-CORE-02. *Obligation.* Reject every source/target mismatch. *Current evidence.* Constructor and negative tests. *Status.* EXECUTABLE VALIDATION. *Reading.* The rejection sites are `compose`, the `Endomap` validator, the `SemigroupoidPath` validator, and catalog evaluation (Chapter 9); the negative fixtures of the category and normal-form suites drive each site to its typed error. Executable validation is the row's ceiling by design: the property is about runtime behavior of the implementation.

PO-TKS8-CORE-03. *Obligation.* Keep one-carrier powers distinct from heterogeneous paths. *Current evidence.* Separate runtime types and negative tests. *Status.* EXECUTABLE VALIDATION. *Reading.* `Endomap` and `SemigroupoidPath` are distinct runtime types; the path type exposes no power operation; the endomap wrapper rejects non-endo arrows with `CompositionTypeError`. The Lean seed's endomap-only powering discipline (Chapter 11) independently mirrors the separation in the model, though the ledger row itself records executable evidence.

PO-TKS8-CORE-04. *Obligation.* $f^{|X|!}$ is idempotent for finite endomaps. *Current evidence.* Standard finite-semigroup result; bounded exhaustive test. *Status.* STANDARD RESULT / FORMALIZATION OPEN. *Reading.* The tail-and-cycle proof is Chapter 5; the exhaustive check covers all 289 endomaps on carriers through size four via the binary powering pathway. Open: a Lean proof of the universal statement (functional-graph decomposition, divisibility of $|X|!$ by all cycle lengths).

PO-TKS8-CORE-05. *Obligation.* Exact deterministic-lift norm formula, $\|A_f\|^2 = \max_y |f^{-1}(y)|$. *Current evidence.* Block-Gram proof; bounded exhaustive structural tests; Lean checks only the exact two-state expansion, not the universal norm or the generic matrix/trace/rank bridge. *Status.* INFORMAL PROOF PRESENT / UNIVERSAL FORMALIZATION OPEN. *Reading.* The fiber-block spectral proof is Chapter 7; the structural suites tie the implemented shortcuts (norm, trace, rank, Gram forms) to matrix reality on the bounded family; the kernel-checked fragment is the two-state constant-lift expansion and noncontractivity (ledger record THM-LIFT-NORM-FIBER-FORMULA; counterexample record THM-LIFT-OCCUPATION-COUNTEREXAMPLE).

PO-TKS8-CORE-06. *Obligation.* Soundness and leastness of the shortlex catalog. *Current evidence.* Finiteness and pruning argument; regression tests. *Status.* INFORMAL CORRECTNESS ARGUMENT / FORMALIZATION OPEN. *Reading.* The pruning-soundness induction and the $|Y|^{|X|}$ finiteness bound are Chapter 8; the regression suite verifies representatives, counts, rejection, and fail-closed resource guards on hand-checkable catalogs (Appendix A works one in full). Open: formalization of the exploration invariants — shortlex-ordered dequeueing and first-insertion leastness.

PO-TKS8-CORE-07. *Obligation.* Independent theorem-prover formalization. *Current evidence.* Pinned Lean 4.32.1 seed and axiom audit at `formal/lean`; exact scope in `LEAN_PROOF_MANIFEST_v0.1.json`. *Status.* PARTIAL / PRESENT. *Reading.* Chapter 11 in full: toolchain pinned by commit, verifier hardcoding `propext` as the sole standard axiom, rejection of project axioms and `sorry`, source-hash checks, Lake build, per-theorem dependency audit. The verified and open scopes are itemized in Chapter 11; the row closes when the open scope does.

PO-TKS8-CORE-08. *Obligation.* Reconcile kernel object/arrow types with the corrected v7.4 theorem records. *Current evidence.* No semantic mapping admitted in this prototype. *Status.* OPEN / NOT LICENSED. *Reading.* The kernel deliberately encodes no Noetic or CAS construct (Chapter 9's boundary; Chapter 12's exclusions). This row is the sole gate through which canonical content could ever reach the substrate, and its licensing requires review, not drafting. The manifest's integration boundary field `cas_semantic_transfer_status: OPEN / NOT LICENSED` is this row's machine-readable form, and the boundary regression test asserts it on every suite run.

PO-TKS8-CORE-09. *Obligation.* Integrate with the active notation compiler. *Current evidence.* Explicitly outside this lane; requires compatibility design and review. *Status.* OPEN. *Reading.* The kernel is an independent reference model; the manifest asserts `active_cas_compiler_modified: false` and `production_runtime_dependency: false`, both pinned by the boundary regression. Integration needs its own design artifact and review lane (Chapter 12).

PO-TKS8-CORE-10. *Obligation.* Independent specialist review. *Current evidence.* No external review receipt. *Status.* OPEN. *Reading.* Every evidence channel is project-authored; this row is the only one addressing that fact. It gates the semantic rows (08, and any semantically loaded 09) and audits statement selection in the formal seed. Nothing blocks its start.

PO-TKS8-CORE-11. *Obligation.* The standard-basis lift preserves identities and composition. *Current evidence.* 1,678 exact bounded matrix-composition cases; Lean entry/basis/support proofs. *Status.* PARTIAL FORMAL VERIFICATION PRESENT / FINITE-SUM MATRIX BRIDGE OPEN. *Reading.* The executable half is the exact integer matrix sweep over all composable typed pairs through size three, including empty carriers (Chapter 10); the formal half is the identity-entrywise, identity-on-basis, typed-basis-composition, and relational-product support lemmas (Chapter 11). Open: finite-sum numerical matrix multiplication and arbitrary-vector linear extension, which would connect the basis-level Lean statements to the entrywise matrix statements the tests check.

3. Reading conventions

Three conventions govern citation of this ledger elsewhere in the repository and in downstream documents. First, *status strings are quoted, not paraphrased*: EXECUTABLE VALIDATION is not "verified," PARTIAL / PRESENT is not "formalized," and STANDARD RESULT is not "proved here." Second, *rows do not aggregate*: no combination of green rows yields a status any single row lacks — in particular, no combination amounts to canonical TKS v8 status, CAS semantic transfer, or specialist review. Third, *changes are events*: a row's status changes only with a new bound artifact snapshot (a revised specification and manifest with new hashes), never by narrative reference. The ledger as printed here is the ledger as of the v0.1 specification and the 2026-08-04 validation snapshot; any later document claiming a different status must exhibit the later snapshot that grounds it.

APPENDIX C

Glossary and Notation Index

This appendix collects the technical vocabulary and symbols of the monograph. Each entry gives a self-contained definition and, in parentheses, the chapters where the notion is introduced or most fully developed (Chapters 1 through 12, Appendices A through C). Terms are alphabetical; symbols follow in a separate index; the status vocabulary of the artifact's claim discipline closes the appendix. Definitions here are summaries for reference — where a glossary entry and a chapter's formal definition could be read differently, the chapter governs.

1. Terms

Adjoint (of the lift). The conjugate transpose A_f^* of the lift matrix; implemented as coordinate gathering, $(A_f^* w)_x = w_{f(x)}$. (Chapters 7, 9; Appendix A.)

Arrow. A total function between two declared finite objects, represented as a mapping tuple of target indices together with its source object, target object, and a presentation name excluded from equality. Implemented as `FiniteArrow`. (Chapters 3, 4, 9.)

Artifact binding. A manifest record tying a file path to its SHA-256 digest, byte count, and declared role, enabling the hash-check half of Gate 1 verification. (Chapters 9, 10.)

Axiom audit. The per-theorem extraction and checking of the axioms a kernel-checked proof term depends on; the seed's verifier admits only `propext` and rejects project axioms and `sorry`. (Chapter 11.)

Binary powering. Computing f^n by iterated squaring over the bits of n, in $O(\log n)$ compositions; the implementation of `Endomap.power`. (Chapters 5, 9; Appendix A.)

Bounded-exhaustive testing. Checking a property on every instance up to an explicit size bound, as opposed to sampling; justified by the small-scope hypothesis. (Chapter 10.)

Carrier. The finite, explicitly ordered sequence of distinct hashable elements of an object; its length is written $|X|$ and its positions are the standard indices. (Chapter 3.)

Catalog (generator catalog). A finite ordered sequence of declared objects together with a finite ordered sequence of typed generators over them; the input to normal-form closure, whose declared orders fix all shortlex ranks. Implemented as `NormalFormCatalog`. (Chapters 8, 9; Appendix A.)

Category laws. Left and right identity neutrality and associativity of composition, holding pointwise for total finite functions. (Chapter 4.)

Claim boundary. A statement, carried alongside a result, delimiting what the result does not establish; rendered in this monograph by the boundary environment and in the manifest by boundary fields. (Chapters 1, 7, 8, 10, 11, 12.)

Closure (denotational). The finite set of all typed denotations reachable by composing catalog generators from declared identities, computed to completion or refused; never returned partially. (Chapters 8, 9; Appendix A.)

Composition. The partial operation $g \circ f$, defined iff the declared target object of f equals the declared source object of g; realized by index chasing on mapping tuples. (Chapters 4, 9.)

Denotation. The extensional finite function an arrow or a typed word stands for — source, target, and mapping tuple, independent of presentation names. (Chapters 4, 8.)

Denotation key. The triple (source, target, mapping) used as the dictionary identity of a denotation in the closure engine; implemented as `FiniteArrow.denotation_key`. (Chapters 8, 9.)

Deterministic lift. The linear map A_f with $A_f e_x = e_{f(x)}$ on standard bases indexed by the ordered carriers; a column-selection zero-one matrix. Implemented as `DeterministicLift`. (Chapter 7; Chapters 9, 10; Appendix A.)

Endomap. An arrow whose declared source and target are the same object, wrapped in the type that alone carries monoid powers. Implemented as `Endomap`. (Chapters 5, 9; Appendix A.)

Error taxonomy. The four exception types of the package: `ValidationError` (malformed data), `CompositionTypeError` (categorical type mismatch), `ClosureLimitExceeded` (resource guard, fail closed), and `UnreachableDenotationError` (denotation outside the generated closure). (Chapter 9.)

Extensional equality. Equality of arrows by source, target, and mapping tuple only, with presentation names excluded; the executable form is dataclass equality with the name field marked non-comparing. (Chapters 4, 9.)

Factorial power (safe finite power). The power $f^{|X|!}$ of a finite endomap, always idempotent because every tail has entered its cycle and every cycle length divides $|X|!$. (Chapter 5; Appendix A.)

Fail-closed. The policy of raising an exception rather than returning partial or degraded output; realized by the closure resource guards and the bridge designs of the downstream consumers. (Chapters 8, 9, 12.)

Fiber. The preimage $f^{-1}(y)$ of a target element; fiber sizes determine the lift's Gram spectrum, norm, trace, and rank. (Chapter 7; Appendix A.)

Formal seed. The pinned, axiom-audited Lean 4 development kernel-checking a bounded scope of the monograph's mathematics; a starting point, not a formalized theory. (Chapter 11.)

Functional graph. The directed graph on a carrier with one edge $x \to f(x)$ per point; its components are trees hanging on unique cycles, the structure behind the factorial-power theorem. (Chapter 5; Appendix A.)

Functor laws (lift). $A_{\mathrm{id}_X} = I$ and $A_{g \circ f} = A_g A_f$; checked exactly on the bounded family and at basis level in Lean. (Chapters 7, 10, 11.)

Gate 1. The Project Coherence stage to which this prototype belongs; Gate 1 verification means hash checks against the manifest plus a full suite rerun. (Chapters 1, 9, 10.)

Generator. A labeled typed arrow declared in a catalog; the alphabet unit of typed words. (Chapters 8, 9.)

Gram matrix (input). $A_f^* A_f$, whose (x, x') entry is one iff $f(x) = f(x')$; block-diagonalizes into all-ones fiber blocks. (Chapter 7; Appendix A.)

Gram matrix (output). $A_f A_f^*$, diagonal with fiber sizes; distinct from the support projector whenever some fiber has size at least two. (Chapters 7, 9; Appendix A.)

Idempotent. An arrow or operator equal to its own square; for a deterministic lift, equivalent to the underlying map fixing every point of its image. Idempotent lifts need not be projectors. (Chapters 5, 7, 9; Appendix A.)

Isometry. An operator preserving inner products, $A^*A = I$; the lift A_f is an isometry iff f is injective. (Chapter 7.)

Kernel (proof kernel). The small trusted core of Lean that re-checks every proof term; the basis of the formal seed's trust story. (Chapter 11.)

Kernel (TKS v8.0 kernel). This artifact: the deliberately small executable reference model of typed finite-function semantics. The two senses of "kernel" are always disambiguated by context. (Chapters 1, 9.)

Manifest. The JSON artifact (TKS-V8-TYPED-CORE-PROTOTYPE v0.1.0) recording status, provenance, integration boundary, formal-verification and validation snapshots, claims, and artifact bindings; it does not hash itself. (Chapters 9, 10, 12.)

Manifest-boundary regression. The test module asserting the manifest's integration boundary fields, making claim drift a test failure. (Chapter 10.)

Noncontractivity fixture. The constant map on two points, whose lift is idempotent with norm $\sqrt{2}$ — the permanent counterexample blocking "lifts are contractive" and "idempotent lifts are projectors." (Chapters 7, 10, 11; Appendix A.)

Normal form. The shortlex-least well-typed word denoting a reachable denotation, relative to a declared, versioned catalog. (Chapter 8; Appendix A.)

Object. A named finite carrier; identity includes both the name and the ordered carrier. Implemented as `FiniteObject`. (Chapter 3.)

Ordered-input discipline. The package-wide rejection, at construction, of sets, mappings, and one-shot iterables wherever order is semantic; the defense against hash-order nondeterminism. (Chapters 3, 9.)

Orthogonal projector. A self-adjoint idempotent; among total deterministic lifts on one carrier, only the identity qualifies. (Chapters 7, 9.)

Partial isometry. An operator isometric on the orthogonal complement of its null space; for total standard-basis lifts, equivalent to injectivity of the underlying map. (Chapters 7, 9.)

Path (semigroupoid path). A nonempty sequence of arrows with matching adjacent objects, possibly across different carriers; has a typed composite and deliberately no power operation. Implemented as `SemigroupoidPath`. (Chapters 6, 9.)

Presentation name. The human-facing label of an arrow, excluded from equality and hashing; bounded in construction cost by the compact integer label scheme. (Chapters 4, 9.)

Pruning soundness. The lemma that expanding only the first representative of each denotation preserves all reachable denotations and all shortlex-least representatives. (Chapter 8; Appendix A.)

Representative-first closure. The exploration algorithm: shortlex-ordered search that stores the first word reaching each denotation and never expands duplicates. (Chapters 8, 9; Appendix A.)

Resource guard. Either of the two caps (accepted denotations; pending frontier) whose breach raises `ClosureLimitExceeded` from the catalog constructor. (Chapters 8, 9; Appendix A.)

Shortlex order. Word order by length first, then lexicographically by generator rank sequence; extended in the implementation by a start-object rank tie-break. (Chapter 8; Appendix A.)

Small-scope hypothesis. The empirical claim that most defects have small counterexamples, warranting exhaustive checking of small instances. (Chapter 10.)

Standard basis (standard indices). The orthonormal basis (e_x) of the Hilbert space attached to an object, indexed by carrier positions $0, \ldots, |X|-1$ in declared order. (Chapters 3, 7.)

Support projector (codomain). The diagonal zero-one matrix marking nonempty fibers — the orthogonal projector onto $\mathrm{im}(A_f)$. (Chapters 7, 9; Appendix A.)

Typed word. A start object together with a sequence of generator labels; syntax validated at construction, well-typedness certified by catalog evaluation; the empty word at X denotes id_X. (Chapters 8, 9; Appendix A.)

Validation snapshot. The manifest record of the 2026-08-04 suite run: result PASS, 20 tests, with the four bounded exhaustive case counts. (Chapter 10.)

Verifier (Gate 1). `scripts/verify_tks_v8_gate1.py`: recompute and compare all artifact hashes, then rerun the executable suite. (Chapter 10.)

Verifier (Lean). `scripts/verify_psr8_lean_gate1.py`: pinned-toolchain Lake build with hardcoded single-axiom policy, `sorry` rejection, source-hash checks, and per-theorem dependency audit. (Chapter 11.)

2. Symbols

Symbol	Meaning	Chapters
X, Y, Z	finite objects (named ordered carriers)	3–8
$\|X\|$	carrier length of X	3, 5, 7
$f : X \to Y$	a total finite-function arrow with declared types	3–8
id_X	the identity arrow on X	4
$g \circ f$	typed composition, defined iff target of f equals source of g	4
f^n	monoid power of an endomap ($f^0 = \mathrm{id}_X$)	5, App. A
$f^{\|X\|!}$	the safe finite (factorial) power, always idempotent	5, App. A
$\|X\|!$	factorial of the carrier size	5
$f^{-1}(y)$	the fiber of f over y	7, App. A
$\mathrm{im}f$, $\mathrm{im}(A_f)$	image of a map; range of a lift	7
H_X	the Hilbert space with standard basis indexed by X	7
e_x	the standard basis vector at carrier position x	7
A_f	the deterministic standard-basis lift of f	7, 9, App. A
A_f^*	the adjoint (conjugate transpose) of A_f	7, App. A
$A_f^* A_f$	the input-collision Gram matrix	7, 9, App. A
$A_f A_f^*$	the output Gram matrix, diagonal of fiber sizes	7, 9, App. A
$\|A_f\|$	the operator norm, $\sqrt{\max_y \|f^{-1}(y)\|}$	7, App. A
$\mathrm{tr}(\cdot)$	matrix trace; $\mathrm{tr}(A_f^* A_f) = \|X\|$	7, App. A
$\mathrm{rank}(\cdot)$	matrix rank; $\mathrm{rank}(A_f) = \|\mathrm{im}f\|$	7, App. A
J_m	the $m \times m$ all-ones matrix, eigenvalues m and 0	7, App. A
I, I_{H_X}	the identity matrix / operator	7
$G = (g_0, \ldots, g_{k-1})$	a declared, ordered generator catalog	8
w, u, v	typed words over a catalog	8, App. A
$[\![w]\!]$	the denotation of a typed word	8
$<_{\mathrm{slex}}$	shortlex order on typed words	8, App. A

Symbol	Meaning	Chapters
$\mathrm{NF}_G(a)$	the shortlex-least word denoting a over catalog G	8, App. A
ε_X	the empty word at object X, denoting id_X	8, App. A
$\lvert Y\rvert^{\lvert X\rvert}$	the count of total maps $X \to Y$; the finiteness bound	8, 10
$\mathcal{C}$	the typed finite-function category of the kernel	2–4
Ob, Hom	objects and hom-sets of a category	2
Set	the category of sets (mathematical background)	2
PSh, Sh	presheaf and sheaf constructions (open scope; companion program)	2, 11, 12

3. Status vocabulary

The artifact's claim discipline uses a fixed vocabulary, quoted from the specification and manifest; these words are load-bearing and are never paraphrased in this monograph.

Gate 1 prototype / in review / not frozen. The artifact's document status: a candidate under review, whose content may change before any freeze. (Chapters 1, 9.)

Ring 4. The admission ring of the artifact and of every research transfer touching it: the outermost ring, farthest from licensed canonical status. (Chapters 1, 12.)

Not licensed. No permission exists to rely on the item as established or to transfer it downstream; distinct from "false" — a true statement can be unlicensed. (Chapters 1, 12; Appendix B.)

Open. The obligation or question has no closing evidence receipt; drafting documents does not close it. (Appendix B.)

Executable validation. Evidence consisting of passing tests of the running implementation, bounded by their scopes. (Chapter 10; Appendix B.)

Partial formal verification present. A pinned, axiom-audited proof-assistant development checks part — and only the named part — of the relevant scope. (Chapter 11; Appendix B.)

Informal proof present / informal correctness argument. A complete human-checked mathematical argument exists without a kernel-checked counterpart. (Chapters 7, 8; Appendix B.)

Standard result. The mathematics is classical and carries no novelty claim; the ledger records provenance, not discovery. (Chapters 5, 12; Appendix B.)

Pass (validation snapshot). The recorded outcome of the 2026-08-04 suite run — 20 tests, four bounded exhaustive families — certifying exactly the bounded behavior and nothing beyond it. (Chapter 10.)

Bibliography

[1] S. Mac Lane and I. Moerdijk, *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*, Springer, 1994.

[2] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998.

[3] The Stacks Project Authors, "Sites and sheaves: the topology defined by a site," Tag 00ZC, https://stacks.math.columbia.edu/tag/00ZC.

[4] The Stacks Project Authors, "Sheafification," Tag 00W1, https://stacks.math.columbia.edu/tag/00W1.

[5] The Stacks Project Authors, "Representable sheaves," Tag 00WO, https://stacks.math.columbia.edu/tag/00WO.

[6] P. T. Johnstone, *Sketches of an Elephant: A Topos Theory Compendium*, Oxford University Press, 2002.

[7] S. Awodey, *Category Theory*, 2nd ed., Oxford University Press, 2010.

[8] E. Riehl, *Category Theory in Context*, Dover, 2016.

[9] N. N. Vorob'ev, "Consistent families of measures and their extensions," *Theory Probab. Appl.* 7 (1962), 147–163.

[10] S. Abramsky and A. Brandenburger, "The sheaf-theoretic structure of non-locality and contextuality," *New J. Phys.* 13 (2011), 113036.

[11] J. Hansen and R. Ghrist, "Toward a spectral theory of cellular sheaves," *J. Appl. Comput. Topol.* 3 (2019), 315–358.

[12] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed., Cambridge University Press, 2013.

[13] J.-É. Pin, "Finite semigroups and recognizable languages: an introduction," in *Semigroups, Formal Languages and Groups*, Kluwer, 1995.

[14] J. M. Howie, *Fundamentals of Semigroup Theory*, Oxford University Press, 1995.

[15] D. Jackson, *Software Abstractions: Logic, Language, and Analysis*, rev. ed., MIT Press, 2012.

[16] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," *CADE 28*, 2021.

[17] R. Hammack, *Book of Proof*, 3rd ed., 2018.

[18] IEEE Std 1076-2019, *VHDL Language Reference Manual*.

[19] D. Harel, "Statecharts: a visual formalism for complex systems," *Sci. Comput. Program.* 8 (1987), 231–274.

[20] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.* 1 (1972), 146–160.

[21] IEEE Std 754-2019, *Floating-Point Arithmetic*.

[22] W3C, *PROV-O: The PROV Ontology*, 2013.

[23] R. G. Sargent, "Verification and validation of simulation models," *J. Simulation* 7 (2013), 12–24.

[24] K. Claessen and J. Hughes, "QuickCheck: a lightweight tool for random testing of Haskell programs," *ICFP* (2000).

[25] S.-i. Amari, *Information Geometry and Its Applications*, Springer, 2016.

[26] N. Ay, J. Jost, H. V. Lê, and L. Schwachhöfer, *Information Geometry*, Springer, 2017.

[27] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed., Cambridge University Press, 2009.

[28] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, 2020.

[29] C. Lamb and S. Zacchiroli, "Reproducible builds: increasing the integrity of software supply chains," *IEEE Software* 39(2), 2022.

[30] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed., Wiley, 2020.

[31] A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014.

[32] JCGM 200:2012, *International Vocabulary of Metrology—Basic and General Concepts and Associated Terms*, 3rd ed.

[33] JCGM 100:2008, *Evaluation of Measurement Data—Guide to the Expression of Uncertainty in Measurement*.

[34] E. Buckingham, "On physically similar systems," *Phys. Rev.* 4 (1914), 345–376.

[35] L. O. Chua, "Memristor—the missing circuit element," *IEEE Trans. Circuit Theory* 18 (1971), 507–519.

[36] L. O. Chua and S. M. Kang, "Memristive devices and systems," *Proc. IEEE* 64 (1976), 209–223.

[37] M. C. Kennedy and A. O'Hagan, "Bayesian calibration of computer models," *J. R. Stat. Soc. Ser. B* 63 (2001), 425–464.

[38] C. Ritz, F. Baty, J. C. Streibig, and D. Gerhard, "Dose-response analysis using R," *PLoS ONE* 10(12), 2015.

[39] A. Zou et al., "Representation engineering: a top-down approach to AI transparency," arXiv:2310.01405, 2023.

[40] N. Elhage et al., "Toy models of superposition," Anthropic, 2022.

[41] A. Vaswani et al., "Attention is all you need," *NeurIPS*, 2017.

[42] R. A. Fisher, "On the mathematical foundations of theoretical statistics," *Philos. Trans. R. Soc. A* 222 (1922), 309–368.

[43] C. R. Rao, "Information and the accuracy attainable in the estimation of statistical parameters," *Bull. Calcutta Math. Soc.* 37 (1945), 81–91.

[44] K. Claessen, "Shrinking and showing functions," *Haskell Symposium*, 2012.

[45] B. Beizer, *Software Testing Techniques*, 2nd ed., Van Nostrand Reinhold, 1990.

[46] D. E. Knuth, *Literate Programming*, CSLI, 1992.